Universidad San Carlos de Guatemala.

Centro Universitario de Occidente.

Curso: Seminario de Sistemas 1.

Catedrático: Ing. Mario Tobar.

Proyecto: Barber Kongs — Sistema de Gestión para Barbería.

Portafolio Técnico

Fase 3

Nombre: Miguel Angel Quemé Gómez

Carné: 202031398

Nombre: Brandon Gustavo Güinac Román

Carné: 201931217

Quetzaltenango, 25 de octubre del 2026.

Índice

DOCUMENTACIÓN TÉCNICA

1. Manual Técnico Integral 4

1. Visión general del sistema 4
2. Arquitectura de despliegue 5
3. Arquitectura del backend 7
4. Flujos principales 12
5. Arquitectura del frontend 15
6. Seguridad 17
7. Calidad, integración continua y flujo de trabajo 17
8. Modelado de datos 18
9. Despliegue, operación y respaldo 20
10. Trazabilidad 22

Anexo A. Puesta en marcha para desarrolladores 23
Anexo B. Documentos complementarios 23

2. Modelo de Datos 25

1. Principios del diseño de datos 25
2. Modelo entidad-relación (Chen) 26
3. Modelo relacional (diagramas de tablas) 29
4. Diccionario de datos 31
5. Reglas de integridad 35
6. Índices 36
7. Seguridad de los datos 37
8. Historial de migraciones y cambios del modelo 37

3. Infraestructura, Despliegue y Backup 39

1. Arquitectura e infraestructura cloud final (4.2) 39
2. Guía de despliegue / Runbook 40
3. Estrategia de backup (5.1) 42

4. Matriz de Trazabilidad 44

1. Resumen 44
2. Matriz de requisitos funcionales 46
3. Matriz de casos de uso 48
4. Matriz de historias de usuario 50
5. Matriz de requisitos no funcionales 51
6. Matriz inversa de endpoints 52
7. Evidencia en Git 54

Anexo A. Pruebas automáticas y requisitos que verifican 57

5. Casos de Uso e Historias de Usuario 65

1. Actores y alcance 65
2. Especificación de casos de uso 66
3. Historias de usuario 76

6. Inventario de Endpoints 79

1. Convenciones de la API 79
2. Objetos que devuelve la API 80
3. Resumen de endpoints 80
4. Detalle por módulo 83
5. Cobertura de pruebas por endpoint 99
6. Hallazgos 100

7. Plan de Pruebas..... 102
1. Objetivos de las pruebas..... 102
2. Alcance..... 102
3. Estrategia de pruebas..... 103
4. Entorno de pruebas..... 103
5. Ejecución y automatización..... 104
6. Casos de prueba..... 105
7. Trazabilidad entre requisitos y pruebas..... 106
8. Resultados de la ejecución..... 107
9. Hallazgos..... 110
10. Limitaciones y riesgos..... 111
11. Criterios de aceptación y plan de mejora..... 111
Anexo A. Catálogo de casos de prueba..... 113

ESCALABILIDAD FUTURA

8. Documentación para Escalabilidad Futura..... 121
1. Declaración de alcance..... 121
2. Punto de partida..... 121
3. Procesos futuros identificados y diagramados..... 122
4. Migraciones necesarias..... 131
5. Tecnologías nuevas y su justificación..... 133
6. Hoja de ruta..... 134
7. Riesgos..... 134
Anexo A. SQL de las migraciones propuestas..... 135
Anexo B. Diagramas BPMN..... 139

MANUAL DE USUARIO

9. Manual de Usuario..... 141
1. Roles de usuario y requisitos..... 141
2. Funciones comunes a todos los roles..... 141
3. Guía del cliente..... 146
4. Guía del barbero..... 150
5. Guía del administrador..... 152
6. Mensajes que puede encontrar..... 162
7. Preguntas frecuentes..... 162

Cada documento conserva su propia numeración de página; el número del índice es la página de este PDF. El PDF incluye marcadores para navegar entre documentos y capítulos.

Manual Técnico Integral

Campo	Detalle
Proyecto	Barber Kongs — Sistema de Gestión para Barbería (Fase 3)
Equipo	Miguel Angel Quemé Gómez · Brandon Gustavo Güinac Román
Versión del documento	1.0 — 25 de octubre de 2026
Código descrito	Backend: repositorio Barber-Kong, rama `main`. Frontend: repositorio Barber-Kong-Frontend, rama `main`.
Contenido	Arquitectura detallada, modelado de datos, despliegue, seguridad, calidad y trazabilidad

1. Visión general del sistema

1.1 Qué hace

Barber Kongs gestiona una barbería de punta a punta: los clientes se registran y agendan, reprograman o cancelan sus citas; los barberos consultan su agenda diaria y semanal y marcan las atenciones; el administrador gestiona barberos, clientes, servicios e inventario y obtiene reportes. Al marcar una cita como atendida, el sistema descuenta automáticamente los insumos que consume el servicio y avisa cuando alguno cae bajo su mínimo.

El alcance funcional está definido por 31 requisitos funcionales y 11 no funcionales, 23 casos de uso y 23 historias de usuario, implementados en 8 módulos de backend y 48 endpoints.

1.2 Tecnologías

Capa	Tecnología	Uso
Frontend	React 19, React Router 7, Vite 8	Aplicación de una sola página (SPA); `oxlint` como linter.
Backend	Node.js 20 (ESM), Express 4	API REST con módulos independientes.
Acceso a datos	`pg` 8 (node-postgres), SQL plano	Sin ORM: cada módulo escribe su SQL parametrizado.
Base de datos	PostgreSQL 16	Supabase en producción; contenedor `postgres:16-alpine` en local y en CI.
Validación	Ajv 8 (+ ajv-formats, ajv-errors)	Esquemas JSON con mensajes en español.
Seguridad	`jsonwebtoken` 9, `bcrypt` 6, `helmet` 8, `cors`	Sesión por JWT, hash de contraseñas, cabeceras seguras.
Correo y reportes	`nodemailer` 10, `exceljs` 4, `pdfkit` 0.20	Notificaciones por correo y exportación a Excel y PDF.
Bitácora	`winston` 3, `morgan`	Registro estructurado de solicitudes y errores.
Pruebas	Vitest 4, Supertest 7, `@vitest/coverage-v8`	Pruebas unitarias y de integración; cobertura.
Infraestructura	Vercel, Supabase, GitHub, GitHub Actions, Docker Compose	Despliegue, base de datos, repositorios, integración continua y entorno local.

2. Arquitectura de despliegue

Barber Kongs es una arquitectura serverless de tres capas, sin servidores propios que administrar: el frontend y el backend viven como dos proyectos separados en Vercel y los datos, en una base PostgreSQL administrada por Supabase.

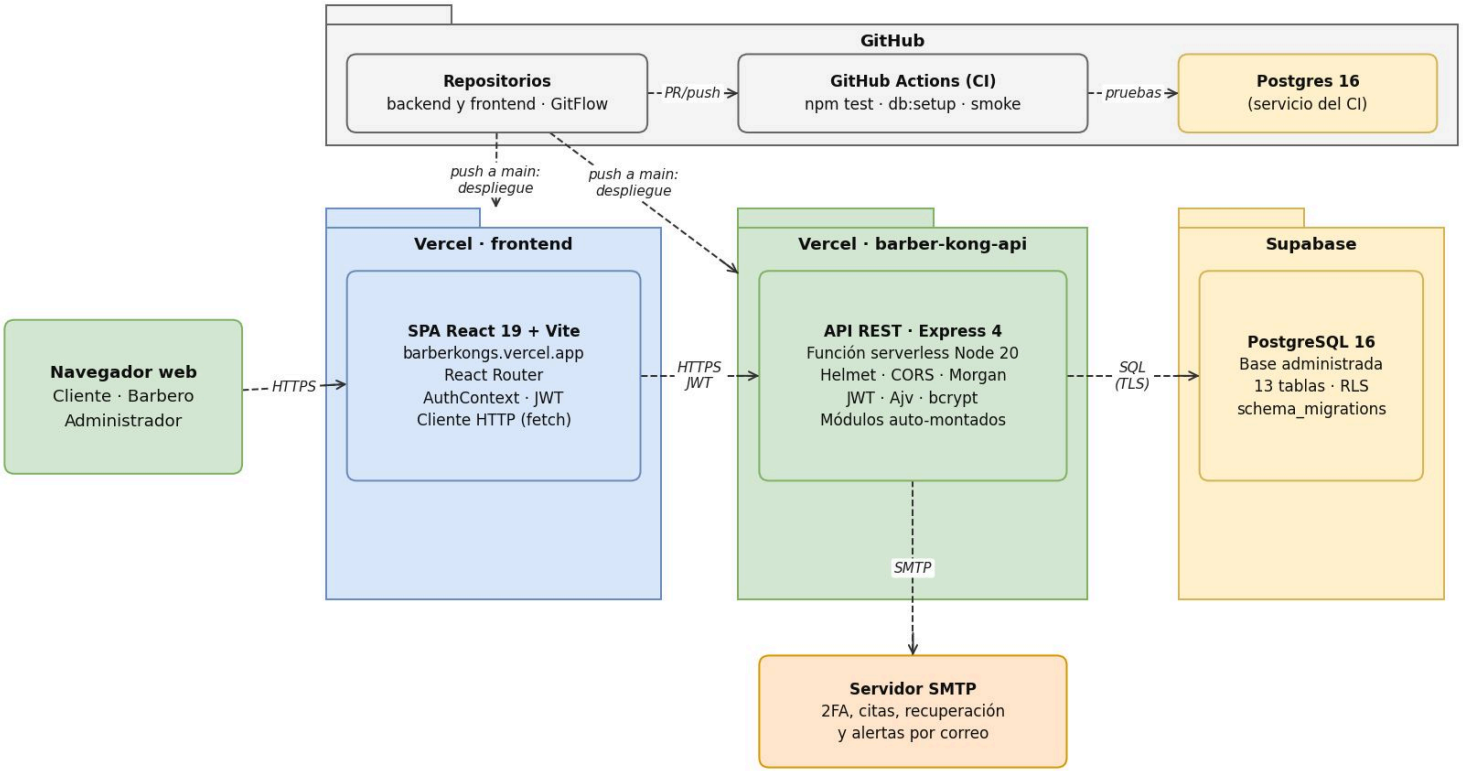


Figura 1. Arquitectura de despliegue.

Componente	Descripción
Navegador	Cliente de todos los roles. Habla con el frontend y con la API únicamente por HTTPS.
Frontend (Vercel)	SPA de React compilada con Vite y servida como contenido estático en `barberkongs.vercel.app`. Las variables `VITE_*` se fijan al compilar.
Backend (Vercel)	Proyecto `barber-kong-api`: la aplicación Express se publica como función serverless (`api/index.js`) y reutiliza el mismo `createApp()` que el servidor local.
Base de datos (Supabase)	PostgreSQL 16 administrado. El backend se conecta con TLS (`DB_SSL=true`) y un pool pequeño (`DB_POOL_MAX`) para no agotar las conexiones bajo concurrencia.

Componente	Descripción
Correo (SMTP)	Transporte SMTP configurable por variables de entorno para códigos 2FA, recuperación de contraseña, confirmaciones de cita y alertas de inventario.
GitHub y Actions	Repositorios del backend y del frontend con GitFlow. Cada pull request y push a `develop` o `main` ejecuta la integración continua de su repositorio (en el backend, pruebas; en el frontend, lint y build). Un push a `main` dispara el despliegue en Vercel y, en el backend, la aplicación de las migraciones.

2.1 Por qué Vercel y Supabase

Se eligieron por dos razones. La primera es el conocimiento del equipo: ya conocía ambas plataformas, lo que reduce la curva de aprendizaje y el tiempo de puesta en producción. La segunda es el costo: el cliente no estuvo dispuesto a pagar el costo adicional que implicaría AWS, mientras que Vercel y Supabase se pueden operar en sus planes gratuitos.

2.2 Entornos

Entorno	Cómo se ejecuta	Base de datos
Local con Docker	`docker compose up --build` levanta base, migraciones, API (puerto 3000) y frontend (puerto 5173).	Contenedor `postgres:16-alpine`, base `barber_kong`.
Local sin Docker	`npm run dev` (backend, `.env.dev`) y `npm run dev` en el frontend (Vite).	Postgres local.
Pruebas	`npm test` con `.env.test`. La suite se niega a correr si `NODE_ENV` no es `test` o si la base no termina en `_test`.	Base `barber_kong_test`.
Integración continua	Backend: flujo `ci.yml` de GitHub Actions con un servicio Postgres 16. Frontend: flujo `ci.yml` con lint y build.	Bases efímeras `barber_kong` y `barber_kong_test` (backend).
Producción	Vercel (frontend y backend) con variables de entorno del proyecto.	Supabase.

3. Arquitectura del backend

3.1 Paquetes y dependencias

El backend se organiza en paquetes con responsabilidades claras. Toda solicitud atraviesa primero la configuración común de Express (`src/server`) y luego el módulo que le corresponde; los módulos usan los paquetes transversales y, en pocos puntos, se apoyan unos en otros.

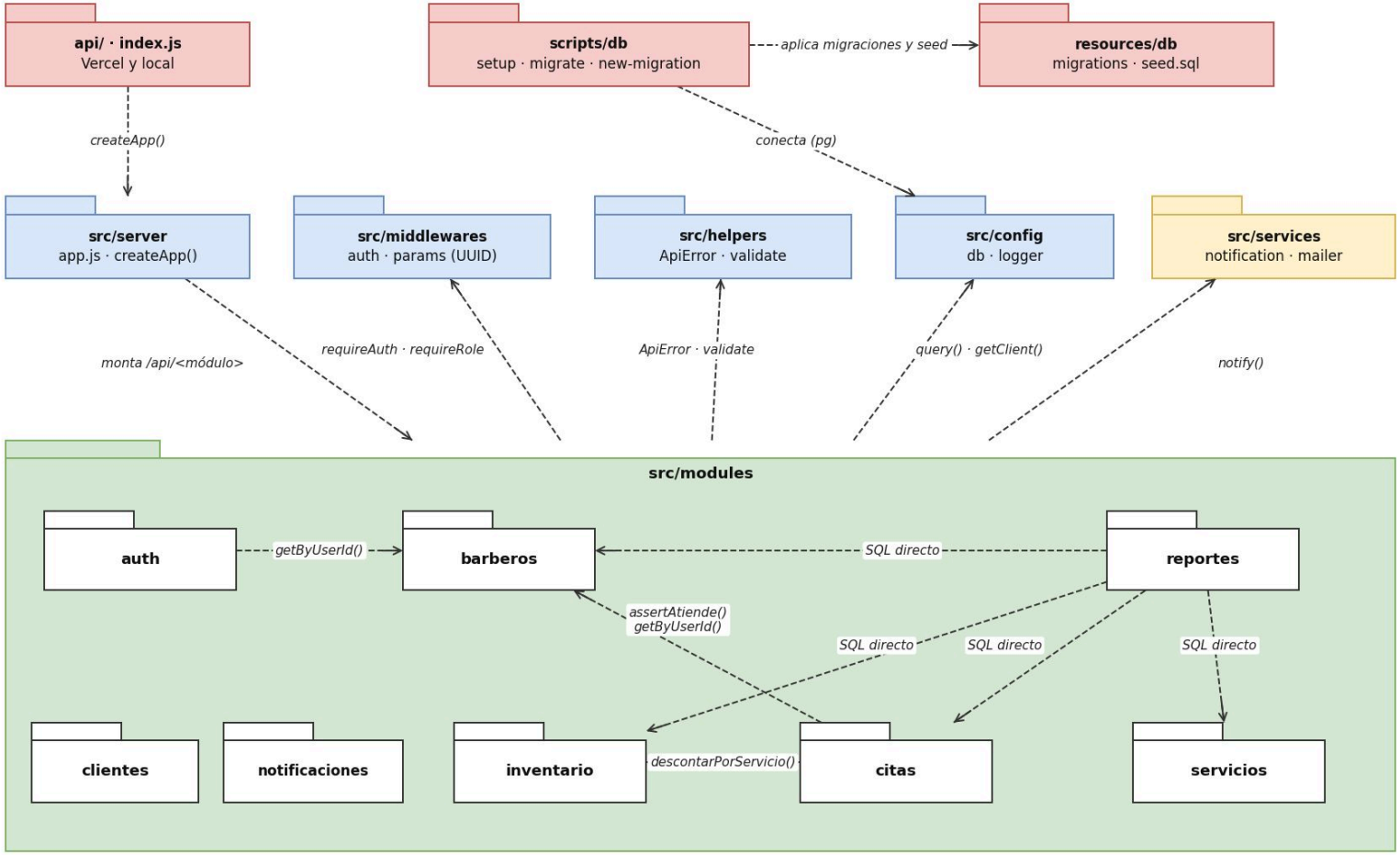


Figura 2. Diagrama de paquetes del backend.

Paquete	Contenido	Lo usan
src/server	app.js con createApp(): arma Express con helmet, cors, morgan, JSON, GET /api/health, el montaje de módulos y el manejo de errores.	index.js (local y Docker) y api/index.js (Vercel)
src/middlewares	auth.middleware.js (requireAuth, requireRole) y params.middleware.js (identificadores UUID).	Las rutas de todos los módulos
src/helpers	ApiError, validate (Ajv) y errorHandler.	Controladores, servicios y src/server
src/config	db.js (pool de pg, query, getClient) y logger.js (winston).	Repositorios, servicios y scripts
src/services	notification.service, mailer y emailTemplate.	auth, citas e inventario
src/modules	Los 8 módulos de negocio.	src/server los monta en /api/<módulo>
resources/db	Migraciones SQL numeradas y seed.sql.	scripts/db
scripts/db	setup, migrate y new-migration.	Desarrollo, CI y despliegue

Dentro de src/modules los módulos son independientes salvo en estos puntos: auth consulta a barberos para completar el perfil del barbero (getByUserId); citas consulta a barberos (assertAtiende, getByUserId) y a inventario (descontarPorServicio, notificarStockBajo); y reportes lee directamente con SQL las tablas de citas, servicios, barberos e inventario. clientes, servicios y notificaciones no dependen de otros módulos.

3.2 Paquetes de un módulo y sus capas

Cada módulo repite la misma estructura, con una sola dirección de dependencia: rutas, controlador, servicio y repositorio. El esquema de validación lo usa el controlador.

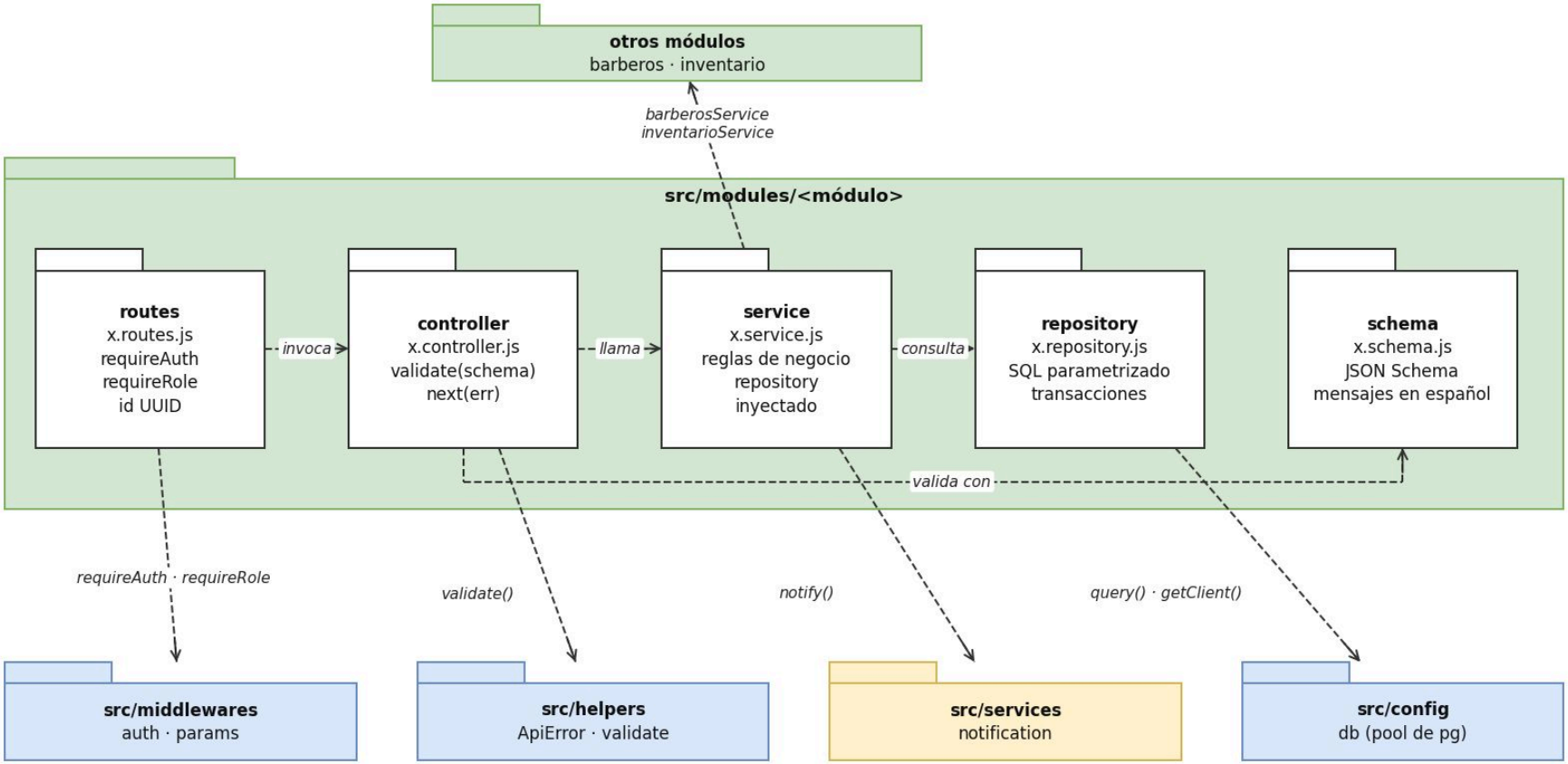


Figura 3. Paquetes de un módulo y sus dependencias.

Capa	Archivo	Responsabilidad
Rutas	`*.routes.js`	Declara los endpoints y aplica `requireAuth` y `requireRole`. Los parámetros de ruta que son identificadores se validan como UUID en un solo lugar (`params.middleware.js`).
Controlador	`*.controller.js`	Valida el cuerpo o la consulta con el esquema Ajv, llama al servicio y responde. Los errores se propagan con `next(err)`.
Esquema	`*.schema.js`	JSON Schema con `additionalProperties: false` y mensajes de error en español.
Servicio	`*.service.js`	Reglas de negocio. Se construye con una fábrica que recibe su repositorio, de modo que las pruebas unitarias inyectan uno falso.
Repositorio	`*.repository.js`	Único lugar con SQL del módulo; consultas parametrizadas y transacciones con `getClient()`.

3.3 Estructura de carpetas

```
index.js                    Arranque local y Docker (app.listen)
api/index.js                Punto de entrada serverless para Vercel
src/
  server/app.js             createApp(): helmet, cors, morgan, JSON, health, módulos, errores
  modules/<módulo>/         routes · controller · schema · service · repository
  modules/index.js          Descubre y monta cada *.routes.js en /api/<módulo>
  middlewares/              auth.middleware.js · params.middleware.js
  helpers/                  ApiError · validate (Ajv) · errorHandler
  services/                 notification · mailer · emailTemplate
  config/                   db.js (pool de pg) · logger.js (winston)
resources/db/               migrations/NNNN_*.sql · seed.sql
scripts/db/                 setup · migrate · new-migration
tests/                      unit/ · integration/ · helpers/ · load/
```

3.4 Módulos

Módulo	Responsabilidad	Endpoints	Tablas principales
auth	Registro, inicio de sesión, doble factor, recuperación de contraseña y sesión actual.	7	`users`, `clientes`, `two_factor_codes`, `password_reset_tokens`
barberos	Alta, edición y baja de barberos; horario semanal; días no disponibles; disponibilidad por fecha.	11	`barberos`, `disponibilidad_barberos`, `ausencias_barbero`
citas	Agendar, reprogramar, cancelar, walk-in, atender y consultar la agenda.	7	`citas`
clientes	Registro, búsqueda, historial, edición y activación de clientes.	7	`clientes`
servicios	Catálogo de servicios y asociación de insumos a cada servicio.	8	`servicios`, `servicio_insumos`
inventario	Insumos, entradas de reabastecimiento, descuento automático y alertas de stock bajo.	4	`insumos`, `insumo_movimientos`
notificaciones	Centro de notificaciones del usuario.	2	`notificaciones`
reportes	Reportes de citas, ingresos y consumo de insumos en PDF y Excel.	1	Lecturas de varias tablas

Además existe `GET /api/health`, que responde `{ "status": "ok" }` y usan la integración continua y el monitoreo.

3.5 Patrones y decisiones de diseño

- **Auto-montaje de módulos:** `modules/index.js` recorre `src/modules`, importa cada `*.routes.js` y lo monta en `/api/<carpeta>`. Agregar un módulo es crear su carpeta; no hay que registrar rutas a mano.
- **Fábricas con inyección de dependencias:** `createXService({ repository })` permite probar las reglas de negocio sin base de datos.

- **SQL plano sin ORM:** cada repositorio escribe consultas parametrizadas; da control total sobre las consultas y evita una dependencia pesada en un entorno serverless.
- **Identificadores UUID generados por la aplicación** (`randomUUID()`), sin valor por defecto en la base.
- **Errores centralizados:** los servicios lanzan `ApiError` con estado y mensaje para el usuario; `errorHandler` es el único que decide la respuesta. Los errores 5xx se registran completos y al cliente solo llega un mensaje genérico.
- **Transacciones solo donde hay varios pasos que deben ir juntos:** por ejemplo, marcar una cita como atendida junto con el descuento de inventario, o crear un barbero con su cuenta y su horario.
- **Los avisos nunca interrumpen el flujo principal:** `notify()` registra la notificación en la base y envía el correo; si el SMTP falla, solo se deja constancia en la bitácora y la operación del usuario continúa.
- **Preparado para serverless:** el `app` de Express se construye una vez por instancia de función y se reutiliza; el pool de conexiones es configurable y la bitácora en archivo se desactiva en Vercel.

3.6 Configuración

Toda la configuración llega por variables de entorno; ninguna credencial vive en el repositorio (`.env.example` solo lista las claves).

Variable	Para qué sirve
`PORT`, `NODE_ENV`	Puerto de escucha local y modo de ejecución (`development`, `test`, `production`).
`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`	Conexión a PostgreSQL.
`DB_SSL`	Activa TLS; necesario con Supabase.
`DB_POOL_MAX`	Tamaño máximo del pool; se recomienda 1 en funciones serverless.
`JWT_SECRET`, `JWT_EXPIRES_IN`	Firma de los tokens de sesión y su vigencia (2 horas por defecto).
`FRONTEND_URL`	Arma el enlace de recuperación de contraseña que se envía por correo.
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM`	Transporte de correo.
`LOG_LEVEL`	Nivel de detalle de la bitácora.
`VITE_API_URL` (frontend)	URL base de la API; se fija al compilar el frontend.

4. Flujos principales

4.1 Agendar una cita

El cliente envía el servicio, el barbero, la fecha y la hora. El controlador valida el cuerpo y el rol; el servicio comprueba que el servicio y el barbero existan y estén activos, que el barbero atienda en ese horario (horario semanal y ausencias) y que no haya traslape con otra cita. Si todo es correcto, crea la cita en estado `confirmada` y avisa al cliente y al barbero.

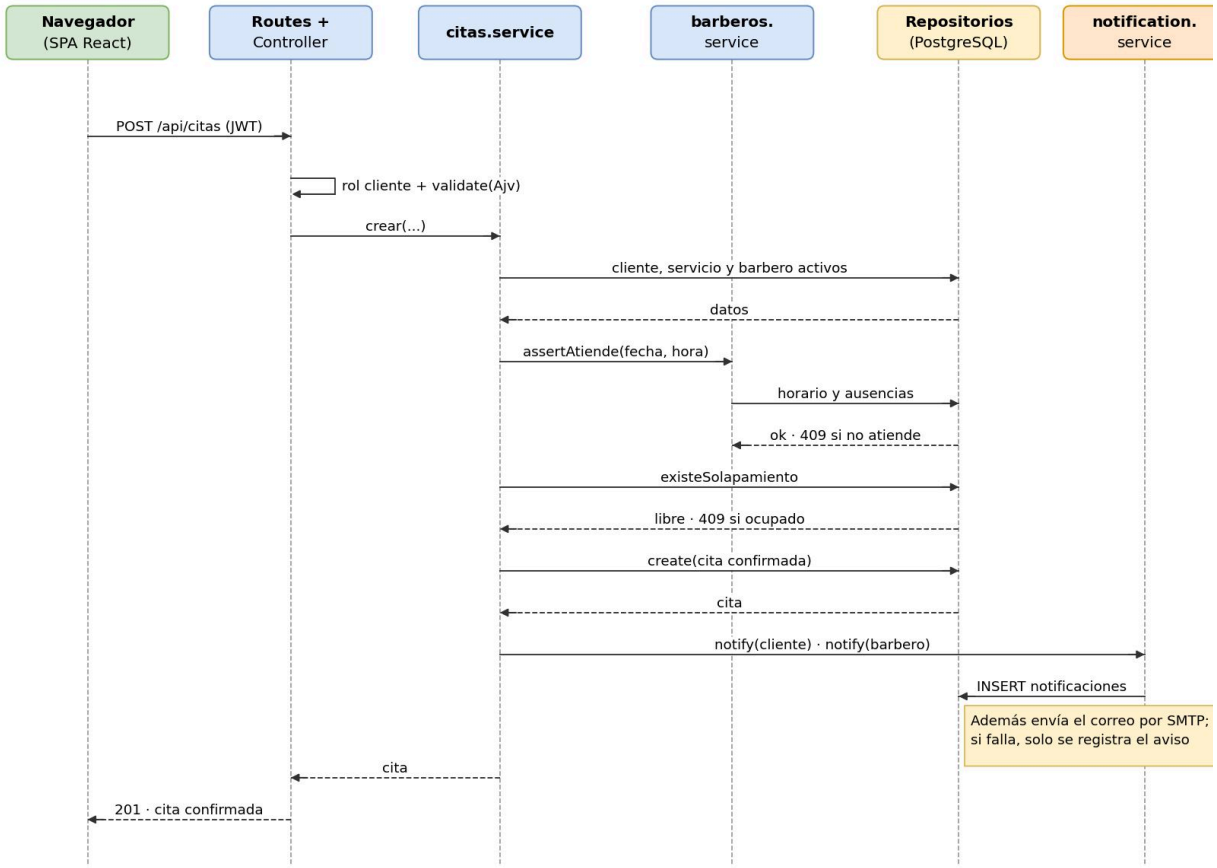


Figura 4. Secuencia de agendar una cita.

4.2 Marcar una cita como atendida

Es el proceso técnico más complejo: cambiar el estado de la cita y descontar los insumos deben confirmarse juntos o ninguno, por lo que corren en una sola transacción con una única conexión. Los avisos de stock bajo se envían solo después de confirmar la transacción.

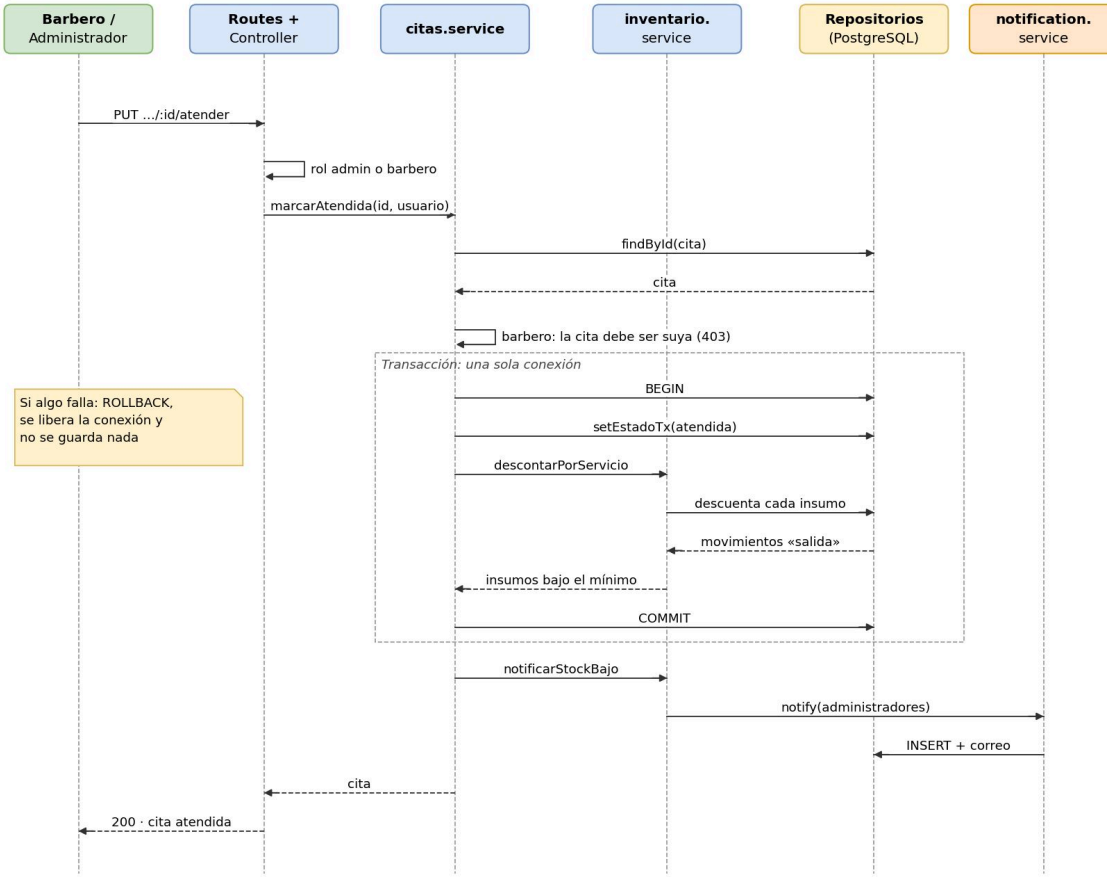


Figura 5. Secuencia de marcar una cita como atendida.

4.3 Inicio de sesión con doble factor

Si la cuenta tiene el doble factor activo, el inicio de sesión no entrega token: genera un código de 6 dígitos de un solo uso, guarda solo su huella (SHA-256) con vencimiento de 10 minutos y lo envía por correo. El token se entrega al verificar el código.

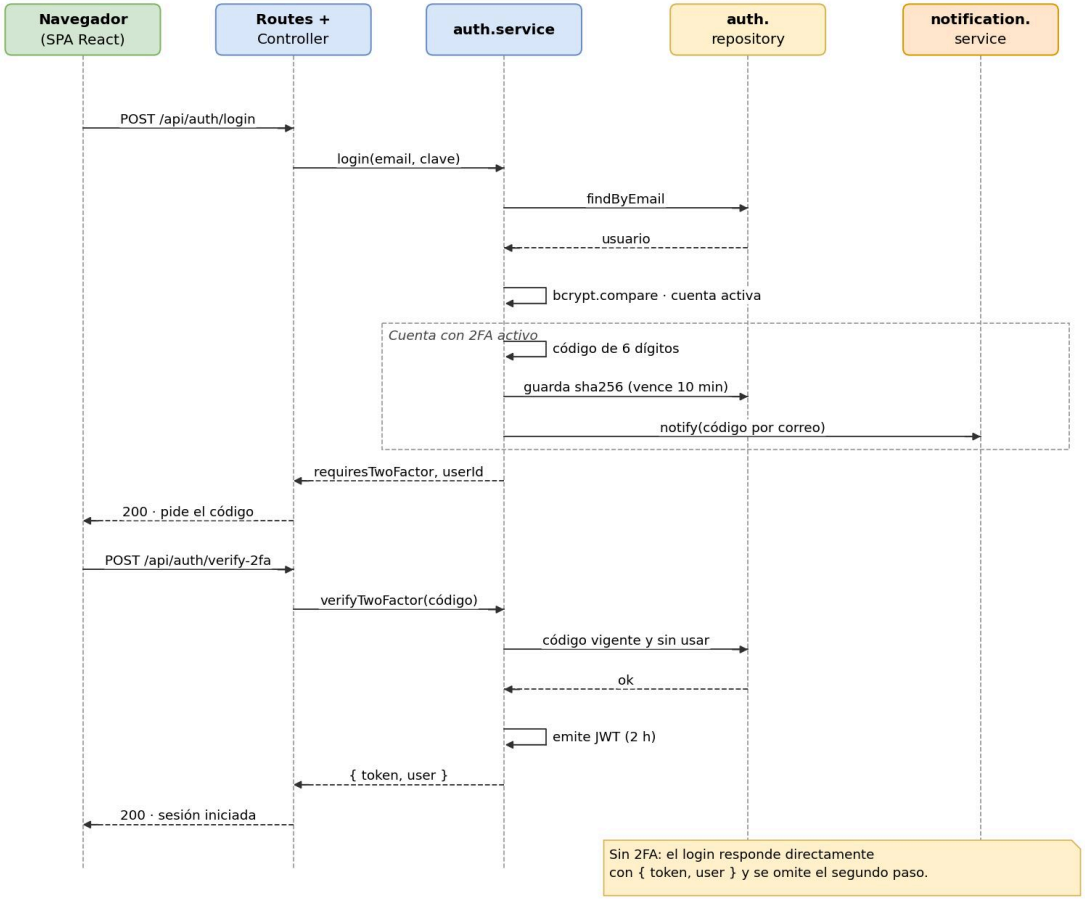


Figura 6. Secuencia del inicio de sesión con 2FA.

4.4 Recuperación de contraseña

El usuario pide el enlace con su correo; el sistema responde lo mismo exista o no la cuenta, para no revelar qué correos están registrados. Si existe, genera un token aleatorio, guarda solo su huella SHA-256 con vencimiento de 1 hora y envía por correo un enlace al frontend. Al restablecer, el token se consume y la nueva contraseña se guarda con bcrypt.

5. Arquitectura del frontend

El frontend es una SPA de React organizada en paquetes por tipo de archivo y, dentro de las páginas, por rol. `App.jsx` declara las rutas; `ProtectedRoute` exige sesión y, cuando se indica, un rol; `AuthContext` guarda el usuario y el token (en `localStorage`) y expone el inicio de sesión, el registro y el cierre de sesión. Todo acceso a la API pasa por `api/client.js`, que añade el token y convierte los errores del servidor en un `ApiClientError` con un mensaje listo para mostrar.

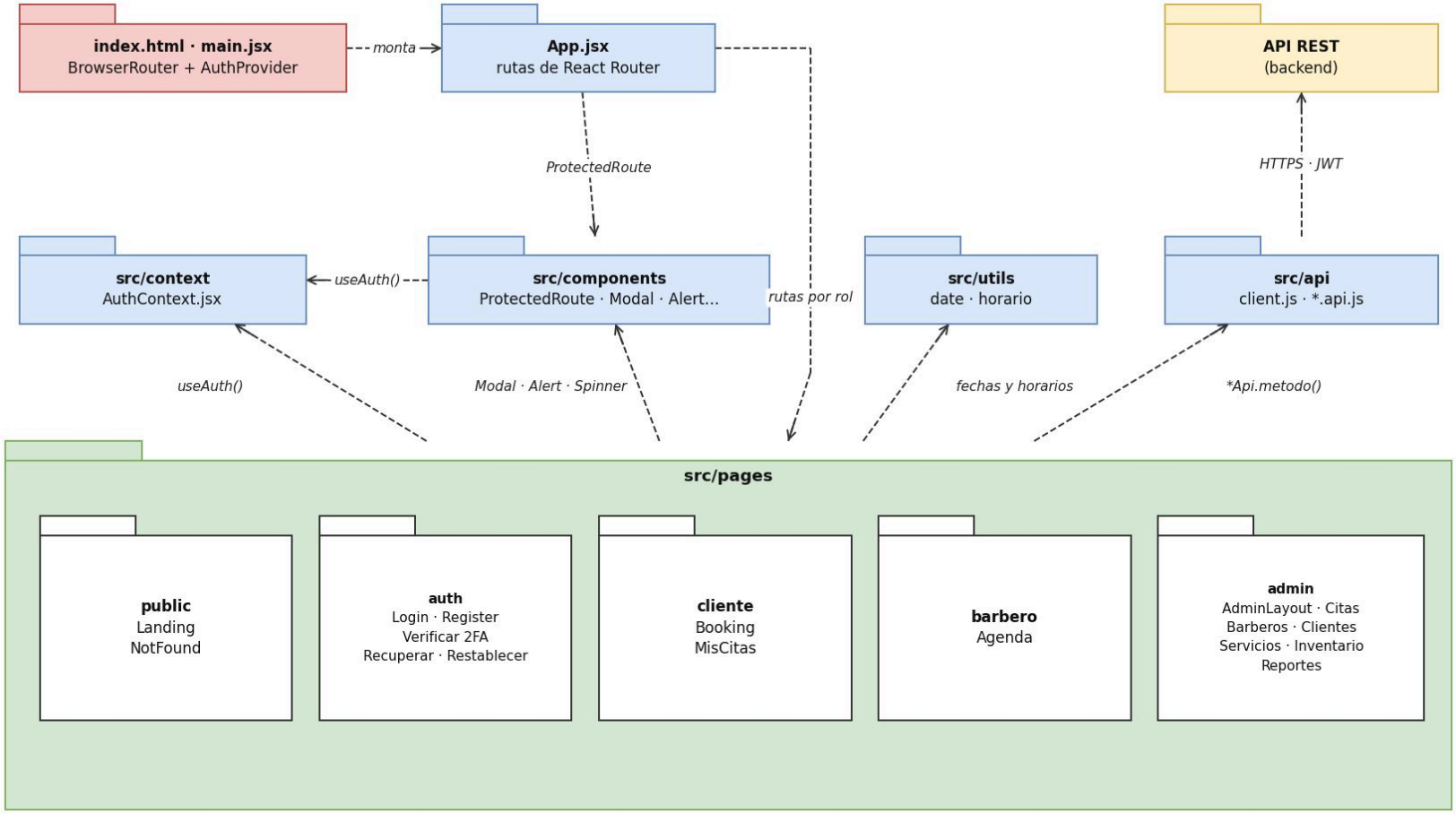


Figura 7. Diagrama de paquetes del frontend.

5.1 Rutas

Ruta	Pantalla	Acceso
`/`	Inicio: servicios, galería, equipo y horario	Público

Ruta	Pantalla	Acceso
`/ingresar`, `/registro`, `/verificar-2fa`, `/recuperar`, `/restablecer`	Autenticación y recuperación de contraseña	Público
`/agendar`, `/mis-citas`	Asistente de 4 pasos y citas del cliente	Cliente
`/barbero/agenda`	Agenda diaria y semanal, walk-in y días no disponibles	Barbero
`/admin/citas`, `/admin/barberos`, `/admin/clientes`, `/admin/servicios`, `/admin/inventario`, `/admin/reportes`	Panel de administración	Administrador
`/cuenta`, `/seguridad`	Cuenta y verificación en dos pasos	Cualquier usuario con sesión
*	Página no encontrada	Público

5.2 Despliegue del frontend

El frontend se compila con `vite build` y se publica como contenido estático. `vercel.json` redirige cualquier ruta a `index.html` para que React Router resuelva la navegación. La URL de la API se incorpora al compilar mediante `VITE_API_URL`.

6. Seguridad

Control	Cómo se implementa	Dónde
Cifrado en tránsito	HTTPS en frontend y API (Vercel termina TLS); conexión a la base con TLS.	Vercel · `config/db.js`
Autenticación	JWT firmado con `JWT_SECRET`, vigencia de 2 horas; se envía como `Authorization: Bearer`.	`auth.service.js` · `auth.middleware.js`
Contraseñas	Hash con bcrypt (10 rondas); nunca se devuelven en las respuestas.	`auth.service.js`
Autorización por rol	Cada ruta protegida declara `requireRole(...)` en el servidor, además de ocultar opciones en la interfaz.	`*.routes.js`
Autorización por propietario	El barbero solo consulta su agenda y solo marca como atendidas sus citas; el cliente solo gestiona las suyas.	`citas.service.js`
Doble factor	Código de 6 dígitos por correo, de un solo uso, guardado como SHA-256, vigencia de 10 minutos; opcional por cuenta.	`auth.service.js`
Recuperación de contraseña	Token aleatorio de un solo uso guardado como SHA-256, vigencia de 1 hora; respuesta idéntica exista o no el correo.	`auth.service.js`
Validación de entradas	Esquemas Ajv que rechazan campos desconocidos; identificadores de ruta validados como UUID; consultas validadas.	`*.schema.js` · `params.middleware.js`
Inyección SQL	Todas las consultas son parametrizadas; no se concatenan valores en el SQL.	`*.repository.js`
Cabeceras HTTP	`helmet()` aplica cabeceras de seguridad; `cors()` expone solo `Content-Disposition` para las descargas.	`server/app.js`
Base de datos	Seguridad a nivel de fila (RLS) activada en 12 de las 13 tablas de negocio (migración 0006): la base no queda expuesta por la API pública de Supabase.	`0006_rls_seguridad.sql`
Secretos	Claves, credenciales de base y de SMTP en variables de entorno de Vercel; nunca en el repositorio.	Vercel · `.env.example`
Errores	Los errores 5xx se registran completos y al cliente solo llega un mensaje genérico.	`errorHandler.js`

7. Calidad, integración continua y flujo de trabajo

7.1 Pruebas

La suite automática del backend tiene 202 casos (101 unitarios y 101 de integración) con Vitest y Supertest. Las pruebas de integración usan la aplicación Express real sobre PostgreSQL 16, con datos propios que se eliminan al terminar y el transporte de correo simulado. La cobertura de líneas es de 96.2 %, se verificó con 10 mutaciones deliberadas (10 detectadas) y un script de carga con 20 usuarios simultáneos midió un p95 de 10.7 ms en la consulta de disponibilidad. El detalle está en el Plan de Pruebas.

Comando	Para qué
`npm test`	Toda la suite (unitarias e integración).
`npm run test:unit` · `npm run test:integration`	Solo un nivel.
`npm run test:cov`	Suite con reporte de cobertura.
`npm run db:setup:test`	Prepara la base `barber_kong_test`.
`npm run test:carga`	Prueba de carga manual (20 usuarios).

7.2 Integración continua

El flujo `.github/workflows/ci.yml` se ejecuta en cada pull request y en cada push a `develop` o `main`:

1. Levanta un servicio PostgreSQL 16 y prepara Node 20.
2. Instala las dependencias con `npm ci`.
3. Valida que el esquema y los datos de ejemplo se apliquen sin errores (`npm run db:setup`).
4. Prepara la base de pruebas y ejecuta toda la suite (`npm test`).
5. Arranca el servidor y comprueba `GET /api/health` como verificación de humo.

El repositorio del frontend tiene su propio `.github/workflows/ci.yml`, con los mismos disparadores: instala con `npm ci` y ejecuta `npm run lint` y `npm run build`. En el backend, el flujo `.github/workflows/migrate.yml` aplica las migraciones en la base de producción en cada push a `main` (sección 9.2).

7.3 Flujo de trabajo con Git

- **GitFlow:** `main` (producción) y `develop` están protegidas; todo cambio entra por pull request revisado por el otro integrante. Las funcionalidades salen de `develop` en ramas `feature/…`; las correcciones urgentes de producción salen de `main` en ramas `hotfix/…` y se fusionan a `main` y a `develop`.
- **Commits:** en español, con Conventional Commits (`feat`, `fix`, `docs`, `test`, `chore`) y, cuando aplica, la referencia al caso de uso o a la historia.
- **Reglas de arquitectura del equipo:** SQL siempre parametrizado; permisos de rol validados en el backend; secretos fuera del código; ids UUID generados por la aplicación; errores con `next(err)`; sin SQL en controladores ni lógica de negocio en repositorios; todo cuerpo validado con esquema; transacciones con `getClient()`.

7.4 Migraciones de base de datos

El esquema se versiona con archivos SQL numerados en `resources/db/migrations/`. El ejecutor propio (`scripts/db/migrate.mjs`) aplica cada archivo una sola vez dentro de una transacción, lo registra en `schema_migrations` con su huella SHA-256, usa un bloqueo consultivo para evitar ejecuciones simultáneas y rechaza que se edite una migración ya aplicada. Para cambiar el esquema siempre se agrega una migración nueva (`npm run db:new -- <descripcion>`); `npm run db:status` muestra qué está aplicado y qué está pendiente.

8. Modelado de datos

El modelo completo, con el diccionario de datos y las reglas de integridad, está en el documento Modelo de Datos. Esta sección resume lo esencial.

8.1 Modelo entidad-relación

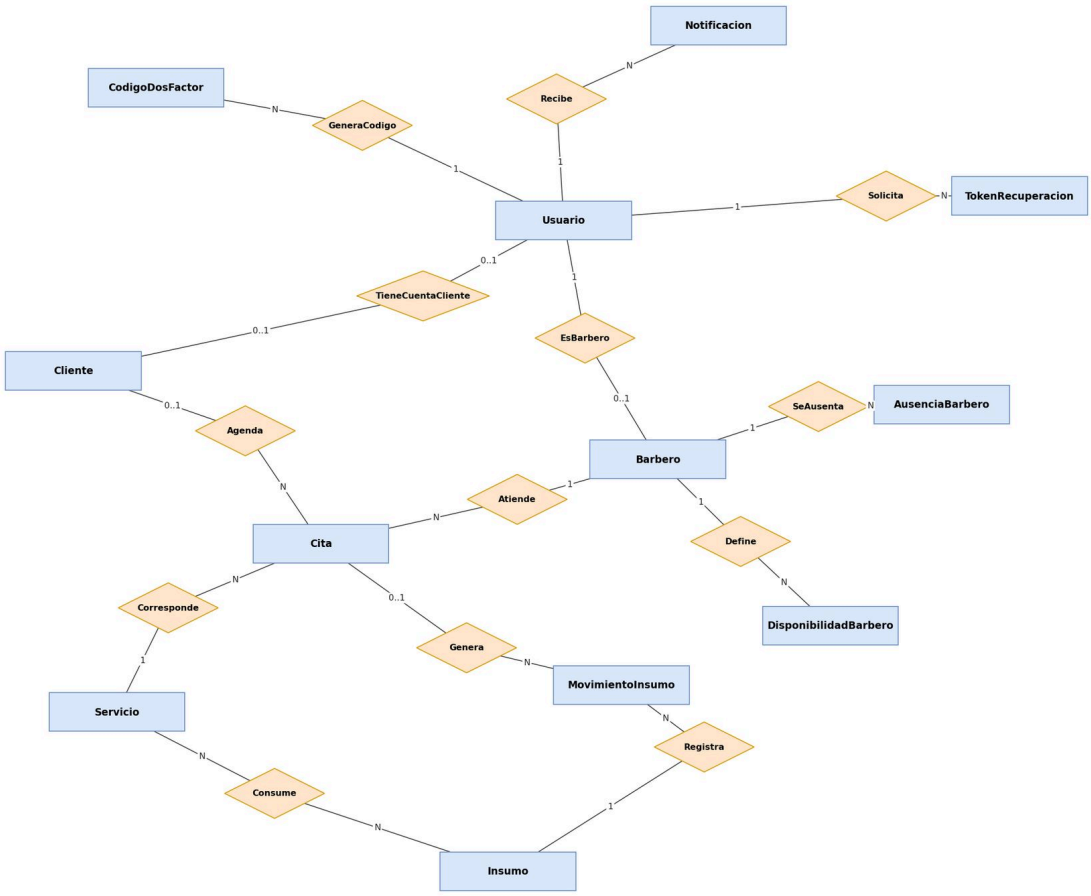


Figura 8. Modelo entidad-relación (vista general).

8.2 Tablas

Tabla	Contenido
`users`	Cuentas de acceso: correo único, hash de contraseña, rol (`admin`, `barbero`, `cliente`), estado y bandera de 2FA.
`clientes`	Perfil del cliente. `user_id` es nulo para clientes atendidos sin cuenta (walk-in).
`barberos`	Perfil del barbero, asociado a su cuenta.
`disponibilidad_barberos`	Bloques de atención semanales de cada barbero (día y rango de horas).
`ausencias_barbero`	Días completos en que el barbero no estará disponible.
`servicios`	Catálogo: nombre, duración y precio.
`insumos`	Existencia, unidad de medida y cantidad mínima de alerta.
`servicio_insumos`	Cuánto consume de cada insumo un servicio.
`citas`	Cita con cliente (opcional), barbero, servicio, fecha, horas y estado (`pendiente`, `confirmada`, `cancelada`, `atendida`).
`insumo_movimientos`	Historial de entradas y salidas de inventario; las salidas se vinculan a la cita.
`notificaciones`	Avisos del sistema por usuario, con su tipo y estado de lectura.
`two_factor_codes`	Huella y vencimiento de los códigos de doble factor.
`password_reset_tokens`	Huella y vencimiento de los enlaces de recuperación.

8.3 Reglas de integridad destacadas

- Identificadores UUID en todas las tablas; llaves foráneas con `ON UPDATE CASCADE` y el `ON DELETE` que corresponde a la regla de negocio.
- Restricciones `CHECK` para los valores permitidos: rol, estado de cita, tipo de movimiento y tipo de notificación; y para el horario (`hora_fin > hora_inicio`, día entre 0 y 6).
- Unicidad: correo de usuario, un perfil de cliente o de barbero por cuenta, una ausencia por barbero y fecha, un insumo por servicio.
- Barberos, clientes y servicios se desactivan en lugar de borrarse, para conservar el historial.

8.4 Migraciones

N.º	Migración	Contenido
0001	core_usuarios_roles	Cuentas, clientes, barberos y horario semanal.
0002	catalogo_servicios_insumos	Servicios, insumos y su asociación.
0003	citas_e_inventario	Citas y movimientos de inventario.
0004	notificaciones_y_auth	Notificaciones, códigos 2FA y tokens de recuperación.
0005	agenda_barbero	Días no disponibles y restricciones de horario.
0006	rls_seguridad	Seguridad a nivel de fila en las tablas de negocio.

9. Despliegue, operación y respaldo

Esta sección reúne la guía de despliegue y el runbook del sistema. El documento Infraestructura, Despliegue y Backup contiene el mismo procedimiento como documento independiente.

9.1 Flujo de despliegue

El despliegue es automático por Git: un push a `main` dispara el build y la publicación en Vercel, por separado para el frontend y para el backend.

Frontend (proyecto frontend)

1. Push a `main` en el repositorio Barber-Kong-Frontend.
2. Vercel detecta el commit y ejecuta `npm install` y el build de Vite, con `frontend` como Root Directory.
3. Si el build pasa, publica automáticamente en `barberkongs.vercel.app`.
4. En paralelo, el flujo `ci.yml` del repositorio ejecuta `npm run lint` y `npm run build` en cada pull request y push a `develop` o `main`; no despliega, solo valida.
5. Las variables `VITE_*` quedan fijas en el bundle al compilar: cambiar una requiere un nuevo despliegue, no basta con guardar el valor.

Backend (proyecto barber-kong-api)

1. Push a `main` en el repositorio Barber-Kong.
2. Vercel instala las dependencias y despliega las funciones serverless (Node y Express).
3. El flujo `migrate.yml` de GitHub Actions aplica las migraciones pendientes en la base de producción (sección 9.2).

Las variables de entorno de cada proyecto están en la sección 3.6; `SMTP_*` sirve a todos los avisos por correo (2FA, recuperación de contraseña, confirmaciones de cita y alertas de inventario).

9.2 Migraciones en producción

El ejecutor propio (`scripts/db/migrate.mjs`) aplica cada archivo de `resources/db/migrations/` una sola vez y lo registra en `schema_migrations` con su huella SHA-256. Un flujo de GitHub Actions (`.github/workflows/migrate.yml`, entorno `production`) lo ejecuta en cada push a `main`: muestra el estado, aplica las migraciones pendientes y vuelve a mostrar el estado. Corre en paralelo al despliegue de Vercel, por lo que cada migración es compatible hacia atrás: agrega columnas o tablas antes de que el código las use y no quita nada que el código en producción todavía necesite. La conexión usa los secretos `PROD_DB_HOST`, `PROD_DB_NAME`, `PROD_MIGRATOR_USER` y `PROD_MIGRATOR_PASSWORD` del entorno, con el pooler de sesión de Supabase (el ejecutor usa `pg_advisory_lock`). El mismo comando puede ejecutarse manualmente:

```
npm run db:migrate   # aplica las migraciones pendientes contra la base de producción
npm run db:status    # revisa qué está aplicado y qué está pendiente
```

9.3 Reversión

- **Código (Vercel):** Dashboard → Deployments → elegir el despliegue anterior que funcionaba → «Promote to Production». Revierte al instante, sin recompilar.
- **Base de datos:** el ejecutor no tiene reversión automática. Deshacer un cambio de esquema es una migración nueva que lo revierte; nunca se edita un archivo ya aplicado (el ejecutor lo rechaza si la huella no coincide).

9.4 Rotación de secretos

1. Generar el nuevo valor (por ejemplo, un `JWT_SECRET` nuevo).
2. Actualizarlo en Vercel → Project Settings → Environment Variables (Production).
3. Volver a desplegar: es obligatorio, porque las variables solo se leen al iniciar o compilar.
4. Si el secreto rotado es `JWT_SECRET`, todas las sesiones activas quedan invalidadas y los usuarios deben iniciar sesión de nuevo.

9.5 Incidentes ocurridos

Síntoma	Causa	Resolución
500 en los endpoints de agenda del barbero	La migración `0005_agenda_barbero.sql` no se había aplicado en producción y el código esperaba las tablas nuevas.	Se ejecutó `npm run db:migrate` contra la base de Supabase de producción.
530 al enviar el correo de recuperación de contraseña	El SMTP de Gmail rechazó la autenticación.	Se generó una contraseña de aplicación (App Password) de Google para la cuenta remitente, que es la de uno de los integrantes del equipo, y se configuró como `SMTP_PASSWORD`.

9.6 Estrategia de respaldo

> **Estado:** la estrategia de respaldo está planificada, pero no se pudo implementar. Hoy no existen copias automáticas fuera de Supabase; lo que sigue es el diseño previsto.

Supabase en el plan Free no incluye copias de seguridad automáticas ni recuperación a un punto en el tiempo (PITR); eso empieza en el plan Pro y la recuperación real es un complemento aparte. Como el proyecto se queda en el plan Free, el respaldo debía armarse fuera de Supabase.

Necesario según el enunciado	¿Supabase Free lo da?	Cómo se suplirá (planificado)
Respaldo completo semanal	No	`pg_dump` completo mediante un workflow programado de GitHub Actions.
Incremental diario	No	`pg_dump` completo cada noche: con el tamaño actual de la base es más simple y confiable que un incremental por filas.

Necesario según el enunciado	¿Supabase Free lo da?	Cómo se suplirá (planificado)
Registro cada 15–30 min y restauración a un punto en el tiempo	No (es PITR real, por WAL, solo en Pro con complemento)	Aproximación: `pg_dump` cada 30 minutos solo de las tablas que cambian durante el día (`citas`, `insumo_movimientos`).

Diseño planificado

- **Ejecutor:** un workflow de GitHub Actions con tres trabajos programados (`schedule: cron`) que corren `pg_dump` contra la cadena de conexión de Supabase, guardada en GitHub Secrets.
- **Almacenamiento:** los archivos `.dump` comprimidos se suben a un lugar distinto de la base que respaldan (por ejemplo, un repositorio privado de GitHub mediante Releases o un bucket gratuito).
- **Retención:** 4 respaldos semanales, 14 diarios y 48 horas de los de 30 minutos, con rotación de los más antiguos.
- **Restauración:** al no haber WAL, solo es posible volver «al momento del respaldo más cercano anterior al incidente»: se restaura el completo o el diario más reciente y, si existen, se reaplican los de las tablas que cambian seguido con `pg_restore --clean --if-exists -d <conexión> respaldo.dump`.

9.7 Observabilidad

- **Bitácora:** `winston` registra errores y eventos; `morgan` registra las solicitudes. En Vercel se escribe en la salida estándar, que Vercel centraliza en sus registros de función.
- **Salud:** `GET /api/health` confirma que el servicio responde.

10. Trazabilidad

La Matriz de Trazabilidad relaciona cada requisito con su caso de uso, historia de usuario, pantalla, endpoint, tablas y pruebas, y recorre el camino inverso desde cada endpoint y cada prueba. Esta sección muestra la vista por caso de uso.

Caso de uso	Historia · Jira	Endpoints	Pruebas (U / I)
UC-01 Iniciar sesión	HU-01 · BK-15	E-02, E-03, E-06, E-07	12 / 7
UC-02 Registrarse	HU-02 · BK-16	E-01	2 / 2
UC-03 Recuperar contraseña	HU-03 · BK-17	E-04, E-05	4 / 2
UC-04 Agendar cita	HU-07 · BK-29	E-08, E-12, E-19, E-40	14 / 12
UC-05 Reprogramar cita	HU-08 · BK-30	E-12, E-21, E-25	0 / 2
UC-06 Cancelar cita	HU-09 · BK-31	E-21, E-22, E-23	2 / 4
UC-07 Consultar agenda	HU-10 · BK-32	E-16, E-17, E-18, E-22	3 / 9
UC-08 Registrar cliente	HU-12 · BK-21	E-29	0 / 3
UC-09 Buscar y filtrar clientes	HU-13 · BK-22	E-26	0 / 2
UC-10 Consultar historial de cliente	HU-14 · BK-23	E-27, E-28	0 / 3
UC-11 Editar o desactivar cliente	HU-15 · BK-24	E-30, E-31, E-32	0 / 4
UC-12 Crear, editar o desactivar servicio	HU-16 · BK-25	E-40, E-42, E-43, E-44	0 / 6
UC-13 Asociar insumos a servicio	HU-17 · BK-26	E-45, E-46, E-47	1 / 4
UC-14 Registrar insumo	HU-18 · BK-27	E-33, E-34, E-35	1 / 5
UC-15 Registrar reabastecimiento	HU-19 · BK-28	E-36	1 / 4
UC-16 Descontar insumos automáticamente	HU-20 · BK-34	E-24	9 / 7
UC-17 Generar alerta de inventario bajo	HU-21 · BK-35	E-24, E-35, E-37	8 / 5
UC-18 Generar y exportar reporte	HU-22 · BK-36	E-39	9 / 4
UC-22 Crear cuenta de barbero	HU-04 · —	E-09, E-13	10 / 4

Caso de uso	Historia · Jira	Endpoints	Pruebas (U / I)
UC-23 Editar cuenta de barbero	HU-05 · —	E-10, E-11, E-14	9 / 1
UC-24 Desactivar cuenta de barbero	HU-06 · BK-20	E-09, E-15	0 / 3
UC-25 Registrar cliente sin reservación (walk-in)	HU-11 · BK-33	E-20	0 / 1
UC-26 Consultar notificaciones	HU-23 · —	E-37, E-38	2 / 6

En total: 31 requisitos funcionales, 23 casos de uso, 23 historias, 48 endpoints y 202 pruebas automáticas, todas asociadas a algún requisito.

Anexo A. Puesta en marcha para desarrolladores

A.1 Con Docker (recomendado)

```
# Clonar Barber-Kong y Barber-Kong-Frontend en la misma carpeta
cd Barber-Kong
docker compose up --build
# Frontend:  http://localhost:5173
# API:       http://localhost:3000/api/health
```

El servicio `migrate` aplica las migraciones pendientes y siembra los datos de demostración antes de que arranque la API. Las cuentas de demostración están descritas en el `README` del repositorio.

A.2 Sin Docker

```
cp .env.example .env.dev      # ajustar la conexión a Postgres
npm install
npm run db:setup             # crea la base, aplica migraciones y siembra datos
npm run dev                  # API en http://localhost:3000

# Frontend
cd ../Barber-Kong-Frontend/frontend
npm install
npm run dev                  # SPA en http://localhost:5173
```

A.3 Nueva migración

```
npm run db:new -- nombre_descriptivo   # crea
resources/db/migrations/NNNN_nombre_descriptivo.sql
# escribir SQL idempotente (IF NOT EXISTS) y luego:
npm run db:migrate
npm run db:status
```

Anexo B. Documentos complementarios

Documento	Contenido
Modelo de Datos	Modelo entidad-relación, modelo relacional y diccionario de datos, con diagramas editables en draw.io.
Inventario de Endpoints	Los 48 endpoints de la API: acceso, cuerpo, respuesta y errores, con la cobertura de pruebas de cada uno.
Plan de Pruebas	Estrategia, entorno, catálogo de casos, cobertura, prueba de mutación, carga y defectos corregidos.
Matriz de Trazabilidad	Requisitos, casos de uso, historias, pantallas, endpoints, pruebas y evidencia en Git.

Documento	Contenido
Documentación de Escalabilidad	Procesos futuros en BPMN, migraciones propuestas y tecnologías nuevas con su justificación.
Infraestructura, Despliegue y Backup	Arquitectura cloud, guía de despliegue y runbook, reversión, rotación de secretos, incidentes y estrategia de respaldo (planificada).
Manual de Usuario	Guía para cliente, barbero y administrador, con capturas de la aplicación.
Diagramas de arquitectura (draw.io)	Archivo `arquitectura-barber-kongs.drawio` con las figuras de las secciones 2 a 5, editables.

Modelo de Datos

Campo	Detalle
Proyecto	Barber Kongs — Sistema de Gestión para Barbería (Fase 3)
Equipo	Miguel Angel Quemé Gómez · Brandon Gustavo Güinac Román
Versión del documento	1.0 — 25 de octubre de 2026
Base de datos	PostgreSQL 16, sin ORM (SQL parametrizado), 13 tablas
Fuente	Migraciones versionadas 0001 a 0006 (carpeta `resources/db/migrations`) y esquema inspeccionado en una base creada con ellas
Diagramas editables	`modelo-er-chen-barber-kong-v2.drawio` (2 páginas) y `diagrama-tablas-barber-kong.drawio` (3 páginas), para abrir y modificar en draw.io

1. Principios del diseño de datos

- **Identificadores UUID generados por la aplicación** (`randomUUID()`), nunca por la base de datos. Así el esquema no depende de extensiones de PostgreSQL que podrían no estar habilitadas en el servicio administrado.
- **Sin ORM:** cada módulo escribe su SQL parametrizado; los valores nunca se concatenan en la consulta.
- **Esquema versionado e idempotente:** cada cambio es un archivo `NNNN_descripcion.sql`; el ejecutor guarda un checksum y se niega a correr si una migración ya aplicada fue editada.
- **Integridad en la base, no solo en la aplicación:** llaves foráneas con acción de borrado explícita, restricciones `UNIQUE` y `CHECK` para estados, roles, horarios y duplicados.
- **Desactivar en lugar de borrar:** clientes, barberos y servicios se marcan inactivos (`is_active`); así se conserva el historial de citas y las llaves `RESTRICT` no estorban.
- **Secretos solo como hash:** contraseñas con bcrypt (costo 10); códigos de dos factores y tokens de recuperación con SHA-256. El valor original nunca se almacena.

2. Modelo entidad-relación (Chen)

2.1 Vista general

La figura 1 muestra las 12 entidades y sus 13 relaciones. Cada etiqueta junto a una entidad indica cuántas instancias de esa entidad se asocian a una instancia de la otra: por ejemplo, un barbero (1) tiene muchas (N) disponibilidades, y una cita tiene 0..1 clientes porque las citas walk-in pueden no tener cliente registrado.

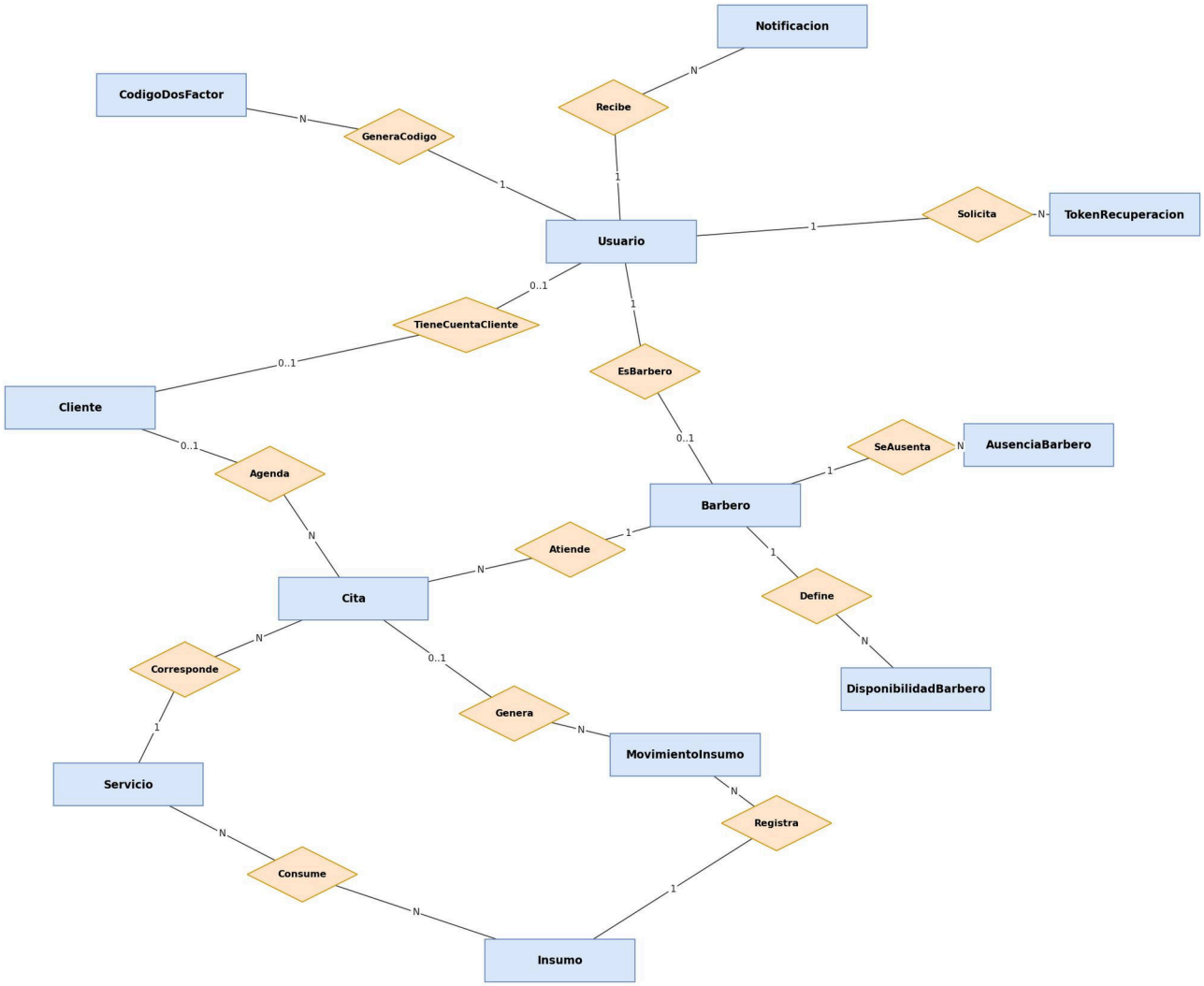


Figura 1. Modelo entidad-relación (vista general, sin atributos).

2.2 Entidades

Entidad	Tabla	Descripción
Usuario	`users`	Cuentas de acceso de los tres roles (administrador, barbero y cliente).
Cliente	`clientes`	Datos de contacto del cliente, con o sin cuenta (clientes walk-in).
Barbero	`barberos`	Perfil del barbero; uno por cuenta con rol barbero.
DisponibilidadBarbero	`disponibilidad_barberos`	Bloques de atención semanales de cada barbero.
AusenciaBarbero	`ausencias_barbero`	Días completos en que un barbero avisa que no atiende.
Servicio	`servicios`	Catálogo de servicios con duración y precio.

Entidad	Tabla	Descripción
Insumo	`insumos`	Inventario de insumos y su umbral de alerta de stock bajo.
(relación Consume)	`servicio_insumos`	Tabla asociativa: cuánto de cada insumo consume un servicio.
Cita	`citas`	Citas agendadas, atendidas o canceladas, incluidas las citas sin reservación (walk-in).
MovimientoInsumo	`insumo_movimientos`	Historial de entradas y salidas de inventario.
Notificacion	`notificaciones`	Centro de notificaciones de cada usuario.
TokenRecuperacion	`password_reset_tokens`	Tokens de un solo uso para recuperar la contraseña.
CodigoDosFactor	`two_factor_codes`	Códigos de un solo uso del segundo factor de autenticación.

La relación Consume (N:M entre Servicio e Insumo) se resuelve con la tabla asociativa `servicio_insumos`, que además guarda la cantidad consumida.

2.3 Relaciones y cardinalidades

Relación	Entidades	Cardinalidad	Llave y acción al borrar	Regla de negocio
TieneCuentaCliente	Usuario – Cliente	0..1 : 0..1	`clientes.user_id`, único y opcional; SET NULL	Un cliente puede no tener cuenta (walk-in).
EsBarbero	Usuario – Barbero	1 : 0..1	`barberos.user_id`, único y obligatorio; CASCADE	Cada barbero tiene exactamente una cuenta.
Define	Barbero – DisponibilidadBarbero	1 : N	`disponibilidad_barberos.barbero_id`; CASCADE	Un barbero define varios bloques de atención.
SeAusenta	Barbero – AusenciaBarbero	1 : N	`ausencias_barbero.barbero_id`; CASCADE	Un barbero puede marcar días sin atender; uno por fecha.
Atiende	Barbero – Cita	1 : N	`citas.barbero_id`; RESTRICT	Un barbero con citas no se elimina; se desactiva.
Agenda	Cliente – Cita	0..1 : N	`citas.cliente_id`, opcional; SET NULL	Una cita pertenece a lo sumo a un cliente.
Corresponde	Servicio – Cita	1 : N	`citas.servicio_id`; RESTRICT	Toda cita es de un servicio.
Consume	Servicio – Insumo	N : N	`servicio_insumos`; servicio CASCADE, insumo RESTRICT	Cada servicio consume cantidades de varios insumos.
Registra	Insumo – MovimientoInsumo	1 : N	`insumo_movimientos.insumo_id`; CASCADE	Cada entrada o salida queda en el historial del insumo.
Genera	Cita – MovimientoInsumo	0..1 : N	`insumo_movimientos.cita_id`, opcional; SET NULL	Atender una cita genera salidas; las entradas no tienen cita.
Recibe	Usuario – Notificacion	1 : N	`notificaciones.user_id`; CASCADE	Un usuario recibe varias notificaciones.
Solicita	Usuario – TokenRecuperacion	1 : N	`password_reset_tokens.user_id`; CASCADE	Un usuario puede pedir varios enlaces de recuperación.
GeneraCodigo	Usuario – CodigoDosFactor	1 : N	`two_factor_codes.user_id`; CASCADE	Cada inicio de sesión con 2FA genera un código.

Cardinalidad "A : B": cuántas instancias de A (por cada una de B) y cuántas de B (por cada una de A), en el orden de la columna Entidades.

2.4 Modelo completo con atributos

La figura 2 reúne entidades, relaciones y atributos. Los atributos subrayados son claves; el subrayado discontinuo marca una clave parcial (el día de la semana de un bloque de horario y la fecha de una ausencia, únicas por barbero). El archivo `modelo-er-chen-barber-kong-v2.drawio` conserva este diagrama en vectores para ampliarlo o editarlo.

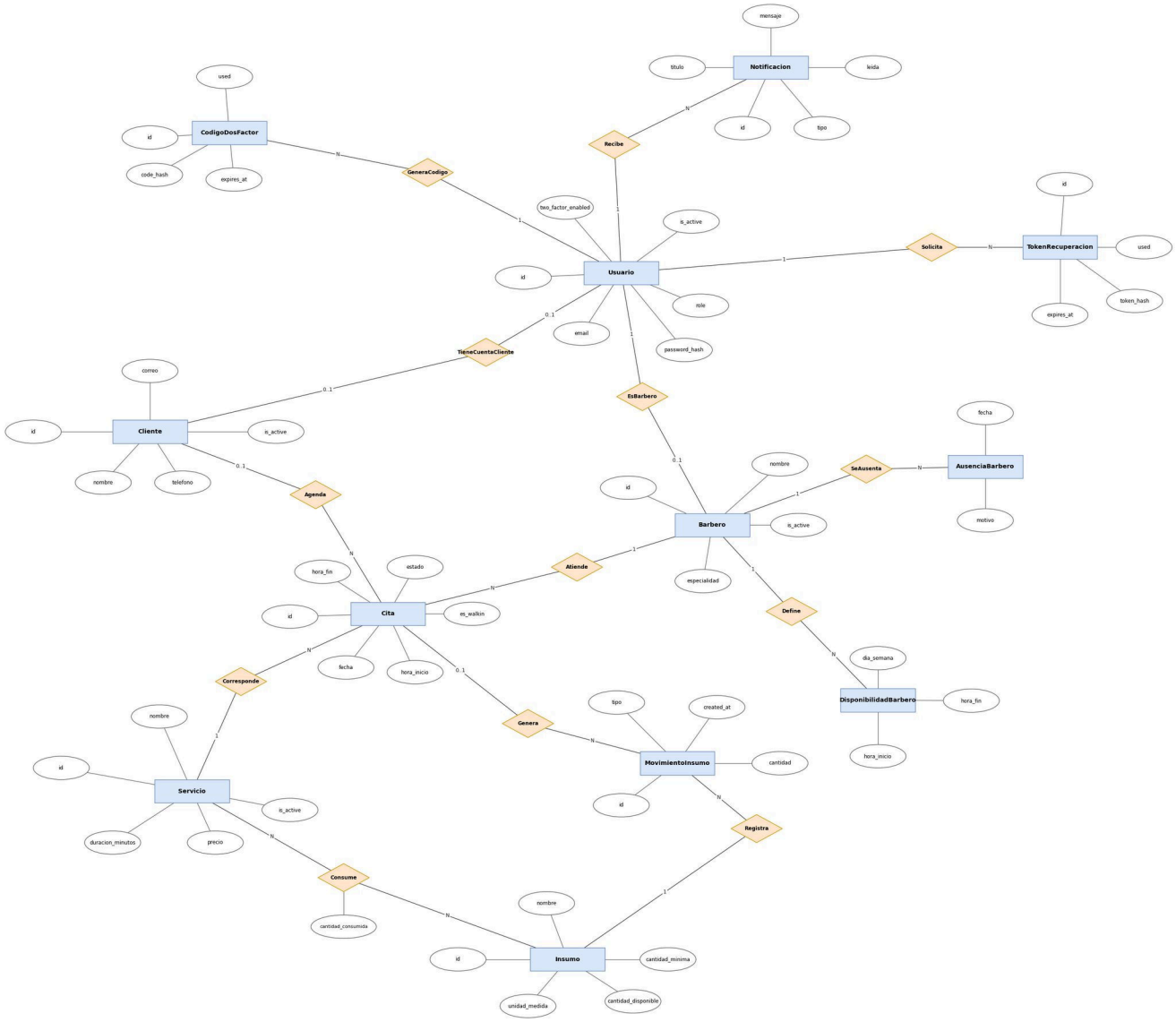


Figura 2. Modelo entidad-relación completo, con atributos.

3. Modelo relacional (diagramas de tablas)

Las figuras 3 y 4 muestran las tablas con sus columnas, tipos, llaves y restricciones. Las líneas con pata de gallo son llaves foráneas: el lado "muchos" está en la tabla que guarda la llave; el rótulo indica la columna y la acción al borrar. Un círculo señala que la llave es opcional (admite nulo). El archivo `diagrama-tablas-barber-kong.drawio` incluye además una página con las 13 tablas juntas.

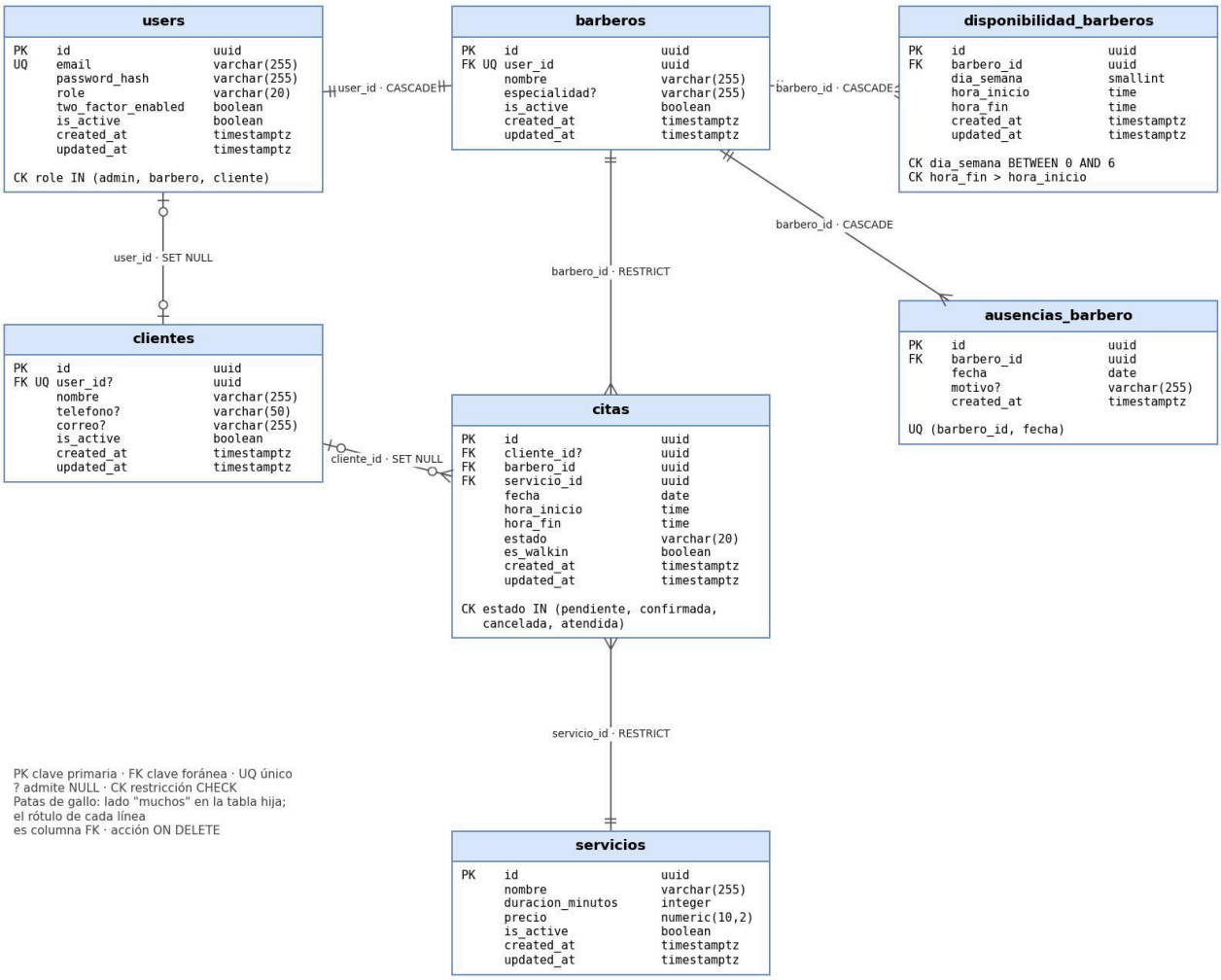


Figura 3. Tablas de usuarios, agenda y citas.

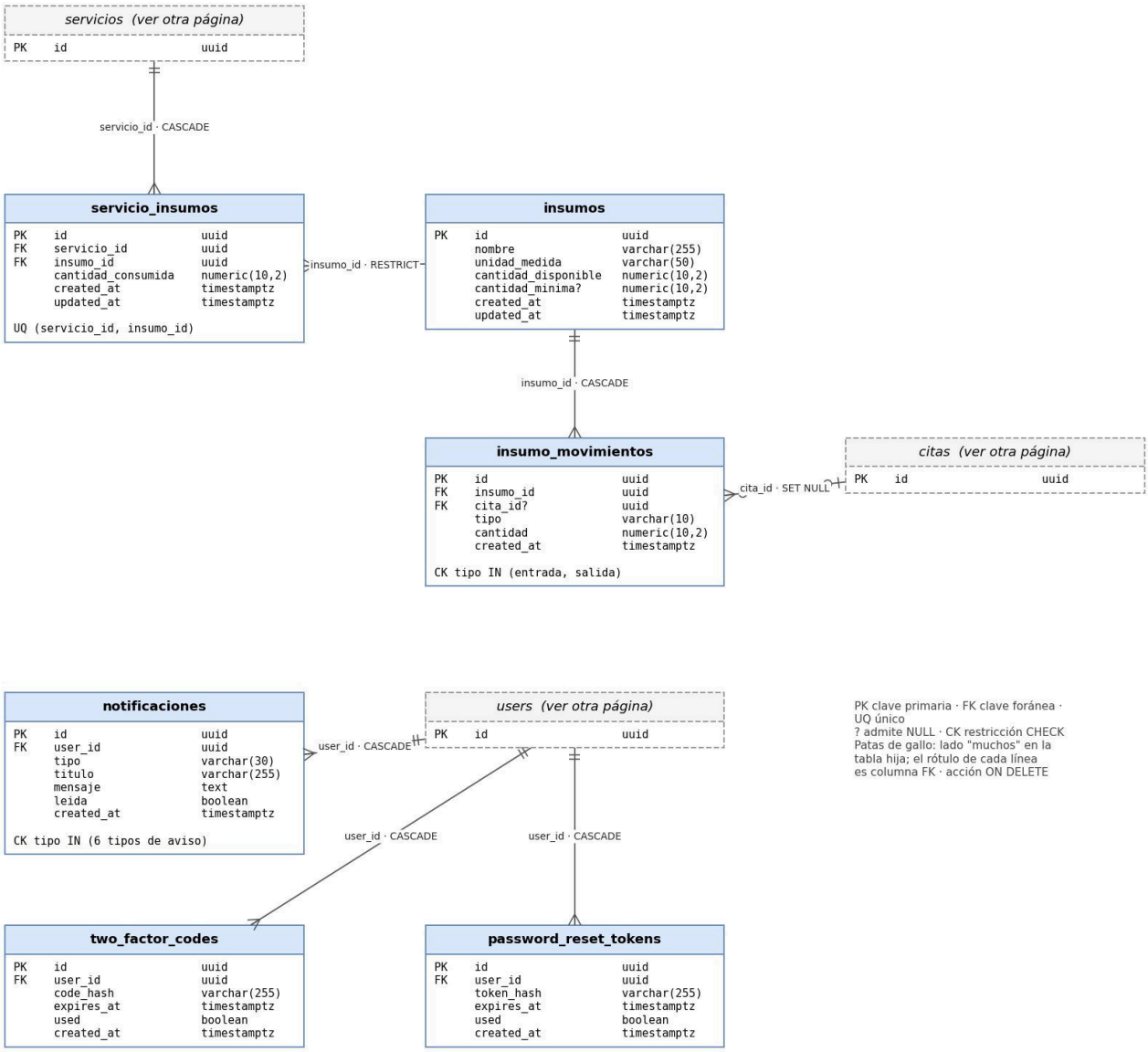


Figura 4. Tablas de inventario, notificaciones y seguridad de la cuenta. Las tablas con borde discontinuo pertenecen a la figura 3.

4. Diccionario de datos

Cada tabla se describe con sus columnas. "Clave": PK llave primaria, FK llave foránea (con la tabla a la que apunta) y UQ valor único. "Nulo": si la columna admite valores nulos.

users

Cuentas de acceso de los tres roles (administrador, barbero y cliente). Entidad: Usuario.

Columna	Tipo	Nulo	Clave	Descripción
`id`	`uuid`	No	PK	Identificador (UUID generado por la aplicación).
`email`	`varchar(255)`	No	UQ	Correo con el que inicia sesión; único.
`password_hash`	`varchar(255)`	No	—	Hash bcrypt de la contraseña; nunca se guarda en claro.
`role`	`varchar(20)`	No	—	Rol de la cuenta: admin, barbero o cliente.
`two_factor_enabled`	`boolean`	No	—	Si la cuenta exige un código de dos factores al iniciar sesión.
`is_active`	`boolean`	No	—	Cuenta activa; una cuenta inactiva no puede iniciar sesión.
`created_at`	`timestamptz`	No	—	Fecha y hora de creación del registro.
`updated_at`	`timestamptz`	No	—	Fecha y hora de la última modificación.

Restricciones: `UNIQUE` (email)

- `CHECK` role IN ('admin', 'barbero', 'cliente')

clientes

Datos de contacto del cliente, con o sin cuenta (clientes walk-in). Entidad: Cliente.

Columna	Tipo	Nulo	Clave	Descripción
`id`	`uuid`	No	PK	Identificador (UUID generado por la aplicación).
`user_id`	`uuid`	Sí	FK → users, UQ	Cuenta del cliente; nulo en clientes walk-in sin cuenta.
`nombre`	`varchar(255)`	No	—	Nombre del cliente.
`telefono`	`varchar(50)`	Sí	—	Teléfono de contacto.
`correo`	`varchar(255)`	Sí	—	Correo de contacto para los avisos.
`is_active`	`boolean`	No	—	Cliente activo; se puede desactivar y reactivar.
`created_at`	`timestamptz`	No	—	Fecha y hora de creación del registro.
`updated_at`	`timestamptz`	No	—	Fecha y hora de la última modificación.

Restricciones: `UNIQUE` (user_id)

barberos

Perfil del barbero; uno por cuenta con rol barbero. Entidad: Barbero.

Columna	Tipo	Nulo	Clave	Descripción
id	uuid	No	PK	Identificador (UUID generado por la aplicación).
user_id	uuid	No	FK → users, UQ	Cuenta de acceso del barbero (una por barbero).
nombre	varchar(255)	No	—	Nombre visible del barbero.
especialidad	varchar(255)	Sí	—	Especialidad (por ejemplo, fade o barba).
is_active	boolean	No	—	Barbero activo: aparece en el catálogo y se le pueden agendar citas.
created_at	timestamptz	No	—	Fecha y hora de creación del registro.
updated_at	timestamptz	No	—	Fecha y hora de la última modificación.

Restricciones: UNIQUE (user_id)

disponibilidad_barberos

Bloques de atención semanales de cada barbero. Entidad: DisponibilidadBarbero.

Columna	Tipo	Nulo	Clave	Descripción
id	uuid	No	PK	Identificador (UUID generado por la aplicación).
barbero_id	uuid	No	FK → barberos	Barbero al que pertenece el bloque.
dia_semana	smallint	No	—	Día de la semana: 0 = domingo … 6 = sábado.
hora_inicio	time	No	—	Inicio del bloque de atención.
hora_fin	time	No	—	Fin del bloque; debe ser posterior al inicio.
created_at	timestamptz	No	—	Fecha y hora de creación del registro.
updated_at	timestamptz	No	—	Fecha y hora de la última modificación.

Restricciones: CHECK dia_semana BETWEEN 0 AND 6

- CHECK hora_fin > hora_inicio (chk_disponibilidad_horas, migración 0005)

ausencias_barbero

Días completos en que un barbero avisa que no atiende. Entidad: AusenciaBarbero.

Columna	Tipo	Nulo	Clave	Descripción
id	uuid	No	PK	Identificador (UUID generado por la aplicación).
barbero_id	uuid	No	FK → barberos	Barbero que no atiende ese día.
fecha	date	No	—	Día completo de ausencia.
motivo	varchar(255)	Sí	—	Motivo opcional.
created_at	timestamptz	No	—	Fecha y hora de creación del registro.

Restricciones: UNIQUE (barbero_id, fecha) (uq_ausencia_barbero_fecha, migración 0005)

servicios

Catálogo de servicios con duración y precio. Entidad: Servicio.

Columna	Tipo	Nulo	Clave	Descripción
id	uuid	No	PK	Identificador (UUID generado por la aplicación).
nombre	varchar(255)	No	—	Nombre del servicio.
duracion_minutos	integer	No	—	Duración estimada; define la hora de fin de la cita.
precio	numeric(10,2)	No	—	Precio del servicio.
is_active	boolean	No	—	Servicio activo en el catálogo público.
created_at	timestamptz	No	—	Fecha y hora de creación del registro.
updated_at	timestamptz	No	—	Fecha y hora de la última modificación.

insumos

Inventario de insumos y su umbral de alerta de stock bajo. Entidad: Insumo.

Columna	Tipo	Nulo	Clave	Descripción
id	uuid	No	PK	Identificador (UUID generado por la aplicación).
nombre	varchar(255)	No	—	Nombre del insumo.
unidad_medida	varchar(50)	No	—	Unidad en que se cuenta (por ejemplo, ml o unidad).
cantidad_disponible	numeric(10,2)	No	—	Existencia actual.
cantidad_minima	numeric(10,2)	Sí	—	Umbral de alerta de stock bajo; nulo = sin alerta.
created_at	timestamptz	No	—	Fecha y hora de creación del registro.
updated_at	timestamptz	No	—	Fecha y hora de la última modificación.

servicio_insumos

Tabla asociativa: cuánto de cada insumo consume un servicio.

Columna	Tipo	Nulo	Clave	Descripción
id	uuid	No	PK	Identificador (UUID generado por la aplicación).
servicio_id	uuid	No	FK → servicios	Servicio que consume el insumo.
insumo_id	uuid	No	FK → insumos	Insumo consumido.
cantidad_consumida	numeric(10,2)	No	—	Cantidad del insumo que consume una cita de ese servicio.
created_at	timestamptz	No	—	Fecha y hora de creación del registro.
updated_at	timestamptz	No	—	Fecha y hora de la última modificación.

Restricciones: `UNIQUE` (servicio_id, insumo_id)

citas

Citas agendadas, atendidas o canceladas, incluidas las citas sin reservación (walk-in). Entidad: Cita.

Columna	Tipo	Nulo	Clave	Descripción
`id`	`uuid`	No	PK	Identificador (UUID generado por la aplicación).
`cliente_id`	`uuid`	Sí	FK → clientes	Cliente que agendó; nulo en citas walk-in anónimas.
`barbero_id`	`uuid`	No	FK → barberos	Barbero que atiende.
`servicio_id`	`uuid`	No	FK → servicios	Servicio solicitado.
`fecha`	`date`	No	—	Día de la cita.
`hora_inicio`	`time`	No	—	Hora de inicio.
`hora_fin`	`time`	No	—	Hora de fin: inicio más la duración del servicio.
`estado`	`varchar(20)`	No	—	pendiente, confirmada, cancelada o atendida.
`es_walkin`	`boolean`	No	—	Si es una cita sin reservación previa.
`created_at`	`timestamptz`	No	—	Fecha y hora de creación del registro.
`updated_at`	`timestamptz`	No	—	Fecha y hora de la última modificación.

Restricciones: `CHECK` estado IN ('pendiente', 'confirmada', 'cancelada', 'atendida')

insumo_movimientos

Historial de entradas y salidas de inventario. Entidad: MovimientoInsumo.

Columna	Tipo	Nulo	Clave	Descripción
`id`	`uuid`	No	PK	Identificador (UUID generado por la aplicación).
`insumo_id`	`uuid`	No	FK → insumos	Insumo afectado.
`cita_id`	`uuid`	Sí	FK → citas	Cita que originó la salida; nulo en las entradas.
`tipo`	`varchar(10)`	No	—	entrada (reabastecimiento) o salida (consumo por cita atendida).
`cantidad`	`numeric(10,2)`	No	—	Cantidad movida.
`created_at`	`timestamptz`	No	—	Momento del movimiento; es la fecha en que se agrupa el reporte de consumo.

Restricciones: `CHECK` tipo IN ('entrada', 'salida')

notificaciones

Centro de notificaciones de cada usuario. Entidad: Notificacion.

Columna	Tipo	Nulo	Clave	Descripción
`id`	`uuid`	No	PK	Identificador (UUID generado por la aplicación).
`user_id`	`uuid`	No	FK → users	Usuario que recibe la notificación.
`tipo`	`varchar(30)`	No	—	Tipo de aviso (6 valores permitidos, ver sección 5).
`titulo`	`varchar(255)`	No	—	Título corto.
`mensaje`	`text`	No	—	Texto de la notificación.
`leida`	`boolean`	No	—	Si el usuario ya la leyó.
`created_at`	`timestamptz`	No	—	Fecha y hora de creación del registro.

Restricciones: `CHECK` tipo IN ('dos_factor', 'recuperacion_password', 'confirmacion_cita', 'reprogramacion_cita', 'cancelacion_cita', 'alerta_inventario')

password_reset_tokens

Tokens de un solo uso para recuperar la contraseña. Entidad: TokenRecuperacion.

Columna	Tipo	Nulo	Clave	Descripción
`id`	`uuid`	No	PK	Identificador (UUID generado por la aplicación).
`user_id`	`uuid`	No	FK → users	Cuenta que pidió recuperar la contraseña.
`token_hash`	`varchar(255)`	No	—	Hash SHA-256 del token enviado por correo; el token no se guarda.
`expires_at`	`timestamptz`	No	—	Vencimiento del enlace.
`used`	`boolean`	No	—	Si ya se usó (un solo uso).
`created_at`	`timestamptz`	No	—	Fecha y hora de creación del registro.

two_factor_codes

Códigos de un solo uso del segundo factor de autenticación. Entidad: CodigoDosFactor.

Columna	Tipo	Nulo	Clave	Descripción
`id`	`uuid`	No	PK	Identificador (UUID generado por la aplicación).
`user_id`	`uuid`	No	FK → users	Cuenta que inicia sesión.
`code_hash`	`varchar(255)`	No	—	Hash SHA-256 del código de 6 dígitos enviado por correo.
`expires_at`	`timestamptz`	No	—	Vencimiento del código.
`used`	`boolean`	No	—	Si ya se usó (un solo uso).
`created_at`	`timestamptz`	No	—	Fecha y hora de creación del registro.

5. Reglas de integridad

5.1 Acciones al borrar (llaves foráneas)

Las llaves definen qué ocurre con los datos dependientes. En la práctica la aplicación desactiva clientes, barberos y servicios en lugar de borrarlos, por lo que estas acciones son una segunda línea de defensa.

Llave foránea	Acción	Efecto
`clientes.user_id` → users	SET NULL	Si se elimina la cuenta, el cliente conserva su historial como cliente sin cuenta.
`barberos.user_id` → users	CASCADE	Eliminar la cuenta elimina al barbero.
`disponibilidad_barberos.barbero_id` → barberos	CASCADE	El horario depende del barbero.
`ausencias_barbero.barbero_id` → barberos	CASCADE	Las ausencias se van con el barbero.
`citas.cliente_id` → clientes	SET NULL	La cita se conserva aunque se elimine al cliente; queda sin cliente.
`citas.barbero_id` → barberos	RESTRICT	No se puede eliminar un barbero que tiene citas.
`citas.servicio_id` → servicios	RESTRICT	No se puede eliminar un servicio que tiene citas.
`servicio_insumos.servicio_id` → servicios	CASCADE	Los insumos asociados dependen del servicio.
`servicio_insumos.insumo_id` → insumos	RESTRICT	No se elimina un insumo que algún servicio consume.

Llave foránea	Acción	Efecto
`insumo_movimientos.insumo_id` → insumos	CASCADE	El historial depende del insumo.
`insumo_movimientos.cita_id` → citas	SET NULL	El movimiento se conserva si se elimina la cita.
`notificaciones.user_id` → users	CASCADE	Las notificaciones dependen de la cuenta.
`password_reset_tokens.user_id` → users	CASCADE	Los tokens dependen de la cuenta.
`two_factor_codes.user_id` → users	CASCADE	Los códigos dependen de la cuenta.

5.2 Valores permitidos y restricciones de dominio

Tabla	Restricción	Qué garantiza
users	`role` ∈ {admin, barbero, cliente}	Solo los tres roles del sistema.
citas	`estado` ∈ {pendiente, confirmada, cancelada, atendida}	Ciclo de vida de la cita. La aplicación hoy crea las citas como "confirmada"; "pendiente" está permitido pero no se usa.
insumo_movimientos	`tipo` ∈ {entrada, salida}	Sentido del movimiento de inventario.
notificaciones	`tipo` ∈ {dos_factor, recuperacion_password, confirmacion_cita, reprogramacion_cita, cancelacion_cita, alerta_inventario}	Los seis avisos que el sistema puede generar.
disponibilidad_barberos	`dia_semana` entre 0 y 6; `hora_fin` > `hora_inicio`	Días válidos y bloques que terminan después de empezar.
ausencias_barbero	`UNIQUE (barbero_id, fecha)`	Un barbero no marca dos veces el mismo día.
servicio_insumos	`UNIQUE (servicio_id, insumo_id)`	Un insumo se asocia una vez por servicio.
users, clientes, barberos	`UNIQUE` en `email` y en `user_id`	Correos no repetidos; una cuenta, un perfil.

6. Índices

Índice	Tabla	Columnas	Para qué sirve
`uq_ausencia_barbero_fecha`	ausencias_barbero	(barbero_id, fecha), único	Evita marcar dos veces el mismo día y acelera "¿está ausente el barbero X en la fecha Y?"
`uq_servicio_insumo`	servicio_insumos	(servicio_id, insumo_id), único	Un insumo se asocia una sola vez por servicio.
`users_email_key`	users	(email), único	Inicio de sesión por correo; correos no repetidos.
`clientes_user_id_key`	clientes	(user_id), único	Una cuenta, a lo sumo un perfil de cliente.
`barberos_user_id_key`	barberos	(user_id), único	Una cuenta, un perfil de barbero.
`idx_citas_barbero_fecha_hora`	citas	(barbero_id, fecha, hora_inicio)	Agenda del barbero y detección de traslapes.
`idx_citas_cliente`	citas	(cliente_id)	Historial y "mis citas" del cliente.
`idx_disponibilidad_barbero_dia`	disponibilidad_barberos	(barbero_id, dia_semana)	Horario de un barbero en un día.
`idx_insumo_movimientos_insumo_fecha`	insumo_movimientos	(insumo_id, created_at)	Historial de un insumo y reporte de consumo.
`idx_notificaciones_user_leida`	notificaciones	(user_id, leida)	Notificaciones sin leer de un usuario.
`idx_password_reset_tokens_user`	password_reset_tokens	(user_id)	Buscar tokens de una cuenta.
`idx_two_factor_codes_user`	two_factor_codes	(user_id)	Buscar códigos de una cuenta.

Además de estos, cada tabla tiene el índice de su llave primaria.

7. Seguridad de los datos

- **Contraseñas, códigos y tokens:** solo se almacenan sus hashes (bcrypt para la contraseña; SHA-256 para el código de dos factores y el token de recuperación). Una fuga de la base no revela estos valores.
- **Row Level Security (RLS):** la migración 0006 activa RLS, sin políticas, en 12 tablas. Supabase expone cada tabla del esquema público por una API REST con una clave pública; con RLS activo y sin políticas, esa clave no puede leer ni escribir nada. El backend se conecta con un rol que se salta RLS, así que su comportamiento no cambia.

Estado de RLS por tabla:

Tabla	RLS	Tabla	RLS
`ausencias_barbero`	SIN RLS	`barberos`	Activo
`citas`	Activo	`clientes`	Activo
`disponibilidad_barberos`	Activo	`insumo_movimientos`	Activo
`insumos`	Activo	`notificaciones`	Activo
`password_reset_tokens`	Activo	`servicio_insumos`	Activo
`servicios`	Activo	`two_factor_codes`	Activo
`users`	Activo		

7.1 Observaciones

ID	Observación	Recomendación
OD-01	**La tabla ausencias_barbero no tiene RLS.** Se creó en la migración 0005 y la 0006 no la incluyó. En un despliegue con Supabase, cualquiera que tenga la clave pública podría leer o modificar los días de ausencia de los barberos (por ejemplo, marcar a un barbero como ausente y bloquear sus citas).	Nueva migración (la 0007) con `ALTER TABLE ausencias_barbero ENABLE ROW LEVEL SECURITY;`, igual que el resto. No se edita la 0006 porque ya está aplicada.
OD-02	El estado `pendiente` de las citas está permitido por la restricción pero la aplicación nunca lo asigna (las citas se crean como `confirmada`).	Documentarlo como estado reservado, o retirarlo en una migración futura si no se va a usar.
OD-03	Cantidades de insumos y consumos son `numeric(10,2)`: admiten decimales y cualquier unidad (ml, unidad).	Es coherente con la definición actual del inventario; si se decide contar solo en unidades enteras, requiere una migración nueva.

8. Historial de migraciones y cambios del modelo

Migración	Qué agrega al esquema
`0001_core_usuarios_roles`	Tablas `users`, `clientes`, `barberos` y `disponibilidad_barberos`.
`0002_catalogo_servicios_insumos`	Tablas `servicios`, `insumos` y `servicio_insumos`.
`0003_citas_e_inventario`	Tablas `citas` e `insumo_movimientos`.
`0004_notificaciones_y_auth`	Tablas `notificaciones`, `password_reset_tokens` y `two_factor_codes`.
`0005_agenda_barbero`	Restricción `chk_disponibilidad_horas` y tabla `ausencias_barbero`.
`0006_rls_seguridad`	Activa RLS en 12 tablas (no incluye `ausencias_barbero`, observación OD-01).

8.1 Cambios respecto al modelo anterior

- **Nueva entidad AusenciaBarbero** y su relación SeAusenta con Barbero (1:N). Es la funcionalidad de días no disponibles de la Fase 2 y no figuraba en el modelo.

- **Restricción de horarios** (`chk_disponibilidad_horas`): un bloque debe terminar después de empezar.
- **Atributo de la relación Consume:** se hizo explícita `cantidad_consumida`, que es parte de la tabla asociativa `servicio_insumos`.
- **Atributos `code_hash` y `token_hash`:** se hizo explícito que del código de dos factores y del token de recuperación solo se guarda el hash.
- **Corrección de cardinalidad en EsBarbero:** estaba invertida respecto al resto del diagrama. La base define `barberos.user_id` único y obligatorio, así que la relación es Usuario 1 : Barbero 0..1 (cada barbero tiene exactamente una cuenta; una cuenta tiene a lo sumo un barbero).
- **Seguridad:** RLS en 12 tablas (migración 0006).

Barber Kongs — Fase 3: Infraestructura, Despliegue y Backup

25 de octubre de 2026 · Brandon Gustavo Güinac Román

1. Arquitectura e infraestructura cloud final (4.2)

Barber Kongs corre en una arquitectura serverless de 3 capas, sin servidor propio que administrar: el frontend y el backend viven como dos proyectos separados en Vercel, y los datos viven en una base Postgres gestionada por Supabase.

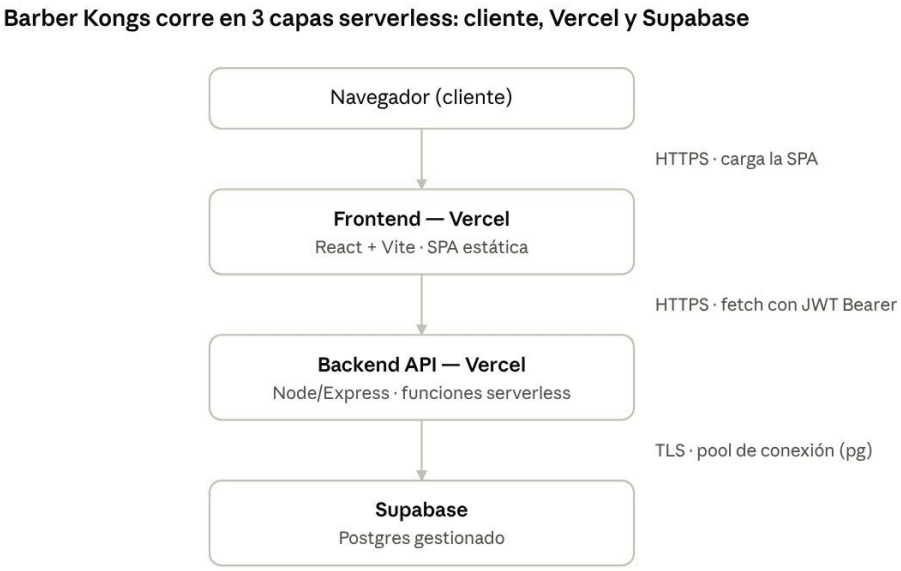


Figura 1. Topología de despliegue: cliente → frontend (Vercel) → backend (Vercel) → base de datos (Supabase).

Capas de seguridad

- **HTTPS en todo el tráfico**: Vercel termina TLS automáticamente en su edge, tanto para el frontend como para la API — no hay certificados que administrar a mano.
- **Autenticación por JWT**: `jsonwebtoken`, firmado con `JWT_SECRET` (variable de entorno), expira a las 2 horas (`JWT_EXPIRES_IN`). El cliente lo manda como `Authorization: Bearer <token>`.
- **Autorización por rol, validada en el servidor**: cada endpoint protegido usa `requireRole(...)` en el backend — el rol (`admin`, `barbero`, `cliente`) nunca se confía solo porque el frontend oculte un botón.
- **Contraseñas con bcrypt**, nunca en texto plano.
- **Cabeceras HTTP endurecidas con Helmet** y **CORS** configurado en el backend (`src/server/app.js`).

- **Secretos fuera del código**: `JWT_SECRET`, credenciales de base de datos y SMTP viven como variables de entorno en Vercel (Production/Preview/Development), nunca en el repo — ver `.env.example` para la lista de claves, sin valores.

Por qué Vercel + Supabase en vez de AWS

La decisión se tomó por dos razones: (1) el equipo ya conocía Vercel y Supabase, lo que reduce la curva de aprendizaje y el tiempo de puesta en producción; y (2) el cliente no estuvo dispuesto a pagar el costo adicional que implicaría AWS, mientras que Vercel y Supabase se pueden operar en sus planes gratuitos.

Dónde corre el frontend

El frontend (React + Vite) está desplegado en **Vercel**, proyecto `frontend`, dominio `barberkongs.vercel.app`. El backend (`barber-kong-api`) es un proyecto de Vercel aparte.

2. Guía de despliegue / Runbook

El despliegue es automático vía Git: un push a `main` dispara el build y la publicación tanto del frontend como del backend en Vercel, por separado.

Despliegue del frontend (proyecto `frontend`)

1. Push a `main` en `github.com/brandonsk8/Barber-Kong-Frontend`.
2. Vercel detecta el commit y corre `npm install` + build de Vite, con Root Directory `frontend`.
3. Si el build pasa, publica automáticamente en `barberkongs.vercel.app`.
4. En paralelo, el flujo ci.yml del repositorio ejecuta npm run lint y npm run build en cada pull request y push a develop o main; no despliega, solo valida.
5. Las variables `VITE_*` quedan fijas en el bundle en tiempo de build — cambiar una requiere redeploy, no solo guardar el valor nuevo.

Despliegue del backend (proyecto `barber-kong-api`)

1. Push a `main` en el repo del backend.
2. Vercel instala dependencias y despliega las funciones serverless (Node/Express).
3. Las migraciones de base de datos las aplica el flujo migrate.yml de GitHub Actions (siguiente sección).

Variables de entorno

Variable	Dónde	Para qué
`VITE_API_URL`	Frontend (Vercel)	URL base de la API que consume la SPA
`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`	Backend (Vercel)	Conexión a Postgres (Supabase)
`DB_SSL`	Backend (Vercel)	true para conectarse a Supabase con TLS
`DB_POOL_MAX`	Backend (Vercel)	Tamaño máximo del pool de conexiones; se

Variable	Dónde	Para qué
		recomienda 1 en funciones serverless
`JWT_SECRET`	Backend (Vercel)	Firma y verificación de tokens de sesión
`JWT_EXPIRES_IN`	Backend (Vercel)	Vigencia del token (2h)
`FRONTEND_URL`	Backend (Vercel)	Arma el link de recuperación de contraseña que se envía por correo
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM`	Backend (Vercel)	Envío de correo: códigos 2FA, recuperación de contraseña, confirmaciones de cita y alertas de inventario
`LOG_LEVEL`	Backend (Vercel)	Nivel de detalle de logs

Aplicar migraciones en producción

El backend usa un runner propio sin ORM (scripts/db/migrate.mjs): cada archivo resources/db/migrations/NNNN_descripcion.sql es idempotente y queda registrado en schema_migrations junto con su checksum. El flujo .github/workflows/migrate.yml (entorno production) lo ejecuta en cada push a main: muestra el estado, aplica las migraciones pendientes y vuelve a mostrar el estado. Corre en paralelo al despliegue de Vercel, por eso cada migración es compatible hacia atrás: agrega columnas o tablas **antes** de que el código las use y no quita nada que el código en producción todavía necesite. La conexión usa los secretos PROD_DB_HOST, PROD_DB_NAME, PROD_MIGRATOR_USER y PROD_MIGRATOR_PASSWORD del entorno, a través del pooler de sesión de Supabase (el runner usa pg_advisory_lock). El mismo comando puede ejecutarse manualmente:

```
npm run db:migrate   # aplica las migraciones pendientes contra la base de producción

npm run db:status    # revisa qué está aplicado/pendiente
```

Rollback

- **Frontend/backend (Vercel):** Dashboard → Deployments → elegir el deployment anterior bueno → "Promote to Production". Revierte el código al instante, sin rebuild.
- **Base de datos:** el runner no tiene `down` automático. Deshacer un cambio de esquema es una migración **nueva** que lo revierte — nunca se edita un archivo ya aplicado (el runner lo rechaza si el checksum no coincide).

Rotación de secretos

1. Generar el nuevo valor (ej. un `JWT_SECRET` nuevo).
2. Actualizarlo en Vercel → Project Settings → Environment Variables (Production).
3. Redeploy — obligatorio, las variables solo se leen al iniciar/buildear.

4. Si el secreto rotado es `JWT_SECRET`, todas las sesiones activas quedan invalidadas y los usuarios deben volver a iniciar sesión.

Incidentes reales

Síntoma	Causa	Resolución
500 en los endpoints de agenda de barbero	La migración `0005_agenda_barbero.sql` no se había aplicado todavía en producción — el código ya esperaba columnas/tablas que la base no tenía	Se corrió `npm run db:migrate` contra la base de Supabase de producción
530 al enviar el correo de recuperación de contraseña	El SMTP de Gmail rechazó la autenticación	Se generó una contraseña de aplicación (App Password) de Google para la cuenta de Gmail remitente, que es la cuenta de uno de los integrantes del equipo, y se configuró como SMTP_PASSWORD.

3. Estrategia de backup (5.1)

Estado: la estrategia de respaldo está planificada, pero no se pudo implementar. Hoy no existen copias automáticas fuera de Supabase; lo que sigue es el diseño previsto.

Supabase en plan **Free** no incluye backups automáticos ni point-in-time recovery (PITR) — eso empieza en el plan Pro, y PITR real es un add-on aparte sobre Pro. Como el proyecto se queda en Free, hay que armar el backup por fuera de Supabase.

Qué cubre Supabase Free y qué no

Necesario según el enunciado	¿Supabase Free lo da?	Cómo se suplirá (planificado)
Full semanal	No	`pg_dump` completo por GitHub Actions (cron semanal)
Incremental diario	No	`pg_dump` diario (la base es pequeña; sin columnas de auditoría en todas las tablas, un incremental real por filas no es confiable — un dump completo diario lo aproxima)
Log cada 15–30 min + restauración a un punto en el tiempo	No (eso es PITR real, vía WAL, solo en Pro+add-on)	Aproximación: `pg_dump` cada 30 min solo de las tablas que cambian seguido (`citas`, `insumo_movimientos`) — no es PITR real, es la alternativa viable en Free

Diseño planificado

- **Ejecutor:** un workflow de GitHub Actions con 3 jobs programados (`schedule: cron`), corriendo `pg_dump` contra la cadena de conexión de Supabase (el secreto vive en GitHub Secrets, nunca en el repo).
- **Full semanal:** `pg_dump --format=custom` de toda la base, un domingo de madrugada.
- **Incremental diario:** `pg_dump --format=custom` completo todas las noches (dado el tamaño actual de la base, es más simple y más confiable que armar un incremental real por filas).
- **"Log" cada 30 min:** `pg_dump --data-only --table=citas --table=insumo_movimientos` — las dos tablas que de verdad cambian seguido durante el día.
- **Almacenamiento:** los `.dump` comprimidos se suben a un bucket/alojamiento aparte de Supabase (ej. un repo privado de GitHub vía Releases, o un bucket gratuito tipo Backblaze B2/S3) — nunca junto a la base que respaldan.
- **Retención:** 4 semanales, 14 diarios, 48 horas de los de 30 min (se van rotando/borrando los más viejos).

Restauración a un punto en el tiempo (aproximada)

Sin WAL no hay un punto exacto en el tiempo — la restauración posible es "al momento del dump más cercano antes del incidente": se restaura el full o incremental más reciente anterior al problema, y después se reaplican (si existen) los dumps de `citas`/`insumo_movimientos` de los 30 minutos siguientes para acercarse más a la hora real.

```
pg_restore --clean --if-exists -d <cadena_de_conexion_destino> backup.dump
```

Matriz de Trazabilidad

Campo	Detalle
Proyecto	Barber Kongs — Sistema de Gestión para Barbería (Fase 3)
Equipo	Miguel Angel Quemé Gómez · Brandon Gustavo Güinac Román
Versión del documento	1.0 — 25 de octubre de 2026
Código evaluado	Backend: rama main (commit a74f76c). Frontend: rama main (commit 34ee80f).
Alcance	31 requisitos funcionales, 11 no funcionales, 23 casos de uso, 23 historias de usuario, 48 endpoints y 202 pruebas automáticas

1. Resumen

Elemento	Total	Cobertura de la cadena
Requisitos funcionales	31 (+2 sin definir)	30 con caso de uso; 31 con prueba de integración
Requisitos no funcionales	11	4 con pruebas automáticas; 2 medidos con el script de carga; el resto, por inspección o por la plataforma
Casos de uso	23	23 con historia de usuario; 21 con pantalla; 23 con prueba de integración
Historias de usuario	23	20 con clave de Jira documentada; 19 validadas como «Cumple» en la Fase 2; 3 no figuran en la validación
Endpoints	48	48 con prueba por HTTP; 43 con consumidor en el frontend; 48 asociados a un caso de uso o requisito
Pruebas automáticas	202	202 asociadas a algún requisito; 101 de integración y 101 unitarias

1.1 Estados de verificación

Estado	Significado
Cubierto	Tiene pruebas automáticas, al menos una de integración (por HTTP).
Cubierto con observaciones	Se cumple y se prueba, con una observación de diseño documentada en el Plan de Pruebas (por ejemplo, O-03).
Parcial	Se cumple solo en parte, o solo tiene pruebas unitarias o inspección.
Sin prueba automática	No tiene ninguna prueba automática.

Las columnas **U** e **I** cuentan las pruebas unitarias y de integración que verifican el elemento. Los defectos **D-xx** son los detectados y corregidos que registra el Plan de Pruebas; **O-xx** y **OE-xx** son observaciones.

1.2 Estado de los requisitos y casos de uso

Estado	Requisitos funcionales	Casos de uso
Cubierto	30	22
Cubierto con observaciones	0	1
Parcial	1	0
Sin prueba automática	0	0
Total	**31**	**23**

1.3 Conclusiones

- **Todo lo planeado tiene un camino hasta el código:** los 23 casos de uso tienen historia, endpoints, tablas y pruebas de integración. Ninguna prueba ni endpoint queda huérfano.
- **30 de los 31 requisitos funcionales están cubiertos** por pruebas de integración y el resto tiene una limitación documentada: 1 parcial (RF-REP-01, observación O-03).
- **Se detectaron y corrigieron 8 defectos** durante las pruebas y el inventario de endpoints (sección 7.1); cada uno dejó una prueba de regresión.

2. Matriz de requisitos funcionales

Para cada requisito funcional: los casos de uso e historias que lo cubren, los endpoints que lo implementan, la pantalla desde la que el usuario lo ejerce y las pruebas automáticas (U = unitarias, I = de integración) que lo verifican. La columna «Defectos corregidos» indica qué defectos detectados (sección 7.1) afectaron a ese requisito.

Requisito	Descripción	Caso de uso	Historia	Endpoints	Pantalla / componente	U	I	Defectos corregidos	Estado
RF-AUT-01	Inicio de sesión con correo y contraseña	UC-01	HU-01	E-02	Login VerifyTwoFactor Seguridad MiCuenta	3	3	—	Cubierto
RF-AUT-02	Emisión y validación de tokens JWT	UC-01	HU-01	E-06	Login VerifyTwoFactor Seguridad MiCuenta	4	2	—	Cubierto
RF-AUT-03	Tres roles con permisos diferenciados	Transversal	—	Todos los protegidos	ProtectedRoute (todas las rutas protegidas)	2	7	—	Cubierto
RF-AUT-04	Crear, editar y desactivar cuentas de barberos	UC-22 UC-23 UC-24	HU-04 HU-05 HU-06	E-09, E-13, E-14, E-15	AdminBarberos	14	9	D-02	Cubierto
RF-AUT-05	Registro autónomo de clientes	UC-02	HU-02	E-01	Register	2	2	—	Cubierto
RF-AUT-06	Recuperación de contraseña por correo	UC-03	HU-03	E-04, E-05	ForgotPassword ResetPassword	4	2	—	Cubierto
RF-AUT-07	Autenticación de dos factores por correo	UC-01	HU-01	E-03, E-07	Login VerifyTwoFactor Seguridad MiCuenta	6	3	—	Cubierto
RF-CIT-01	Agendar cita (barbero, servicio, fecha y hora)	UC-04	HU-07	E-08, E-10, E-12, E-19, E-40	Booking	8	7	D-05	Cubierto
RF-CIT-02	Solo días de atención (domingo a jueves por defecto)	UC-04	HU-07	E-11, E-12, E-19	Booking	4	1	D-01	Cubierto
RF-CIT-03	Sin citas en el horario de almuerzo	UC-04	HU-07	E-19, E-25	Booking	1	1	—	Cubierto
RF-CIT-04	Solo dentro del horario de atención	UC-04	HU-07	E-12, E-19, E-25	Booking	4	1	—	Cubierto
RF-CIT-05	Reprogramar validando disponibilidad	UC-05	HU-08	E-25	MisCitas	0	2	—	Cubierto

Requisito	Descripción	Caso de uso	Historia	Endpoints	Pantalla / componente	U	I	Defectos corregidos	Estado
RF-CIT-06	Cancelar (cliente o administrador)	UC-06	HU-09	E-23	MisCitas AdminCitas	2	4	—	Cubierto
RF-CIT-07	Impedir la doble reserva del mismo barbero	UC-04 UC-25	HU-07 HU-11	E-19, E-20, E-25	Booking AdminCitas Agenda	1	3	—	Cubierto
RF-CIT-08	Notificar confirmación, reprogramación y cancelación	UC-04 UC-05 UC-06	HU-07 HU-08 HU-09	E-19, E-23, E-25, E-37, E-38	Booking MisCitas AdminCitas NotificationsBell	11	6	—	Cubierto
RF-CIT-09	El barbero consulta su agenda diaria y semanal	UC-07	HU-10	E-22	Agenda	3	9	D-03 D-07	Cubierto
RF-CLI-01	Registrar datos básicos del cliente	UC-08	HU-12	E-29	—	0	3	—	Cubierto
RF-CLI-02	Historial de citas por cliente	UC-10	HU-14	E-21, E-28	AdminClientes	0	3	—	Cubierto
RF-CLI-03	Buscar y filtrar clientes	UC-09	HU-13	E-26	AdminClientes	0	2	—	Cubierto
RF-CLI-04	Editar o desactivar clientes	UC-11	HU-15	E-30, E-31, E-32	AdminClientes	0	4	—	Cubierto
RF-SER-01	Crear, editar y desactivar servicios	UC-12	HU-16	E-42, E-43, E-44	AdminServicios	0	6	—	Cubierto
RF-SER-02	Asociar servicios a insumos	UC-13	HU-17	E-45, E-46, E-47	AdminServicios	1	4	—	Cubierto
RF-SER-03	Catálogo de servicios activos al agendar	UC-04	HU-07	E-40, E-41	Booking	0	3	—	Cubierto
RF-INV-01	Registrar insumos (nombre, cantidad, unidad)	UC-14	HU-18	E-33, E-34, E-35	AdminInventario	1	5	—	Cubierto
RF-INV-02	Descuento automático al completar una cita	UC-16	HU-20	E-24	Agenda	9	7	D-04	Cubierto
RF-INV-03	Registrar reabastecimiento (entradas)	UC-15	HU-19	E-36	AdminInventario	1	4	—	Cubierto
RF-INV-04	Alerta de stock bajo	UC-17	HU-21	E-24, E-35, E-37	AdminInventario NotificationsBell	8	5	D-08	Cubierto
RF-REP-01	Reporte de citas por rango de fechas	UC-18	HU-22	E-39	AdminReportes	3	2	O-03	Parcial
RF-REP-02	Reporte de ingresos por servicio y barbero	UC-18	HU-22	E-39	AdminReportes	3	2	—	Cubierto
RF-REP-03	Reporte de consumo de insumos	UC-18	HU-22	E-39	AdminReportes	3	2	—	Cubierto
RF-REP-04	Exportar cada reporte a PDF y a Excel	UC-18	HU-22	E-39	AdminReportes	7	2	—	Cubierto

RF-AUT-03 es transversal: lo aplican todas las rutas protegidas y ningún caso de uso lo lista.

Requisitos citados por los casos de uso pero no definidos

Requisito citado	Dónde se cita	Pruebas U / I	Situación
RF-CIT-10	UC-25	0 / 1	Referenciado por UC-25 (walk-in); no figura en la lista de requisitos
RF-NOT-01	UC-26	2 / 6	Referenciado por UC-26 (notificaciones); no figura en la lista de requisitos

3. Matriz de casos de uso

Cada caso de uso con su historia de usuario, los requisitos de los que deriva, la pantalla, los endpoints, las tablas que toca y el resultado de sus pruebas. Un asterisco indica un requisito citado por el caso de uso pero no definido en la lista (sección 2).

Caso de uso	Actor	Requisitos	Historia · Jira	Pantalla / componente	Endpoints	Tablas	U	I	Estado
UC-01 Iniciar sesión	Cliente, Barbero, Administrador	RF-AUT-01 RF-AUT-02 RF-AUT-07	HU-01 BK-15	Login (/ingresar) VerifyTwoFactor (/verificar-2fa) Seguridad y MiCuenta (2FA)	E-02, E-03, E-06, E-07	users two_factor_codes notificaciones	12	7	Cubierto
UC-02 Registrarse	Cliente	RF-AUT-05	HU-02 BK-16	Register (/registro)	E-01	users clientes	2	2	Cubierto
UC-03 Recuperar contraseña	Cliente, Barbero, Administrador	RF-AUT-06	HU-03 BK-17	ForgotPassword (/recuperar) ResetPassword (/restablecer)	E-04, E-05	users password_reset_tokens	4	2	Cubierto
UC-04 Agendar cita	Cliente	RF-CIT-01 RF-CIT-02 RF-CIT-03 RF-CIT-04 RF-CIT-07 RF-CIT-08 RF-SER-03	HU-07 BK-29	Booking (/agendar)	E-08, E-12, E-19, E-40	citas servicios barberos disponibilidad_barberos ausencias_barbero notificaciones	14	12	Cubierto D-01, D-05
UC-05 Reprogramar cita	Cliente	RF-CIT-05 RF-CIT-08	HU-08 BK-30	MisCitas, modal Reprogramar (/mis-citas)	E-12, E-21, E-25	citas notificaciones	0	2	Cubierto D-01
UC-06 Cancelar cita	Cliente, Administrador	RF-CIT-06 RF-CIT-08	HU-09 BK-31	MisCitas (/mis-citas) AdminCitas (/admin/citas)	E-21, E-22, E-23	citas notificaciones	2	4	Cubierto
UC-07 Consultar agenda	Barbero	RF-CIT-09	HU-10 BK-32	Agenda (/barbero/agenda), vista diaria y semanal	E-16, E-17, E-18, E-22	citas ausencias_barbero disponibilidad_barberos	3	9	Cubierto D-03, D-07
UC-08 Registrar cliente	Administrador	RF-CLI-01	HU-12 BK-21	Sin pantalla propia	E-29	clientes	0	3	Cubierto (API sin pantalla)

Caso de uso	Actor	Requisitos	Historia · Jira	Pantalla / componente	Endpoints	Tablas	U	I	Estado
UC-09 Buscar y filtrar clientes	Administrador	RF-CLI-03	HU-13 BK-22	AdminClientes, búsqueda (/admin/clientes)	E-26	clientes	0	2	Cubierto
UC-10 Consultar historial de cliente	Administrador	RF-CLI-02	HU-14 BK-23	AdminClientes, botón Historial	E-27, E-28	clientes citas	0	3	Cubierto
UC-11 Editar o desactivar cliente	Administrador	RF-CLI-04	HU-15 BK-24	AdminClientes, editar y Activos/Inactivos	E-30, E-31, E-32	clientes	0	4	Cubierto
UC-12 Crear, editar o desactivar servicio	Administrador	RF-SER-01	HU-16 BK-25	AdminServicios (/admin/servicios)	E-40, E-42, E-43, E-44	servicios	0	6	Cubierto
UC-13 Asociar insumos a servicio	Administrador	RF-SER-02	HU-17 BK-26	AdminServicios, modal Insumos	E-45, E-46, E-47	servicio_insumos insumos	1	4	Cubierto
UC-14 Registrar insumo	Administrador	RF-INV-01	HU-18 BK-27	AdminInventario (/admin/inventario), alta y edición	E-33, E-34, E-35	insumos	1	5	Cubierto D-08
UC-15 Registrar reabastecimiento	Administrador	RF-INV-03	HU-19 BK-28	AdminInventario, acción Registrar entrada	E-36	insumos insumo_movimientos	1	4	Cubierto
UC-16 Descontar insumos automáticamente	Sistema (Barbero marca la cita)	RF-INV-02	HU-20 BK-34	Proceso del sistema; se dispara desde Agenda (botón Atendida)	E-24	citas servicio_insumos insumos insumo_movimientos	9	7	Cubierto D-04 (proceso del sistema)
UC-17 Generar alerta de inventario bajo	Sistema (Administrador recibe)	RF-INV-04	HU-21 BK-35	AdminInventario (fila resaltada) Correo y campana de notificaciones	E-24, E-35, E-37	insumos notificaciones users	8	5	Cubierto
UC-18 Generar y exportar reporte	Administrador	RF-REP-01 RF-REP-02 RF-REP-03 RF-REP-04	HU-22 BK-36	AdminReportes (/admin/reportes)	E-39	citas servicios barberos insumo_movimientos insumos	9	4	Cubierto con observaciones O-03
UC-22 Crear cuenta de barbero	Administrador	RF-AUT-04	HU-04 —	AdminBarberos, con editor de horario (/admin/barberos)	E-09, E-13	users barberos disponibilidad_barberos	10	4	Cubierto
UC-23 Editar cuenta de barbero	Administrador	RF-AUT-04	HU-05 —	AdminBarberos, editar con horario y ausencias	E-10, E-11, E-14	barberos disponibilidad_barberos	9	1	Cubierto

Caso de uso	Actor	Requisitos	Historia · Jira	Pantalla / componente	Endpoints	Tablas	U	I	Estado
UC-24 Desactivar cuenta de barbero	Administrador	RF-AUT-04	HU-06 BK-20	AdminBarberos (/admin/barberos)	E-09, E-15	barberos	0	3	Cubierto D-02
UC-25 Registrar cliente sin reservación (walk-in)	Administrador, Barbero	RF-CIT-10* RF-CIT-07	HU-11 BK-33	AdminCitas y Agenda, botón + Walk-in	E-20	citas clientes	0	1	Cubierto D-01
UC-26 Consultar notificaciones	Cliente, Barbero, Administrador	RF-NOT-01*	HU-23 —	Campana de notificaciones (barra de navegación)	E-37, E-38	notificaciones	2	6	Cubierto

4. Matriz de historias de usuario

Las 23 historias del plan de Jira, con la pantalla que las implementa según la validación de la Fase 2 y su situación actual. La relación entre historia y caso de uso es 1 a 1.

Historia	Jira	Épica	Sprint	Caso de uso	Historia de usuario	Pantalla / componente (validación Fase 2)	Validación Fase 2	Observación actual
HU-01	BK-15	EP-01	1	UC-01	Iniciar sesión con correo/contraseña y 2FA opcional por correo	Login.jsx + VerifyTwoFactor.jsx	Cumple	—
HU-02	BK-16	EP-01	1	UC-02	Registrarse como cliente nuevo	Register.jsx	Cumple	—
HU-03	BK-17	EP-01	1	UC-03	Recuperar contraseña por enlace al correo	ForgotPassword.jsx + ResetPassword.jsx	Cumple	—
HU-04	—	EP-01	1	UC-22	Crear la cuenta de un barbero con su horario de disponibilidad	AdminBarberos.jsx + HorarioEditor.jsx	No figura	Implementada (PR #18 backend, PR #13 frontend)
HU-05	—	EP-01	1	UC-23	Editar los datos o el horario de un barbero	AdminBarberos.jsx + HorarioEditor.jsx	No figura	Implementada (PR #18 backend, PR #13 frontend)
HU-06	BK-20	EP-01	1	UC-24	Desactivar la cuenta de un barbero	AdminBarberos.jsx	Cumple	Defecto D-02 corregido
HU-07	BK-29	EP-02	2	UC-04	Agendar una cita (servicio, barbero, fecha y hora)	Booking.jsx	Cumple	—
HU-08	BK-30	EP-02	2	UC-05	Reprogramar una cita ya agendada	MisCitas.jsx, modal Reprogramar	Cumple	—
HU-09	BK-31	EP-02	2	UC-06	Cancelar una cita (cliente o administrador)	MisCitas.jsx + AdminCitas.jsx	Cumple	—
HU-10	BK-32	EP-02	2	UC-07	Consultar la agenda diaria y semanal del barbero	Agenda.jsx	Cumple	Defectos D-03 y D-07 corregidos
HU-11	BK-33	EP-02	2	UC-25	Registrar atención walk-in (sin cita previa)	AdminCitas.jsx + Agenda.jsx, botón + Walk-in	Cumple	Nota de la validación: no es literal al UC-25 (no consulta quién está libre ahora mismo)
HU-12	BK-21	EP-03	1	UC-08	Registrar cliente atendido presencialmente	AdminClientes.jsx	Cumple	Ya no tiene pantalla: el botón + Nuevo cliente se retiró el 24-oct (PR #11)

Historia	Jira	Épica	Sprint	Caso de uso	Historia de usuario	Pantalla / componente (validación Fase 2)	Validación Fase 2	Observación actual
HU-13	BK-22	EP-03	1	UC-09	Buscar y filtrar clientes	AdminClientes.jsx, campo de búsqueda	Cumple	—
HU-14	BK-23	EP-03	1	UC-10	Consultar el historial de citas de un cliente	AdminClientes.jsx, botón Historial	Cumple	—
HU-15	BK-24	EP-03	1	UC-11	Editar, desactivar o reactivar un cliente	AdminClientes.jsx, Activos/Inactivos	Cumple	Reactivar quedó operativo con la corrección del PR #10 (filtro ?estado=)
HU-16	BK-25	EP-04	1	UC-12	Crear, editar o desactivar servicios	AdminServicios.jsx	Cumple	—
HU-17	BK-26	EP-04	1	UC-13	Asociar los insumos que consume cada servicio	AdminServicios.jsx, modal Insumos	Cumple	—
HU-18	BK-27	EP-05	1	UC-14	Registrar un nuevo insumo en el inventario	AdminInventario.jsx	Cumple	—
HU-19	BK-28	EP-05	1	UC-15	Registrar el reabastecimiento de un insumo	AdminInventario.jsx, Registrar entrada	Cumple	—
HU-20	BK-34	EP-05	2	UC-16	Descontar insumos automáticamente al atender una cita	Proceso del sistema (citas.service.js)	N/A	Sin pantalla por diseño; defecto D-04 corregido
HU-21	BK-35	EP-05	2	UC-17	Alertar cuando un insumo cae bajo el mínimo	AdminInventario.jsx + correo	Cumple	—
HU-22	BK-36	EP-06	3	UC-18	Generar y exportar reportes (PDF o Excel)	AdminReportes.jsx	Cumple	—
HU-23	—	EP-07	3	UC-26	Consultar notificaciones y marcarlas como leídas	NotificationsBell.jsx	No figura	Implementada (PR #3 backend)

5. Matriz de requisitos no funcionales

Los requisitos no funcionales se verifican de maneras distintas: pruebas automáticas, el script de carga, inspección del código o la propia plataforma de despliegue. La columna «Verificación» indica cuál.

Requisito	Descripción	Evidencia	Verificación	Estado	Observación
RNF-REN-01	Consulta de disponibilidad en menos de 2 s	Script de carga `npm run test:carga`: disponibilidad con p95 de 10.7 ms y máximo de 93.7 ms (20 usuarios, base local).	Carga manual	Cubierto (carga manual)	Medido en local; no se ha medido contra Vercel + Supabase.
RNF-REN-02	Al menos 20 usuarios concurrentes	Mismo script: 20 usuarios durante 15 s, 48,905 solicitudes, 0 errores.	Carga manual	Cubierto (carga manual)	Medido en local; no se ha medido contra Vercel + Supabase.
RNF-DIS-01	Disponibilidad de al menos 95 % mensual	`GET /api/health` (E-48) responde 200 y lo usa la integración continua; hosting en Vercel y Supabase.	Prueba de integración + plataforma	Parcial	No hay monitoreo de disponibilidad que mida el porcentaje mensual; queda en el alcance del despliegue.
RNF-DIS-02	Página de error clara si el servicio no está disponible	`NotFound.jsx` atiende rutas inexistentes (404); los errores de la API se muestran con `Alert`.	Inspección del código	Parcial	No existe una pantalla específica para «servicio no disponible» (caída del backend o de la base).
RNF-SEG-01	Contraseñas con hash seguro (bcrypt)	2 pruebas: el registro no expone el hash y las contraseñas se guardan con bcrypt.	Automática	Cubierto	—

Requisito	Descripción	Evidencia	Verificación	Estado	Observación
RNF-SEG-02	Permisos de rol validados en cada endpoint del backend	24 pruebas de acceso por rol (401/403) en todos los módulos.	Automática	Cubierto	Valida el rol en cada endpoint y, para el barbero, también que la cita o la agenda sea la suya (defectos D-02 a D-04 corregidos).
RNF-SEG-03	Transmisión por HTTPS	El despliegue en Vercel entrega el frontend y la API únicamente por HTTPS.	Plataforma	Cubierto por la plataforma	No es verificable con las pruebas locales; se comprueba en el despliegue.
RNF-SEG-04	Sesiones (JWT) que expiran	2 pruebas: se rechazan tokens vencidos y de firma inválida. Vigencia de 2 h por defecto (`JWT_EXPIRES_IN`).	Automática	Cubierto	La vigencia configurable no se mide; solo se prueba el rechazo del token vencido.
RNF-USA-01	Interfaz responsiva (móvil y escritorio)	`index.html` declara el viewport y la hoja de estilos tiene 3 consultas `@media`.	Inspección del código	Parcial	Se verifica por inspección del código; el lint y la compilación del frontend corren en el CI.
RNF-USA-02	Agendar en no más de 4 pasos	`Booking.jsx` define 4 pasos: Servicio, Barbero, Fecha y hora, Confirmar.	Inspección del código	Cubierto por diseño	Se cumple por la estructura del componente; no hay prueba de interfaz.
RNF-USA-03	Mensajes de error comprensibles	12 pruebas verifican mensajes en español en los errores de validación.	Automática	Cubierto con observaciones	Algunos mensajes siguen siendo técnicos o en inglés (OE-01). Los ids de ruta y las consultas inválidas ya responden 400 en español (D-06, D-07).

6. Matriz inversa de endpoints

El camino contrario: desde cada endpoint hasta el caso de uso o requisito que lo justifica, la pantalla que lo usa y lo que muestran las pruebas. Las solicitudes son las registradas en la ejecución de la suite de integración.

ID	Endpoint	Casos de uso	Requisitos	Consumido por (frontend)	Solicitudes	Caminos probados	Defectos corregidos
E-01	**POST** `/auth/register`	UC-02	RF-AUT-05	Register	3	2xx · 4xx	—
E-02	**POST** `/auth/login`	UC-01	RF-AUT-01	Login	43	2xx · 4xx	—
E-03	**POST** `/auth/verify-2fa`	UC-01	RF-AUT-07	VerifyTwoFactor	3	2xx · 4xx	—
E-04	**POST** `/auth/forgot-password`	UC-03	RF-AUT-06	ForgotPassword	2	2xx	—
E-05	**POST** `/auth/reset-password`	UC-03	RF-AUT-06	ResetPassword	2	2xx · 4xx	—
E-06	**GET** `/auth/me`	UC-01	RF-AUT-02	AuthContext (sesión)	3	2xx · 4xx	—
E-07	**PUT** `/auth/2fa`	UC-01	RF-AUT-07	Seguridad, MiCuenta	5	2xx · 4xx	—
E-08	**GET** `/barberos`	UC-04	RF-CIT-01	Booking, AdminCitas, Landing	3	2xx	D-05
E-09	**GET** `/barberos/admin`	UC-22, UC-24	RF-AUT-04	AdminBarberos	5	2xx · 4xx	—
E-10	**GET** `/barberos/:id`	UC-23	RF-CIT-01	AdminBarberos	5	2xx · 4xx	D-05, D-06

ID	Endpoint	Casos de uso	Requisitos	Consumido por (frontend)	Solicitudes	Caminos probados	Defectos corregidos
E-11	**GET** `/barberos/:id/horario`	UC-23	RF-CIT-02	Sin consumidor	4	2xx · 4xx	D-06
E-12	**GET** `/barberos/:id/disponibilidad`	UC-04, UC-05	RF-CIT-01, RF-CIT-02, RF-CIT-04	Booking	2	2xx	D-06
E-13	**POST** `/barberos`	UC-22	RF-AUT-04	AdminBarberos	10	2xx · 4xx	—
E-14	**PUT** `/barberos/:id`	UC-23	RF-AUT-04	AdminBarberos	3	2xx · 4xx	D-06
E-15	**DELETE** `/barberos/:id`	UC-24	RF-AUT-04	AdminBarberos	5	2xx · 4xx	D-02, D-06
E-16	**GET** `/barberos/:id/ausencias`	UC-07	Agenda del barbero	AusenciasBarbero	5	2xx · 4xx	D-06
E-17	**POST** `/barberos/:id/ausencias`	UC-07	Agenda del barbero	AusenciasBarbero	7	2xx · 4xx	D-06
E-18	**DELETE** `/barberos/:id/ausencias/:ausenciaId`	UC-07	Agenda del barbero	AusenciasBarbero	4	2xx · 4xx	D-06
E-19	**POST** `/citas`	UC-04	RF-CIT-01, RF-CIT-02, RF-CIT-03, RF-CIT-04, RF-CIT-07, RF-CIT-08	Booking	46	2xx · 4xx	D-01
E-20	**POST** `/citas/walkin`	UC-25	RF-CIT-07, RF-CIT-10	AdminCitas, Agenda	3	2xx · 4xx	D-01
E-21	**GET** `/citas/mias`	UC-05, UC-06	RF-CLI-02	MisCitas	3	2xx	—
E-22	**GET** `/citas`	UC-06, UC-07	RF-CIT-09	AdminCitas, Agenda	23	2xx · 4xx	D-03, D-07
E-23	**PUT** `/citas/:id/cancelar`	UC-06	RF-CIT-06, RF-CIT-08	MisCitas, AdminCitas	8	2xx · 4xx	D-06
E-24	**PUT** `/citas/:id/atender`	UC-16, UC-17	RF-INV-02, RF-INV-04	Agenda	15	2xx · 4xx · 5xx	D-04, D-06
E-25	**PUT** `/citas/:id`	UC-05	RF-CIT-03, RF-CIT-04, RF-CIT-05, RF-CIT-07, RF-CIT-08	MisCitas	3	2xx · 4xx	D-01, D-06
E-26	**GET** `/clientes`	UC-09	RF-CLI-03	AdminClientes	10	2xx · 4xx	—
E-27	**GET** `/clientes/:id`	UC-10	—	Sin consumidor	4	2xx · 4xx	D-06
E-28	**GET** `/clientes/:id/historial`	UC-10	RF-CLI-02	AdminClientes	4	2xx · 4xx	D-06
E-29	**POST** `/clientes`	UC-08	RF-CLI-01	Sin consumidor	12	2xx · 4xx	—
E-30	**PUT** `/clientes/:id`	UC-11	RF-CLI-04	AdminClientes	3	2xx · 4xx	D-06
E-31	**PUT** `/clientes/:id/activar`	UC-11	RF-CLI-04	AdminClientes	2	2xx · 4xx	D-06
E-32	**DELETE** `/clientes/:id`	UC-11	RF-CLI-04	AdminClientes	3	2xx · 4xx	D-06
E-33	**GET** `/inventario`	UC-14	RF-INV-01	AdminInventario, AdminServicios	5	2xx · 4xx	—

ID	Endpoint	Casos de uso	Requisitos	Consumido por (frontend)	Solicitudes	Caminos probados	Defectos corregidos
E-34	**POST** `/inventario`	UC-14	RF-INV-01	AdminInventario	2	2xx · 4xx	—
E-35	**PUT** `/inventario/:id`	UC-14, UC-17	RF-INV-01, RF-INV-04	AdminInventario	7	2xx · 4xx	D-06, D-08
E-36	**POST** `/inventario/:id/entrada`	UC-15	RF-INV-03	AdminInventario	6	2xx · 4xx	D-06
E-37	**GET** `/notificaciones`	UC-17, UC-26	RF-CIT-08, RF-INV-04, RF-NOT-01	NotificationsBell	10	2xx · 4xx	—
E-38	**PUT** `/notificaciones/:id/leer`	UC-26	RF-CIT-08, RF-NOT-01	NotificationsBell	5	2xx · 4xx	D-06
E-39	**GET** `/reportes`	UC-18	RF-REP-01, RF-REP-02, RF-REP-03, RF-REP-04	AdminReportes	15	2xx · 4xx	—
E-40	**GET** `/servicios`	UC-04, UC-12	RF-CIT-01, RF-SER-03	Booking, AdminCitas, AdminServicios, Agenda, Landing	3	2xx	—
E-41	**GET** `/servicios/:id`	—	RF-SER-03	Sin consumidor	4	2xx · 4xx	D-06
E-42	**POST** `/servicios`	UC-12	RF-SER-01	AdminServicios	10	2xx · 4xx	—
E-43	**PUT** `/servicios/:id`	UC-12	RF-SER-01	AdminServicios	5	2xx · 4xx	D-06
E-44	**DELETE** `/servicios/:id`	UC-12	RF-SER-01	AdminServicios	3	2xx · 4xx	D-06
E-45	**GET** `/servicios/:id/insumos`	UC-13	RF-SER-02	AdminServicios	4	2xx · 4xx	D-06
E-46	**POST** `/servicios/:id/insumos`	UC-13	RF-SER-02	AdminServicios	7	2xx · 4xx	D-06
E-47	**DELETE** `/servicios/:id/insumos/:insumoId`	UC-13	RF-SER-02	AdminServicios	4	2xx · 4xx	D-06
E-48	**GET** `/health`	—	RNF-DIS-01	Integración continua (no es el frontend)	1	2xx	—

D-06 (ids que no eran UUID) afectaba a los 25 endpoints que reciben un id en la ruta. «Sin consumidor» indica que ninguna pantalla del frontend lo usa hoy.

7. Evidencia en Git

Cada funcionalidad se ancla a un commit y a su pull request. El historial muestra que el código llegó a `develop` mediante pull requests, salvo la base inicial de la Fase 2 de cada repositorio, que se subió directamente.

Repo	Elemento implementado	Commit	Fecha	Autor	Pull request	Casos de uso / requisitos
Backend	Base de la Fase 2: auth, citas, barberos, clientes e inventario	e207f67	20-sep-2026	Brandonsk8	Sin PR (directo a develop, previo al PR #1)	UC-01 a UC-07, UC-08 a UC-11, UC-14 a UC-17, UC-22 a UC-25
Backend	Historial y reactivar clientes	1a934a5	20-sep-2026	OkKid-A	PR #1	UC-10, UC-11
Backend	Reportes de citas, ingresos e insumos	19af664	20-sep-2026	OkKid-A	PR #2	UC-18 (EP-06)

Repo	Elemento implementado	Commit	Fecha	Autor	Pull request	Casos de uso / requisitos
Backend	Centro de notificaciones	80b8d42	20-sep-2026	OkKid-A	PR #3	UC-26 (EP-07)
Backend	2FA activable por cuenta	0596012	20-sep-2026	OkKid-A	PR #4	RF-AUT-07 (UC-01)
Backend	Asociar insumos a servicio	e0b0f7d	20-sep-2026	OkKid-A	PR #6	UC-13
Backend	Migraciones de BD versionadas	dfc4fa1	21-sep-2026	OkKid-A	PR #8	Modelo de datos
Backend	Filtro de estado para reactivar clientes (BK-24)	f676eae	21-sep-2026	Brandonsk8	PR #10	UC-11 (HU-15)
Backend	Plantilla de correo con marca	5e97bc0	24-oct-2026	OkKid-A	PR #17	UC-26, RF-CIT-08
Backend	Horario semanal al crear y editar barbero	2666f25	24-oct-2026	OkKid-A	PR #18	UC-22, UC-23
Backend	Días en que el barbero no estará disponible	1b381c5	24-oct-2026	OkKid-A	PR #18	Agenda del barbero (UC-07)
Backend	Avisar al barbero cuando se agenda una cita	35929bf	24-oct-2026	OkKid-A	PR #19	UC-04, UC-26
Backend	Estilos de los reportes en Excel y PDF	be313c2	24-oct-2026	OkKid-A	PR #20	UC-18
Backend	Despliegue en Vercel + Supabase y marca Barber Kongs	60d2e26	25-oct-2026	Brandonsk8	PR #21	RNF-DIS-01, RNF-SEG-03
Backend	Pruebas automáticas del backend	3f9331e	25-oct-2026	OkKid-A	PR #25 y #27	Todas (Plan de Pruebas)
Backend	Pruebas de los endpoints sin cobertura HTTP (rama feature/testing-endpoints)	f583d98	25-oct-2026	OkKid-A	PR #29	Inventario de Endpoints
Backend	Correcciones D-02 a D-05: baja de barbero, agenda propia, atender solo lo propio, catálogo sin correo	9e5feb8, a014f5b	25-oct-2026	OkKid-A	PR #29 (hotfix/acceso-y-datos-expuestos)	RF-AUT-04, RF-CIT-01, RF-CIT-09, RF-INV-02
Backend	Correcciones D-06 a D-08: ids UUID, consulta de citas e insumo sin mínimo	7da88f4, 5e07f5e, df04891	25-oct-2026	OkKid-A	PR #30 (hotfix/validacion)	RNF-USA-03, RF-CIT-09, RF-INV-04
Backend	Aplicación automática de las migraciones en producción (migrate.yml)	23e79b5	25-oct-2026	Brandonsk8	Sin PR (directo a main)	Infraestructura y Despliegue
Frontend	Frontend inicial según los mockups de la Fase 2	88c4a59	20-sep-2026	Brandonsk8	Sin PR (directo a develop)	Pantallas base de UC-01 a UC-18
Frontend	Historial/desactivar clientes y reprogramar/cancelar citas	3f5446a	20-sep-2026	OkKid-A	PR #1	UC-05, UC-06, UC-10, UC-11
Frontend	Pantalla para activar o desactivar el 2FA	f29170b	20-sep-2026	OkKid-A	PR #2	UC-01 (RF-AUT-07)
Frontend	Reportes conectados al contrato real	b8d6078	20-sep-2026	OkKid-A	PR #3	UC-18
Frontend	Gestión de insumos por servicio	f76cc1e	21-sep-2026	OkKid-A	PR #4	UC-13
Frontend	HU pendientes: barberos, reprogramar, cancelar (admin), reportes, notificaciones, mi cuenta	e55dd89	21-sep-2026	Brandonsk8	PR #7	UC-22 a UC-24, UC-25, UC-26
Frontend	Editor de horario semanal de barbero	d085527	24-oct-2026	OkKid-A	PR #13	UC-22, UC-23
Frontend	Días no disponibles del barbero	4a82505	24-oct-2026	OkKid-A	PR #13	UC-07 (agenda)

Repo	Elemento implementado	Commit	Fecha	Autor	Pull request	Casos de uso / requisitos
Frontend	Se retira la opción de nuevo cliente en Administración > Clientes	fda4c8a	24-oct-2026	OkKid-A	PR #11	UC-08
Frontend	Se quita el botón Insumos duplicado en Servicios (hotfix)	fbf520b	25-oct-2026	OkKid-A	PR #16	UC-13
Frontend	Integración continua del frontend: lint y build (ci.yml)	11c763a	25-oct-2026	Brandonsk8	Sin PR (directo a main)	Infraestructura y Despliegue

La numeración de los pull requests es independiente en cada repositorio. En «Pruebas automáticas del backend» se indica el commit de fusión del PR #25.

7.1 Defectos corregidos y requisitos afectados

Defecto	Descripción	Requisitos	Casos de uso	Historias	Endpoints	Prueba de regresión	Corregido en
D-01	El aviso de horario decía «no atiende los domingo» (sin plural)	RF-CIT-02	UC-04 UC-05 UC-25	HU-07 HU-08 HU-11	E-19, E-20, E-25	unit/barberos.service	develop (commit 3e31a52)
D-02	Un barbero desactivado aún podía iniciar sesión	RF-AUT-04	UC-24	HU-06	E-15	int/permisos-y-datos-expuestos	hotfix/acceso-y-datos-expuestos
D-03	Un barbero consultaba la agenda y las citas de otros barberos	RF-CIT-09	UC-07	HU-10	E-22	int/permisos-y-datos-expuestos, unit/citas.service	hotfix/acceso-y-datos-expuestos
D-04	Un barbero marcaba atendida la cita de otro barbero	RF-INV-02	UC-16	HU-20	E-24	int/permisos-y-datos-expuestos, unit/citas.service	hotfix/acceso-y-datos-expuestos
D-05	El catálogo público exponía el correo y el user_id del barbero	RF-CIT-01	UC-04	HU-07	E-08, E-10	int/permisos-y-datos-expuestos	hotfix/acceso-y-datos-expuestos
D-06	Los ids de ruta que no son UUID producían 500	Transversal	—	—	25 endpoints con id en la ruta	int/validacion-de-entrada	hotfix/validacion
D-07	La consulta de GET /api/citas no se validaba (500)	RF-CIT-09	UC-07	HU-10	E-22	int/validacion-de-entrada	hotfix/validacion
D-08	No se podía quitar el mínimo de un insumo (null)	RF-INV-04	UC-14	HU-18	E-35	int/validacion-de-entrada	hotfix/validacion

Anexo A. Pruebas automáticas y requisitos que verifican

Las 202 pruebas de la suite, con su identificador, el archivo donde están (`unit` o `int` = integración) y los requisitos o casos de uso que verifican. Es la vista inversa de las matrices: desde la prueba hasta lo que respalda.

Prueba	Archivo	Qué verifica	Requisitos y casos de uso
PI-001	int/auth	registra un cliente, no expone el hash y deja su perfil de cliente creado	RF-AUT-05, RF-CLI-01, RNF-SEG-01, UC-02
PI-002	int/auth	el mismo correo no se puede registrar dos veces (409) y los datos inválidos dan 400 en español	RF-AUT-05, RNF-USA-03, UC-02
PI-003	int/auth	login correcto entrega token y perfil; /me lo reconoce; sin token o con token falso es 401	RF-AUT-01, RF-AUT-02, UC-01
PI-004	int/auth	contraseña incorrecta y correo inexistente dan el mismo 401	RF-AUT-01, UC-01
PI-005	int/auth	una cuenta desactivada no entra aunque la contraseña sea correcta	RF-AUT-01, RF-AUT-04, UC-01
PI-006	int/auth	con 2FA activo el login no da token: manda un código al correo y solo con ese código se entra	RF-AUT-07
PI-007	int/auth	manda el enlace, permite cambiar la contraseña una vez y la anterior deja de servir	RF-AUT-06, UC-03
PI-008	int/auth	con un correo que no existe responde igual y no manda ningún correo	RF-AUT-06, UC-03
PI-009	int/auth	un cliente no puede usar endpoints de administración	RF-AUT-03, RNF-SEG-02
PI-010	int/auth	sin sesión esas mismas rutas responden 401	RF-AUT-02, RF-AUT-03, RNF-SEG-02
PI-011	int/auth	un barbero tampoco administra inventario ni reportes	RF-AUT-03, RNF-SEG-02
PI-012	int/barberos-servicios	el administrador crea un servicio y queda visible en el catálogo público	RF-SER-01, RF-SER-03, UC-12
PI-013	int/barberos-servicios	valida los datos con mensajes en español: obligatorios, duración entera y precio no negativo	RF-SER-01, RNF-USA-03, UC-12
PI-014	int/barberos-servicios	editar cambia solo lo enviado; el administrador inexistente da 404	RF-SER-01, UC-12
PI-015	int/barberos-servicios	cliente y barbero no pueden crear ni editar	RF-SER-01, RNF-SEG-02
PI-016	int/barberos-servicios	crea un barbero con su cuenta y, sin horario, le asigna el de la barbería (domingo a jueves, 9:00-19:00)	RF-AUT-04, RF-CIT-02, UC-22
PI-017	int/barberos-servicios	con horario propio lo guarda tal cual y también puede reemplazarlo al editar	RF-AUT-04, UC-22, UC-23
PI-018	int/barberos-servicios	rechaza un correo repetido (409), un horario inválido y datos incompletos	RF-AUT-04, UC-22, RNF-USA-03
PI-019	int/barberos-servicios	desactivar saca al barbero del listado público pero el administrador aún lo ve	RF-AUT-04, UC-24
PI-020	int/barberos-servicios	un barbero desactivado ya no se puede reservar (404)	RF-AUT-04, UC-24, RF-CIT-01
PI-021	int/barberos-servicios	inexistente da 404 y solo el administrador gestiona barberos	RF-AUT-04, RF-AUT-03, RNF-SEG-02
PI-022	int/barberos-servicios	consulta de un día: trae las citas de ese barbero ordenadas por hora, con cliente y servicio	RF-CIT-09, UC-07
PI-023	int/barberos-servicios	listado por rango (semanal): incluye varios días y se puede filtrar por barbero	RF-CIT-09, UC-07

Prueba	Archivo	Qué verifica	Requisitos y casos de uso
PI-024	int/barberos-servicios	una cita cancelada sigue en la agenda pero marcada como cancelada	RF-CIT-09, UC-07
PI-025	int/barberos-servicios	sin parámetros suficientes responde 400 y un cliente no puede ver la agenda (403)	RF-CIT-09, UC-07
PI-026	int/citas	crea la cita confirmada, calcula la hora de fin y avisa por correo al cliente y al barbero	RF-CIT-01, RF-CIT-08, UC-04, UC-26
PI-027	int/citas	el mismo horario ya no se puede reservar (409), pero uno que no se traslapa sí	RF-CIT-07
PI-028	int/citas	otro barbero sí está libre a la misma hora	RF-CIT-07
PI-029	int/citas	rechaza fechas pasadas, horas fuera de horario y el receso del almuerzo	RF-CIT-03, RF-CIT-04
PI-030	int/citas	valida el cuerpo: formato de hora, uuid y campos de más	RF-CIT-01, RNF-USA-03
PI-031	int/citas	servicio o barbero inexistentes dan 404	RF-CIT-01
PI-032	int/citas	solo el cliente agenda: barbero y administrador no pueden usar este endpoint	RF-CIT-01, RF-AUT-03, RNF-SEG-02
PI-033	int/citas	un día marcado como ausente no se puede reservar y la disponibilidad lo refleja	Agenda del barbero
PI-034	int/citas	un barbero no puede gestionar la agenda de otro, pero el administrador sí	Agenda del barbero, RNF-SEG-02
PI-035	int/citas	no deja marcar dos veces el mismo día (409)	Agenda del barbero
PI-036	int/citas	otro cliente no puede cancelar ni reprogramar una cita ajena (403)	RF-CIT-05, RF-CIT-06, RNF-SEG-02
PI-037	int/citas	el dueño reprograma a otra hora libre y la cita conserva su estado	RF-CIT-05, UC-05
PI-038	int/citas	cancelar libera el horario para otro cliente y es idempotente	RF-CIT-06, RF-CIT-07, UC-06
PI-039	int/citas	"mis citas" solo trae las del cliente que consulta	RF-CLI-02
PI-040	int/citas	descuenta la cantidad configurada, deja el movimiento ligado a la cita y no vuelve a descontar	RF-INV-02, UC-16
PI-041	int/citas	también descuenta cantidades en ml o decimales: 30 ml de un stock de 500 dejan 470	RF-INV-02, UC-16
PI-042	int/citas	un servicio con varios insumos los descuenta todos	RF-INV-02, UC-16
PI-043	int/citas	si el stock queda por debajo del mínimo, avisa por correo a los administradores (después de confirmar)	RF-INV-04, UC-17
PI-044	int/citas	sin quedar por debajo del mínimo no hay alerta	RF-INV-04, UC-17
PI-045	int/citas	es una transacción: si el descuento falla la cita sigue confirmada y el stock intacto	RF-INV-02, UC-16
PI-046	int/citas	una cita cancelada no se puede marcar como atendida y no descuenta nada	RF-INV-02, RF-CIT-06
PI-047	int/citas	un cliente no puede marcar citas como atendidas	RF-AUT-03, RNF-SEG-02
PI-048	int/citas	una cita atendida ya no se puede cancelar	RF-CIT-06
PI-049	int/citas	el barbero registra una cita sin cuenta y con nombre; el cliente no puede	UC-25
PI-050	int/clientes	solo el administrador entra: cliente y barbero reciben 403, sin sesión 401	RF-AUT-03, RNF-SEG-02

Prueba	Archivo	Qué verifica	Requisitos y casos de uso
PI-051	int/clientes	el administrador registra un cliente sin cuenta con nombre, teléfono y correo opcionales	RF-CLI-01, UC-08
PI-052	int/clientes	valida los datos: nombre obligatorio, correo con formato y campos desconocidos	RF-CLI-01, RNF-USA-03, UC-08
PI-053	int/clientes	editar cambia solo los campos enviados y conserva el resto	RF-CLI-04, UC-11
PI-054	int/clientes	un cliente inexistente da 404 al consultar, editar, activar o desactivar	RF-CLI-04, UC-11
PI-055	int/clientes	busca por nombre o por teléfono sin distinguir mayúsculas	RF-CLI-03, UC-09
PI-056	int/clientes	por defecto lista solo activos; "inactivos" y "todos" permiten encontrar a los desactivados	RF-CLI-03, RF-CLI-04, UC-09
PI-057	int/clientes	desactivar y volver a activar cambia el estado y conserva los datos	RF-CLI-04, UC-11
PI-058	int/clientes	muestra las citas del cliente con servicio, barbero y estado, la más reciente primero	RF-CLI-02, UC-10
PI-059	int/clientes	un cliente sin citas tiene historial vacío y uno inexistente da 404	RF-CLI-02, UC-10
PI-060	int/endpoints-restantes	responde 200 con el estado del servicio, sin sesión	RNF-DIS-01
PI-061	int/endpoints-restantes	activar el 2FA exige sesión, el campo `enabled` y la contraseña actual	RF-AUT-07, UC-01
PI-062	int/endpoints-restantes	con la contraseña correcta activa el 2FA y el siguiente inicio de sesión pide código; se puede desactivar	RF-AUT-07, UC-01
PI-063	int/endpoints-restantes	devuelve el barbero con su horario semanal	RF-CIT-01
PI-064	int/endpoints-restantes	el barbero lista sus ausencias y el administrador también; otro barbero recibe 403	Agenda del barbero
PI-065	int/endpoints-restantes	quitar una ausencia responde 204, vuelve a habilitar el día y repetirlo da 404	Agenda del barbero
PI-066	int/endpoints-restantes	el administrador lista los insumos y encuentra el que acaba de crear	RF-INV-01, UC-14
PI-067	int/endpoints-restantes	editar cambia solo lo enviado y un insumo inexistente da 404	RF-INV-01, RF-INV-04
PI-068	int/endpoints-restantes	quita el insumo del servicio (204), no toca el insumo y repetirlo da 404; solo el administrador	RF-SER-02, UC-13
PI-069	int/inventario-reportes	crea un insumo en cero; el stock entra después con una entrada	RF-INV-01, RF-INV-03, UC-14
PI-070	int/inventario-reportes	el stock no se puede fijar directamente al crear: solo entradas y consumos lo mueven	RF-INV-01
PI-071	int/inventario-reportes	rechaza entradas en cero o negativas y mínimos negativos, sin tocar el stock	RF-INV-03, RF-INV-04
PI-072	int/inventario-reportes	una entrada suma la cantidad (también decimales) y queda en el historial de movimientos	RF-INV-03, UC-15
PI-073	int/inventario-reportes	un insumo inexistente da 404	RF-INV-03
PI-074	int/inventario-reportes	el administrador asocia un insumo con una cantidad positiva y no con cero ni negativa	RF-SER-02, UC-13
PI-075	int/inventario-reportes	asociar otra vez el mismo insumo actualiza la cantidad en vez de duplicar la fila	RF-SER-02, UC-13
PI-076	int/inventario-reportes	solo el administrador lo configura	RF-SER-02, RNF-SEG-02
PI-077	int/inventario-reportes	es público para leer; crear, editar y desactivar exige administrador	RF-SER-01, RF-SER-03, RNF-SEG-02

Prueba	Archivo	Qué verifica	Requisitos y casos de uso
PI-078	int/inventario-reportes	un servicio desactivado desaparece del catálogo público	RF-SER-01, RF-SER-03, UC-12
PI-079	int/inventario-reportes	cada tipo se exporta en Excel y PDF con el tipo de contenido y el nombre de archivo correctos	RF-REP-04, UC-18
PI-080	int/inventario-reportes	el reporte de citas refleja las citas reales del periodo y el de insumos, la cantidad consumida	RF-REP-01, RF-REP-02, RF-REP-03
PI-081	int/inventario-reportes	un periodo sin datos igual descarga el archivo	RF-REP-04
PI-082	int/inventario-reportes	valida los parámetros: rango invertido, tipo y formato desconocidos	RF-REP-01, RF-REP-02, RF-REP-03, RNF-USA-03
PI-083	int/notificaciones	exige sesión	RF-CIT-08, UC-26, RNF-SEG-02
PI-084	int/notificaciones	agendar deja una notificación sin leer para el cliente y otra para el barbero	RF-CIT-08, UC-26
PI-085	int/notificaciones	cada usuario solo ve las suyas	RF-CIT-08, UC-26
PI-086	int/notificaciones	marcar como leída cambia el estado y no afecta a las demás	RF-CIT-08, UC-26
PI-087	int/notificaciones	no se puede marcar una notificación ajena (403) ni una inexistente (404)	RF-CIT-08, UC-26
PI-088	int/permisos-y-datos-expuestos	un barbero desactivado por el administrador ya no puede iniciar sesión	RF-AUT-04, UC-24, RNF-SEG-02
PI-089	int/permisos-y-datos-expuestos	un barbero consulta su propia agenda, del día y por rango	RF-CIT-09, UC-07
PI-090	int/permisos-y-datos-expuestos	un barbero no puede consultar la agenda ni las citas de otro barbero	RF-CIT-09, UC-07, RNF-SEG-02
PI-091	int/permisos-y-datos-expuestos	el administrador sigue viendo la agenda de cualquier barbero y el listado global	RF-CIT-09, UC-07
PI-092	int/permisos-y-datos-expuestos	un barbero no puede marcar como atendida la cita de otro barbero	RF-INV-02, UC-16, RNF-SEG-02
PI-093	int/permisos-y-datos-expuestos	un barbero sí marca como atendida su propia cita	RF-INV-02, UC-16
PI-094	int/permisos-y-datos-expuestos	no expone correos ni identificadores de cuenta	RF-CIT-01, RNF-SEG-02
PI-095	int/permisos-y-datos-expuestos	el administrador sigue viendo el correo de cada barbero en su listado	RF-AUT-04, UC-22
PI-096	int/validacion-de-entrada	un id que no es UUID responde 400 con mensaje en español, en todos los módulos	Transversal, RNF-USA-03
PI-097	int/validacion-de-entrada	un UUID válido que no existe sigue dando 404	Transversal, RNF-USA-03
PI-098	int/validacion-de-entrada	las rutas fijas que comparten prefijo con :id no se confunden con un id	Transversal, RNF-USA-03
PI-099	int/validacion-de-entrada	fechas o ids inválidos responden 400 con mensaje en español	RF-CIT-09, UC-07, RNF-USA-03
PI-100	int/validacion-de-entrada	una consulta válida sigue funcionando y sin parámetros pide cuáles usar	RF-CIT-09, UC-07, RNF-USA-03
PI-101	int/validacion-de-entrada	cantidad_minima null quita el mínimo; omitirla lo conserva; un número lo cambia	RF-INV-01, RF-INV-04, UC-14
PU-001	unit/auth.service	crea una cuenta con rol cliente y guarda la contraseña con hash, no en claro	RF-AUT-05, RNF-SEG-01
PU-002	unit/auth.service	un correo ya registrado devuelve 409	RF-AUT-05, UC-02
PU-003	unit/auth.service	no distingue entre correo inexistente y contraseña incorrecta (mismo 401 y mismo mensaje)	RF-AUT-01, UC-01

Prueba	Archivo	Qué verifica	Requisitos y casos de uso
PU-004	unit/auth.service	una cuenta desactivada no puede entrar	RF-AUT-01, RF-AUT-04
PU-005	unit/auth.service	sin 2FA devuelve un JWT firmado con id, correo y rol	RF-AUT-01, RF-AUT-02, RNF-SEG-04
PU-006	unit/auth.service	con 2FA no entrega token: guarda el código con hash y lo manda por correo	RF-AUT-07
PU-007	unit/auth.service	un barbero recibe además su barberoId en el perfil (distinto del id de usuario)	RF-AUT-03
PU-008	unit/auth.service	un código incorrecto o vencido devuelve 401	RF-AUT-07
PU-009	unit/auth.service	un código válido entrega el token y queda marcado como usado	RF-AUT-07
PU-010	unit/auth.service	activar o desactivar el 2FA exige la contraseña actual	RF-AUT-07
PU-011	unit/auth.service	responde igual exista o no el correo y, si no existe, no manda nada	RF-AUT-06, UC-03
PU-012	unit/auth.service	el enlace lleva un token cuyo hash es lo único que se guarda	RF-AUT-06, UC-03
PU-013	unit/auth.service	un token inválido devuelve 400; uno válido cambia la contraseña y se marca como usado	RF-AUT-06, UC-03
PU-014	unit/barberos.service	sin horario usa el de la barbería: domingo a jueves, 9:00-19:00	RF-AUT-04, RF-CIT-02, UC-22
PU-015	unit/barberos.service	rechaza un bloque cuyo fin no es posterior al inicio, indicando cuál	RF-AUT-04, UC-22
PU-016	unit/barberos.service	rechaza bloques traslapados del mismo día, pero permite mañana y tarde seguidos	RF-AUT-04, UC-22
PU-017	unit/barberos.service	un correo ya registrado devuelve 409	RF-AUT-04, UC-22
PU-018	unit/barberos.service	con horario lo valida y lo manda al repositorio; sin horario no lo toca	RF-AUT-04, UC-23
PU-019	unit/barberos.service	un horario inválido no llega a guardarse	RF-AUT-04, UC-23
PU-020	unit/barberos.service	devuelve 404 si el barbero no existe	RF-AUT-04, UC-23
PU-021	unit/barberos.service	acepta una cita dentro del bloque, incluso pegada a los extremos	RF-CIT-04
PU-022	unit/barberos.service	rechaza una cita que empieza antes o termina después del bloque	RF-CIT-04
PU-023	unit/barberos.service	rechaza un día en que el barbero no atiende y nombra el día	RF-CIT-02, RF-CIT-04
PU-024	unit/barberos.service	rechaza un día marcado como ausencia, aunque el horario lo permita	Agenda del barbero, RF-CIT-01
PU-025	unit/barberos.service	con dos bloques, no deja agendar en el hueco entre ambos	RF-CIT-03
PU-026	unit/barberos.service	rechaza una cita que cruza la medianoche	RF-CIT-04
PU-027	unit/barberos.service	solo el administrador o el propio barbero pueden gestionar la agenda	Agenda del barbero
PU-028	unit/barberos.service	no deja marcar un día que ya pasó	Agenda del barbero
PU-029	unit/barberos.service	no cancela citas por detrás: con citas ese día responde 409 con la cantidad	Agenda del barbero
PU-030	unit/barberos.service	un día ya marcado devuelve 409	Agenda del barbero

Prueba	Archivo	Qué verifica	Requisitos y casos de uso
PU-031	unit/barberos.service	quitar una ausencia que no existe devuelve 404	Agenda del barbero
PU-032	unit/barberos.service	un día de ausencia no ofrece horas: trabaja=false, ausente=true, sin bloques	RF-CIT-01, RF-CIT-02, Agenda del barbero
PU-033	unit/barberos.service	devuelve los bloques del día y las horas ocupadas por citas, ordenadas	RF-CIT-01, RF-CIT-02, Agenda del barbero
PU-034	unit/citas.service	crea la cita confirmada con la hora de fin según la duración del servicio	RF-CIT-01
PU-035	unit/citas.service	rechaza una fecha pasada sin consultar nada más	RF-CIT-01
PU-036	unit/citas.service	un servicio o un barbero inactivo o inexistente devuelve 404	RF-CIT-01
PU-037	unit/citas.service	respeta el horario y las ausencias del barbero: si assertAtiende falla, no se crea nada	RF-CIT-01
PU-038	unit/citas.service	evita la doble reserva: horario solapado devuelve 409	RF-CIT-07
PU-039	unit/citas.service	un usuario sin perfil de cliente no puede agendar	RF-CIT-01
PU-040	unit/citas.service	avisa al cliente y también al barbero asignado	RF-CIT-08, UC-26
PU-041	unit/citas.service	el barbero se entera aunque el cliente no tenga correo	RF-CIT-08, UC-26
PU-042	unit/citas.service	marca la cita y descuenta dentro de la misma transacción, y avisa solo después del COMMIT	RF-INV-02, UC-16
PU-043	unit/citas.service	si el descuento falla se revierte todo, se libera la conexión y no se avisa nada	RF-INV-02, UC-16
PU-044	unit/citas.service	una cita ya atendida no vuelve a descontar (ni abre transacción)	RF-INV-02, UC-16
PU-045	unit/citas.service	una cita cancelada no se puede marcar como atendida; una inexistente da 404	RF-INV-02, UC-16
PU-046	unit/citas.service	un barbero solo marca como atendidas sus propias citas	RF-INV-02, UC-16, RNF-SEG-02
PU-047	unit/citas.service	un barbero consulta su propia agenda, del día o por rango	RF-CIT-09, UC-07, RNF-SEG-02
PU-048	unit/citas.service	un barbero no puede ver la agenda de otro ni el listado de todos	RF-CIT-09, UC-07, RNF-SEG-02
PU-049	unit/citas.service	el administrador consulta la agenda de cualquiera y el listado global	RF-CIT-09, UC-07, RNF-SEG-02
PU-050	unit/citas.service	un cliente no puede cancelar la cita de otro cliente	RF-CIT-06, RNF-SEG-02
PU-051	unit/citas.service	una cita atendida no se puede cancelar; el administrador sí puede cancelar una confirmada	RF-CIT-06, RNF-SEG-02
PU-052	unit/emailTemplate	dos_factor usa "Barber Kongs" y nunca la marca vieja	RF-AUT-07
PU-053	unit/emailTemplate	recuperacion_password usa "Barber Kongs" y nunca la marca vieja	RF-AUT-06
PU-054	unit/emailTemplate	confirmacion_cita usa "Barber Kongs" y nunca la marca vieja	RF-CIT-08
PU-055	unit/emailTemplate	reprogramacion_cita usa "Barber Kongs" y nunca la marca vieja	RF-CIT-08
PU-056	unit/emailTemplate	cancelacion_cita usa "Barber Kongs" y nunca la marca vieja	RF-CIT-08
PU-057	unit/emailTemplate	alerta_inventario usa "Barber Kongs" y nunca la marca vieja	RF-INV-04

Prueba	Archivo	Qué verifica	Requisitos y casos de uso
PU-058	unit/emailTemplate	muestra el código de 2FA y su vencimiento	RF-AUT-07
PU-059	unit/emailTemplate	arma el detalle de la cita con fecha larga y hora de 12 horas	RF-CIT-08
PU-060	unit/emailTemplate	el aviso al barbero muestra el cliente y no repite al barbero; los textos se pueden reemplazar	RF-CIT-08
PU-061	unit/emailTemplate	sin `datos` igual sale presentable con el mensaje	RF-CIT-08
PU-062	unit/emailTemplate	escapa el HTML de los valores dinámicos	Transversal
PU-063	unit/emailTemplate	no pone como botón un enlace que no sea http(s)	Transversal
PU-064	unit/emailTemplate	formatFecha no depende de la zona horaria (fecha solo-día)	RF-CIT-08
PU-065	unit/emailTemplate	formatHora pasa a 12 horas con a. m./p. m.	RF-CIT-08
PU-066	unit/emailTemplate	devuelve el valor tal cual si no se puede interpretar	RF-CIT-08
PU-067	unit/inventario.service	descuenta cada insumo del servicio con la cantidad configurada y ligado a la cita	RF-INV-02, RF-INV-04, UC-16
PU-068	unit/inventario.service	un servicio sin insumos asociados no descuenta nada ni genera alertas	RF-INV-02, RF-INV-04, UC-16
PU-069	unit/inventario.service	devuelve alerta solo de los insumos que quedaron por debajo del mínimo	RF-INV-02, RF-INV-04, UC-16
PU-070	unit/inventario.service	un insumo sin mínimo configurado nunca genera alerta, aunque llegue a cero o negativo	RF-INV-02, RF-INV-04, UC-16
PU-071	unit/inventario.service	no hace nada si no hay alertas (ni siquiera consulta a los administradores)	RF-INV-04, UC-17
PU-072	unit/inventario.service	avisa a cada administrador activo por cada insumo con stock bajo	RF-INV-04, UC-17
PU-073	unit/reportes.formato	reporte de citas: título con la marca, resumen, encabezados y datos reales ordenables	RF-REP-01, RF-REP-04
PU-074	unit/reportes.formato	reporte de ingresos: una hoja por servicio y otra por barbero, con totales numéricos	RF-REP-02, RF-REP-04
PU-075	unit/reportes.formato	reporte de insumos: cantidad consumida como número real	RF-REP-03, RF-REP-04
PU-076	unit/reportes.formato	un periodo sin datos muestra el aviso en vez de una tabla vacía	RF-REP-04
PU-077	unit/reportes.formato	genera un PDF válido con la marca en los metadatos	RF-REP-04
PU-078	unit/reportes.formato	con muchas filas pagina en vez de cortar la tabla	RF-REP-04
PU-079	unit/reportes.formato	un periodo sin datos igual produce un PDF válido	RF-REP-04
PU-080	unit/utilidades	cada atajo trae su código HTTP	Transversal
PU-081	unit/utilidades	muestra al usuario el mensaje de los errores 4xx	RNF-USA-03
PU-082	unit/utilidades	en un 500 nunca filtra el detalle interno	Transversal
PU-083	unit/utilidades	sin encabezado, con esquema distinto de Bearer o con token basura responde 401	RF-AUT-02
PU-084	unit/utilidades	rechaza un token firmado con otro secreto o ya vencido	RF-AUT-02, RNF-SEG-04

Prueba	Archivo	Qué verifica	Requisitos y casos de uso
PU-085	unit/utilidades	con un token válido deja pasar y expone el usuario en req.user	RF-AUT-02
PU-086	unit/utilidades	requireRole: permite los roles de la lista, da 403 a otros y 401 si no hay sesión	RF-AUT-03, RNF-SEG-02
PU-087	unit/validaciones	devuelve los datos válidos y lanza ApiError 400 con el mensaje si no lo son	Transversal
PU-088	unit/validaciones	rechaza campos que el schema no conoce (additionalProperties: false)	Transversal
PU-089	unit/validaciones	las entradas son números mayores que cero (admiten decimales, p. ej. ml)	RF-INV-03
PU-090	unit/validaciones	el mínimo de un insumo es >= 0 (o null / ausente para no tener alerta)	RF-INV-04
PU-091	unit/validaciones	un insumo exige nombre y unidad de medida, y no acepta el stock al crearlo	RF-INV-01
PU-092	unit/validaciones	un servicio consume una cantidad mayor que cero de cada insumo	RF-SER-02
PU-093	unit/validaciones	acepta un horario válido y también que no venga (se usa el de la barbería)	RF-AUT-04, UC-22, UC-23
PU-094	unit/validaciones	rechaza hora con formato inválido con un mensaje que señala el campo	RF-AUT-04, UC-22, UC-23
PU-095	unit/validaciones	rechaza hora fuera de rango con un mensaje que señala el campo	RF-AUT-04, UC-22, UC-23
PU-096	unit/validaciones	rechaza día fuera de 0-6 con un mensaje que señala el campo	RF-AUT-04, UC-22, UC-23
PU-097	unit/validaciones	rechaza día no entero con un mensaje que señala el campo	RF-AUT-04, UC-22, UC-23
PU-098	unit/validaciones	rechaza un bloque incompleto y un horario vacío	RF-AUT-04, UC-22, UC-23
PU-099	unit/validaciones	una ausencia exige una fecha AAAA-MM-DD	Agenda del barbero
PU-100	unit/validaciones	acepta cualquier combinación de tipo y formato válidos	RF-REP-01, RF-REP-02, RF-REP-03
PU-101	unit/validaciones	rechaza tipo o formato desconocidos y fechas mal escritas	RF-REP-01, RF-REP-02, RF-REP-03

Casos de Uso e Historias de Usuario

Campo	Detalle
Proyecto	Barber Kongs — Sistema de Gestión para Barbería (Fase 3)
Equipo	Miguel Angel Quemé Gómez · Brandon Gustavo Güinac Román
Versión del documento	2.0 — 25 de octubre de 2026
Contenido	23 casos de uso y 23 historias de usuario, organizados en 7 épicas y 3 sprints

1. Actores y alcance

1.1 Actores

Actor	Descripción
Cliente	Persona que agenda, reprograma o cancela sus propias citas y consulta su historial de servicios.
Barbero	Presta los servicios, consulta su agenda diaria y semanal y marca las citas como atendidas.
Administrador	Gestiona clientes, servicios, inventario, barberos y reportes; también puede cancelar citas.
Sistema	Actor secundario que ejecuta automáticamente el descuento de inventario y la generación de alertas de stock bajo, sin intervención directa de un usuario.

1.2 Matriz actor – caso de uso

Caso de uso	Cliente	Barbero	Administrador	Sistema
UC-01 — Iniciar sesión	X	X	X	
UC-02 — Registrarse	X			
UC-03 — Recuperar contraseña	X	X	X	
UC-04 — Agendar cita	X			
UC-05 — Reprogramar cita	X			
UC-06 — Cancelar cita	X		X	
UC-07 — Consultar agenda		X		
UC-08 — Registrar cliente			X	
UC-09 — Buscar y filtrar clientes			X	
UC-10 — Consultar historial de cliente			X	
UC-11 — Editar o desactivar cliente			X	
UC-12 — Crear, editar o desactivar servicio			X	
UC-13 — Asociar insumos a servicio			X	
UC-14 — Registrar insumo			X	
UC-15 — Registrar reabastecimiento de insumo			X	
UC-16 — Descontar insumos automáticamente		X		X
UC-17 — Generar alerta de inventario bajo			X	X
UC-18 — Generar y exportar reporte			X	
UC-22 — Crear cuenta de barbero			X	

Caso de uso	Cliente	Barbero	Administrador	Sistema
UC-23 — Editar cuenta de barbero			X	
UC-24 — Desactivar cuenta de barbero			X	
UC-25 — Registrar cliente sin reservación (walk-in)		X	X	
UC-26 — Consultar notificaciones	X	X	X	

El barbero participa en UC-16 como actor secundario (marca la cita como atendida) y el administrador en UC-17 (recibe la alerta). La numeración conserva los identificadores de la Fase 1; los números UC-19 a UC-21 no se utilizan.

1.3 Requisitos añadidos al catálogo

Dos casos de uso agregados durante el desarrollo se apoyan en requisitos que se incorporan al catálogo de requisitos funcionales:

Requisito	Descripción	Caso de uso
RF-CIT-10	El sistema debe permitir registrar la atención inmediata de un cliente que llega sin cita previa (walk-in).	UC-25
RF-NOT-01	El sistema debe permitir a cualquier usuario autenticado consultar las notificaciones que se le enviaron y marcarlas como leídas.	UC-26

2. Especificación de casos de uso

Los casos de uso se agrupan por épica, que corresponde a cada módulo del sistema.

EP-01 — Autenticación y Cuentas

UC-01 — Iniciar sesión

Identificador	**UC-01** · HU-01 (BK-15) · EP-01
Nombre	Iniciar sesión
Actor(es)	Cliente, Barbero, Administrador
Requisitos relacionados	RF-AUT-01, RF-AUT-02, RF-AUT-07
Descripción	Permite a un usuario registrado autenticarse en el sistema mediante correo y contraseña. Si la cuenta tiene habilitada la autenticación de dos factores (2FA) — opcional, configurable por cuenta — el sistema exige además un código de verificación por correo antes de establecer la sesión.
Precondiciones	El usuario cuenta con una cuenta registrada en el sistema (y, si tiene 2FA habilitado, acceso a su correo electrónico).
Flujo principal	1. El usuario accede a la pantalla de inicio de sesión. 2. El usuario ingresa su correo electrónico y contraseña. 3. El sistema valida las credenciales contra la base de datos. 4. El sistema verifica si la cuenta tiene habilitada la autenticación de dos factores (2FA). 5. (Solo si 2FA está habilitado) El sistema genera un código de verificación de un solo uso y lo envía al correo del usuario (RF-AUT-07). 6. (Solo si 2FA está habilitado) El usuario recibe e ingresa el código de verificación. 7. (Solo si 2FA está habilitado) El sistema valida el código de verificación. 8. El sistema emite un token JWT y establece la sesión. 9. El sistema redirige al usuario al panel correspondiente a su rol.
Flujos alternativos	• 3a. Credenciales inválidas: el sistema muestra un mensaje de error genérico y permite reintentar. • 4a. La cuenta no tiene 2FA habilitado: el sistema omite los pasos 5–7 y continúa directamente a emitir el token (paso 8). • 7a. Código de verificación inválido o expirado: el sistema muestra un error y permite reingresar el código o solicitar uno nuevo.
Postcondiciones	El usuario queda autenticado con una sesión activa asociada a su rol, tras superar la contraseña y, si aplica, el segundo factor de verificación.

UC-02 — Registrarse

Identificador	**UC-02** · HU-02 (BK-16) · EP-01
Nombre	Registrarse
Actor(es)	Cliente
Requisitos relacionados	RF-AUT-05
Descripción	Permite a un nuevo cliente crear una cuenta de forma autónoma.
Precondiciones	El correo electrónico ingresado no debe estar previamente registrado.
Flujo principal	1. El cliente accede a la pantalla de registro. 2. El cliente ingresa nombre, teléfono, correo y contraseña. 3. El sistema valida el formato de los datos y que el correo no esté en uso. 4. El sistema crea la cuenta con el rol Cliente. 5. El sistema confirma el registro y permite iniciar sesión.
Flujos alternativos	• 3a. Correo ya registrado: el sistema muestra un error y solicita otro correo.
Postcondiciones	Se crea una nueva cuenta de cliente en el sistema; a partir de ese momento el cliente puede agendar citas (UC-04).

UC-03 — Recuperar contraseña

Identificador	**UC-03** · HU-03 (BK-17) · EP-01
Nombre	Recuperar contraseña
Actor(es)	Cliente, Barbero, Administrador
Requisitos relacionados	RF-AUT-06
Descripción	Permite a un usuario restablecer su contraseña cuando la ha olvidado.
Precondiciones	El usuario cuenta con una cuenta registrada con correo válido.
Flujo principal	1. El usuario solicita la recuperación desde la pantalla de inicio de sesión. 2. El usuario ingresa su correo electrónico. 3. El sistema envía un enlace o código de recuperación al correo. 4. El usuario define una nueva contraseña a través del enlace. 5. El sistema actualiza la contraseña utilizando hash seguro.
Flujos alternativos	• 2a. Correo no registrado: el sistema muestra un mensaje genérico, sin revelar si el correo existe.
Postcondiciones	La contraseña del usuario queda actualizada.

UC-22 — Crear cuenta de barbero

Identificador	**UC-22** · HU-04 · EP-01
Nombre	Crear cuenta de barbero
Actor(es)	Administrador
Requisitos relacionados	RF-AUT-04
Descripción	Permite al administrador dar de alta la cuenta de un nuevo barbero.
Precondiciones	El administrador ha iniciado sesión.
Flujo principal	1. El administrador accede a la sección de barberos. 2. El administrador ingresa los datos del barbero y su horario de disponibilidad. 3. El sistema crea la cuenta con el rol Barbero.
Flujos alternativos	• 2a. Correo ya registrado: el sistema muestra un error y solicita otro correo.
Postcondiciones	Se crea una nueva cuenta de barbero en el sistema; a partir de ese momento puede ser seleccionado por los clientes al agendar una cita (UC-04).

UC-23 — Editar cuenta de barbero

Identificador	**UC-23** · HU-05 · EP-01
Nombre	Editar cuenta de barbero
Actor(es)	Administrador
Requisitos relacionados	RF-AUT-04
Descripción	Permite al administrador modificar los datos o el horario de disponibilidad de un barbero existente.
Precondiciones	El administrador ha iniciado sesión y la cuenta del barbero existe.
Flujo principal	1. El administrador selecciona la cuenta del barbero. 2. El administrador edita los datos o el horario de disponibilidad. 3. El sistema guarda los cambios.
Flujos alternativos	• 2a. Datos inválidos: el sistema impide guardar y señala los campos con error.
Postcondiciones	Los datos del barbero quedan actualizados.

UC-24 — Desactivar cuenta de barbero

Identificador	**UC-24** · HU-06 (BK-20) · EP-01
Nombre	Desactivar cuenta de barbero
Actor(es)	Administrador
Requisitos relacionados	RF-AUT-04
Descripción	Permite al administrador desactivar la cuenta de un barbero que deja de laborar.
Precondiciones	El administrador ha iniciado sesión y la cuenta del barbero existe.
Flujo principal	1. El administrador selecciona la cuenta del barbero. 2. El administrador selecciona la opción de desactivar. 3. El sistema marca la cuenta como inactiva y la excluye de la asignación de nuevas citas.
Flujos alternativos	• 2a. El administrador cancela la operación: la cuenta permanece activa.
Postcondiciones	La lista de barberos activos queda actualizada y la cuenta del barbero ya no puede iniciar sesión.

EP-02 — Gestión de Citas

UC-04 — Agendar cita

Identificador	**UC-04** · HU-07 (BK-29) · EP-02
Nombre	Agendar cita
Actor(es)	Cliente
Requisitos relacionados	RF-CIT-01, RF-CIT-02, RF-CIT-03, RF-CIT-04, RF-CIT-07, RF-CIT-08, RF-SER-03
Descripción	Permite al cliente reservar una cita seleccionando servicio, barbero, fecha y hora disponibles.
Precondiciones	El cliente ha iniciado sesión.
Flujo principal	1. El cliente selecciona un servicio del catálogo activo. 2. El cliente selecciona un barbero disponible. 3. El sistema muestra los horarios disponibles (domingo a jueves, 9:00–19:00, excluyendo 13:00–14:00). 4. El cliente selecciona fecha y hora. 5. El sistema valida que el barbero no tenga otra cita en ese horario. 6. El sistema registra la cita y notifica la confirmación al cliente y al barbero asignado.
Flujos alternativos	• 3a. El cliente intenta ver viernes o sábado: el sistema no los muestra como disponibles. • 5a. Horario ya ocupado: el sistema informa el conflicto y solicita elegir otro horario.
Postcondiciones	Se crea una nueva cita en estado "confirmada"; el barbero la ve reflejada de inmediato en su agenda (UC-07).

UC-05 — Reprogramar cita

Identificador	**UC-05** · HU-08 (BK-30) · EP-02
Nombre	Reprogramar cita
Actor(es)	Cliente
Requisitos relacionados	RF-CIT-05, RF-CIT-08
Descripción	Permite al cliente cambiar la fecha u hora de una cita ya agendada.
Precondiciones	Existe una cita previamente agendada por el cliente.
Flujo principal	1. El cliente selecciona la cita a reprogramar desde su historial. 2. El cliente elige una nueva fecha/hora dentro del horario permitido. 3. El sistema valida la disponibilidad del barbero para el nuevo horario. 4. El sistema actualiza la cita y notifica el cambio al cliente.
Flujos alternativos	• 3a. Horario no disponible: el sistema solicita elegir otro horario.
Postcondiciones	La cita queda actualizada con la nueva fecha/hora; el cambio se refleja en la agenda del barbero (UC-07).

UC-06 — Cancelar cita

Identificador	**UC-06** · HU-09 (BK-31) · EP-02
Nombre	Cancelar cita
Actor(es)	Cliente, Administrador
Requisitos relacionados	RF-CIT-06, RF-CIT-08
Descripción	Permite cancelar una cita activa, liberando el horario del barbero.
Precondiciones	Existe una cita activa asociada al cliente.
Flujo principal	1. El actor selecciona la cita a cancelar. 2. El sistema solicita confirmación de la cancelación. 3. El sistema marca la cita como "cancelada" y libera el horario del barbero. 4. El sistema notifica la cancelación al cliente.
Flujos alternativos	• 2a. El actor no confirma: la operación se cancela y la cita permanece sin cambios.
Postcondiciones	La cita queda marcada como cancelada y el horario queda disponible nuevamente para el barbero (visible en UC-07).

UC-07 — Consultar agenda

Identificador	**UC-07** · HU-10 (BK-32) · EP-02
Nombre	Consultar agenda
Actor(es)	Barbero
Requisitos relacionados	RF-CIT-09
Descripción	Permite al barbero visualizar sus citas asignadas por día o semana; es la vista donde se reflejan las citas agendadas, reprogramadas o canceladas (UC-04, UC-05, UC-06). El barbero recibe además un aviso cuando se le agenda una cita (UC-04).
Precondiciones	El barbero ha iniciado sesión.
Flujo principal	1. El barbero accede a su panel de agenda. 2. El sistema muestra las citas del período seleccionado, con cliente y servicio.
Flujos alternativos	• 2a. No hay citas registradas: el sistema muestra un estado vacío.
Postcondiciones	No aplica (consulta de solo lectura).

UC-25 — Registrar cliente sin reservación (walk-in)

Identificador	**UC-25** · HU-11 (BK-33) · EP-02
Nombre	Registrar cliente sin reservación (walk-in)
Actor(es)	Administrador, Barbero
Requisitos relacionados	RF-CIT-10, RF-CIT-07
Descripción	Permite atender de inmediato a un cliente que llega a la barbería sin una cita previamente agendada, siempre que el barbero elegido esté libre en el horario indicado. Si se escribe el nombre del cliente se registra un cliente sin cuenta; si no, la atención queda anónima. Da como resultado una cita "confirmada" igual que si se hubiera agendado con anticipación (UC-04), y de ahí en adelante sigue el mismo flujo de atención (UC-16).
Precondiciones	El cliente se encuentra físicamente en la barbería.
Flujo principal	1. El administrador (o el barbero) abre el registro de walk-in. 2. Selecciona el servicio, el barbero (el administrador) y la hora, y opcionalmente escribe el nombre del cliente. 3. El sistema valida que el barbero atienda en ese horario y no tenga otra cita. 4. El sistema registra una cita en estado "confirmada", marcada como walk-in.
Flujos alternativos	• 3a. El horario no está disponible: el sistema lo informa y se elige otra hora o se agenda una cita normal (UC-04). • 2a. No se escribe el nombre del cliente: la cita walk-in se registra de forma anónima, sin datos de cliente.
Postcondiciones	Se crea una cita walk-in confirmada, con o sin cliente asociado; el barbero la atiende de inmediato y la marca como atendida al finalizar (UC-16), igual que una cita agendada con anticipación.

EP-03 — Gestión de Clientes

UC-08 — Registrar cliente

Identificador	**UC-08** · HU-12 (BK-21) · EP-03
Nombre	Registrar cliente
Actor(es)	Administrador
Requisitos relacionados	RF-CLI-01
Descripción	Permite al administrador registrar los datos básicos de un nuevo cliente que no cuenta con una cuenta propia (por ejemplo, un cliente atendido presencialmente).
Precondiciones	El administrador ha iniciado sesión.
Flujo principal	1. El administrador accede a la sección de clientes. 2. El administrador ingresa nombre, teléfono y correo del cliente. 3. El sistema valida el formato de los datos. 4. El sistema crea el registro del cliente.
Flujos alternativos	• 3a. Datos incompletos o inválidos: el sistema impide guardar y señala los campos con error.
Postcondiciones	Se crea un nuevo registro de cliente en el sistema; a partir de ese momento el administrador puede agendarle citas en su nombre (UC-04).

UC-09 — Buscar y filtrar clientes

Identificador	**UC-09** · HU-13 (BK-22) · EP-03
Nombre	Buscar y filtrar clientes
Actor(es)	Administrador
Requisitos relacionados	RF-CLI-03
Descripción	Permite al administrador localizar clientes específicos dentro del listado general.
Precondiciones	El administrador ha iniciado sesión y existen clientes registrados.
Flujo principal	1. El administrador accede al listado de clientes. 2. El administrador ingresa un criterio de búsqueda (nombre o teléfono). 3. El sistema filtra y muestra los clientes que coinciden.
Flujos alternativos	• 3a. Sin resultados de búsqueda: el sistema informa que no se encontraron coincidencias.
Postcondiciones	Se muestra un listado filtrado de clientes (no aplica cambio de estado).

UC-10 — Consultar historial de cliente

Identificador	**UC-10** · HU-14 (BK-23) · EP-03
Nombre	Consultar historial de cliente
Actor(es)	Administrador
Requisitos relacionados	RF-CLI-02
Descripción	Permite al administrador revisar las citas y servicios recibidos previamente por un cliente; el historial se construye automáticamente a partir de las citas agendadas, reprogramadas, canceladas o atendidas del cliente (UC-04, UC-05, UC-06, UC-16).
Precondiciones	El administrador ha iniciado sesión y el cliente existe en el sistema.
Flujo principal	1. El administrador selecciona un cliente del listado. 2. El sistema muestra el historial de citas y servicios asociados al cliente.
Flujos alternativos	• 2a. El cliente no tiene historial: el sistema muestra un estado vacío.
Postcondiciones	No aplica (consulta de solo lectura).

UC-11 — Editar o desactivar cliente

Identificador	**UC-11** · HU-15 (BK-24) · EP-03
Nombre	Editar o desactivar cliente
Actor(es)	Administrador
Requisitos relacionados	RF-CLI-04
Descripción	Permite al administrador modificar los datos de un cliente o desactivar su registro.
Precondiciones	El administrador ha iniciado sesión y el cliente existe en el sistema.
Flujo principal	1. El administrador selecciona el cliente a modificar. 2. El administrador edita los datos o selecciona la opción de desactivar. 3. El sistema guarda los cambios o marca el registro como inactivo.
Flujos alternativos	• 2a. El administrador cancela la operación: no se guardan cambios.
Postcondiciones	Los datos del cliente quedan actualizados o el cliente queda desactivado.

EP-04 — Gestión de Servicios

UC-12 — Crear, editar o desactivar servicio

Identificador	**UC-12** · HU-16 (BK-25) · EP-04
Nombre	Crear, editar o desactivar servicio
Actor(es)	Administrador
Requisitos relacionados	RF-SER-01
Descripción	Permite al administrador mantener el catálogo de servicios ofrecidos por la barbería.
Precondiciones	El administrador ha iniciado sesión.
Flujo principal	1. El administrador crea un nuevo servicio o selecciona uno existente. 2. El administrador ingresa o edita nombre, duración estimada y precio. 3. El administrador activa o desactiva el servicio. 4. El sistema guarda el servicio y lo refleja en el catálogo según su estado.
Flujos alternativos	• 2a. Datos incompletos: el sistema impide guardar y señala los campos faltantes.
Postcondiciones	El catálogo de servicios queda actualizado; solo los servicios activos aparecen como opción al agendar una cita (UC-04).

UC-13 — Asociar insumos a servicio

Identificador	**UC-13** · HU-17 (BK-26) · EP-04
Nombre	Asociar insumos a servicio
Actor(es)	Administrador
Requisitos relacionados	RF-SER-02
Descripción	Permite al administrador definir qué insumos del inventario consume un servicio al realizarse.
Precondiciones	El administrador ha iniciado sesión y el servicio y los insumos ya existen en el sistema.
Flujo principal	1. El administrador selecciona un servicio existente. 2. El administrador selecciona uno o varios insumos y la cantidad que consume el servicio. 3. El sistema guarda la asociación entre el servicio y sus insumos.
Flujos alternativos	• 2a. El insumo seleccionado no existe: el sistema solicita registrarlo primero (UC-14).
Postcondiciones	El servicio queda asociado a los insumos que consume; esa asociación es la que usa el sistema para descontar inventario automáticamente al atender una cita (UC-16).

EP-05 — Gestión de Inventario

UC-14 — Registrar insumo

Identificador	**UC-14** · HU-18 (BK-27) · EP-05
Nombre	Registrar insumo
Actor(es)	Administrador
Requisitos relacionados	RF-INV-01
Descripción	Permite al administrador dar de alta un nuevo insumo en el inventario.
Precondiciones	El administrador ha iniciado sesión.
Flujo principal	1. El administrador accede a la sección de inventario. 2. El administrador ingresa nombre, cantidad disponible y unidad de medida del insumo. 3. El sistema guarda el nuevo insumo en el inventario.
Flujos alternativos	• 2a. Datos incompletos: el sistema impide guardar y señala los campos faltantes.
Postcondiciones	Se crea un nuevo insumo en el inventario.

UC-15 — Registrar reabastecimiento de insumo

Identificador	**UC-15** · HU-19 (BK-28) · EP-05
Nombre	Registrar reabastecimiento de insumo
Actor(es)	Administrador
Requisitos relacionados	RF-INV-03
Descripción	Permite al administrador registrar el ingreso de existencias de un insumo ya registrado.
Precondiciones	El administrador ha iniciado sesión y el insumo ya existe en el inventario.
Flujo principal	1. El administrador selecciona el insumo a reabastecer. 2. El administrador ingresa la cantidad que ingresa. 3. El sistema incrementa la cantidad disponible del insumo.
Flujos alternativos	• 2a. Cantidad inválida (negativa o cero): el sistema rechaza el registro.
Postcondiciones	La cantidad disponible del insumo queda actualizada.

UC-16 — Descontar insumos automáticamente

Identificador	**UC-16** · HU-20 (BK-34) · EP-05
Nombre	Descontar insumos automáticamente
Actor(es)	Sistema (secundario: Barbero, quien marca la cita como atendida)
Requisitos relacionados	RF-INV-02
Descripción	Descuenta del inventario los insumos asociados a los servicios de una cita al completarse esta.
Precondiciones	La cita fue marcada como atendida y sus servicios tienen insumos asociados.
Flujo principal	1. El barbero marca la cita como atendida. 2. El sistema identifica los insumos asociados a los servicios de la cita. 3. El sistema descuenta del inventario la cantidad correspondiente de cada insumo.
Flujos alternativos	• 3a. Insumo con existencia insuficiente: el sistema registra el descuento, deja la existencia en cero o negativo, y evalúa la alerta de inventario bajo (UC-17).
Postcondiciones	El inventario refleja el consumo real de insumos de la cita atendida.

UC-17 — Generar alerta de inventario bajo

Identificador	**UC-17** · HU-21 (BK-35) · EP-05
Nombre	Generar alerta de inventario bajo
Actor(es)	Sistema (secundario: Administrador, quien recibe la alerta)
Requisitos relacionados	RF-INV-04
Descripción	Notifica al administrador cuando la existencia de un insumo cae por debajo de un mínimo configurable.
Precondiciones	El insumo tiene un mínimo configurado.
Flujo principal	1. El sistema evalúa la existencia del insumo tras cada movimiento de inventario. 2. El sistema detecta que la existencia es menor al mínimo configurado. 3. El sistema genera una alerta visible para el administrador.
Flujos alternativos	• 1a. El insumo no tiene mínimo configurado: el sistema no genera alerta.
Postcondiciones	El administrador queda notificado del nivel bajo del insumo.

EP-06 — Reportes

UC-18 — Generar y exportar reporte

Identificador	**UC-18** · HU-22 (BK-36) · EP-06
Nombre	Generar y exportar reporte
Actor(es)	Administrador
Requisitos relacionados	RF-REP-01, RF-REP-02, RF-REP-03, RF-REP-04
Descripción	Permite al administrador generar cualquiera de los reportes del sistema (citas atendidas/canceladas/reprogramadas, ingresos por servicio y barbero, o consumo de insumos) y exportarlo. Es un único caso de uso porque el tipo de reporte es solo un parámetro de selección (un input), no un flujo distinto: los cuatro casos de uso originales (Generar reporte de citas/ingresos/insumos y Exportar reporte) se consolidaron aquí.
Precondiciones	El administrador ha iniciado sesión.
Flujo principal	1. El administrador selecciona el tipo de reporte (citas, ingresos o consumo de insumos) y el rango de fechas. 2. El sistema genera el reporte con los datos correspondientes al tipo seleccionado. 3. El administrador selecciona el formato de exportación (PDF y Excel). 4. El sistema genera el archivo descargable en el formato seleccionado.
Flujos alternativos	• 2a. Sin datos en el período seleccionado: el sistema genera un reporte vacío con esa indicación. • 3a. Formato no soportado: el sistema restringe la selección a los formatos disponibles.
Postcondiciones	Se genera un archivo descargable con la información del reporte del tipo y período seleccionados.

EP-07 — Notificaciones

UC-26 — Consultar notificaciones

Identificador	**UC-26** · HU-23 · EP-07
Nombre	Consultar notificaciones
Actor(es)	Cliente, Barbero, Administrador
Requisitos relacionados	RF-NOT-01
Descripción	Permite a cualquier usuario autenticado consultar el listado de notificaciones que el sistema le ha enviado (confirmaciones de cita, códigos de verificación, alertas de inventario, etc.), indicando cuáles no ha leído todavía.
Precondiciones	El usuario ha iniciado sesión.
Flujo principal	1. El usuario abre el centro de notificaciones (campana de la barra superior). 2. El sistema lista las notificaciones del usuario, de la más reciente a la más antigua, e indica cuántas no ha leído. 3. El usuario presiona «Marcar leída» en una notificación. 4. El sistema la marca como leída.
Flujos alternativos	• 2a. No hay notificaciones registradas: el sistema muestra un estado vacío indicándolo.
Postcondiciones	La notificación consultada queda marcada como leída; el listado refleja el estado actualizado.

3. Historias de usuario

3.1 Épicas

Épica	Nombre	Casos de uso	Historias	Story points
EP-01	Autenticación y Cuentas	UC-01, 02, 03, 22, 23, 24	6	18
EP-02	Gestión de Citas	UC-04, 05, 06, 07, 25	5	24
EP-03	Gestión de Clientes	UC-08, 09, 10, 11	4	9
EP-04	Gestión de Servicios	UC-12, 13	2	6
EP-05	Gestión de Inventario	UC-14, 15, 16, 17	4	12
EP-06	Reportes	UC-18	1	8
EP-07	Notificaciones	UC-26	1	3
Total			**23**	**80**

3.2 Sprints

El proyecto se organizó en tres sprints de tres semanas. El orden respeta las dependencias entre módulos: autenticación y catálogos antes de citas, y citas antes del descuento automático de inventario.

Sprint	Semanas	Objetivo	Historias	SP
Sprint 1	1–3	Cimientos: cuentas y catálogos maestros (login, registro, 2FA, barberos, clientes, servicios, insumos)	HU-01 a HU-06, HU-12 a HU-19	37
Sprint 2	4–6	Núcleo del negocio: flujo completo de citas e inventario automático	HU-07 a HU-11, HU-20, HU-21	32
Sprint 3	7–9	Cierre: reportes, notificaciones, pruebas de integración y carga, y despliegue final	HU-22, HU-23	11

3.3 Backlog

HU	Historia de usuario	Prioridad	SP	Sprint
HU-01	Como usuario registrado, quiero iniciar sesión con correo/contraseña y un segundo factor por correo (si está habilitado), para acceder de forma segura al panel de mi rol. (UC-01)	Highest	5	1
HU-02	Como cliente nuevo, quiero registrarme con mis datos y un correo no usado previamente, para poder agendar citas por mi cuenta. (UC-02)	High	3	1
HU-03	Como usuario registrado, quiero recuperar mi contraseña mediante un enlace enviado a mi correo, para volver a acceder si la olvido. (UC-03)	Medium	3	1
HU-04	Como administrador, quiero crear la cuenta de un nuevo barbero con su horario de disponibilidad, para que pueda ser asignado a citas. (UC-22)	High	3	1
HU-05	Como administrador, quiero editar los datos o el horario de un barbero existente, para mantener su información actualizada. (UC-23)	Medium	2	1
HU-06	Como administrador, quiero desactivar la cuenta de un barbero que deja de laborar, para excluirlo de la asignación de nuevas citas. (UC-24)	Medium	2	1
HU-07	Como cliente, quiero agendar una cita eligiendo servicio, barbero, fecha y hora disponibles, para reservar mi turno. (UC-04)	Highest	8	2
HU-08	Como cliente, quiero reprogramar una cita ya agendada a un nuevo horario disponible, para ajustarla si cambia mi disponibilidad. (UC-05)	High	5	2
HU-09	Como cliente o administrador, quiero cancelar una cita activa, para liberar el horario del barbero. (UC-06)	High	3	2
HU-10	Como barbero, quiero consultar mi agenda diaria y semanal, para saber qué citas tengo asignadas. (UC-07)	High	3	2
HU-11	Como administrador o barbero, quiero registrar la atención inmediata de un cliente sin cita previa (walk-in), para atenderlo si hay un barbero disponible. (UC-25)	Medium	5	2
HU-12	Como administrador, quiero registrar los datos básicos de un cliente atendido presencialmente, para poder agendarle citas en su nombre. (UC-08)	Medium	2	1
HU-13	Como administrador, quiero buscar y filtrar clientes por nombre o teléfono, para ubicarlos rápidamente. (UC-09)	Low	2	1
HU-14	Como administrador, quiero consultar el historial de citas y servicios de un cliente, para conocer su actividad previa. (UC-10)	Medium	3	1
HU-15	Como administrador, quiero editar o desactivar el registro de un cliente, para mantener actualizada la base de clientes. (UC-11)	Low	2	1
HU-16	Como administrador, quiero crear, editar o desactivar los servicios del catálogo, para mantener actualizada la oferta. (UC-12)	High	3	1
HU-17	Como administrador, quiero asociar los insumos que consume cada servicio, para que el sistema los descuente automáticamente al atender una cita. (UC-13)	High	3	1
HU-18	Como administrador, quiero registrar un nuevo insumo en el inventario, para llevar control de las existencias. (UC-14)	High	2	1
HU-19	Como administrador, quiero registrar el reabastecimiento de un insumo existente, para reflejar el ingreso de nuevas existencias. (UC-15)	Medium	2	1
HU-20	Como sistema, quiero descontar automáticamente los insumos de una cita al marcarse como atendida, para mantener el inventario sincronizado. (UC-16)	High	5	2
HU-21	Como administrador, quiero recibir una alerta cuando un insumo caiga bajo un mínimo configurable, para reabastecerlo a tiempo. (UC-17)	Medium	3	2

HU	Historia de usuario	Prioridad	SP	Sprint
HU-22	Como administrador, quiero generar y exportar (PDF o Excel) reportes de citas, ingresos o consumo de insumos por rango de fechas, para analizar el desempeño de la barbería. (UC-18)	Medium	8	3
HU-23	Como usuario autenticado, quiero consultar mis notificaciones (confirmaciones, códigos, alertas) y marcarlas como leídas, para mantenerme al tanto de la actividad relevante a mi cuenta. (UC-26)	Low	3	3

3.4 Implementación de cada historia

Pantalla o proceso del sistema que implementa cada historia, con su clave en Jira.

HU	Jira	Caso de uso	Pantalla / componente
HU-01	BK-15	UC-01	Login (/ingresar); VerifyTwoFactor (/verificar-2fa); Seguridad y MiCuenta (2FA)
HU-02	BK-16	UC-02	Register (/registro)
HU-03	BK-17	UC-03	ForgotPassword (/recuperar); ResetPassword (/restablecer)
HU-04	—	UC-22	AdminBarberos, con editor de horario (/admin/barberos)
HU-05	—	UC-23	AdminBarberos, editar con horario y ausencias
HU-06	BK-20	UC-24	AdminBarberos (/admin/barberos)
HU-07	BK-29	UC-04	Booking (/agendar)
HU-08	BK-30	UC-05	MisCitas, modal Reprogramar (/mis-citas)
HU-09	BK-31	UC-06	MisCitas (/mis-citas); AdminCitas (/admin/citas)
HU-10	BK-32	UC-07	Agenda (/barbero/agenda), vista diaria y semanal
HU-11	BK-33	UC-25	AdminCitas y Agenda, botón + Walk-in
HU-12	BK-21	UC-08	Sin pantalla propia
HU-13	BK-22	UC-09	AdminClientes, búsqueda (/admin/clientes)
HU-14	BK-23	UC-10	AdminClientes, botón Historial
HU-15	BK-24	UC-11	AdminClientes, editar y Activos/Inactivos
HU-16	BK-25	UC-12	AdminServicios (/admin/servicios)
HU-17	BK-26	UC-13	AdminServicios, modal Insumos
HU-18	BK-27	UC-14	AdminInventario (/admin/inventario), alta y edición
HU-19	BK-28	UC-15	AdminInventario, acción Registrar entrada
HU-20	BK-34	UC-16	Proceso del sistema; se dispara desde Agenda (botón Atendida)
HU-21	BK-35	UC-17	AdminInventario (fila resaltada); Correo y campana de notificaciones
HU-22	BK-36	UC-18	AdminReportes (/admin/reportes)
HU-23	—	UC-26	Campana de notificaciones (barra de navegación)

Inventario de Endpoints de la API

Campo	Detalle
Proyecto	Barber Kongs — Sistema de Gestión para Barbería (Fase 3)
Equipo	Miguel Angel Quemé Gómez · Brandon Gustavo Güinac Román
Versión del documento	1.0 — 25 de octubre de 2026
Código evaluado	Repositorio Barber-Kong (backend), rama `main`, commit `a74f76c`
Alcance	48 endpoints en 8 módulos más el de estado, bajo el prefijo `/api`
Fuente	Rutas, controladores y esquemas de validación extraídos del código; cobertura medida con las 344 solicitudes reales que hacen las pruebas de integración

1. Convenciones de la API

1.1 Generales

- **Prefijo y formato:** todas las rutas cuelgan de `/api` y usan JSON (UTF-8). Cada módulo se monta solo desde su carpeta: `src/modules/<módulo>` queda en `/api/<módulo>`.
- **Identificadores:** UUID generados por la aplicación. Los parámetros de ruta llamados `id`, `ausenciaId` e `insumoId` son UUID.
- **Fechas y horas:** las fechas viajan como `AAAA-MM-DD`; las horas de entrada como `HH:MM`. Las citas devuelven la hora como `HH:MM:SS` y el horario del barbero como `HH:MM`. Los instantes (`created_at`) son ISO 8601.
- **Números:** los importes y cantidades (`precio`, `cantidad_*`) se devuelven como texto con dos decimales, por ejemplo `"80.00"`.
- **Desactivar frente a eliminar:** `DELETE` sobre barberos, clientes y servicios solo los desactiva (200 con el recurso); `DELETE` de una ausencia o de una asociación servicio-insumo elimina el registro (204 sin cuerpo).

1.2 Autenticación y roles

- **Sesión:** se envía el JWT en el encabezado `Authorization: Bearer <token>`. El token lleva `id`, `email` y `rol` y vence a las 2 horas por defecto (`JWT_EXPIRES_IN`).
- **Inicio de sesión en dos pasos:** si la cuenta tiene 2FA, `login` responde `requiresTwoFactor: true` y el token se obtiene con `verify-2fa`.
- **Roles:** `admin`, `barbero` y `cliente`. Cada ruta valida el rol en el servidor (no solo en la interfaz).
- **Niveles de acceso:** *Público* (sin sesión), *Cualquier rol* (con sesión; algunos solo sobre datos propios) o un conjunto de roles.

1.3 Errores y validación

Todos los errores responden `{ "message": "..." }`, con el texto pensado para mostrarse al usuario. Los errores internos nunca devuelven detalles técnicos.

Código	Significado	Cuándo ocurre
400	Solicitud inválida	El cuerpo o la consulta no cumple el esquema (campos de más, faltantes o con formato inválido) falla una regla de negocio simple (por ejemplo, fecha pasada) o un identificador de la ruta no es un UUID válido.
401	No autenticado	Falta el token, es inválido o venció; o las credenciales son incorrectas.
403	Sin permiso	El rol no puede usar el endpoint o el recurso pertenece a otro usuario.

Código	Significado	Cuándo ocurre
404	No encontrado	El recurso no existe, o la ruta no existe.
409	Conflicto	La operación choca con el estado actual: horario ocupado, correo repetido, día ya marcado, cita ya atendida.
500	Error interno	Error no previsto. El mensaje es genérico: "Ocurrió un error inesperado."

Los cuerpos se validan con esquemas (Ajv) que rechazan campos desconocidos. Las tablas de cuerpo de la sección 4 muestran el tipo, si es obligatorio y las reglas de cada campo.

2. Objetos que devuelve la API

Objeto	Campos principales	Lo devuelven
Usuario	`id`, `email`, `role`, `twoFactorEnabled` y, para barberos, `barberoId`	register, login, verify-2fa, me, 2fa
Cita	`id`, `cliente_id`, `barbero_id`, `servicio_id`, `fecha`, `hora_inicio`, `hora_fin`, `estado` (pendiente, confirmada, cancelada o atendida), `es_walkin`, y del detalle: `servicio_nombre`, `precio`, `duracion_minutos`, `barbero_nombre`, `cliente_nombre`	Endpoints de citas
Barbero	`id`, `nombre`, `especialidad`, `is_active`; el detalle añade `horario` y el listado del administrador, `email`	Endpoints de barberos
Ausencia	`id`, `barbero_id`, `fecha`, `motivo`, `created_at`	Ausencias del barbero
Cliente	`id`, `user_id`, `nombre`, `telefono`, `correo`, `is_active`; en listados y ficha, `citas_totales` y `ultima_visita`	Endpoints de clientes
Servicio	`id`, `nombre`, `duracion_minutos`, `precio`, `is_active`	Endpoints de servicios
Insumo	`id`, `nombre`, `unidad_medida`, `cantidad_disponible`, `cantidad_minima`	Endpoints de inventario
Notificación	`id`, `user_id`, `tipo`, `titulo`, `mensaje`, `leida`, `created_at`	Notificaciones

3. Resumen de endpoints

Acceso: √ puede usarlo; √* solo sobre sus propios datos (un cliente sobre sus citas; un barbero sobre su agenda; cada usuario sobre sus notificaciones) y el administrador sobre todos. Los endpoints públicos no exigen sesión.

Autenticación y cuentas

ID	Método	Ruta	Púb	Cli	Bar	Adm	Descripción	Requisito
E-01	**POST**	`/auth/register`	√	—	—	—	Crea una cuenta de cliente y su perfil de cliente	UC-02 · RF-AUT-05
E-02	**POST**	`/auth/login`	√	—	—	—	Inicia sesión con correo y contraseña	UC-01 · RF-AUT-01, RF-AUT-02
E-03	**POST**	`/auth/verify-2fa`	√	—	—	—	Completa el inicio de sesión con el código de 6 dígitos recibido por correo	RF-AUT-07
E-04	**POST**	`/auth/forgot-password`	√	—	—	—	Envía por correo un enlace para restablecer la contraseña	UC-03 · RF-AUT-06
E-05	**POST**	`/auth/reset-password`	√	—	—	—	Cambia la contraseña con el token del enlace recibido	UC-03 · RF-AUT-06
E-06	**GET**	`/auth/me`	—	√	√	√	Devuelve el perfil de la sesión actual: id, correo, rol, estado del 2FA y, para barberos, su `barberoId`	RF-AUT-02
E-07	**PUT**	`/auth/2fa`	—	√	√	√	Activa o desactiva el segundo factor de la cuenta	RF-AUT-07

Barberos, agenda y horario

ID	Método	Ruta	Púb	Cli	Bar	Adm	Descripción	Requisito
E-08	**GET**	`/barberos`	√	—	—	—	Lista pública de barberos activos, para elegir con quién agendar	RF-CIT-01
E-09	**GET**	`/barberos/admin`	—	—	—	√	Lista todos los barberos, incluidos los inactivos, para que el administrador pueda reactivarlos	UC-22 a UC-24
E-10	**GET**	`/barberos/:id`	√	—	—	—	Detalle público de un barbero con su horario semanal	RF-CIT-01
E-11	**GET**	`/barberos/:id/horario`	√	—	—	—	Horario semanal del barbero (bloques de atención por día)	RF-CIT-02 a RF-CIT-04
E-12	**GET**	`/barberos/:id/disponibilidad`	√	—	—	—	Disponibilidad de un barbero en una fecha: sus bloques de atención y las horas ya ocupadas por citas	RF-CIT-01, RNF-REN-01
E-13	**POST**	`/barberos`	—	—	—	√	Crea un barbero con su cuenta (rol barbero) y su horario	UC-22 · RF-AUT-04
E-14	**PUT**	`/barberos/:id`	—	—	—	√	Edita nombre y especialidad	UC-23 · RF-AUT-04
E-15	**DELETE**	`/barberos/:id`	—	—	—	√	Desactiva al barbero (no lo elimina): sale del catálogo público, no se le pueden agendar citas y su cuenta de acceso también se desactiva, por lo que ya no puede iniciar sesión	UC-24 · RF-AUT-04
E-16	**GET**	`/barberos/:id/ausencias`	—	—	√*	√*	Lista las ausencias del barbero de hoy en adelante	Agenda del barbero
E-17	**POST**	`/barberos/:id/ausencias`	—	—	√*	√*	Marca un día completo como no disponible	Agenda del barbero
E-18	**DELETE**	`/barberos/:id/ausencias/:ausenciaId`	—	—	√*	√*	Quita una ausencia y vuelve a habilitar ese día	Agenda del barbero

Citas

ID	Método	Ruta	Púb	Cli	Bar	Adm	Descripción	Requisito
E-19	**POST**	`/citas`	—	√	—	—	El cliente agenda una cita	UC-04 · RF-CIT-01 a RF-CIT-04, RF-CIT-07, RF-CIT-08
E-20	**POST**	`/citas/walkin`	—	—	√	√	Administrador o barbero registran una cita sin reservación previa	UC-25
E-21	**GET**	`/citas/mias`	—	√	—	—	Lista las citas del cliente autenticado, la más reciente primero	UC-04, RF-CLI-02
E-22	**GET**	`/citas`	—	—	√*	√*	Agenda para barbero o administrador	UC-07 · RF-CIT-09
E-23	**PUT**	`/citas/:id/cancelar`	—	√*	√*	√*	Cancela una cita	UC-06 · RF-CIT-06

ID	Método	Ruta	Púb	Cli	Bar	Adm	Descripción	Requisito
E-24	**PUT**	`/citas/:id/atender`	—	—	√*	√*	Marca la cita como atendida (un barbero solo puede con las suyas; el administrador, con cualquiera) y descuenta los insumos del servicio dentro de una sola transacción; si algún insumo queda bajo su mínimo avisa a los administradores después de confirmar	UC-16, UC-17 · RF-INV-02, RF-INV-04
E-25	**PUT**	`/citas/:id`	—	√*	√*	√*	Reprograma una cita a otra fecha y hora, validando de nuevo el horario, las ausencias y los traslapes	UC-05 · RF-CIT-05

Clientes

ID	Método	Ruta	Púb	Cli	Bar	Adm	Descripción	Requisito
E-26	**GET**	`/clientes`	—	—	—	√	Lista y busca clientes	UC-09 · RF-CLI-03
E-27	**GET**	`/clientes/:id`	—	—	—	√	Ficha de un cliente con su total de citas y la fecha de su última visita	UC-10
E-28	**GET**	`/clientes/:id/historial`	—	—	—	√	Historial de citas del cliente con servicio, barbero y estado, la más reciente primero	UC-10 · RF-CLI-02
E-29	**POST**	`/clientes`	—	—	—	√	Registra un cliente sin cuenta (por ejemplo, un cliente que llega sin reservar)	UC-08 · RF-CLI-01
E-30	**PUT**	`/clientes/:id`	—	—	—	√	Edita nombre, teléfono o correo; solo cambia los campos enviados	UC-11 · RF-CLI-04
E-31	**PUT**	`/clientes/:id/activar`	—	—	—	√	Reactiva a un cliente desactivado	UC-11 · RF-CLI-04
E-32	**DELETE**	`/clientes/:id`	—	—	—	√	Desactiva al cliente (no lo elimina); conserva su historial	UC-11 · RF-CLI-04

Inventario

ID	Método	Ruta	Púb	Cli	Bar	Adm	Descripción	Requisito
E-33	**GET**	`/inventario`	—	—	—	√	Lista los insumos con su existencia y su mínimo de alerta	RF-INV-01
E-34	**POST**	`/inventario`	—	—	—	√	Registra un insumo	UC-14 · RF-INV-01
E-35	**PUT**	`/inventario/:id`	—	—	—	√	Edita nombre, unidad o mínimo de alerta; solo cambia los campos enviados	RF-INV-01, RF-INV-04
E-36	**POST**	`/inventario/:id/entrada`	—	—	—	√	Registra un reabastecimiento: suma la cantidad a la existencia y deja el movimiento en el historial	UC-15 · RF-INV-03

Notificaciones

ID	Método	Ruta	Púb	Cli	Bar	Adm	Descripción	Requisito
E-37	**GET**	`/notificaciones`	—	√*	√*	√*	Lista las notificaciones del usuario autenticado, la más reciente primero	UC-26
E-38	**PUT**	`/notificaciones/:id/leer`	—	√*	√*	√*	Marca una notificación propia como leída	UC-26

Reportes

ID	Método	Ruta	Púb	Cli	Bar	Adm	Descripción	Requisito
E-39	**GET**	`/reportes`	—	—	—	√	Genera un reporte de citas, ingresos o consumo de insumos en el rango indicado y lo devuelve como archivo descargable	UC-18 · RF-REP-01 a RF-REP-04

Servicios

ID	Método	Ruta	Púb	Cli	Bar	Adm	Descripción	Requisito
E-40	**GET**	`/servicios`	√	—	—	—	Catálogo público de servicios activos	RF-SER-03
E-41	**GET**	`/servicios/:id`	√	—	—	—	Detalle público de un servicio	RF-SER-03
E-42	**POST**	`/servicios`	—	—	—	√	Crea un servicio con nombre, duración y precio	UC-12 · RF-SER-01
E-43	**PUT**	`/servicios/:id`	—	—	—	√	Edita nombre, duración o precio; solo cambia los campos enviados	UC-12 · RF-SER-01
E-44	**DELETE**	`/servicios/:id`	—	—	—	√	Desactiva el servicio (no lo elimina): deja de aparecer en el catálogo público	UC-12 · RF-SER-01
E-45	**GET**	`/servicios/:id/insumos`	—	—	—	√	Lista los insumos que consume un servicio y en qué cantidad	UC-13 · RF-SER-02
E-46	**POST**	`/servicios/:id/insumos`	—	—	—	√	Asocia un insumo al servicio con la cantidad que consume	UC-13 · RF-SER-02
E-47	**DELETE**	`/servicios/:id/insumos/:insumoId`	—	—	—	√	Quita la asociación de un insumo con el servicio	UC-13 · RF-SER-02

Estado del servicio

ID	Método	Ruta	Púb	Cli	Bar	Adm	Descripción	Requisito
E-48	**GET**	`/health`	√	—	—	—	Comprueba que el servicio responde	RNF-DIS

4. Detalle por módulo

Autenticación y cuentas

Registro, inicio de sesión, segundo factor y recuperación de contraseña. La mayoría son públicos.

E-01 · POST /api/auth/register

Acceso	Público
Descripción	Crea una cuenta de cliente y su perfil de cliente. El rol siempre es `cliente`.
Requisito	UC-02 · RF-AUT-05
Cuerpo	`nombre` — texto, obligatorio; 1–255 caracteres `email` — correo, obligatorio; máx. 255 caracteres `telefono` — texto, opcional; máx. 50 caracteres `password` — texto, obligatorio; 8–72 caracteres
Respuesta	201 · `{ user }` con id, correo, rol y estado del 2FA.
Errores	400 — Datos inválidos (nombre obligatorio, correo válido, contraseña de 8 a 72 caracteres). 409 — Ya existe una cuenta con ese correo.
Pruebas	3 solicitudes · 201×1, 400×1, 409×1

E-02 · POST /api/auth/login

Acceso	Público
Descripción	Inicia sesión con correo y contraseña. Si la cuenta tiene 2FA no entrega token: genera un código de 6 dígitos y lo envía por correo.
Requisito	UC-01 · RF-AUT-01, RF-AUT-02
Cuerpo	`email` — correo, obligatorio `password` — texto, obligatorio
Respuesta	200 · sin 2FA: `{ requiresTwoFactor: false, token, user }`; con 2FA: `{ requiresTwoFactor: true, userId }`.
Errores	400 — Datos inválidos. 401 — Correo o contraseña incorrectos (mismo mensaje si el correo no existe, la contraseña es errónea o la cuenta está desactivada).
Pruebas	43 solicitudes · 200×38, 401×5

E-03 · POST /api/auth/verify-2fa

Acceso	Público
Descripción	Completa el inicio de sesión con el código de 6 dígitos recibido por correo. El código es de un solo uso y vence.
Requisito	RF-AUT-07
Cuerpo	`userId` — UUID, obligatorio `code` — texto, obligatorio; 6 dígitos
Respuesta	200 · `{ token, user }`.
Errores	400 — Datos inválidos (el código debe tener 6 dígitos). 401 — El código es incorrecto o ya expiró.
Pruebas	3 solicitudes · 200×1, 401×2

E-04 · POST /api/auth/forgot-password

Acceso	Público
Descripción	Envía por correo un enlace para restablecer la contraseña. Responde lo mismo exista o no el correo, para no revelar qué cuentas existen.
Requisito	UC-03 · RF-AUT-06
Cuerpo	`email` — correo, obligatorio
Respuesta	200 · `{ message }`.
Errores	400 — Correo con formato inválido.
Pruebas	2 solicitudes · 200×2

E-05 · POST /api/auth/reset-password

Acceso	Público
Descripción	Cambia la contraseña con el token del enlace recibido. El token es de un solo uso y vence.
Requisito	UC-03 · RF-AUT-06
Cuerpo	`token` — texto, obligatorio; mín. 10 caracteres `password` — texto, obligatorio; 8–72 caracteres
Respuesta	200 · `{ message }`.
Errores	400 — Datos inválidos, o "El enlace no es válido o ya expiró."
Pruebas	2 solicitudes · 200×1, 400×1

E-06 · GET /api/auth/me

Acceso	Cualquier rol
Descripción	Devuelve el perfil de la sesión actual: id, correo, rol, estado del 2FA y, para barberos, su `barberoId`.
Requisito	RF-AUT-02
Respuesta	200 · `{ user }`.
Errores	401 — Sin sesión o token inválido o vencido. 404 — Usuario no encontrado.
Pruebas	3 solicitudes · 200×1, 401×2

E-07 · PUT /api/auth/2fa

Acceso	Cualquier rol
Descripción	Activa o desactiva el segundo factor de la cuenta. Exige la contraseña actual.
Requisito	RF-AUT-07
Cuerpo	`enabled` — booleano, obligatorio `password` — texto, obligatorio
Respuesta	200 · `{ user }` con el nuevo estado.
Errores	400 — Datos inválidos. 401 — Sin sesión o "Contraseña incorrecta."
Pruebas	5 solicitudes · 200×2, 400×1, 401×2

Barberos, agenda y horario

Catálogo público de barberos, gestión por el administrador y ausencias del barbero.

E-08 · GET /api/barberos

Acceso	Público
Descripción	Lista pública de barberos activos, para elegir con quién agendar.
Requisito	RF-CIT-01
Respuesta	200 · lista de `{ id, nombre, especialidad, is_active }`, ordenada por nombre. No incluye el correo del barbero.
Errores	Sin errores de negocio (solo 500 por fallo interno).
Pruebas	3 solicitudes · 200×3

E-09 · GET /api/barberos/admin

Acceso	Administrador
Descripción	Lista todos los barberos, incluidos los inactivos, para que el administrador pueda reactivarlos.
Requisito	UC-22 a UC-24
Respuesta	200 · lista de barberos (activos e inactivos).
Errores	401 — Sin sesión. 403 — Rol sin permiso.
Pruebas	5 solicitudes · 200×3, 401×1, 403×1

E-10 · GET /api/barberos/:id

Acceso	Público
Descripción	Detalle público de un barbero con su horario semanal.
Requisito	RF-CIT-01
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · `{ id, nombre, especialidad, is_active, horario: [{ dia_semana, hora_inicio, hora_fin }] }`. No incluye el correo ni el id de la cuenta.
Errores	400 — El identificador de la ruta no es un UUID válido. 404 — Barbero no encontrado.
Pruebas	5 solicitudes · 200×2, 400×1, 404×2

E-11 · GET /api/barberos/:id/horario

Acceso	Público
Descripción	Horario semanal del barbero (bloques de atención por día).
Requisito	RF-CIT-02 a RF-CIT-04
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · lista de `{ dia_semana, hora_inicio, hora_fin }` con horas en formato HH:MM.
Errores	400 — El identificador de la ruta no es un UUID válido. 404 — Barbero no encontrado.
Pruebas	4 solicitudes · 200×3, 400×1

E-12 · GET /api/barberos/:id/disponibilidad

Acceso	Público
Descripción	Disponibilidad de un barbero en una fecha: sus bloques de atención y las horas ya ocupadas por citas.
Requisito	RF-CIT-01, RNF-REN-01
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400. `fecha` (consulta): Obligatoria, AAAA-MM-DD.
Consulta	`fecha` — fecha, obligatorio; AAAA-MM-DD
Respuesta	200 · `{ ocupadas: ["HH:MM", …], trabaja, bloques: [{ hora_inicio, hora_fin }], ausente }`.
Errores	400 — El identificador de la ruta no es un UUID válido. 400 — Fecha faltante o inválida. 404 — Barbero no encontrado.
Pruebas	2 solicitudes · 200×2

E-13 · POST /api/barberos

Acceso	Administrador
Descripción	Crea un barbero con su cuenta (rol barbero) y su horario. Sin horario usa el de la barbería: domingo a jueves de 9:00 a 19:00.
Requisito	UC-22 · RF-AUT-04
Cuerpo	`nombre` — texto, obligatorio; 1–255 caracteres `especialidad` — texto, opcional; máx. 255 caracteres `email` — correo, obligatorio; máx. 255 caracteres `password` — texto, obligatorio; 8–72 caracteres `horario` — lista, opcional; 1–28 bloques `horario[].dia_semana` — entero, obligatorio; 0–6 `horario[].hora_inicio` — texto, obligatorio; HH:MM `horario[].hora_fin` — texto, obligatorio; HH:MM
Respuesta	201 · barbero creado.
Errores	400 — Datos inválidos o un bloque cuyo fin no es posterior al inicio, o bloques traslapados del mismo día. 409 — Ya existe una cuenta con ese correo.
Pruebas	10 solicitudes · 201×6, 400×2, 403×1, 409×1

E-14 · PUT /api/barberos/:id

Acceso	Administrador
Descripción	Edita nombre y especialidad. Si se envía `horario`, reemplaza todo el horario semanal del barbero.
Requisito	UC-23 · RF-AUT-04
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Cuerpo	`nombre` — texto, opcional; 1–255 caracteres `especialidad` — texto, opcional; máx. 255 caracteres `horario` — lista, opcional; 1–28 bloques `horario[].dia_semana` — entero, obligatorio; 0–6 `horario[].hora_inicio` — texto, obligatorio; HH:MM `horario[].hora_fin` — texto, obligatorio; HH:MM
Respuesta	200 · barbero con su horario.
Errores	400 — El identificador de la ruta no es un UUID válido. 400 — Datos inválidos u horario inválido. 404 — Barbero no encontrado.
Pruebas	3 solicitudes · 200×1, 403×1, 404×1

E-15 · DELETE /api/barberos/:id

Acceso	Administrador
Descripción	Desactiva al barbero (no lo elimina): sale del catálogo público, no se le pueden agendar citas y su cuenta de acceso también se desactiva, por lo que ya no puede iniciar sesión.
Requisito	UC-24 · RF-AUT-04
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · barbero con `is_active: false`.
Errores	400 — El identificador de la ruta no es un UUID válido. 404 — Barbero no encontrado.
Pruebas	5 solicitudes · 200×3, 403×1, 404×1

E-16 · GET /api/barberos/:id/ausencias

Acceso	Administrador, Barbero (solo sobre datos propios; el administrador, sobre todos)
Descripción	Lista las ausencias del barbero de hoy en adelante. El administrador ve las de cualquiera; un barbero solo las propias.
Requisito	Agenda del barbero
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · lista de `{ id, barbero_id, fecha, motivo, created_at }`.
Errores	400 — El identificador de la ruta no es un UUID válido. 403 — Solo puedes gestionar tu propia agenda. 404 — Barbero no encontrado.
Pruebas	5 solicitudes · 200×2, 401×1, 403×2

E-17 · POST /api/barberos/:id/ausencias

Acceso	Administrador, Barbero (solo sobre datos propios; el administrador, sobre todos)
Descripción	Marca un día completo como no disponible. No permite días pasados ni días que ya tengan citas.
Requisito	Agenda del barbero
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Cuerpo	`fecha` — fecha, obligatorio; AAAA-MM-DD `motivo` — texto, opcional; máx. 255 caracteres
Respuesta	201 · ausencia creada.
Errores	400 — El identificador de la ruta no es un UUID válido. 400 — Datos inválidos o "No puedes marcar un día que ya pasó." 403 — Solo puedes gestionar tu propia agenda. 404 — Barbero no encontrado. 409 — Ya hay citas ese día, o "Ese día ya está marcado como no disponible."
Pruebas	7 solicitudes · 201×5, 403×1, 409×1

E-18 · DELETE /api/barberos/:id/ausencias/:ausenciaId

Acceso	Administrador, Barbero (solo sobre datos propios; el administrador, sobre todos)
Descripción	Quita una ausencia y vuelve a habilitar ese día. Elimina el registro de verdad.
Requisito	Agenda del barbero
Parámetros	`id` (ruta): UUID del barbero. `ausenciaId` (ruta): UUID de la ausencia.
Respuesta	204 · sin cuerpo.
Errores	400 — El identificador de la ruta no es un UUID válido. 403 — Solo puedes gestionar tu propia agenda. 404 — Esa ausencia no existe.
Pruebas	4 solicitudes · 204×1, 400×1, 403×1, 404×1

Citas

Agendar, reprogramar, cancelar, atender y consultar la agenda. Todas exigen sesión.

E-19 · POST /api/citas

Acceso	Cliente
Descripción	El cliente agenda una cita. Calcula la hora de fin según el servicio, valida el horario del barbero, sus ausencias y los traslapes, y avisa por correo y en pantalla al cliente y al barbero.
Requisito	UC-04 · RF-CIT-01 a RF-CIT-04, RF-CIT-07, RF-CIT-08
Cuerpo	`servicio_id` — UUID, obligatorio `barbero_id` — UUID, obligatorio `fecha` — fecha, obligatorio; AAAA-MM-DD `hora_inicio` — texto, obligatorio; HH:MM
Respuesta	201 · cita con `estado: "confirmada"` y datos del servicio y del barbero.
Errores	400 — Datos inválidos, fecha pasada o falta el perfil de cliente. 403 — Solo el cliente puede agendar. 404 — Servicio o barbero inexistente o inactivo. 409 — Horario ocupado, fuera del horario del barbero, día sin atención o barbero ausente.
Pruebas	46 solicitudes · 201×33, 400×4, 403×2, 404×3, 409×4

E-20 · POST /api/citas/walkin

Acceso	Administrador, Barbero
Descripción	Administrador o barbero registran una cita sin reservación previa. Con `nombre_cliente` se crea un cliente sin cuenta; sin él la cita queda anónima.
Requisito	UC-25
Cuerpo	`servicio_id` — UUID, obligatorio `barbero_id` — UUID, obligatorio `fecha` — fecha, obligatorio; AAAA-MM-DD `hora_inicio` — texto, obligatorio; HH:MM `nombre_cliente` — texto, opcional; máx. 255 caracteres
Respuesta	201 · cita con `es_walkin: true`.
Errores	400 — Datos inválidos o fecha pasada. 403 — Rol sin permiso. 404 — Servicio o barbero inexistente o inactivo. 409 — Horario ocupado o fuera del horario del barbero.
Pruebas	3 solicitudes · 201×2, 403×1

E-21 · GET /api/citas/mias

Acceso	Cliente
Descripción	Lista las citas del cliente autenticado, la más reciente primero.
Requisito	UC-04, RF-CLI-02
Respuesta	200 · lista de citas del cliente (vacía si no tiene perfil).
Errores	403 — Solo el cliente.
Pruebas	3 solicitudes · 200×3

E-22 · GET /api/citas

Acceso	Administrador, Barbero (solo sobre datos propios; el administrador, sobre todos)
Descripción	Agenda para barbero o administrador. Con `barbero_id` y `fecha` devuelve la agenda de un día; con `desde` y `hasta` (y `barbero_id` opcional) devuelve un rango, para la vista semanal. Un barbero solo puede consultar su propia agenda (con su `barbero_id`); el listado sin `barbero_id` es del administrador.
Requisito	UC-07 · RF-CIT-09
Parámetros	`barbero_id` (consulta): UUID del barbero. `fecha` (consulta): Día AAAA-MM-DD (con `barbero_id`). `desde`, `hasta` (consulta): Rango AAAA-MM-DD (alternativa a `fecha`).
Consulta	`barbero_id` — UUID, opcional `fecha` — fecha, opcional; AAAA-MM-DD `desde` — fecha, opcional; AAAA-MM-DD `hasta` — fecha, opcional; AAAA-MM-DD
Respuesta	200 · lista de citas ordenadas por fecha y hora, con servicio, barbero y cliente.
Errores	400 — Faltan los parámetros ("Especificá barbero_id+fecha, o desde+hasta.") o las fechas y el `barbero_id` tienen un formato inválido. 403 — Rol sin permiso, o un barbero que consulta la agenda de otro o el listado general.
Pruebas	23 solicitudes · 200×11, 400×6, 401×2, 403×4

E-23 · PUT /api/citas/:id/cancelar

Acceso	Cualquier rol (solo sobre datos propios; el administrador, sobre todos)
Descripción	Cancela una cita. El cliente solo las suyas; administrador y barbero, cualquiera. Cancelar una cita ya cancelada no falla.
Requisito	UC-06 · RF-CIT-06
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · cita con `estado: "cancelada"`.
Errores	400 — El identificador de la ruta no es un UUID válido. 403 — No tienes permiso sobre esta cita. 404 — Cita no encontrada. 409 — Esta cita ya fue atendida, no se puede cancelar.
Pruebas	8 solicitudes · 200×5, 400×1, 403×1, 409×1

E-24 · PUT /api/citas/:id/atender

Acceso	Administrador, Barbero (solo sobre datos propios; el administrador, sobre todos)
Descripción	Marca la cita como atendida (un barbero solo puede con las suyas; el administrador, con cualquiera) y descuenta los insumos del servicio dentro de una sola transacción; si algún insumo queda bajo su mínimo avisa a los administradores después de confirmar. Repetirlo no vuelve a descontar.
Requisito	UC-16, UC-17 · RF-INV-02, RF-INV-04
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · cita con `estado: "atendida"`.
Errores	400 — El identificador de la ruta no es un UUID válido. 403 — Rol sin permiso, o un barbero que intenta atender la cita de otro barbero. 404 — Cita no encontrada. 409 — Una cita cancelada no se puede marcar como atendida.
Pruebas	15 solicitudes · 200×10, 400×1, 403×2, 409×1, 500×1

E-25 · PUT /api/citas/:id

Acceso	Cualquier rol (solo sobre datos propios; el administrador, sobre todos)
Descripción	Reprograma una cita a otra fecha y hora, validando de nuevo el horario, las ausencias y los traslapes. El cliente solo las suyas.
Requisito	UC-05 · RF-CIT-05
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Cuerpo	`fecha` — fecha, obligatorio; AAAA-MM-DD `hora_inicio` — texto, obligatorio; HH:MM
Respuesta	200 · cita actualizada.
Errores	400 — El identificador de la ruta no es un UUID válido. 400 — Datos inválidos o fecha pasada. 403 — No tienes permiso sobre esta cita. 404 — Cita no encontrada. 409 — Horario ocupado o fuera de horario, o la cita ya no se puede reprogramar (cancelada o atendida).
Pruebas	3 solicitudes · 200×1, 400×1, 403×1

Clientes

Gestión de clientes. Solo para el administrador.

E-26 · GET /api/clientes

Acceso	Administrador
Descripción	Lista y busca clientes. Por defecto solo los activos.
Requisito	UC-09 · RF-CLI-03
Parámetros	`search` (consulta): Texto a buscar en nombre o teléfono, sin distinguir mayúsculas. `estado` (consulta): `activos` (por defecto), `inactivos` o `todos`. Un valor distinto se trata como `activos`.
Respuesta	200 · lista de clientes con `citas_totales` y `ultima_visita`, ordenada por nombre.
Errores	401 — Sin sesión. 403 — Solo el administrador.
Pruebas	10 solicitudes · 200×7, 401×1, 403×2

E-27 · GET /api/clientes/:id

Acceso	Administrador
Descripción	Ficha de un cliente con su total de citas y la fecha de su última visita.
Requisito	UC-10
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · cliente.
Errores	400 — El identificador de la ruta no es un UUID válido. 404 — Cliente no encontrado.
Pruebas	4 solicitudes · 200×1, 400×1, 404×2

E-28 · GET /api/clientes/:id/historial

Acceso	Administrador
Descripción	Historial de citas del cliente con servicio, barbero y estado, la más reciente primero.
Requisito	UC-10 · RF-CLI-02
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · lista de citas (vacía si no tiene).
Errores	400 — El identificador de la ruta no es un UUID válido. 404 — Cliente no encontrado.
Pruebas	4 solicitudes · 200×2, 400×1, 404×1

E-29 · POST /api/clientes

Acceso	Administrador
Descripción	Registra un cliente sin cuenta (por ejemplo, un cliente que llega sin reservar).
Requisito	UC-08 · RF-CLI-01
Cuerpo	`nombre` — texto, obligatorio; 1–255 caracteres `telefono` — texto, opcional; máx. 50 caracteres `correo` — correo, opcional; máx. 255 caracteres
Respuesta	201 · cliente creado (`user_id` nulo).
Errores	400 — Datos inválidos.
Pruebas	12 solicitudes · 201×8, 400×3, 403×1

E-30 · PUT /api/clientes/:id

Acceso	Administrador
Descripción	Edita nombre, teléfono o correo; solo cambia los campos enviados.
Requisito	UC-11 · RF-CLI-04
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Cuerpo	`nombre` — texto, opcional; 1–255 caracteres `telefono` — texto, opcional; máx. 50 caracteres `correo` — correo, opcional; máx. 255 caracteres
Respuesta	200 · cliente actualizado.
Errores	400 — El identificador de la ruta no es un UUID válido. 400 — Datos inválidos. 404 — Cliente no encontrado.
Pruebas	3 solicitudes · 200×1, 400×1, 404×1

E-31 · PUT /api/clientes/:id/activar

Acceso	Administrador
Descripción	Reactiva a un cliente desactivado.
Requisito	UC-11 · RF-CLI-04
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · cliente con `is_active: true`.
Errores	400 — El identificador de la ruta no es un UUID válido. 404 — Cliente no encontrado.
Pruebas	2 solicitudes · 200×1, 404×1

E-32 · DELETE /api/clientes/:id

Acceso	Administrador
Descripción	Desactiva al cliente (no lo elimina); conserva su historial.
Requisito	UC-11 · RF-CLI-04
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · cliente con `is_active: false`.
Errores	400 — El identificador de la ruta no es un UUID válido. 404 — Cliente no encontrado.
Pruebas	3 solicitudes · 200×2, 404×1

Inventario

Catálogo de insumos y reabastecimiento. Solo para el administrador. El descuento por cita atendida ocurre dentro de `PUT /api/citas/:id/atender`.

E-33 · GET /api/inventario

Acceso	Administrador
Descripción	Lista los insumos con su existencia y su mínimo de alerta.
Requisito	RF-INV-01
Respuesta	200 · lista de insumos.
Errores	401 — Sin sesión. 403 — Solo el administrador.
Pruebas	5 solicitudes · 200×2, 401×1, 403×2

E-34 · POST /api/inventario

Acceso	Administrador
Descripción	Registra un insumo. La existencia inicia en cero; el stock entra después con una entrada.
Requisito	UC-14 · RF-INV-01
Cuerpo	`nombre` — texto, obligatorio; 1–255 caracteres `unidad_medida` — texto, obligatorio; 1–50 caracteres `cantidad_minima` — número o null, opcional; ≥ 0
Respuesta	201 · insumo creado.
Errores	400 — Datos inválidos (también si se intenta enviar la existencia).
Pruebas	2 solicitudes · 201×1, 400×1

E-35 · PUT /api/inventario/:id

Acceso	Administrador
Descripción	Edita nombre, unidad o mínimo de alerta; solo cambia los campos enviados.
Requisito	RF-INV-01, RF-INV-04
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Cuerpo	`nombre` — texto, opcional; 1–255 caracteres `unidad_medida` — texto, opcional; 1–50 caracteres `cantidad_minima` — número o null, opcional; ≥ 0
Respuesta	200 · insumo actualizado.
Errores	400 — El identificador de la ruta no es un UUID válido. 400 — Datos inválidos. 404 — Insumo no encontrado.
Observación	Enviar `cantidad_minima: null` quita el mínimo (y con él la alerta de stock bajo); omitir el campo lo conserva.
Pruebas	7 solicitudes · 200×4, 400×2, 404×1

E-36 · POST /api/inventario/:id/entrada

Acceso	Administrador
Descripción	Registra un reabastecimiento: suma la cantidad a la existencia y deja el movimiento en el historial.
Requisito	UC-15 · RF-INV-03
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Cuerpo	`cantidad` — número, obligatorio; > 0
Respuesta	200 · insumo con la existencia actualizada.
Errores	400 — El identificador de la ruta no es un UUID válido. 400 — Cantidad inválida (debe ser mayor que cero). 404 — Insumo no encontrado.
Pruebas	6 solicitudes · 200×2, 400×3, 404×1

Notificaciones

Centro de notificaciones del usuario autenticado.

E-37 · GET /api/notificaciones

Acceso	Cualquier rol (solo sobre datos propios; el administrador, sobre todos)
Descripción	Lista las notificaciones del usuario autenticado, la más reciente primero.
Requisito	UC-26
Respuesta	200 · lista de notificaciones propias.
Errores	401 — Sin sesión.
Pruebas	10 solicitudes · 200×8, 401×2

E-38 · PUT /api/notificaciones/:id/leer

Acceso	Cualquier rol (solo sobre datos propios; el administrador, sobre todos)
Descripción	Marca una notificación propia como leída.
Requisito	UC-26
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · notificación con `leida: true`.
Errores	400 — El identificador de la ruta no es un UUID válido. 403 — No tienes permiso sobre esta notificación. 404 — Notificación no encontrada.
Pruebas	5 solicitudes · 200×1, 400×1, 401×1, 403×1, 404×1

Reportes

Descarga de reportes en PDF y Excel. Solo para el administrador.

E-39 · GET /api/reportes

Acceso	Administrador
Descripción	Genera un reporte de citas, ingresos o consumo de insumos en el rango indicado y lo devuelve como archivo descargable.
Requisito	UC-18 · RF-REP-01 a RF-REP-04
Consulta	`tipo` — texto (valores fijos), obligatorio; citas | ingresos | insumos `formato` — texto (valores fijos), obligatorio; pdf | excel `desde` — fecha, obligatorio; AAAA-MM-DD `hasta` — fecha, obligatorio; AAAA-MM-DD
Respuesta	200 · archivo binario. `Content-Type`: `application/pdf` o `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`; `Content-Disposition`: `attachment; filename="reporte-{tipo}-{desde}-a-{hasta}.{pdf|xlsx}"`.
Errores	400 — Parámetros faltantes o inválidos, o la fecha "desde" es posterior a "hasta". 403 — Solo el administrador.
Pruebas	15 solicitudes · 200×10, 400×3, 403×2

Servicios

Catálogo público y administración de servicios y de los insumos que consume cada uno.

E-40 · GET /api/servicios

Acceso	Público
Descripción	Catálogo público de servicios activos.
Requisito	RF-SER-03
Respuesta	200 · lista de servicios activos ordenada por nombre.
Errores	Sin errores de negocio (solo 500 por fallo interno).
Pruebas	3 solicitudes · 200×3

E-41 · GET /api/servicios/:id

Acceso	Público
Descripción	Detalle público de un servicio.
Requisito	RF-SER-03
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · servicio.
Errores	400 — El identificador de la ruta no es un UUID válido. 404 — Servicio no encontrado.
Pruebas	4 solicitudes · 200×1, 400×1, 404×2

E-42 · POST /api/servicios

Acceso	Administrador
Descripción	Crea un servicio con nombre, duración y precio.
Requisito	UC-12 · RF-SER-01
Cuerpo	`nombre` — texto, obligatorio; 1–255 caracteres `duracion_minutos` — entero, obligatorio; ≥ 1 `precio` — número, obligatorio; ≥ 0
Respuesta	201 · servicio creado.
Errores	400 — Datos inválidos (mensajes en español para los campos obligatorios).
Pruebas	10 solicitudes · 201×2, 400×5, 403×3

E-43 · PUT /api/servicios/:id

Acceso	Administrador
Descripción	Edita nombre, duración o precio; solo cambia los campos enviados.
Requisito	UC-12 · RF-SER-01
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Cuerpo	`nombre` — texto, opcional; 1–255 caracteres `duracion_minutos` — entero, opcional; ≥ 1 `precio` — número, opcional; ≥ 0
Respuesta	200 · servicio actualizado.
Errores	400 — El identificador de la ruta no es un UUID válido. 400 — Datos inválidos. 404 — Servicio no encontrado.
Pruebas	5 solicitudes · 200×1, 400×1, 403×2, 404×1

E-44 · DELETE /api/servicios/:id

Acceso	Administrador
Descripción	Desactiva el servicio (no lo elimina): deja de aparecer en el catálogo público.
Requisito	UC-12 · RF-SER-01
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · servicio con `is_active: false`.
Errores	400 — El identificador de la ruta no es un UUID válido. 404 — Servicio no encontrado.
Pruebas	3 solicitudes · 200×1, 401×1, 404×1

E-45 · GET /api/servicios/:id/insumos

Acceso	Administrador
Descripción	Lista los insumos que consume un servicio y en qué cantidad.
Requisito	UC-13 · RF-SER-02
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Respuesta	200 · lista de `{ insumo_id, cantidad_consumida, nombre, unidad_medida }`.
Errores	400 — El identificador de la ruta no es un UUID válido. 404 — Servicio no encontrado.
Pruebas	4 solicitudes · 200×3, 400×1

E-46 · POST /api/servicios/:id/insumos

Acceso	Administrador
Descripción	Asocia un insumo al servicio con la cantidad que consume. Si ya estaba asociado, actualiza la cantidad en lugar de duplicar.
Requisito	UC-13 · RF-SER-02
Parámetros	`id` (ruta): Identificador (UUID) del recurso; si no es un UUID válido responde 400.
Cuerpo	`insumo_id` — UUID, obligatorio `cantidad_consumida` — número, obligatorio; > 0
Respuesta	201 · asociación creada o actualizada.
Errores	400 — El identificador de la ruta no es un UUID válido. 400 — Datos inválidos o "Ese insumo no existe." 404 — Servicio no encontrado.
Pruebas	7 solicitudes · 201×3, 400×2, 403×2

E-47 · DELETE /api/servicios/:id/insumos/:insumoId

Acceso	Administrador
Descripción	Quita la asociación de un insumo con el servicio. No modifica el insumo.
Requisito	UC-13 · RF-SER-02
Parámetros	`id` (ruta): UUID del servicio. `insumoId` (ruta): UUID del insumo.
Respuesta	204 · sin cuerpo.
Errores	400 — El identificador de la ruta no es un UUID válido. 404 — Ese insumo no está asociado a este servicio.
Pruebas	4 solicitudes · 204×1, 400×1, 403×1, 404×1

Estado del servicio

Comprobación de disponibilidad.

E-48 · GET /api/health

Acceso	Público
Descripción	Comprueba que el servicio responde. Lo usa el flujo de integración continua y sirve para monitoreo.
Requisito	RNF-DIS
Respuesta	200 · `{ "status": "ok" }`.
Errores	Sin errores de negocio (solo 500 por fallo interno).
Pruebas	1 solicitudes · 200×1

5. Cobertura de pruebas por endpoint

Las pruebas de integración hicieron 344 solicitudes reales a la API, que se registraron y se asociaron a cada ruta. **48 de 48 endpoints se ejercitan por HTTP**; 48 se prueban por su camino exitoso y 42 también por algún camino de error (4xx).

Módulo	Endpoints	Ejercitados	Solicitudes	Con camino exitoso	Con camino de error
Autenticación y cuentas	7	7	61	7	6
Barberos, agenda y horario	11	11	53	11	9
Citas	7	7	101	7	6
Clientes	7	7	38	7	7
Inventario	4	4	20	4	4
Notificaciones	2	2	15	2	2
Reportes	1	1	15	1	1
Servicios	8	8	40	8	7
Estado del servicio	1	1	1	1	0
Total	**48**	**48**	**344**	**48**	**42**

Endpoints sin ninguna prueba de camino de error: POST /auth/forgot-password, GET /barberos, GET /barberos/:id/disponibilidad, GET /citas/mias, GET /servicios, GET /health.

Un 500 observado en `PUT /citas/:id/atender` es intencional: una prueba fuerza un fallo del descuento para comprobar que la transacción se revierte.

6. Hallazgos

6.1 Defectos detectados y corregidos

Detectados al inventariar los endpoints o al probarlos. Todos están corregidos y cada uno tiene una prueba de regresión en la suite. Mantienen la numeración del Plan de Pruebas (D-01, el plural del aviso de horario, solo aparece allí).

ID	Severidad	Endpoints	Defecto	Corrección aplicada
D-02	Media	DELETE /api/barberos/:id	Desactivar un barbero no desactivaba su cuenta: podía iniciar sesión y gestionar su agenda.	Se desactiva también `users.is_active` en la misma transacción.
D-03	Media	GET /api/citas	Un barbero podía consultar la agenda y el listado de citas de otros barberos, con nombres de clientes.	El barbero solo consulta su propio `barbero_id`; el listado general es del administrador.
D-04	Media	PUT /api/citas/:id/atender	Un barbero podía marcar atendida la cita de otro barbero, lo que además descontaba insumos.	Se comprueba que la cita pertenezca al barbero de la sesión (el administrador, cualquiera).
D-05	Media	GET /api/barberos, GET /api/barberos/:id	El catálogo público exponía el correo de cada barbero (su usuario de acceso) y, en el detalle, su `user_id`.	Los endpoints públicos devuelven solo id, nombre, especialidad y estado.
D-06	Media	Los 25 endpoints que reciben un id en la ruta	Un id que no era UUID producía un error de PostgreSQL y la API respondía 500.	Un middleware común valida los parámetros de ruta y responde 400.
D-07	Media	GET /api/citas	Los parámetros de consulta no se validaban: fechas o ids inválidos respondían 500.	Esquema de validación de la consulta; responde 400 con mensaje en español.
D-08	Baja	PUT /api/inventario/:id	El esquema aceptaba `cantidad_minima: null` pero el valor se ignoraba: no se podía quitar un mínimo ya fijado.	Se distingue "campo ausente" de `null` al actualizar.

6.2 Observaciones

ID	Observación	Recomendación
OE-01	Mensajes técnicos o en inglés en algunas validaciones: un JSON mal formado responde `Unexpected end of JSON input`, un cuerpo al que le faltan campos obligatorios sin mensaje propio responde `#/required must have required property 'email'` y una fecha inválida en disponibilidad responde `/fecha must match format "date"`. Los campos con mensaje propio sí están en español.	RNF-USA-03: agregar mensajes en español a todos los esquemas y al manejo de JSON inválido.
OE-02	Ningún listado tiene paginación (clientes, servicios, insumos, notificaciones, citas por rango): devuelven todos los registros.	Aceptable con el volumen de una barbería; considerar límites si el historial crece.
OE-03	No hay límite de intentos en `login`, `verify-2fa` ni `forgot-password`, y CORS permite cualquier origen (`cors()` sin opciones).	Agregar limitación de intentos y restringir los orígenes al frontend; ya figura en el plan de mejora.
OE-04	Existen `activate` en los servicios de barberos y de servicios, pero ninguna ruta los expone: un barbero o servicio desactivado no se puede reactivar por la API. Los clientes sí tienen `PUT /api/clientes/:id/activar`.	Exponer la reactivación si la interfaz la necesita.
OE-05	Los endpoints DELETE de barberos, clientes y servicios desactivan (responden 200 con el recurso); los DELETE de ausencias y de asociaciones servicio-insumo sí eliminan (responden 204).	Convención a tener presente al consumir la API (ver sección 1).

Plan de Pruebas

Campo	Detalle
Proyecto	Barber Kongs — Sistema de Gestión para Barbería (Fase 3)
Equipo	Miguel Angel Quemé Gómez · Brandon Gustavo Güinac Román
Versión del documento	1.0 — 25 de octubre de 2026
Código evaluado	Repositorio Barber-Kong (backend), rama `main` (incluye las correcciones descritas en la sección 9); commit evaluado `a74f76c`
Resultado global	202 casos automáticos: 202 superados · cobertura de líneas 96,2 % · 10 de 10 mutaciones detectadas · 8 defectos detectados durante las pruebas y corregidos

1. Objetivos de las pruebas

- Comprobar que cada requisito funcional del backend se comporta como lo describe la Fase 1 y que las reglas de negocio no se rompen al cambiar el código.
- Garantizar que los permisos por rol se validan en el servidor (RNF-SEG-02) y no solo en la interfaz.
- Verificar el proceso técnico más complejo del sistema: marcar una cita como atendida, descontar los insumos y avisar del stock bajo dentro de una transacción.
- Detectar regresiones automáticamente antes de fusionar a `develop`, mediante el pipeline de integración continua.
- Medir el rendimiento frente a los requisitos no funcionales RNF-REN-01 y RNF-REN-02.

2. Alcance

2.1 Dentro del alcance

Módulo	Qué se verifica
Autenticación y roles	Registro, inicio de sesión, JWT, doble factor (2FA), recuperación de contraseña, desactivación de cuentas y permisos por rol.
Clientes	Registro por el administrador, edición, búsqueda por nombre o teléfono, activar y desactivar, e historial de citas.
Citas	Agendar, reprogramar, cancelar, reservas sobre un mismo horario, fechas pasadas, horario de atención del barbero, citas sin reservación (walk-in) y permisos sobre la agenda de cada barbero.
Barberos, agenda y horario	Alta, edición y desactivación de barberos, horario por barbero, días no disponibles (ausencias), disponibilidad por fecha y consulta de agenda diaria y semanal.
Atención de cita e inventario automático	Transacción de marcar atendida, descuento de insumos, alerta de stock bajo y reversión ante fallos.
Inventario y servicios	Alta de insumos, entradas, alta y edición de servicios, asociación de insumos a servicios y catálogo público.
Reportes	Contenido real de los archivos Excel y PDF, periodos sin datos y validación de parámetros.
Correo y notificaciones	Plantillas de correo (marca, contenido, escape de HTML), envío tras eventos del negocio y centro de notificaciones del usuario.
Transversal	Manejo de errores, validación de entradas (Ajv) y formato de mensajes en español.

2.2 Fuera del alcance y cómo se mitiga

Elemento	Motivo / mitigación actual
Interfaz del frontend	Las pantallas se revisan manualmente y el flujo de CI del frontend valida el lint y la compilación en cada Pull Request.
Infraestructura en la nube (Vercel y Supabase)	La suite corre sobre PostgreSQL 16 local y en CI. No valida la configuración de producción; esto se cubre en la documentación de despliegue.
Envío real de correo (SMTP)	El transporte se reemplaza por un simulado. El envío real se comprueba manualmente tras desplegar; un error de credenciales ya se presentó en producción (SMTP rechazado por falta de variables de entorno).
Rendimiento en producción	La prueba de carga se ejecutó en local; no incluye la latencia de red hacia la nube.
Pruebas de penetración	No se realizaron. Los controles de seguridad se verifican a nivel funcional (roles, JWT, hash de contraseñas).

3. Estrategia de pruebas

3.1 Niveles de prueba

Nivel	Qué valida	Cómo	Pruebas
Unitario	Reglas de negocio de cada servicio y utilidades puras, sin base de datos.	Los servicios se construyen con una fábrica que recibe su repositorio (`createXService({ repository })`); en las pruebas se inyecta un repositorio falso.	101
Integración	La API completa (rutas, validación, permisos, servicios, SQL real).	Aplicación Express real (`createApp()`) invocada con Supertest sobre PostgreSQL 16, con datos propios que se eliminan al terminar.	101
Carga	Rendimiento con 20 usuarios simultáneos.	Script manual `npm run test:carga` contra la aplicación real en un puerto local.	1 escenario
Mutación (manual)	Que las pruebas realmente detectan errores.	Se introduce un error deliberado en el código, se ejecuta la suite y se restaura el original.	10 mutaciones

3.2 Principios de diseño de las pruebas

- **Aisladas y repetibles:** cada prueba crea sus propios datos con nombres únicos y el prefijo `zz-test`, y se borran al terminar; el resultado no depende del orden ni de datos previos.
- **Sin efectos externos:** el transporte de correo está simulado para todos los archivos; ninguna prueba envía mensajes reales.
- **Casos negativos primero:** además del flujo correcto se prueban rechazos (403, 409, 400, 404), permisos cruzados entre usuarios y entradas inválidas.
- **Mensajes en español:** las validaciones comprueban el texto que verá el usuario, no solo el código HTTP (RNF-USA-03).
- **Regresión:** todo defecto corregido debe dejar una prueba que lo habría detectado (ejemplo: el plural del día en el aviso de horario, defecto D-01).
- **Seguridad del entorno:** la suite se niega a ejecutarse si `NODE_ENV` no es `test` o si el nombre de la base de datos no termina en `_test`, para no alterar datos de desarrollo ni de producción.

4. Entorno de pruebas

Componente	Valor
Sistema operativo y equipo	Linux 7.0 (x64), AMD Ryzen 9 7900X, 24 hilos, 30 GB de RAM

Componente	Valor
Node.js	v20.19.0
Base de datos	PostgreSQL 16.15 (contenedor Docker, base `barber_kong_test`); en CI, imagen `postgres:16-alpine`
Marco de pruebas	Vitest 4.1.11 (ejecución secuencial de archivos para evitar interferencia en la base)
Pruebas HTTP	Supertest 7
Cobertura	@vitest/coverage-v8
Esquema y datos base	Migraciones versionadas 0001–0006 y `seed.sql`, aplicadas con `npm run db:setup:test`
Configuración	Archivo `.env.test` (no se versiona); en CI se genera durante el flujo
Correo	Transporte SMTP simulado (`vi.mock`); se inspeccionan los mensajes que se habrían enviado

5. Ejecución y automatización

5.1 Comandos

Comando	Descripción
`npm run db:setup:test`	Crea o actualiza la base de pruebas con migraciones y datos base.
`npm test`	Ejecuta la suite completa (unitarias e integración).
`npm run test:unit`	Solo unitarias; no requieren base de datos.
`npm run test:integration`	Solo integración.
`npm run test:cov`	Suite con reporte de cobertura de código.
`npm run test:carga`	Carga con 20 usuarios y reservas simultáneas (manual).

5.2 Integración continua

El flujo `ci.yml` de GitHub Actions se ejecuta en cada Pull Request y en cada push hacia `develop` y `main`. Sobre un servicio PostgreSQL 16 realiza, en orden: instalación de dependencias (`npm ci`), validación de migraciones y datos base, preparación de la base de pruebas, ejecución de `npm test` y un arranque del servidor con verificación de `/api/health`. Una prueba fallida bloquea la integración del cambio. El repositorio del frontend tiene su propio flujo `ci.yml`, que ejecuta `npm run lint` y `npm run build` en cada Pull Request y push hacia `develop` y `main`.

Evidencia. Tras fusionar el Pull Request #28 (de `develop` a `main`), el flujo se ejecutó en GitHub Actions sobre `main` (ejecución #63, commit `31d48fe`) y terminó con éxito en 52 segundos. El reporte de Vitest que publica el propio flujo muestra la suite de ese momento: 15 archivos de prueba y 177 casos. La suite actual (17 archivos y 202 casos) se ejecuta con el mismo flujo en cada Pull Request.

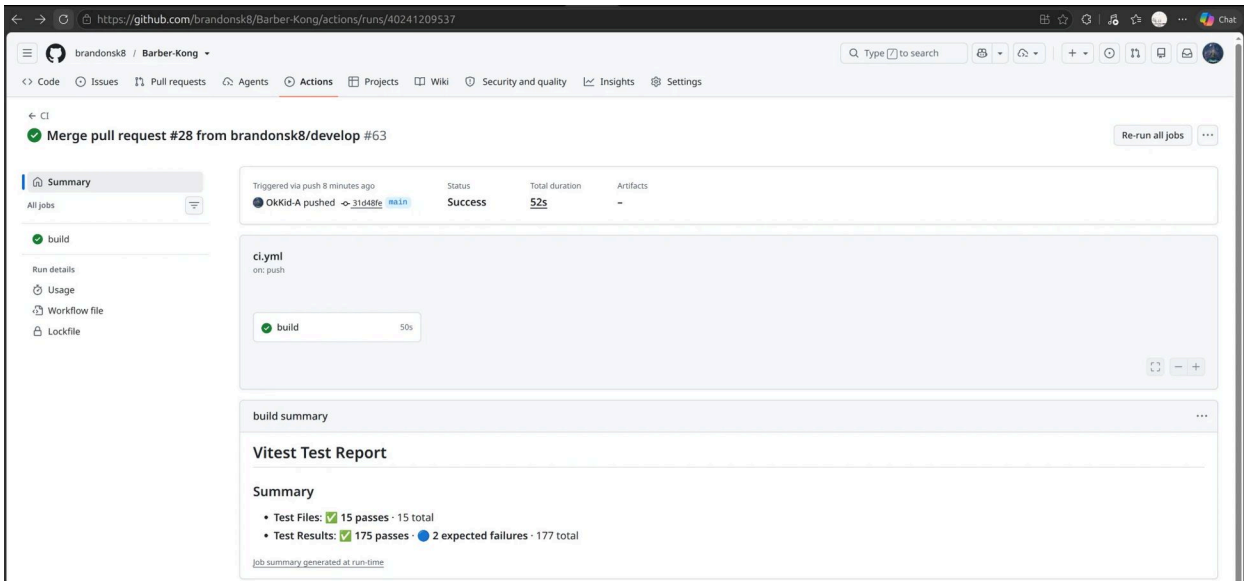


Figura 1. Ejecución de CI en GitHub Actions sobre `main` (run #63): reporte de Vitest de la suite de ese momento (15 archivos, 177 casos).

6. Casos de prueba

La suite contiene **202 casos automáticos** (101 unitarios y 101 de integración). El catálogo completo, con identificador, descripción y requisito asociado, está en el Anexo A. Los identificadores `PU-###` corresponden a pruebas unitarias y `PI-###` a pruebas de integración. La distribución por módulo es la siguiente:

Módulo	Unitarias	Integración	Total
Autenticación y roles	17	13	30
Clientes	0	10	10
Citas	10	17	27
Barberos, agenda y horario	30	19	49
Atención de cita e inventario automático	11	9	20
Inventario y servicios	4	18	22
Reportes	9	4	13
Correo y notificaciones	15	5	20
Transversal (errores y validación)	5	6	11
TOTAL	**101**	**101**	**202**

Casos representativos del proceso central (marcar atendida y descontar inventario):

ID	Caso	Resultado esperado
PI-040	Marcar atendida una cita de un servicio que consume un insumo	La cita queda atendida, el stock baja la cantidad configurada, se registra un movimiento de salida ligado a la cita y marcarla de nuevo no vuelve a descontar.

ID	Caso	Resultado esperado
PI-045	Fallo forzado durante el descuento	La cita sigue confirmada, el stock no cambia y no queda movimiento: la transacción se revierte completa.
PI-043	Stock por debajo del mínimo tras atender	Los administradores reciben la alerta por correo y en el centro de notificaciones, solo después de confirmar la transacción.
PI-046	Marcar atendida una cita cancelada	Respuesta 409 y el inventario no cambia.

7. Trazabilidad entre requisitos y pruebas

Cada prueba está asociada a los requisitos que verifica. La tabla resume, para cada requisito, cuántas pruebas lo cubren y su estado. De los 31 requisitos funcionales, **30 están cubiertos**, **1 parcialmente** y **0 sin pruebas automáticas**. Los estados se explican en la columna de observaciones.

Requisito	Descripción	PU	PI	Estado	Observación
RF-AUT-01	Inicio de sesión con correo y contraseña	3	3	Cubierto	—
RF-AUT-02	Emisión y validación de tokens JWT	4	2	Cubierto	—
RF-AUT-03	Tres roles con permisos diferenciados	2	7	Cubierto	—
RF-AUT-04	Crear, editar y desactivar cuentas de barberos	14	9	Cubierto	Desactivar al barbero lo oculta del catálogo, bloquea nuevas reservas y cierra su acceso al sistema.
RF-AUT-05	Registro autónomo de clientes	2	2	Cubierto	—
RF-AUT-06	Recuperación de contraseña por correo	4	2	Cubierto	—
RF-AUT-07	Autenticación de dos factores por correo	6	3	Cubierto	—
RF-CIT-01	Agendar cita (barbero, servicio, fecha y hora)	8	7	Cubierto	—
RF-CIT-02	Solo días de atención del barbero (domingo a jueves por defecto)	4	1	Cubierto	—
RF-CIT-03	Sin citas en el horario de almuerzo	1	1	Cubierto	El receso se aplica porque el horario del barbero se define por bloques de atención (9:00–13:00 y 14:00–19:00 en las pruebas); se verifica rechazando una cita que cruza las 13:00.
RF-CIT-04	Solo dentro del horario de atención	4	1	Cubierto	—
RF-CIT-05	Reprogramar validando la disponibilidad	0	2	Cubierto	—
RF-CIT-06	Cancelar (cliente o administrador)	2	4	Cubierto	—
RF-CIT-07	Impedir la doble reserva del mismo barbero	1	3	Cubierto	Se verifica con solicitudes sucesivas sobre el mismo horario.
RF-CIT-08	Notificar confirmación, reprogramación y cancelación	11	6	Cubierto	Correo simulado en las pruebas; el envío SMTP real se valida a mano.
RF-CIT-09	El barbero consulta su agenda diaria y semanal	3	9	Cubierto	Un barbero solo consulta su propia agenda; el listado general es del administrador.
RF-CLI-01	Registrar datos básicos del cliente	0	3	Cubierto	—
RF-CLI-02	Historial de citas por cliente	0	3	Cubierto	—
RF-CLI-03	Buscar y filtrar clientes	0	2	Cubierto	—
RF-CLI-04	Editar o desactivar clientes	0	4	Cubierto	—

Requisito	Descripción	PU	PI	Estado	Observación
RF-SER-01	Crear, editar y desactivar servicios	0	6	Cubierto	—
RF-SER-02	Asociar servicios a insumos	1	4	Cubierto	—
RF-SER-03	Catálogo público de servicios activos	0	3	Cubierto	—
RF-INV-01	Registrar insumos (nombre, cantidad, unidad)	1	5	Cubierto	—
RF-INV-02	Descuento automático al completar una cita	9	7	Cubierto	—
RF-INV-03	Registrar reabastecimiento (entradas)	1	4	Cubierto	—
RF-INV-04	Alerta de stock bajo	8	5	Cubierto	—
RF-REP-01	Reporte de citas por rango de fechas	3	2	Parcial	El reporte no puede contar citas "reprogramadas" (observación O-03).
RF-REP-02	Reporte de ingresos por servicio y barbero	3	2	Cubierto	—
RF-REP-03	Reporte de consumo de insumos	3	2	Cubierto	—
RF-REP-04	Exportar cada reporte a PDF y a Excel	7	2	Cubierto	—
RNF-REN-01	Consulta de disponibilidad en menos de 2 s	0	0	Cubierto (carga manual)	Script de carga: p95 de 10,7 ms (local).
RNF-REN-02	Al menos 20 usuarios concurrentes	0	0	Cubierto (carga manual)	Script de carga: 20 usuarios, 48905 solicitudes, 0 errores (local).
RNF-SEG-01	Contraseñas con hash seguro (bcrypt)	1	1	Cubierto	—
RNF-SEG-02	Permisos de rol validados en cada endpoint del backend	7	17	Cubierto	La validación es por rol y, para el barbero, también por propietario: solo ve y atiende sus propias citas.
RNF-SEG-04	Expiración de sesiones (JWT)	2	0	Parcial	Se prueba el rechazo de tokens vencidos y la firma; la duración configurable no se mide.
RNF-USA-03	Mensajes de error comprensibles	1	11	Cubierto	—
Agenda*	Agenda del barbero: horario propio y días no disponibles (funcionalidad nueva de la Fase 2, sin RF original)	9	5	Cubierto	Funcionalidad añadida en la Fase 2.

*PU = pruebas unitarias · PI = pruebas de integración. * La agenda del barbero (horario propio y días no disponibles) es funcionalidad añadida en la Fase 2 y no tiene requisito propio en la Fase 1.*

8. Resultados de la ejecución

Ejecución del 25 de octubre de 2026 sobre el commit `a74f76c`.

8.1 Resumen

Indicador	Resultado
Archivos de prueba	17 (todos en verde)
Casos ejecutados	202
Casos superados / fallidos	202 / 0
Duración de la suite completa	≈ 6.2 s
Estabilidad	Ejecutada de forma repetida (varias corridas consecutivas) sin resultados distintos, también partiendo de una base de datos vacía como en CI.

8.2 Resultados por archivo

Archivo	Nivel	Casos	Superados	Duración
integration/auth.test.js	Integración	11	11	325 ms
integration/barberos-servicios.test.js	Integración	14	14	455 ms
integration/citas.test.js	Integración	24	24	262 ms
integration/clientes.test.js	Integración	10	10	106 ms
integration/endpoints-restantes.test.js	Integración	9	9	95 ms
integration/inventario-reportes.test.js	Integración	14	14	209 ms
integration/notificaciones.test.js	Integración	5	5	29 ms
integration/permisos-y-datos-expuestos.test.js	Integración	8	8	86 ms
integration/validacion-de-entrada.test.js	Integración	6	6	57 ms
unit/auth.service.test.js	Unitario	13	13	160 ms
unit/barberos.service.test.js	Unitario	20	20	124 ms
unit/citas.service.test.js	Unitario	18	18	11 ms
unit/emailTemplate.test.js	Unitario	15	15	12 ms
unit/inventario.service.test.js	Unitario	6	6	5 ms
unit/reportes.formato.test.js	Unitario	7	7	115 ms
unit/utilidades.test.js	Unitario	7	7	8 ms
unit/validaciones.test.js	Unitario	15	15	17 ms

8.3 Cobertura de código

La cobertura global es de **95,5 % de sentencias**, **83,1 % de ramas**, **97,3 % de funciones** y **96,2 % de líneas**. El porcentaje es una referencia de qué código ejecutan las pruebas; no mide por sí solo su calidad, que se evalúa con la prueba de mutación.

Módulo	Sentencias	Ramas	Funciones	Líneas
src/config	86,7 %	43,8 %	75,0 %	85,7 %
src/helpers	100,0 %	100,0 %	100,0 %	100,0 %
src/middlewares	100,0 %	100,0 %	100,0 %	100,0 %
src/modules	94,1 %	88,9 %	100,0 %	100,0 %
src/modules/auth	96,5 %	78,7 %	100,0 %	98,5 %
src/modules/barberos	93,8 %	90,4 %	98,2 %	93,8 %
src/modules/citas	96,7 %	81,8 %	100,0 %	98,2 %
src/modules/clientes	93,4 %	88,9 %	95,5 %	93,1 %
src/modules/inventario	90,8 %	76,0 %	90,0 %	91,5 %
src/modules/notificaciones	96,0 %	87,5 %	100,0 %	95,8 %
src/modules/reportes	99,0 %	85,1 %	100,0 %	99,3 %
src/modules/servicios	89,9 %	77,8 %	96,2 %	91,6 %
src/server	92,3 %	50,0 %	75,0 %	92,3 %
src/services	94,3 %	81,2 %	92,9 %	94,8 %

Módulo	Sentencias	Ramas	Funciones	Líneas
TOTAL	95,5 %	83,1 %	97,3 %	96,2 %

Cierre de brechas. Una primera ejecución con 146 casos arrojó 78,1 % de líneas y evidenció cuatro áreas con cobertura baja: clientes, notificaciones, servicios y los controladores de barberos. Se agregaron 56 casos de integración que las cubren y los resultados de este documento corresponden a la suite completa. Comparación de cobertura de líneas:

Área	Primera ejecución	Suite completa
Clientes	19,4 %	93,1 %
Notificaciones	25,0 %	95,8 %
Servicios	57,8 %	91,6 %
Barberos	58,0 %	93,8 %
Global	**78,1 %**	**96,2 %**

Los módulos con menor cobertura de líneas son ahora config (85,7 %), inventario (91,5 %), servicios (91,6 %). El transporte de correo (`services/mailer.js`) aparece sin cobertura porque se simula a propósito en todas las pruebas.

8.4 Prueba de mutación

Para comprobar que las pruebas detectan errores y no solo recorren el código, se introdujo un defecto deliberado a la vez en reglas críticas, se ejecutó la suite completa y se restauró el código. Una mutación es "detectada" cuando al menos una prueba falla.

#	Error introducido	Archivo	Resultado
1	El umbral de stock bajo usa <= en vez de <	`inventario/inventario.service.js`	Detectada (1 prueba falló)
2	Se permite agendar en fechas pasadas	`citas/citas.service.js`	Detectada (2 pruebas fallaron)
3	Un cliente puede cancelar la cita de otro	`citas/citas.service.js`	Detectada (2 pruebas fallaron)
4	No se hace ROLLBACK si falla el descuento de insumos	`citas/citas.service.js`	Detectada (2 pruebas fallaron)
5	POST /api/citas sin restricción de rol	`citas/citas.routes.js`	Detectada (1 prueba falló)
6	Reportes accesibles para cualquier usuario	`reportes/reportes.routes.js`	Detectada (2 pruebas fallaron)
7	Una cuenta desactivada puede iniciar sesión	`auth/auth.service.js`	Detectada (2 pruebas fallaron)
8	Se ignora el traslape de horarios (doble reserva)	`citas/citas.repository.js`	Detectada (1 prueba falló)
9	Se puede marcar como leída la notificación de otro usuario	`notificaciones/notificaciones.service.js`	Detectada (1 prueba falló)
10	El listado de clientes incluye a los desactivados por defecto	`clientes/clientes.repository.js`	Detectada (1 prueba falló)

Resultado: **10 de 10 mutaciones detectadas**; no sobrevivió ninguna.

8.5 Pruebas de carga

Se ejecutó `npm run test:carga` contra la aplicación real y la base de pruebas, en un equipo local (sin latencia de red hacia la nube). Se mantuvieron 20 usuarios simultáneos durante 15 segundos alternando entre tres consultas públicas.

Consulta	Solicitudes	p50	p95	p99	Máximo
Disponibilidad de un barbero	16304	8,3 ms	10,7 ms	14,8 ms	93,7 ms
Catálogo de servicios	16301	4,5 ms	6,5 ms	8,4 ms	71,2 ms
Lista de barberos	16300	4,9 ms	6,8 ms	8,6 ms	75,3 ms

Requisito	Criterio	Medido	Resultado
RNF-REN-01	Consulta de disponibilidad < 2 s	p95 = 10,7 ms	Cumple
RNF-REN-02	20 usuarios concurrentes sin degradación	48905 solicitudes (3259,7/s), 0 errores	Cumple

Estos valores son una cota optimista: se midieron en local. Deben repetirse contra el despliegue real antes de afirmar el cumplimiento en producción.

9. Hallazgos

9.1 Defectos detectados y corregidos

Las pruebas y el inventario de endpoints detectaron los defectos siguientes. Todos se corrigieron y cada uno dejó una prueba de regresión que forma parte de la suite. D-01 lo detectó una prueba unitaria; D-02 a D-05 aparecieron al probar los permisos entre usuarios; D-06 a D-08, al inventariar los endpoints.

ID	Defecto	Severidad	Corrección	Prueba de regresión
D-01	El aviso de horario decía "no atiende los domingo" (sin plural).	Baja	Función `enPlural` (commit `3e31a52`).	`unit/barberos.service`
D-02	Un barbero desactivado por el administrador seguía pudiendo iniciar sesión y gestionar su agenda (RF-AUT-04, RNF-SEG-02).	Media	Desactivar al barbero desactiva también su cuenta (`users.is_active`) en la misma transacción.	`integration/permisos-y-datos-expuestos`
D-03	Un barbero podía consultar la agenda y las citas de otros barberos, con los nombres de sus clientes (RF-CIT-09, RNF-SEG-02).	Media	En `GET /api/citas` el barbero solo consulta su propio `barbero_id`; el listado general es del administrador.	`integration/permisos-y-datos-expuestos`, `unit/citas.service`
D-04	Un barbero podía marcar como atendida la cita de otro barbero, y con ello descontar insumos (RF-INV-02).	Media	Se comprueba que la cita pertenezca al barbero de la sesión; el administrador puede con cualquiera.	`integration/permisos-y-datos-expuestos`, `unit/citas.service`
D-05	El catálogo público de barberos exponía el correo de cada barbero y, en el detalle, el id de su cuenta (RNF-SEG-02).	Media	Los endpoints públicos devuelven solo id, nombre, especialidad y estado; el administrador conserva el correo en su listado.	`integration/permisos-y-datos-expuestos`
D-06	Un id de ruta que no era UUID producía un error de PostgreSQL y la API respondía 500 (en los 25 endpoints con id en la ruta).	Media	Un middleware común valida los parámetros de ruta y responde 400 con un mensaje en español.	`integration/validacion-de-entrada`
D-07	`GET /api/citas` no validaba la consulta: fechas o ids inválidos respondían 500 (RNF-USA-03).	Media	Esquema de validación de la consulta; responde 400 con un mensaje en español.	`integration/validacion-de-entrada`
D-08	`PUT /api/inventario/:id` con `cantidad_minima: null` se ignoraba: no se podía quitar un mínimo ya fijado (RF-INV-04).	Baja	Se distingue "campo ausente" de `null`; con `null` se quita el mínimo y con él la alerta.	`integration/validacion-de-entrada`

Las correcciones se integraron a `main` mediante los Pull Requests #29 (`hotfix/acceso-y-datos-expuestos`, D-02 a D-05) y #30 (`hotfix/validacion`, D-06 a D-08). Ninguna cambió el esquema de la base de datos.

9.2 Observaciones sobre el comportamiento

Comportamientos que las pruebas confirmaron y que conviene tener presentes al documentar o al hacer cambios. No son defectos.

ID	Observación
O-01	El stock de un insumo siempre nace en cero: no se puede fijar al crearlo, solo aumenta con entradas y disminuye con consumos de citas atendidas.
O-02	Asociar de nuevo el mismo insumo a un servicio actualiza la cantidad que consume; no se rechaza ni se duplica la fila.
O-03	El reporte de citas agrupa por estado (pendiente, confirmada, cancelada, atendida). Una reprogramación solo cambia fecha y hora de la misma cita, por lo que RF-REP-01 no puede contar citas "reprogramadas". Es una decisión de diseño documentada en el código.
O-04	El reporte de consumo de insumos agrupa por la fecha en que se registró el descuento (el momento de marcar la cita como atendida), no por la fecha de la cita.
O-05	Las cantidades de insumos admiten decimales y unidades como mililitros; el sistema descuenta directamente la cantidad configurada en cada servicio.

10. Limitaciones y riesgos

Área	Brecha	Riesgo	Siguiente paso
Frontend	Sin pruebas automáticas.	Errores de interfaz no detectados antes de la entrega.	Pruebas de componentes y recorrido de extremo a extremo del flujo de agendar.
Entorno de producción	La suite no corre contra Vercel y Supabase.	Diferencias de configuración (variables, SSL, migraciones sin aplicar) solo se ven al desplegar.	Lista de verificación posterior al despliegue y prueba de humo contra producción.
Correo real	SMTP simulado en las pruebas.	Credenciales faltantes o inválidas en producción (ya ocurrió).	Verificación manual del envío tras cada despliegue.
Rendimiento	Medido solo en local.	La latencia real con la nube puede ser mayor.	Repetir `test:carga` contra el despliegue.
Seguridad	Sin pruebas de penetración ni límite de intentos de inicio de sesión.	Fuerza bruta sobre el login; CORS abierto.	Limitar intentos y restringir orígenes; documentar en el plan de seguridad.

11. Criterios de aceptación y plan de mejora

11.1 Criterios para fusionar un cambio

- `npm test` debe terminar sin fallos en el flujo de CI del Pull Request.
- Todo cambio de una regla de negocio incluye la prueba correspondiente; todo defecto corregido incluye su prueba de regresión.
- El otro integrante del equipo revisa y aprueba el Pull Request (flujo GitFlow definido en la Fase 1).
- No se desactiva ni debilita la guarda que impide ejecutar las pruebas contra bases de datos que no terminen en `_test`.

11.2 Plan de mejora priorizado

Prioridad	Acción	Resultado esperado
1	Ejecutar `test:carga` y una prueba de humo contra el despliegue real.	Evidencia de RNF-REN en producción.
2	Pruebas del frontend (componentes y flujo de agendar).	Cobertura automática de la interfaz.

Prioridad	Acción	Resultado esperado
3	Limitar los intentos de inicio de sesión y restringir los orígenes permitidos (CORS), con sus pruebas.	Cerrar las debilidades de seguridad conocidas.
4	Actualizar las acciones del flujo de CI (`checkout` y `setup-node`).	Eliminar la advertencia de obsolescencia de Node 20 que GitHub muestra en cada ejecución.

Anexo A. Catálogo de casos de prueba

Todos los casos se ejecutan automáticamente con `npm test`. El resultado corresponde a la ejecución descrita en la sección 8.

Autenticación y roles

ID	Descripción del caso	Requisitos	Resultado
PI-001	Registra un cliente, no expone el hash y deja su perfil de cliente creado	RF-AUT-05, RF-CLI-01, RNF-SEG-01, UC-02	Superado
PI-002	El mismo correo no se puede registrar dos veces (409) y los datos inválidos dan 400 en español	RF-AUT-05, RNF-USA-03, UC-02	Superado
PI-003	Login correcto entrega token y perfil; /me lo reconoce; sin token o con token falso es 401	RF-AUT-01, RF-AUT-02, UC-01	Superado
PI-004	Contraseña incorrecta y correo inexistente dan el mismo 401	RF-AUT-01, UC-01	Superado
PI-005	Una cuenta desactivada no entra aunque la contraseña sea correcta	RF-AUT-01, RF-AUT-04, UC-01	Superado
PI-006	Con 2FA activo el login no da token: manda un código al correo y solo con ese código se entra	RF-AUT-07	Superado
PI-007	Manda el enlace, permite cambiar la contraseña una vez y la anterior deja de servir	RF-AUT-06, UC-03	Superado
PI-008	Con un correo que no existe responde igual y no manda ningún correo	RF-AUT-06, UC-03	Superado
PI-009	Un cliente no puede usar endpoints de administración	RF-AUT-03, RNF-SEG-02	Superado
PI-010	Sin sesión esas mismas rutas responden 401	RF-AUT-02, RF-AUT-03, RNF-SEG-02	Superado
PI-011	Un barbero tampoco administra inventario ni reportes	RF-AUT-03, RNF-SEG-02	Superado
PI-061	Activar el 2FA exige sesión, el campo `enabled` y la contraseña actual	RF-AUT-07, UC-01	Superado
PI-062	Con la contraseña correcta activa el 2FA y el siguiente inicio de sesión pide código; se puede desactivar	RF-AUT-07, UC-01	Superado
PU-001	Crea una cuenta con rol cliente y guarda la contraseña con hash, no en claro	RF-AUT-05, RNF-SEG-01	Superado
PU-002	Un correo ya registrado devuelve 409	RF-AUT-05, UC-02	Superado
PU-003	No distingue entre correo inexistente y contraseña incorrecta (mismo 401 y mismo mensaje)	RF-AUT-01, UC-01	Superado
PU-004	Una cuenta desactivada no puede entrar	RF-AUT-01, RF-AUT-04	Superado
PU-005	Sin 2FA devuelve un JWT firmado con id, correo y rol	RF-AUT-01, RF-AUT-02, RNF-SEG-04	Superado
PU-006	Con 2FA no entrega token: guarda el código con hash y lo manda por correo	RF-AUT-07	Superado
PU-007	Un barbero recibe además su barberoId en el perfil (distinto del id de usuario)	RF-AUT-03	Superado
PU-008	Un código incorrecto o vencido devuelve 401	RF-AUT-07	Superado
PU-009	Un código válido entrega el token y queda marcado como usado	RF-AUT-07	Superado
PU-010	Activar o desactivar el 2FA exige la contraseña actual	RF-AUT-07	Superado
PU-011	Responde igual exista o no el correo y, si no existe, no manda nada	RF-AUT-06, UC-03	Superado
PU-012	El enlace lleva un token cuyo hash es lo único que se guarda	RF-AUT-06, UC-03	Superado
PU-013	Un token inválido devuelve 400; uno válido cambia la contraseña y se marca como usado	RF-AUT-06, UC-03	Superado

ID	Descripción del caso	Requisitos	Resultado
PU-083	Sin encabezado, con esquema distinto de Bearer o con token basura responde 401	RF-AUT-02	Superado
PU-084	Rechaza un token firmado con otro secreto o ya vencido	RF-AUT-02, RNF-SEG-04	Superado
PU-085	Con un token válido deja pasar y expone el usuario en req.user	RF-AUT-02	Superado
PU-086	RequireRole: permite los roles de la lista, da 403 a otros y 401 si no hay sesión	RF-AUT-03, RNF-SEG-02	Superado

Clientes

ID	Descripción del caso	Requisitos	Resultado
PI-050	Solo el administrador entra: cliente y barbero reciben 403, sin sesión 401	RF-AUT-03, RNF-SEG-02	Superado
PI-051	El administrador registra un cliente sin cuenta con nombre, teléfono y correo opcionales	RF-CLI-01, UC-08	Superado
PI-052	Valida los datos: nombre obligatorio, correo con formato y campos desconocidos	RF-CLI-01, RNF-USA-03, UC-08	Superado
PI-053	Editar cambia solo los campos enviados y conserva el resto	RF-CLI-04, UC-11	Superado
PI-054	Un cliente inexistente da 404 al consultar, editar, activar o desactivar	RF-CLI-04, UC-11	Superado
PI-055	Busca por nombre o por teléfono sin distinguir mayúsculas	RF-CLI-03, UC-09	Superado
PI-056	Por defecto lista solo activos; "inactivos" y "todos" permiten encontrar a los desactivados	RF-CLI-03, RF-CLI-04, UC-09	Superado
PI-057	Desactivar y volver a activar cambia el estado y conserva los datos	RF-CLI-04, UC-11	Superado
PI-058	Muestra las citas del cliente con servicio, barbero y estado, la más reciente primero	RF-CLI-02, UC-10	Superado
PI-059	Un cliente sin citas tiene historial vacío y uno inexistente da 404	RF-CLI-02, UC-10	Superado

Citas

ID	Descripción del caso	Requisitos	Resultado
PI-026	Crea la cita confirmada, calcula la hora de fin y avisa por correo al cliente y al barbero	RF-CIT-01, RF-CIT-08, UC-04, UC-26	Superado
PI-027	El mismo horario ya no se puede reservar (409), pero uno que no se traslapa sí	RF-CIT-07	Superado
PI-028	Otro barbero sí está libre a la misma hora	RF-CIT-07	Superado
PI-029	Rechaza fechas pasadas, horas fuera de horario y el receso del almuerzo	RF-CIT-03, RF-CIT-04	Superado
PI-030	Valida el cuerpo: formato de hora, uuid y campos de más	RF-CIT-01, RNF-USA-03	Superado
PI-031	Servicio o barbero inexistentes dan 404	RF-CIT-01	Superado
PI-032	Solo el cliente agenda: barbero y administrador no pueden usar este endpoint	RF-CIT-01, RF-AUT-03, RNF-SEG-02	Superado
PI-036	Otro cliente no puede cancelar ni reprogramar una cita ajena (403)	RF-CIT-05, RF-CIT-06, RNF-SEG-02	Superado
PI-037	El dueño reprograma a otra hora libre y la cita conserva su estado	RF-CIT-05, UC-05	Superado
PI-038	Cancelar libera el horario para otro cliente y es idempotente	RF-CIT-06, RF-CIT-07, UC-06	Superado
PI-039	"mis citas" solo trae las del cliente que consulta	RF-CLI-02	Superado
PI-049	El barbero registra una cita sin cuenta y con nombre; el cliente no puede	UC-25	Superado
PI-089	Un barbero consulta su propia agenda, del día y por rango	RF-CIT-09, UC-07	Superado

ID	Descripción del caso	Requisitos	Resultado
PI-090	Un barbero no puede consultar la agenda ni las citas de otro barbero	RF-CIT-09, UC-07, RNF-SEG-02	Superado
PI-091	El administrador sigue viendo la agenda de cualquier barbero y el listado global	RF-CIT-09, UC-07	Superado
PI-092	Un barbero no puede marcar como atendida la cita de otro barbero	RF-INV-02, UC-16, RNF-SEG-02	Superado
PI-093	Un barbero sí marca como atendida su propia cita	RF-INV-02, UC-16	Superado
PU-034	Crea la cita confirmada con la hora de fin según la duración del servicio	RF-CIT-01	Superado
PU-035	Rechaza una fecha pasada sin consultar nada más	RF-CIT-01	Superado
PU-036	Un servicio o un barbero inactivo o inexistente devuelve 404	RF-CIT-01	Superado
PU-037	Respeta el horario y las ausencias del barbero: si assertAtiende falla, no se crea nada	RF-CIT-01	Superado
PU-038	Evita la doble reserva: horario solapado devuelve 409	RF-CIT-07	Superado
PU-039	Un usuario sin perfil de cliente no puede agendar	RF-CIT-01	Superado
PU-040	Avisa al cliente y también al barbero asignado	RF-CIT-08, UC-26	Superado
PU-041	El barbero se entera aunque el cliente no tenga correo	RF-CIT-08, UC-26	Superado
PU-050	Un cliente no puede cancelar la cita de otro cliente	RF-CIT-06, RNF-SEG-02	Superado
PU-051	Una cita atendida no se puede cancelar; el administrador sí puede cancelar una confirmada	RF-CIT-06, RNF-SEG-02	Superado

Barberos, agenda y horario

ID	Descripción del caso	Requisitos	Resultado
PI-016	Crea un barbero con su cuenta y, sin horario, le asigna el de la barbería (domingo a jueves, 9:00-19:00)	RF-AUT-04, RF-CIT-02, UC-22	Superado
PI-017	Con horario propio lo guarda tal cual y también puede reemplazarlo al editar	RF-AUT-04, UC-22, UC-23	Superado
PI-018	Rechaza un correo repetido (409), un horario inválido y datos incompletos	RF-AUT-04, UC-22, RNF-USA-03	Superado
PI-019	Desactivar saca al barbero del listado público pero el administrador aún lo ve	RF-AUT-04, UC-24	Superado
PI-020	Un barbero desactivado ya no se puede reservar (404)	RF-AUT-04, UC-24, RF-CIT-01	Superado
PI-021	Inexistente da 404 y solo el administrador gestiona barberos	RF-AUT-04, RF-AUT-03, RNF-SEG-02	Superado
PI-022	Consulta de un día: trae las citas de ese barbero ordenadas por hora, con cliente y servicio	RF-CIT-09, UC-07	Superado
PI-023	Listado por rango (semanal): incluye varios días y se puede filtrar por barbero	RF-CIT-09, UC-07	Superado
PI-024	Una cita cancelada sigue en la agenda pero marcada como cancelada	RF-CIT-09, UC-07	Superado
PI-025	Sin parámetros suficientes responde 400 y un cliente no puede ver la agenda (403)	RF-CIT-09, UC-07	Superado
PI-033	Un día marcado como ausente no se puede reservar y la disponibilidad lo refleja	Agenda del barbero*	Superado
PI-034	Un barbero no puede gestionar la agenda de otro, pero el administrador sí	Agenda del barbero*, RNF-SEG-02	Superado
PI-035	No deja marcar dos veces el mismo día (409)	Agenda del barbero*	Superado
PI-063	Devuelve el barbero con su horario semanal	RF-CIT-01	Superado
PI-064	El barbero lista sus ausencias y el administrador también; otro barbero recibe 403	Agenda del barbero*	Superado

ID	Descripción del caso	Requisitos	Resultado
PI-065	Quitar una ausencia responde 204, vuelve a habilitar el día y repetirlo da 404	Agenda del barbero*	Superado
PI-088	Un barbero desactivado por el administrador ya no puede iniciar sesión	RF-AUT-04, UC-24, RNF-SEG-02	Superado
PI-094	No expone correos ni identificadores de cuenta	RF-CIT-01, RNF-SEG-02	Superado
PI-095	El administrador sigue viendo el correo de cada barbero en su listado	RF-AUT-04, UC-22	Superado
PU-014	Sin horario usa el de la barbería: domingo a jueves, 9:00-19:00	RF-AUT-04, RF-CIT-02, UC-22	Superado
PU-015	Rechaza un bloque cuyo fin no es posterior al inicio, indicando cuál	RF-AUT-04, UC-22	Superado
PU-016	Rechaza bloques traslapados del mismo día, pero permite mañana y tarde seguidos	RF-AUT-04, UC-22	Superado
PU-017	Un correo ya registrado devuelve 409	RF-AUT-04, UC-22	Superado
PU-018	Con horario lo valida y lo manda al repositorio; sin horario no lo toca	RF-AUT-04, UC-23	Superado
PU-019	Un horario inválido no llega a guardarse	RF-AUT-04, UC-23	Superado
PU-020	Devuelve 404 si el barbero no existe	RF-AUT-04, UC-23	Superado
PU-021	Acepta una cita dentro del bloque, incluso pegada a los extremos	RF-CIT-04	Superado
PU-022	Rechaza una cita que empieza antes o termina después del bloque	RF-CIT-04	Superado
PU-023	Rechaza un día en que el barbero no atiende y nombra el día	RF-CIT-02, RF-CIT-04	Superado
PU-024	Rechaza un día marcado como ausencia, aunque el horario lo permita	Agenda del barbero*, RF-CIT-01	Superado
PU-025	Con dos bloques, no deja agendar en el hueco entre ambos	RF-CIT-03	Superado
PU-026	Rechaza una cita que cruza la medianoche	RF-CIT-04	Superado
PU-027	Solo el administrador o el propio barbero pueden gestionar la agenda	Agenda del barbero*	Superado
PU-028	No deja marcar un día que ya pasó	Agenda del barbero*	Superado
PU-029	No cancela citas por detrás: con citas ese día responde 409 con la cantidad	Agenda del barbero*	Superado
PU-030	Un día ya marcado devuelve 409	Agenda del barbero*	Superado
PU-031	Quitar una ausencia que no existe devuelve 404	Agenda del barbero*	Superado
PU-032	Un día de ausencia no ofrece horas: trabaja=false, ausente=true, sin bloques	RF-CIT-01, RF-CIT-02, Agenda del barbero*	Superado
PU-033	Devuelve los bloques del día y las horas ocupadas por citas, ordenadas	RF-CIT-01, RF-CIT-02, Agenda del barbero*	Superado
PU-047	Un barbero consulta su propia agenda, del día o por rango	RF-CIT-09, UC-07, RNF-SEG-02	Superado
PU-048	Un barbero no puede ver la agenda de otro ni el listado de todos	RF-CIT-09, UC-07, RNF-SEG-02	Superado
PU-049	El administrador consulta la agenda de cualquiera y el listado global	RF-CIT-09, UC-07, RNF-SEG-02	Superado
PU-093	Acepta un horario válido y también que no venga (se usa el de la barbería)	RF-AUT-04, UC-22, UC-23	Superado
PU-094	Rechaza hora con formato inválido con un mensaje que señala el campo	RF-AUT-04, UC-22, UC-23	Superado
PU-095	Rechaza hora fuera de rango con un mensaje que señala el campo	RF-AUT-04, UC-22, UC-23	Superado
PU-096	Rechaza día fuera de 0-6 con un mensaje que señala el campo	RF-AUT-04, UC-22, UC-23	Superado
PU-097	Rechaza día no entero con un mensaje que señala el campo	RF-AUT-04, UC-22, UC-23	Superado
PU-098	Rechaza un bloque incompleto y un horario vacío	RF-AUT-04, UC-22, UC-23	Superado
PU-099	Una ausencia exige una fecha AAAA-MM-DD	Agenda del barbero*	Superado

Atención de cita e inventario automático

ID	Descripción del caso	Requisitos	Resultado
PI-040	Descuenta la cantidad configurada, deja el movimiento ligado a la cita y no vuelve a descontar	RF-INV-02, UC-16	Superado
PI-041	También descuenta cantidades en ml o decimales: 30 ml de un stock de 500 dejan 470	RF-INV-02, UC-16	Superado
PI-042	Un servicio con varios insumos los descuenta todos	RF-INV-02, UC-16	Superado
PI-043	Si el stock queda por debajo del mínimo, avisa por correo a los administradores (después de confirmar)	RF-INV-04, UC-17	Superado
PI-044	Sin quedar por debajo del mínimo no hay alerta	RF-INV-04, UC-17	Superado
PI-045	Es una transacción: si el descuento falla la cita sigue confirmada y el stock intacto	RF-INV-02, UC-16	Superado
PI-046	Una cita cancelada no se puede marcar como atendida y no descuenta nada	RF-INV-02, RF-CIT-06	Superado
PI-047	Un cliente no puede marcar citas como atendidas	RF-AUT-03, RNF-SEG-02	Superado
PI-048	Una cita atendida ya no se puede cancelar	RF-CIT-06	Superado
PU-042	Marca la cita y descuenta dentro de la misma transacción, y avisa solo después del COMMIT	RF-INV-02, UC-16	Superado
PU-043	Si el descuento falla se revierte todo, se libera la conexión y no se avisa nada	RF-INV-02, UC-16	Superado
PU-044	Una cita ya atendida no vuelve a descontar (ni abre transacción)	RF-INV-02, UC-16	Superado
PU-045	Una cita cancelada no se puede marcar como atendida; una inexistente da 404	RF-INV-02, UC-16	Superado
PU-046	Un barbero solo marca como atendidas sus propias citas	RF-INV-02, UC-16, RNF-SEG-02	Superado
PU-067	Descuenta cada insumo del servicio con la cantidad configurada y ligado a la cita	RF-INV-02, RF-INV-04, UC-16	Superado
PU-068	Un servicio sin insumos asociados no descuenta nada ni genera alertas	RF-INV-02, RF-INV-04, UC-16	Superado
PU-069	Devuelve alerta solo de los insumos que quedaron por debajo del mínimo	RF-INV-02, RF-INV-04, UC-16	Superado
PU-070	Un insumo sin mínimo configurado nunca genera alerta, aunque llegue a cero o negativo	RF-INV-02, RF-INV-04, UC-16	Superado
PU-071	No hace nada si no hay alertas (ni siquiera consulta a los administradores)	RF-INV-04, UC-17	Superado
PU-072	Avisa a cada administrador activo por cada insumo con stock bajo	RF-INV-04, UC-17	Superado

Inventario y servicios

ID	Descripción del caso	Requisitos	Resultado
PI-012	El administrador crea un servicio y queda visible en el catálogo público	RF-SER-01, RF-SER-03, UC-12	Superado
PI-013	Valida los datos con mensajes en español: obligatorios, duración entera y precio no negativo	RF-SER-01, RNF-USA-03, UC-12	Superado
PI-014	Editar cambia solo lo enviado; el administrador inexistente da 404	RF-SER-01, UC-12	Superado
PI-015	Cliente y barbero no pueden crear ni editar	RF-SER-01, RNF-SEG-02	Superado
PI-066	El administrador lista los insumos y encuentra el que acaba de crear	RF-INV-01, UC-14	Superado
PI-067	Editar cambia solo lo enviado y un insumo inexistente da 404	RF-INV-01, RF-INV-04	Superado
PI-068	Quita el insumo del servicio (204), no toca el insumo y repetirlo da 404; solo el administrador	RF-SER-02, UC-13	Superado

ID	Descripción del caso	Requisitos	Resultado
PI-069	Crea un insumo en cero; el stock entra después con una entrada	RF-INV-01, RF-INV-03, UC-14	Superado
PI-070	El stock no se puede fijar directamente al crear: solo entradas y consumos lo mueven	RF-INV-01	Superado
PI-071	Rechaza entradas en cero o negativas y mínimos negativos, sin tocar el stock	RF-INV-03, RF-INV-04	Superado
PI-072	Una entrada suma la cantidad (también decimales) y queda en el historial de movimientos	RF-INV-03, UC-15	Superado
PI-073	Un insumo inexistente da 404	RF-INV-03	Superado
PI-074	El administrador asocia un insumo con una cantidad positiva y no con cero ni negativa	RF-SER-02, UC-13	Superado
PI-075	Asociar otra vez el mismo insumo actualiza la cantidad en vez de duplicar la fila	RF-SER-02, UC-13	Superado
PI-076	Solo el administrador lo configura	RF-SER-02, RNF-SEG-02	Superado
PI-077	Es público para leer; crear, editar y desactivar exige administrador	RF-SER-01, RF-SER-03, RNF-SEG-02	Superado
PI-078	Un servicio desactivado desaparece del catálogo público	RF-SER-01, RF-SER-03, UC-12	Superado
PI-101	Cantidad_minima null quita el mínimo; omitirla lo conserva; un número lo cambia	RF-INV-01, RF-INV-04, UC-14	Superado
PU-089	Las entradas son números mayores que cero (admiten decimales, p. ej. ml)	RF-INV-03	Superado
PU-090	El mínimo de un insumo es >= 0 (o null / ausente para no tener alerta)	RF-INV-04	Superado
PU-091	Un insumo exige nombre y unidad de medida, y no acepta el stock al crearlo	RF-INV-01	Superado
PU-092	Un servicio consume una cantidad mayor que cero de cada insumo	RF-SER-02	Superado

Reportes

ID	Descripción del caso	Requisitos	Resultado
PI-079	Cada tipo se exporta en Excel y PDF con el tipo de contenido y el nombre de archivo correctos	RF-REP-04, UC-18	Superado
PI-080	El reporte de citas refleja las citas reales del periodo y el de insumos, la cantidad consumida	RF-REP-01, RF-REP-02, RF-REP-03	Superado
PI-081	Un periodo sin datos igual descarga el archivo	RF-REP-04	Superado
PI-082	Valida los parámetros: rango invertido, tipo y formato desconocidos	RF-REP-01, RF-REP-02, RF-REP-03, RNF-USA-03	Superado
PU-073	Reporte de citas: título con la marca, resumen, encabezados y datos reales ordenables	RF-REP-01, RF-REP-04	Superado
PU-074	Reporte de ingresos: una hoja por servicio y otra por barbero, con totales numéricos	RF-REP-02, RF-REP-04	Superado
PU-075	Reporte de insumos: cantidad consumida como número real	RF-REP-03, RF-REP-04	Superado
PU-076	Un periodo sin datos muestra el aviso en vez de una tabla vacía	RF-REP-04	Superado
PU-077	Genera un PDF válido con la marca en los metadatos	RF-REP-04	Superado
PU-078	Con muchas filas pagina en vez de cortar la tabla	RF-REP-04	Superado
PU-079	Un periodo sin datos igual produce un PDF válido	RF-REP-04	Superado
PU-100	Acepta cualquier combinación de tipo y formato válidos	RF-REP-01, RF-REP-02, RF-REP-03	Superado

ID	Descripción del caso	Requisitos	Resultado
PU-101	Rechaza tipo o formato desconocidos y fechas mal escritas	RF-REP-01, RF-REP-02, RF-REP-03	Superado

Correo y notificaciones

ID	Descripción del caso	Requisitos	Resultado
PI-083	Exige sesión	RF-CIT-08, UC-26, RNF-SEG-02	Superado
PI-084	Agendar deja una notificación sin leer para el cliente y otra para el barbero	RF-CIT-08, UC-26	Superado
PI-085	Cada usuario solo ve las suyas	RF-CIT-08, UC-26	Superado
PI-086	Marcar como leída cambia el estado y no afecta a las demás	RF-CIT-08, UC-26	Superado
PI-087	No se puede marcar una notificación ajena (403) ni una inexistente (404)	RF-CIT-08, UC-26	Superado
PU-052	Dos_factor usa "Barber Kongs" y nunca la marca vieja	RF-AUT-07	Superado
PU-053	Recuperacion_password usa "Barber Kongs" y nunca la marca vieja	RF-AUT-06	Superado
PU-054	Confirmacion_cita usa "Barber Kongs" y nunca la marca vieja	RF-CIT-08	Superado
PU-055	Reprogramacion_cita usa "Barber Kongs" y nunca la marca vieja	RF-CIT-08	Superado
PU-056	Cancelacion_cita usa "Barber Kongs" y nunca la marca vieja	RF-CIT-08	Superado
PU-057	Alerta_inventario usa "Barber Kongs" y nunca la marca vieja	RF-INV-04	Superado
PU-058	Muestra el código de 2FA y su vencimiento	RF-AUT-07	Superado
PU-059	Arma el detalle de la cita con fecha larga y hora de 12 horas	RF-CIT-08	Superado
PU-060	El aviso al barbero muestra el cliente y no repite al barbero; los textos se pueden reemplazar	RF-CIT-08	Superado
PU-061	Sin `datos` igual sale presentable con el mensaje	RF-CIT-08	Superado
PU-062	Escapa el HTML de los valores dinámicos	Transversal	Superado
PU-063	No pone como botón un enlace que no sea http(s)	Transversal	Superado
PU-064	FormatFecha no depende de la zona horaria (fecha solo-día)	RF-CIT-08	Superado
PU-065	FormatHora pasa a 12 horas con a. m./p. m.	RF-CIT-08	Superado
PU-066	Devuelve el valor tal cual si no se puede interpretar	RF-CIT-08	Superado

Transversal (errores y validación)

ID	Descripción del caso	Requisitos	Resultado
PI-060	Responde 200 con el estado del servicio, sin sesión	RNF-DIS-01	Superado
PI-096	Un id que no es UUID responde 400 con mensaje en español, en todos los módulos	Transversal, RNF-USA-03	Superado
PI-097	Un UUID válido que no existe sigue dando 404	Transversal, RNF-USA-03	Superado
PI-098	Las rutas fijas que comparten prefijo con :id no se confunden con un id	Transversal, RNF-USA-03	Superado
PI-099	Fechas o ids inválidos responden 400 con mensaje en español	RF-CIT-09, UC-07, RNF-USA-03	Superado
PI-100	Una consulta válida sigue funcionando y sin parámetros pide cuáles usar	RF-CIT-09, UC-07, RNF-USA-03	Superado
PU-080	Cada atajo trae su código HTTP	Transversal	Superado
PU-081	Muestra al usuario el mensaje de los errores 4xx	RNF-USA-03	Superado
PU-082	En un 500 nunca filtra el detalle interno	Transversal	Superado

ID	Descripción del caso	Requisitos	Resultado
PU-087	Devuelve los datos válidos y lanza ApiError 400 con el mensaje si no lo son	Transversal	Superado
PU-088	Rechaza campos que el schema no conoce (additionalProperties: false)	Transversal	Superado

Documentación para Escalabilidad Futura

Campo	Detalle
Proyecto	Barber Kongs — Sistema de Gestión para Barbería (Fase 3)
Equipo	Miguel Angel Quemé Gómez · Brandon Gustavo Güinac Román
Versión del documento	1.0 — 25 de octubre de 2026
Contenido	Procesos futuros identificados y diagramados (BPMN), migraciones necesarias y tecnologías nuevas con su justificación
Archivos editables	Carpeta `bpmn/escalabilidad/`: 6 procesos y un consolidado, para abrir en Camunda Modeler
Validación	Los 7 archivos BPMN se cargaron en bpmn-js sin advertencias; las 11 migraciones propuestas se aplicaron en PostgreSQL 16 sobre las migraciones 0001 a 0006, se repitieron para comprobar que son idempotentes y se probó cada regla con datos

1. Declaración de alcance

Declaración de alcance. El sistema se entrega para un negocio que hoy opera con un barbero y está construido para crecer a varios sin cambiar código: dar de alta un barbero es una operación de la pantalla de administración, y cada barbero tiene su propio horario y sus propios días no disponibles. Los barberos de ejemplo del repositorio son datos de demostración. Lo que sí requiere desarrollo es lo que describe este documento: reasignar citas, cobrar en línea, liquidar comisiones, recordar y anotar en lista de espera, y abrir sucursales.

2. Punto de partida

2.1 Lo que ya escala

- **Alta de barberos sin tocar código:** la tabla de barberos, el horario por barbero y las ausencias ya existen y se gestionan desde la pantalla de administración.
- **API sin estado en funciones serverless:** cada solicitud es independiente, por lo que el plan de Vercel absorbe más tráfico sin cambios.
- **Rendimiento medido:** con 20 usuarios simultáneos la consulta de disponibilidad tuvo un p95 de 10,7 ms y no hubo errores en 48 905 solicitudes, en local (RNF-REN-01 y RNF-REN-02).
- **Evolución controlada del modelo de datos:** migraciones versionadas con checksum, seguridad por filas (RLS) y una suite de 202 pruebas automáticas que corre en cada cambio.

2.2 Lo que limita el crecimiento hoy

Al analizar el código y al probarlo aparecieron límites que conviene resolver antes o durante el crecimiento. Los identificadores OE-xx corresponden a las observaciones del Inventario de Endpoints.

Limitación	Evidencia	Dónde se resuelve
Correo dentro de la petición	El envío espera la respuesta del servidor SMTP (hasta 5 s por intento) antes de responder al usuario, y se usa una cuenta de correo personal con límites diarios.	Cola de notificaciones (0016) y servicio de correo transaccional.
Disponibilidad sin caché	Cada consulta hace 4 consultas SQL, por barbero y por fecha. Hoy no es un problema: p95 de 10,7 ms con 20 usuarios simultáneos en local.	Sin cambios hasta que la carga lo exija; entonces, caché.
Listados sin paginación (OE-02)	Clientes, servicios, insumos, notificaciones y citas por rango devuelven todos los registros.	Paginación en la API cuando el historial crezca.

Limitación	Evidencia	Dónde se resuelve
Sin límite de intentos y CORS abierto (OE-03)	Inicio de sesión, verificación del 2FA y recuperación de contraseña no limitan intentos; la API acepta cualquier origen.	Limitación de tasa con Redis y restringir CORS.
Un solo negocio	No existe el concepto de sucursal: servicios, precios, inventario y horario son globales, y todos los barberos hacen todos los servicios.	Migraciones 0008, 0009 y 0010.
Infraestructura de una región y una base	Plan gratuito de la base (conexiones, almacenamiento y respaldos limitados), una conexión por función serverless (`DB_POOL_MAX=1`) y una sola base como punto de falla.	Plan de pago de la base y réplicas de lectura si hay reportes pesados.

3. Procesos futuros identificados y diagramados

3.1 Criterios y convenciones

Se identificaron los procesos que el crecimiento hace necesarios y que hoy no existen o son manuales. Están diagramados con la misma notación que los 21 procesos actuales del proyecto: un pool por proceso, carriles por actor, eventos de inicio (verde) y de fin (rojo), compuertas exclusivas (amarillo) y eventos de mensaje (azul). Además se distinguen las tareas de una persona (icono de persona) de las que ejecuta el sistema solo (engranaje), y se usa un evento de temporizador cuando el proceso lo dispara el tiempo.

Cada diagrama incluye una anotación de trazabilidad con las migraciones y tecnologías que lo habilitan. Los archivos están en `bpmn/escalabilidad/` y se abren en Camunda Modeler.

3.2 Resumen

#	Proceso	Disparador	Qué resuelve	Archivo
1	Reasignar las citas de un barbero que se ausenta o se da de baja	El administrador registra una ausencia o desactiva a un barbero	Evita citas confirmadas con un barbero que ya no atiende	reasignar-citas.bpmn
2	Pago en línea de la cita (anticipo)	El cliente elige pagar al confirmar	Reduce las inasistencias y habilita el cobro anticipado	pago-en-linea.bpmn
3	Comisiones y liquidación de barberos	Cierre del periodo (quincenal o mensual)	Calcula y paga lo que corresponde a cada barbero sin hojas de cálculo	comisiones-y-liquidacion.bpmn
4	Recordatorios automáticos de cita	Temporizador cada hora	Reduce las inasistencias y libera horarios a tiempo	recordatorios-automaticos.bpmn
5	Lista de espera para horarios liberados	Se libera un horario	Aprovecha las cancelaciones en lugar de perderlas	lista-de-espera.bpmn
6	Alta de una nueva sucursal	El negocio abre una sucursal	Permite crecer a varias sedes con catálogo e inventario propios	alta-de-sucursal.bpmn

3.3 Descripción de cada proceso

3.3.1 Reasignar las citas de un barbero que se ausenta o se da de baja

Aspecto	Detalle
Disparador	Un barbero se ausenta o se da de baja (lo registra el administrador).
Actores	Administrador, Sistema y Cliente.
Resultado	Cada cita afectada se reasigna a otro barbero con el consentimiento del cliente, o queda marcada para que el cliente la reprograme o la cancele. Nunca queda una cita confirmada con un barbero inactivo.
Reglas clave	Solo se propone un barbero activo que haga ese servicio, esté libre a esa fecha y hora y no tenga una ausencia. El cliente debe aceptar. Se guarda de qué barbero viene la cita y se avisa a ambos barberos y al cliente.
Sistema actual	Cierra el hueco del UC-24 (desactivar un barbero con citas futuras) y completa las ausencias del barbero; reutiliza los avisos de cancelación y reprogramación de RF-CIT-08.
Migraciones	0009 (servicios por barbero), 0013 (trazabilidad de citas) y 0016 (cola de notificaciones).
Tecnologías	Cola de notificaciones (outbox) y correo transaccional.
Diagrama	Figura 1 (sección 3.4)

3.3.2 Pago en línea de la cita (anticipo)

Aspecto	Detalle
Disparador	El cliente confirma su cita y elige pagar en línea.
Actores	Cliente, Sistema y Pasarela de pago (servicio externo).
Resultado	Cita confirmada con comprobante, o pago fallido con el horario liberado.
Reglas clave	El horario se retiene mientras dura el pago y vence si no se completa. El aviso de la pasarela (webhook) se valida por su firma y se procesa una sola vez. El sistema nunca almacena datos de tarjeta.
Sistema actual	Extiende el UC-04. Hoy no hay cobro: el precio solo alimenta el reporte de ingresos estimados. El estado `pendiente` de las citas, que el esquema permite y la aplicación no usa (OD-02), serviría para retener el horario.
Migraciones	0014 (pagos y eventos de la pasarela) y 0018 (auditoría).
Tecnologías	Pasarela de pago con webhooks firmados y correo transaccional.
Diagrama	Figura 2 (sección 3.4)

3.3.3 Comisiones y liquidación de barberos

Aspecto	Detalle
Disparador	Cierre del periodo (temporizador).
Actores	Sistema, Administrador y Barbero.
Resultado	Liquidación cerrada por barbero, comprobante en PDF y pago registrado.
Reglas clave	Solo cuentan las citas atendidas. Una cita se liquida una sola vez. Se aplica la comisión vigente a la fecha de la cita. El administrador puede ajustar con un motivo, que queda en la auditoría; una liquidación cerrada no se edita.
Sistema actual	Reutiliza el estado `atendida` y el precio del servicio, y extiende el UC-18 con reportes de comisiones.
Migraciones	0015 (comisiones y liquidaciones) y 0018 (auditoría).
Tecnologías	Tarea programada (Vercel Cron: una ejecución diaria alcanza) y la generación de PDF que ya existe en reportes.
Diagrama	Figura 3 (sección 3.4)

3.3.4 Recordatorios automáticos de cita

Aspecto	Detalle
Disparador	Temporizador cada hora.
Actores	Sistema y Cliente.
Resultado	El cliente recibe un recordatorio 24 horas antes; si cancela, el horario se libera y activa la lista de espera.
Reglas clave	Un solo recordatorio por cita (queda marcado). Solo para citas confirmadas. La cancelación aplica la misma regla que el UC-06.
Sistema actual	Usa las notificaciones del UC-26 y reduce las inasistencias.
Migraciones	0013 (marca de recordatorio enviado) y 0016 (cola de notificaciones).
Tecnologías	pg_cron y pg_net de Supabase, patrón outbox, Amazon SES o Resend y, opcionalmente, WhatsApp. El plan Hobby de Vercel solo permite una tarea programada por día, por eso no sirve para tareas cada hora.
Diagrama	Figura 4 (sección 3.4)

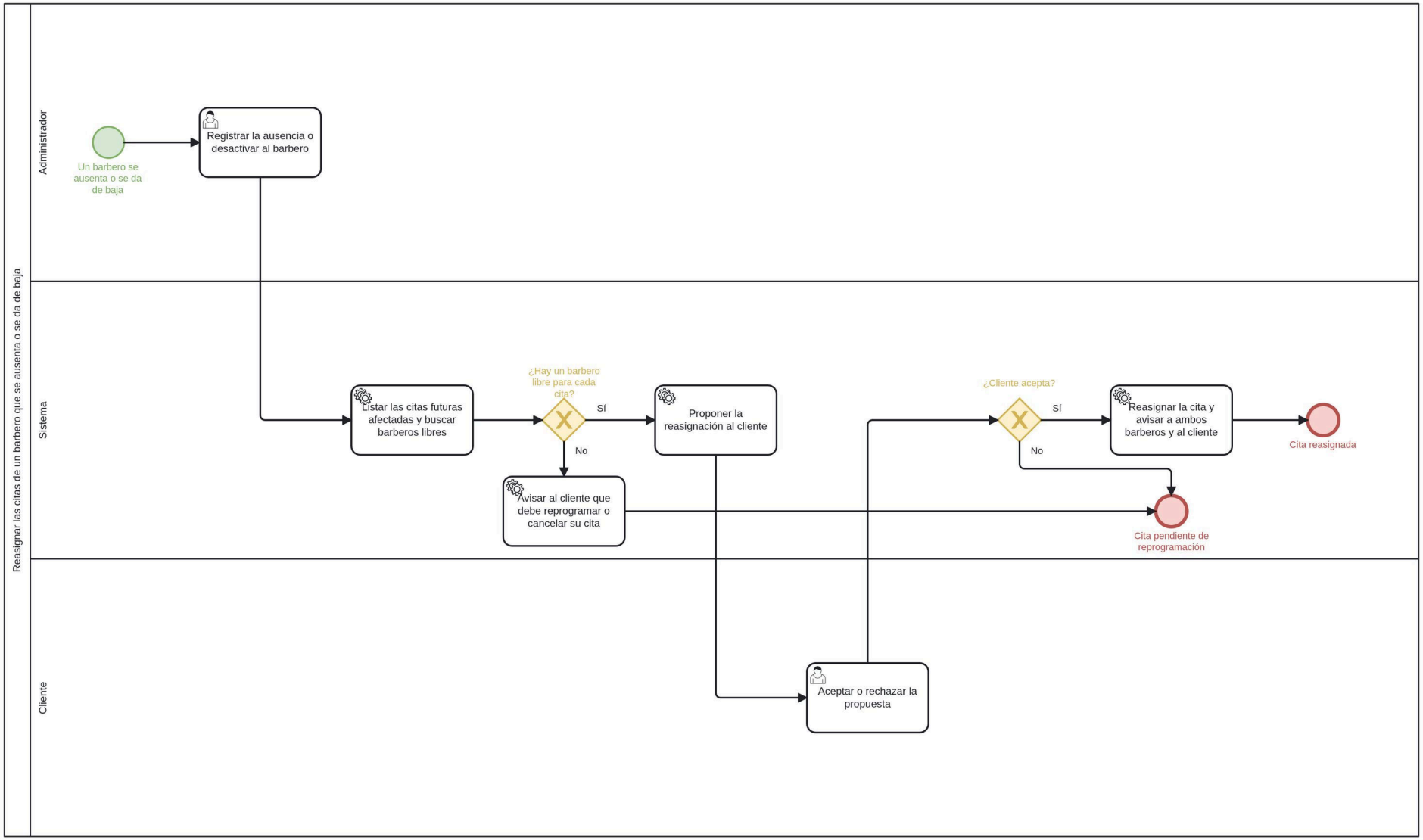
3.3.5 Lista de espera para horarios liberados

Aspecto	Detalle
Disparador	Se libera un horario (cita cancelada o barbero que vuelve a estar disponible).
Actores	Sistema y Cliente en espera.
Resultado	El horario se ofrece por orden de llegada con una vigencia de 15 minutos; se reasigna o se libera para todos.
Reglas clave	Compatibilidad por servicio, fecha, rango horario y barbero (o cualquiera). Una oferta a la vez; si vence o se rechaza, pasa a la siguiente solicitud.
Sistema actual	La solicitud se registraría desde la pantalla de agendar cuando no hay horario disponible.
Migraciones	0017 (lista de espera) y 0016 (cola de notificaciones).
Tecnologías	Patrón outbox y pg_cron para vencer las ofertas.
Diagrama	Figura 5 (sección 3.4)

3.3.6 Alta de una nueva sucursal

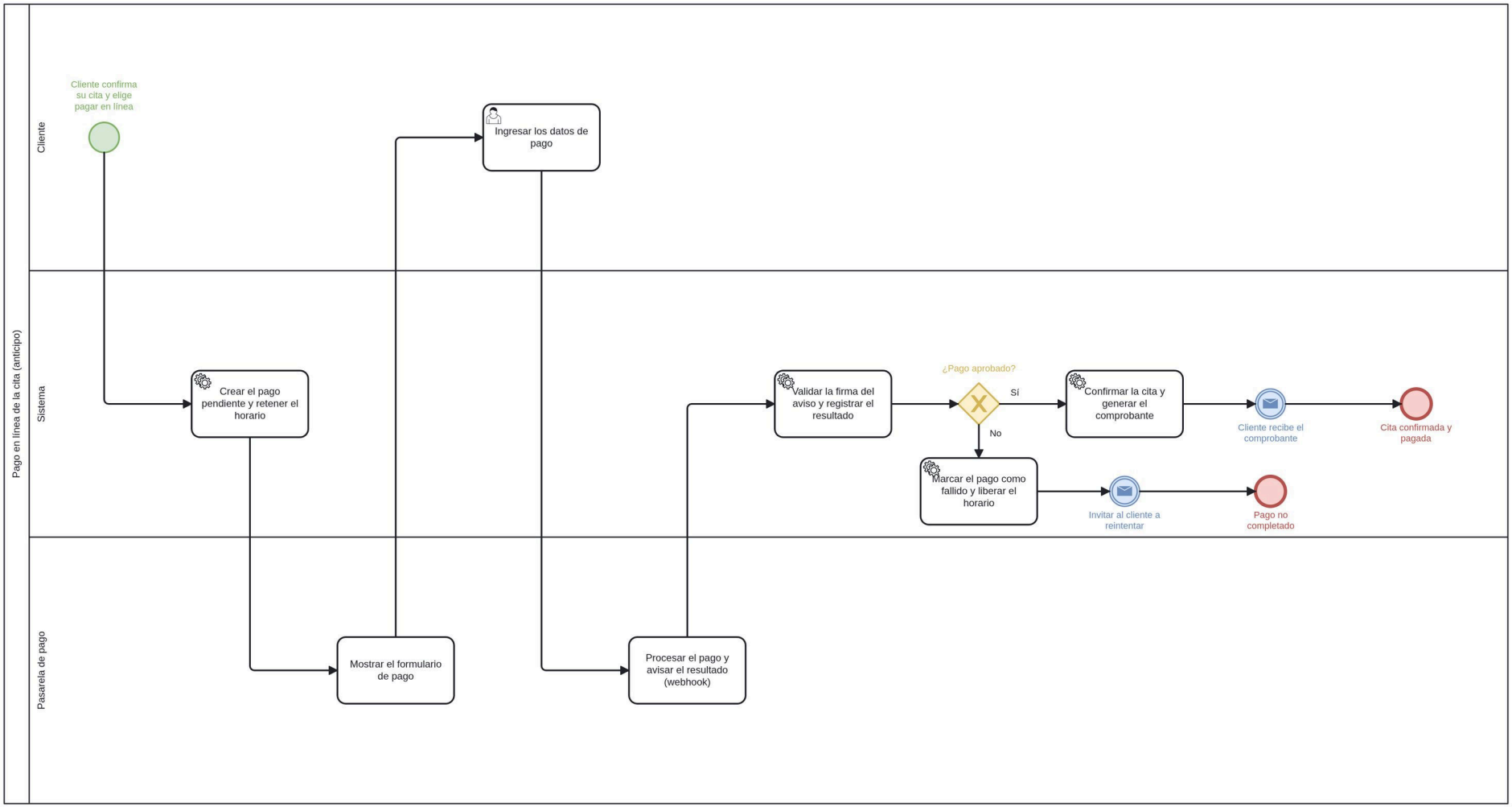
Aspecto	Detalle
Disparador	El negocio abre una nueva sucursal.
Actores	Administrador general, Sistema y Encargado de sucursal.
Resultado	Sucursal operando con sus barberos, servicios, precios e inventario propios.
Reglas clave	Cada sucursal tiene su catálogo y sus precios; cada barbero pertenece a una sucursal; el encargado solo ve su sucursal. Los reportes se pueden filtrar por sucursal o consolidar.
Sistema actual	Hoy todo es de un solo negocio. Requiere el rol nuevo `encargado` y la sucursal en barberos, citas e inventario.
Migraciones	0008 (sucursales y rol de encargado), 0010 (servicios y precio por sucursal) y 0009 (servicios por barbero).
Tecnologías	Una sola base con `sucursal_id` y seguridad por filas (RLS) por sucursal.
Diagrama	Figura 6 (sección 3.4)

3.4 Diagramas BPMN de los procesos futuros



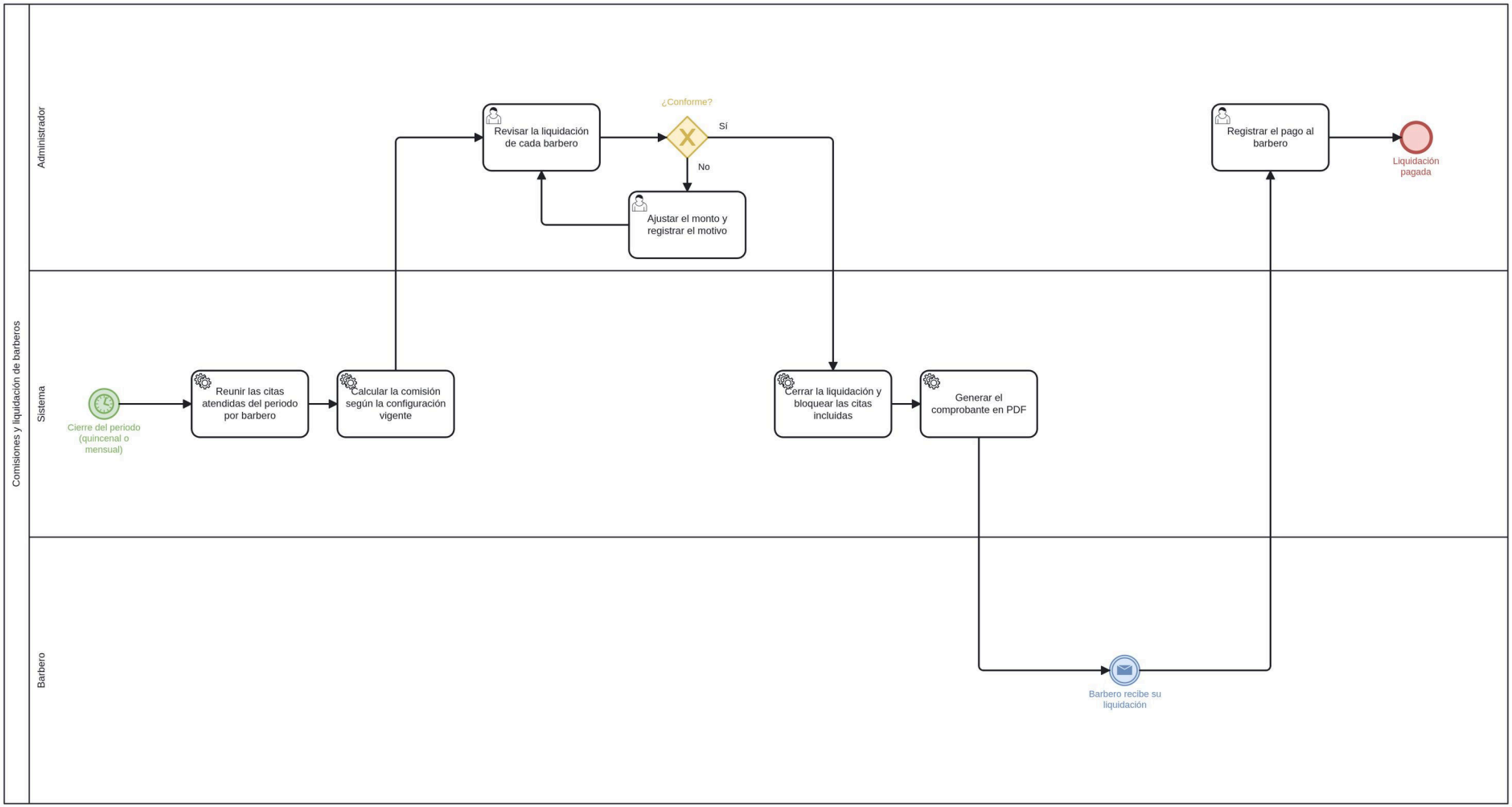
Trazabilidad: proceso futuro. Resuelve el hueco del UC-24 (desactivar barbero con citas futuras) y de las ausencias del barbero. Migraciones: 0013 (trazabilidad de citas) y 0009 (servicios por barbero). Tecnologías: cola de correos (outbox) y Resend/SES.

Figura 1. Reasignar las citas de un barbero que se ausenta o se da de baja.



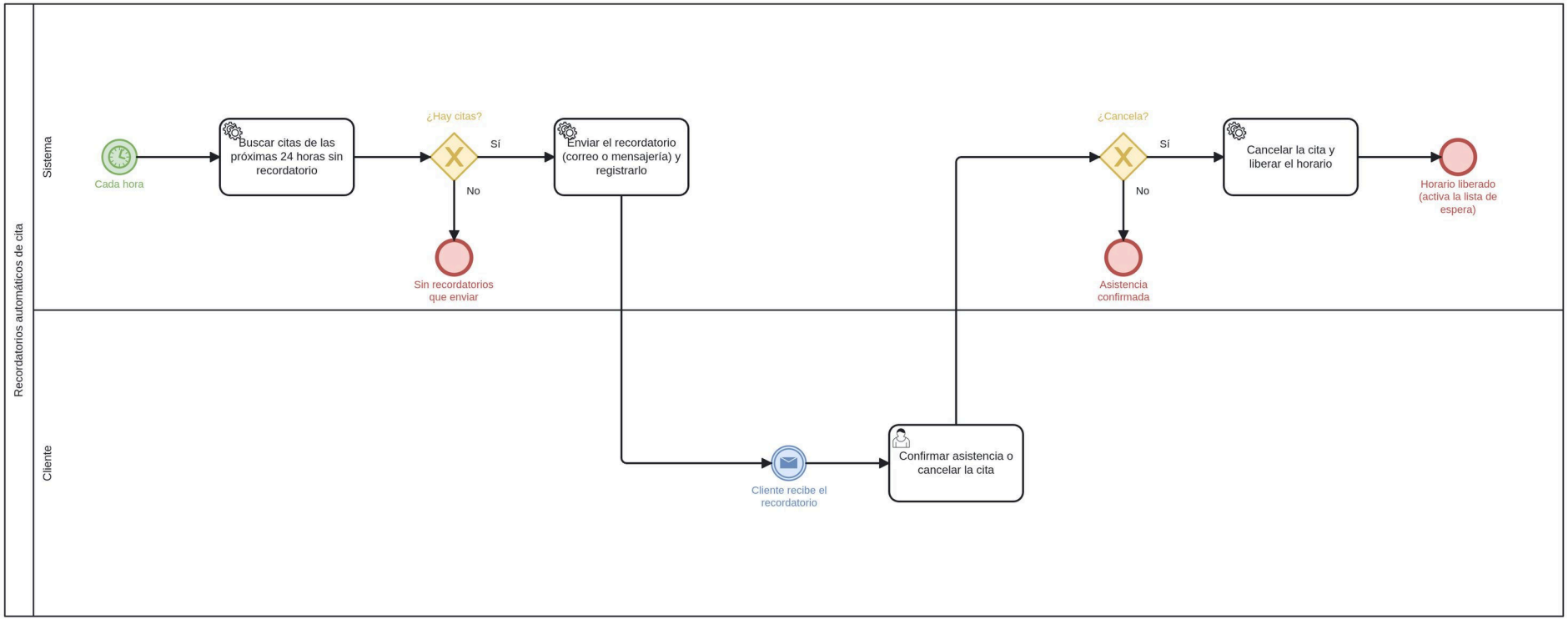
Trazabilidad: proceso futuro (pagos). Migración 0014 (pagos y eventos de webhook con idempotencia). Tecnologías: pasarela de pago con webhooks firmados; el sistema nunca almacena datos de tarjeta.

Figura 2. Pago en línea de la cita (anticipo).



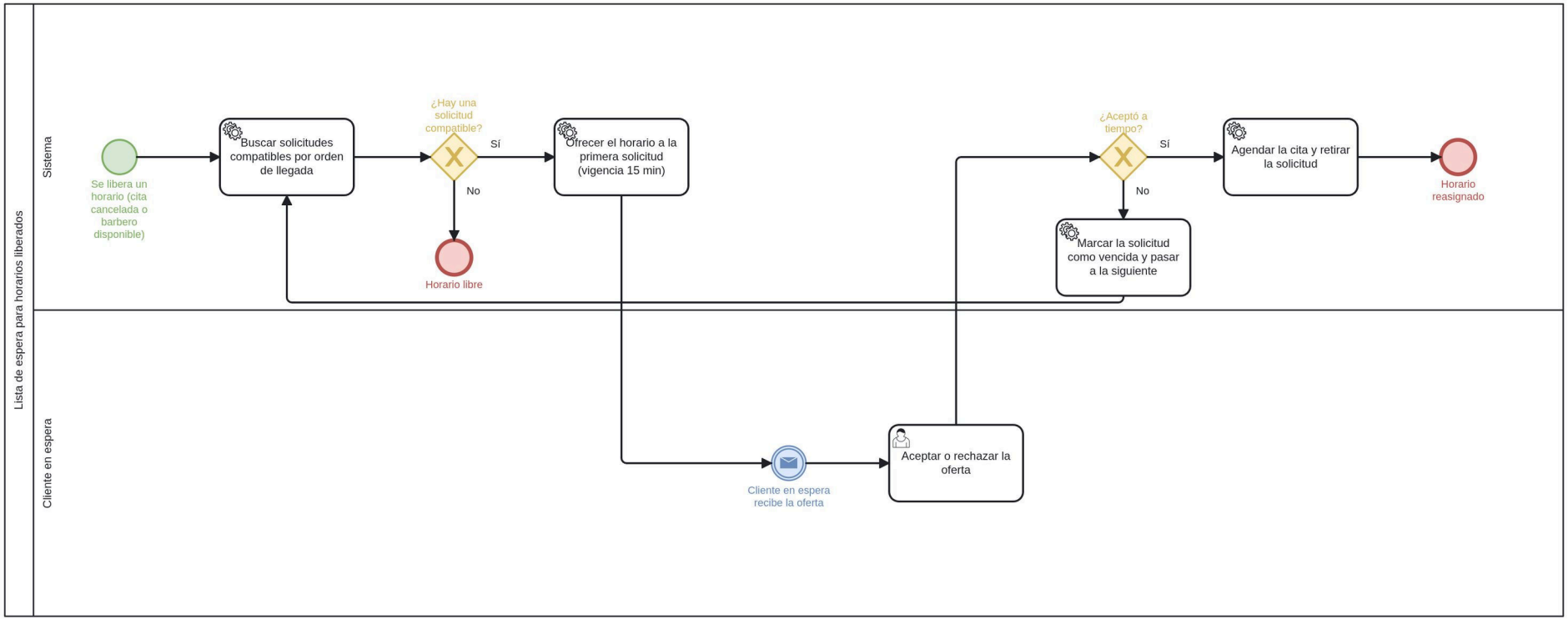
Trazabilidad: proceso futuro (comisiones). Migración 0015 (configuración de comisión, liquidaciones y detalle con una sola liquidación por cita). Tecnologías: tarea programada (Vercel Cron; una ejecución diaria alcanza) y generación de PDF ya existente en el módulo de reportes.

Figura 3. Comisiones y liquidación de barberos.



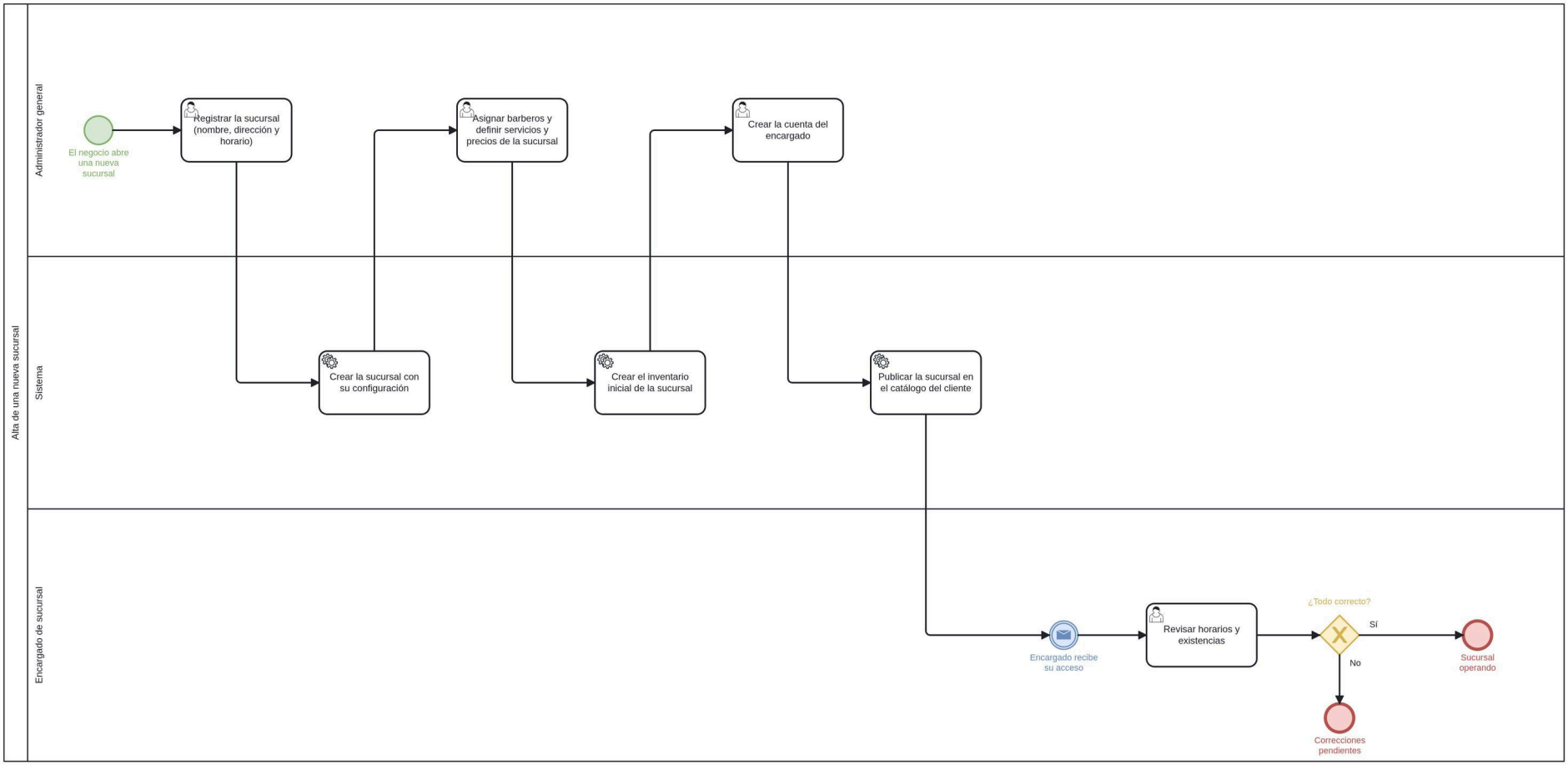
Trazabilidad: proceso futuro. Reduce las inasistencias. Migraciones: 0013 (recordatorio_enviado_at en citas) y 0016 (cola de notificaciones). Tecnologías: pg_cron + pg_net de Supabase (Vercel Cron en plan Hobby solo permite una ejecución diaria), patrón outbox, Amazon SES o Resend y, opcionalmente, WhatsApp.

Figura 4. Recordatorios automáticos de cita.



Trazabilidad: proceso futuro. La solicitud se registra desde la pantalla de agendar cuando no hay horario. Migraciones 0017 (lista_espera) y 0016 (cola de notificaciones). Tecnologías: patrón outbox y pg_cron para vencer las ofertas.

Figura 5. Lista de espera para horarios liberados.



Trazabilidad: proceso futuro (multi-sucursal). Migraciones: 0008 (sucursales y sucursal_id) y 0010 (servicios por sucursal). Decisión: una sola base con sucursal_id y RLS por sucursal, en lugar de una base por sucursal.

Figura 6. Alta de una nueva sucursal.

3.5 Impacto sobre los procesos actuales

Los procesos futuros modifican varios de los 21 diagramas existentes, que se conservan tal como se entregaron en las fases anteriores. Cambios previstos:

Proceso actual	Qué cambia
UC-04 Agendar cita	El cliente elige sucursal y solo ve barberos que hacen el servicio, con su duración y precio; puede pagar en línea o anotarse en la lista de espera si no hay horario.
UC-05 Reprogramar y UC-06 Cancelar	Registran quién cancela y por qué, avisan siempre al barbero y al cliente, y al liberar el horario activan la lista de espera.
UC-16 Descontar insumos	La cita atendida pasa a ser también la base de la liquidación del barbero.
UC-22 a UC-24 Gestión de barberos	Al crear un barbero se le asignan sucursal y servicios; al desactivarlo se dispara la reasignación de sus citas futuras.
UC-12 y UC-13 Servicios	El precio puede variar por sucursal y por barbero.
UC-14 y UC-15 Inventario	El inventario pasa a ser por sucursal.
UC-18 Reportes	Filtro por sucursal y nuevos reportes de pagos y comisiones.
UC-26 Notificaciones	Se alimentan de la cola de notificaciones en lugar de enviarse dentro de la petición.

4. Migraciones necesarias

4.1 Principios

- **Mismas reglas que las migraciones actuales:** un archivo `NNNN_descripcion.sql` por cambio, idempotente, con identificadores UUID generados por la aplicación (las filas iniciales llevan un id fijo) y registro con checksum; nunca se edita una migración ya aplicada.
- **Expandir y contraer:** primero se agregan columnas opcionales y se rellenan; el código empieza a usarlas; y solo después una migración posterior las vuelve obligatorias (`NOT NULL`). Así cada despliegue es compatible con la versión anterior del código.
- **Seguridad por filas desde el primer día:** toda tabla nueva activa RLS y, cuando exista el rol de aplicación de mínimo privilegio, crea su política para ese rol.
- **Ejecución automática:** las aplica el pipeline en cada despliegue. Para deshacer un cambio se escribe una migración que lo revierte o se restaura el respaldo tomado antes de aplicarla.
- **Numeración:** la 0007 queda reservada para activar RLS en `ausencias_barbero` (hallazgo OD-01). Las propuestas empiezan en la 0008; el número final depende del orden en que se desarrollen.

Validación. Las 11 migraciones se aplicaron sobre las migraciones 0001 a 0006 en PostgreSQL 16, se repitieron para comprobar la idempotencia y se verificó cada regla con datos: la restricción rechaza citas traslapadas pero permite las contiguas o las canceladas; el horario de 9:00 a 19:00 queda en dos bloques; una cita no se liquida dos veces; un aviso repetido de la pasarela se rechaza; y el rol `encargado` se acepta mientras un rol inexistente se rechaza. En esa prueba se encontró y corrigió un error de idempotencia de la 0011. El SQL completo está en el Anexo A.

4.2 Resumen

Nº	Nombre	Qué agrega	Habilita	Riesgo	Depende de
0008	sucursales_base	Tabla de sucursales; `sucursal_id` en barberos, citas e insumos con relleno; rol `encargado`.	Varias sedes	Medio	—
0009	barbero_servicios	Qué servicios hace cada barbero, con duración y precio propios opcionales; relleno con todos.	Barberos con especialidad; reasignación	Bajo	—
0010	sucursal_servicios	Servicios y precio por sucursal.	Catálogo por sede	Bajo	0008

Nº	Nombre	Qué agrega	Habilita	Riesgo	Depende de
0011	citas_sin_traslape	Restricción de exclusión: un barbero no puede tener dos citas traslapadas.	Garantiza RF-CIT-07 (sin doble reserva) desde la base	Medio	—
0012	horario_con_receso	Parte en dos bloques los horarios que cruzan 13:00-14:00.	Garantiza RF-CIT-03 (receso del almuerzo) en los horarios	Bajo	—
0013	citas_trazabilidad	Quién y por qué cancela, de qué barbero se reasignó y recordatorio enviado.	Reasignar, recordar, auditar cancelaciones	Bajo	—
0014	pagos	Pagos en línea y registro de eventos de la pasarela con idempotencia.	Cobro anticipado	Medio	—
0015	comisiones_liquidaciones	Comisión por barbero, liquidaciones y detalle con una sola liquidación por cita.	Liquidación de barberos	Medio	—
0016	cola_notificaciones	Cola de notificaciones (outbox) con reintentos.	Correo asíncrono, recordatorios, lista de espera	Bajo	—
0017	lista_espera	Solicitudes de lista de espera con estado y vigencia de la oferta.	Lista de espera	Bajo	0016
0018	auditoria	Bitácora de operaciones sensibles.	Pagos, liquidaciones, cambios de rol	Bajo	—

4.3 Cambios de código asociados

Migración	Módulos del backend y del frontend afectados
0008	Módulos barberos, citas, inventario, auth (rol encargado) y reportes; selector de sucursal en el frontend.
0009	Módulo barberos (alta y edición con servicios) y citas (validar que el barbero haga el servicio y usar su duración y precio).
0010	Módulos servicios, citas y reportes.
0011	Módulo citas: capturar el error de la restricción y responder 409 en lugar de 500.
0012	Módulo barberos (horario por defecto con dos bloques) y el seed.
0013	Módulo citas (registrar quién cancela y avisar al barbero) y notificaciones.
0014	Módulo nuevo de pagos con su webhook; citas (estado `pendiente` mientras se paga).
0015	Módulo nuevo de liquidaciones y reportes.
0016	El servicio de notificaciones escribe en la cola en lugar de enviar; endpoint protegido para la tarea que envía.
0017	Módulo nuevo de lista de espera; citas (al cancelar, ofrecer el horario).
0018	Servicios de pagos y de liquidaciones.

4.4 Fase de contracción

Cuando el código ya escriba `sucursal_id` en barberos, citas e insumos, una migración posterior debe ejecutar `ALTER TABLE ... ALTER COLUMN sucursal_id SET NOT NULL` en cada una y activar las políticas de RLS por sucursal. No se incluye aquí porque depende de que el código ya esté desplegado.

5. Tecnologías nuevas y su justificación

Cada tecnología responde a una necesidad concreta y se adopta cuando se cumple su disparador, no antes. Donde se menciona un proveedor concreto sin haberlo contratado, se indica qué falta verificar.

Necesidad	Tecnología	Alternativas	Justificación	Cuándo
Tareas programadas	Supabase pg_cron + pg_net, que llaman a un endpoint protegido de la API	Vercel Cron; GitHub Actions con `schedule`	No necesita servidor propio y la base ya está en Supabase. El plan gratuito de Vercel solo permite una ejecución diaria, insuficiente para recordatorios cada hora. Confirmar en el panel que `pg_cron` esté habilitado.	Al activar recordatorios y ofertas de la lista de espera
Cola de notificaciones	Patrón outbox en PostgreSQL (tabla `cola_notificaciones`)	BullMQ con Redis; Amazon SQS	Reutiliza la base, permite reintentos y evita que el usuario espere al servidor de correo. BullMQ exige un proceso permanente, que no encaja con funciones serverless.	Primer paso: sacar el correo de la petición
Correo transaccional	Amazon SES (ya previsto en la Fase 1) o Resend	SendGrid; el SMTP de Gmail actual	Dominio verificado (SPF, DKIM y DMARC) para llegar a la bandeja de entrada, límites de envío mayores y avisos de rebote. Una cuenta personal de Gmail no es adecuada para producción.	Antes de enviar recordatorios
Limitación de tasa y caché	Upstash Redis (serverless) con su librería de limitación de tasa	Contador en PostgreSQL; memoria del proceso	En serverless cada invocación puede ser otra instancia, así que un contador en memoria no sirve. Cierra OE-03 (inicio de sesión, 2FA y recuperación) y sirve de caché de disponibilidad.	Limitación de tasa: ya. Caché: si el p95 de disponibilidad se acerca a 1 s
Pagos	Pasarela con webhooks firmados y tokenización (candidatas por evaluar: Stripe, PayPal, Recurrente o Pagadito)	Cobro en el local	Criterios: disponibilidad en Guatemala, moneda GTQ, comisión, entorno de pruebas y que nunca se toquen datos de tarjeta. La disponibilidad y las comisiones de cada proveedor deben verificarse antes de decidir.	Cuando el negocio decida cobrar anticipos
Recordatorios por mensajería	WhatsApp Business (API en la nube de Meta) o Twilio	Solo correo	Canal de mayor alcance que el correo para un recordatorio. Requiere plantillas aprobadas y tiene costo por mensaje, por verificar.	Si el correo no reduce las inasistencias
Observabilidad	Sentry (errores del frontend y del backend) y un monitor de disponibilidad externo	Solo los logs de Vercel	Hoy los errores 500 solo se ven si alguien revisa los logs. Avisos automáticos permitirían detectarlos antes de que los vea un usuario.	Ya
Pruebas de carga y de interfaz	k6 (umbrales en el pipeline) y Playwright (flujo de agendar)	Script de carga propio; sin pruebas del frontend	Convierte el script manual en una verificación automática con límites (p95) y cubre la interfaz, que hoy no tiene pruebas.	Antes de cada sucursal nueva
Base de datos	Supabase plan Pro (respaldos diarios y recuperación a un punto en el tiempo como complemento)	Respaldos propios con `pg_dump`, ya planificados	Más conexiones y almacenamiento, y respaldos gestionados. Pasa a ser necesario cuando haya datos reales de clientes de varias sedes.	Al abrir la segunda sucursal
Multi-sucursal	Una sola base con `sucursal_id` y RLS por sucursal	Una base por sucursal	Migraciones únicas, reportes consolidados y menor costo. A cambio el aislamiento es lógico, no físico; se revisa si hubiera sedes en países con otra regulación.	Con la migración 0008
Seguridad de la cadena de suministro	gitleaks y Dependabot (o `npm audit`) en el pipeline	Revisión manual	Evita publicar secretos y detecta dependencias vulnerables.	Ya

5.1 Decisiones de arquitectura

- **Outbox en lugar de una cola externa.** Las funciones serverless no mantienen procesos permanentes, así que un trabajador como BullMQ no encaja. Escribir el aviso en una tabla dentro de la misma transacción y enviarlo después evita perder avisos y quita la espera del correo de la petición.

- **Tareas programadas desde la base.** El plan Hobby de Vercel limita las tareas programadas a una por día y con precisión de una hora; `pg_cron` ejecuta con precisión de un minuto y no depende del plan de Vercel. Para las tareas diarias, como la liquidación, Vercel Cron alcanza.
- **Una sola base para todas las sucursales.** Mantiene un único conjunto de migraciones y permite reportes consolidados; el aislamiento entre sucursales lo da RLS en la base, no solo el código.
- **Pagos idempotentes.** Cada aviso de la pasarela se registra con su identificador único; si llega repetido, no se vuelve a procesar. Los datos de tarjeta nunca pasan por el sistema.

6. Hoja de ruta

Fase	Tamaño del negocio	Contenido	Migraciones	Tecnologías	Criterio de salida
0. Antes de crecer	Un barbero activo	Corregir los defectos abiertos del Plan de Pruebas y del Inventario de Endpoints; limitación de tasa y restringir CORS; errores y disponibilidad con avisos.	0007 (RLS de ausencias), 0011, 0012	Upstash Redis, Sentry, monitor externo	Sin defectos de severidad alta abiertos
1. Varios barberos	2 a 5 barberos	Servicios por barbero, trazabilidad de citas, cola de notificaciones, recordatorios y reasignación de citas.	0009, 0013, 0016	pg_cron + pg_net, Amazon SES o Resend	Correo fuera de la petición; reasignación probada
2. Lista de espera y cobros	Más de 5 barberos o muchas cancelaciones	Lista de espera, pagos en línea y auditoría.	0017, 0014, 0018	Pasarela de pago, k6	Webhooks idempotentes y conciliados
3. Comisiones y sucursales	Segunda sede	Liquidación de barberos, sucursales y rol de encargado; contraer las columnas nuevas (`NOT NULL`).	0015, 0008, 0010	RLS por sucursal, Supabase Pro, Playwright	Cada sucursal ve solo sus datos

7. Riesgos

Riesgo	Por qué importa	Mitigación
Límites de los planes gratuitos (Vercel, Supabase)	Una ejecución diaria de Vercel Cron; conexiones y almacenamiento de la base acotados.	Usar pg_cron; subir de plan antes de la segunda sede.
Migraciones con relleno masivo (0008, 0009)	Pueden bloquear tablas o dejar datos a medias.	Patrón expandir y contraer; probar sobre una copia; respaldo previo.
Restricción de exclusión (0011)	Falla si ya hay citas traslapadas.	La migración revisa y aborta con un mensaje claro; limpiar los datos antes.
Pagos y datos personales	Dinero real y datos de clientes.	Webhooks firmados e idempotentes, auditoría, sin guardar tarjetas y revisión de la normativa de datos personales.
Complejidad del multi-sucursal	Todo consulta debe filtrar por sucursal.	RLS por sucursal en la base, para que un olvido en el código no filtre datos.
Dependencia de servicios externos	Pasarela, correo y mensajería pueden fallar.	Cola con reintentos y un estado visible para el administrador.

Anexo A. SQL de las migraciones propuestas

Es exactamente el SQL que se aplicó en la validación descrita en la sección 4.1. Las propuestas son borradores: antes de aplicarlas se debe revisar su orden y activar, tabla por tabla, la política de RLS del rol de aplicación.

0008_sucursales_base.sql

```
-- Migración 0008: sucursales y rol de encargado. Fase "expandir": columnas nuevas opcionales y relleno con
la sucursal inicial.
CREATE TABLE IF NOT EXISTS sucursales (
  id          UUID PRIMARY KEY,
  nombre      VARCHAR(255) NOT NULL,
  direccion   VARCHAR(255),
  telefono    VARCHAR(50),
  is_active   BOOLEAN NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- La sucursal inicial usa un id fijo: la migración debe ser reproducible y la base no genera UUID.
INSERT INTO sucursales (id, nombre)
VALUES ('00000000-0000-4000-8000-000000000001', 'Principal')
ON CONFLICT (id) DO NOTHING;

ALTER TABLE barberos ADD COLUMN IF NOT EXISTS sucursal_id UUID REFERENCES sucursales(id) ON UPDATE CASCADE ON
DELETE RESTRICT;
ALTER TABLE citas    ADD COLUMN IF NOT EXISTS sucursal_id UUID REFERENCES sucursales(id) ON UPDATE CASCADE ON
DELETE RESTRICT;
ALTER TABLE insumos  ADD COLUMN IF NOT EXISTS sucursal_id UUID REFERENCES sucursales(id) ON UPDATE CASCADE ON
DELETE RESTRICT;

UPDATE barberos SET sucursal_id = '00000000-0000-4000-8000-000000000001' WHERE sucursal_id IS NULL;
UPDATE citas    SET sucursal_id = '00000000-0000-4000-8000-000000000001' WHERE sucursal_id IS NULL;
UPDATE insumos  SET sucursal_id = '00000000-0000-4000-8000-000000000001' WHERE sucursal_id IS NULL;

-- Nuevo rol "encargado": solo ve y gestiona su sucursal (RLS por sucursal en una fase posterior).
ALTER TABLE users DROP CONSTRAINT IF EXISTS users_role_check;
ALTER TABLE users ADD CONSTRAINT users_role_check CHECK (role IN ('admin', 'barbero', 'cliente',
'encargado'));
ALTER TABLE users ADD COLUMN IF NOT EXISTS sucursal_id UUID REFERENCES sucursales(id) ON UPDATE CASCADE ON
DELETE SET NULL;

CREATE INDEX IF NOT EXISTS idx_barberos_sucursal ON barberos (sucursal_id);
CREATE INDEX IF NOT EXISTS idx_citas_sucursal_fecha ON citas (sucursal_id, fecha);

ALTER TABLE sucursales ENABLE ROW LEVEL SECURITY;
```

0009_barbero_servicios.sql

```
-- Migración 0009: qué servicios hace cada barbero, con duración y precio propios opcionales.
CREATE TABLE IF NOT EXISTS barbero_servicios (
  id               UUID PRIMARY KEY,
  barbero_id       UUID NOT NULL REFERENCES barberos(id) ON UPDATE CASCADE ON DELETE CASCADE,
  servicio_id      UUID NOT NULL REFERENCES servicios(id) ON UPDATE CASCADE ON DELETE CASCADE,
  duracion_minutos  INTEGER CHECK (duracion_minutos > 0),    -- NULL = usa la del servicio
  precio           NUMERIC(10, 2) CHECK (precio >= 0),     -- NULL = usa el del servicio
  is_active        BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT uq_barbero_servicio UNIQUE (barbero_id, servicio_id)
);

-- Relleno: hoy todos los barberos hacen todos los servicios, así que nada cambia para el cliente.
INSERT INTO barbero_servicios (id, barbero_id, servicio_id)
SELECT md5(b.id::text || '-' || s.id::text)::uuid, b.id, s.id
FROM barberos b CROSS JOIN servicios s
ON CONFLICT (id) DO NOTHING;

ALTER TABLE barbero_servicios ENABLE ROW LEVEL SECURITY;
```

0010_sucursal_servicios.sql

```
-- Migración 0010: servicios y precio por sucursal (depende de 0008).
CREATE TABLE IF NOT EXISTS sucursal_servicios (
  id           UUID PRIMARY KEY,
  sucursal_id  UUID NOT NULL REFERENCES sucursales(id) ON UPDATE CASCADE ON DELETE CASCADE,
  servicio_id  UUID NOT NULL REFERENCES servicios(id) ON UPDATE CASCADE ON DELETE CASCADE,
  precio       NUMERIC(10, 2) CHECK (precio >= 0),          -- NULL = usa el precio base del servicio
  is_active    BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT uq_sucursal_servicio UNIQUE (sucursal_id, servicio_id)
);

INSERT INTO sucursal_servicios (id, sucursal_id, servicio_id)
SELECT md5(su.id::text || '-' || se.id::text)::uuid, su.id, se.id
FROM sucursales su CROSS JOIN servicios se
ON CONFLICT (id) DO NOTHING;

ALTER TABLE sucursal_servicios ENABLE ROW LEVEL SECURITY;
```

0011_citas_sin_traslape.sql

```
-- Migración 0011: la base impide que un barbero tenga dos citas traslapadas (RF-CIT-07).
CREATE EXTENSION IF NOT EXISTS btree_gist;

-- Antes de crear la restricción se comprueba que los datos actuales la cumplan; si no,
-- la migración falla con un mensaje claro en vez de dejar la base a medias.
DO $$
BEGIN
  IF EXISTS (
    SELECT 1 FROM citas a JOIN citas b
      ON a.id < b.id AND a.barbero_id = b.barbero_id AND a.fecha = b.fecha
     AND a.estado <> 'cancelada' AND b.estado <> 'cancelada'
     AND (a.hora_inicio, a.hora_fin) OVERLAPS (b.hora_inicio, b.hora_fin)
  ) THEN
    RAISE EXCEPTION 'Hay citas traslapadas; corrígelas antes de aplicar la migración 0011';
  END IF;
END $$;

DO $$
BEGIN
  ALTER TABLE citas ADD CONSTRAINT excl_citas_sin_traslape EXCLUDE USING gist (
    barbero_id WITH =,
    (tsrange(fecha + hora_inicio, fecha + hora_fin)) WITH &&
  ) WHERE (estado <> 'cancelada');
EXCEPTION WHEN duplicate_object OR duplicate_table THEN NULL;
END $$;
```

0012_horario_con_receso.sql

```
-- Migración 0012: separa en dos bloques (mañana y tarde) los horarios que cruzan 13:00-14:00 (RF-CIT-03).
-- Idempotente: tras aplicarse, ningún bloque cumple la condición y no se vuelve a tocar nada.
INSERT INTO disponibilidad_barberos (id, barbero_id, dia_semana, hora_inicio, hora_fin)
SELECT md5(d.id::text || '-tarde')::uuid, d.barbero_id, d.dia_semana, '14:00', d.hora_fin
FROM disponibilidad_barberos d
WHERE d.hora_inicio < '13:00' AND d.hora_fin > '14:00'
ON CONFLICT (id) DO NOTHING;

UPDATE disponibilidad_barberos
SET hora_fin = '13:00', updated_at = now()
WHERE hora_inicio < '13:00' AND hora_fin > '14:00';
```

0013_citas_trazabilidad.sql

```
-- Migración 0013: trazabilidad de cancelaciones y reasignaciones, y marca de recordatorio enviado.
ALTER TABLE citas ADD COLUMN IF NOT EXISTS cancelada_por UUID REFERENCES users(id) ON UPDATE CASCADE ON
DELETE SET NULL;
ALTER TABLE citas ADD COLUMN IF NOT EXISTS cancelada_at TIMESTAMPTZ;
ALTER TABLE citas ADD COLUMN IF NOT EXISTS motivo_cancelacion VARCHAR(255);
ALTER TABLE citas ADD COLUMN IF NOT EXISTS reasignada_desde_barbero_id UUID REFERENCES barberos(id) ON UPDATE
CASCADE ON DELETE SET NULL;
ALTER TABLE citas ADD COLUMN IF NOT EXISTS recordatorio_enviado_at TIMESTAMPTZ;
```

```
-- Índice parcial: la tarea de recordatorios solo mira las citas confirmadas que aún no se avisaron.
CREATE INDEX IF NOT EXISTS idx_citas_recordatorio
  ON citas (fecha, hora_inicio)
  WHERE estado = 'confirmada' AND recordatorio_enviado_at IS NULL;
```

0014_pagos.sql

```
-- Migración 0014: pagos en línea y registro de eventos del proveedor (idempotencia de webhooks).
CREATE TABLE IF NOT EXISTS pagos (
  id                  UUID PRIMARY KEY,
  cita_id             UUID NOT NULL REFERENCES citas(id) ON UPDATE CASCADE ON DELETE RESTRICT,
  monto               NUMERIC(10, 2) NOT NULL CHECK (monto >= 0),
  moneda              CHAR(3) NOT NULL DEFAULT 'GTQ',
  proveedor           VARCHAR(30) NOT NULL,
  referencia_externa  VARCHAR(255),
  estado              VARCHAR(20) NOT NULL DEFAULT 'pendiente'
                      CHECK (estado IN ('pendiente', 'aprobado', 'fallido', 'reembolsado')),
  created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS idx_pagos_cita ON pagos (cita_id);
CREATE UNIQUE INDEX IF NOT EXISTS uq_pagos_referencia ON pagos (proveedor, referencia_externa) WHERE
referencia_externa IS NOT NULL;

CREATE TABLE IF NOT EXISTS pago_eventos (
  id           UUID PRIMARY KEY,
  proveedor    VARCHAR(30) NOT NULL,
  evento_id    VARCHAR(255) NOT NULL,
  pago_id      UUID REFERENCES pagos(id) ON UPDATE CASCADE ON DELETE SET NULL,
  recibido_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_pago_evento UNIQUE (proveedor, evento_id)   -- un aviso repetido no se procesa dos veces
);

ALTER TABLE pagos ENABLE ROW LEVEL SECURITY;
ALTER TABLE pago_eventos ENABLE ROW LEVEL SECURITY;
```

0015_comisiones_liquidaciones.sql

```
-- Migración 0015: comisiones por barbero y liquidaciones por periodo.
CREATE TABLE IF NOT EXISTS barbero_comisiones (
  id             UUID PRIMARY KEY,
  barbero_id     UUID NOT NULL REFERENCES barberos(id) ON UPDATE CASCADE ON DELETE CASCADE,
  tipo           VARCHAR(12) NOT NULL CHECK (tipo IN ('porcentaje', 'fijo')),
  valor          NUMERIC(10, 2) NOT NULL CHECK (valor >= 0),
  vigente_desde  DATE NOT NULL,
  vigente_hasta  DATE,
  CONSTRAINT chk_comision_vigencia CHECK (vigente_hasta IS NULL OR vigente_hasta >= vigente_desde)
);

CREATE TABLE IF NOT EXISTS liquidaciones (
  id             UUID PRIMARY KEY,
  barbero_id     UUID NOT NULL REFERENCES barberos(id) ON UPDATE CASCADE ON DELETE RESTRICT,
  periodo_desde  DATE NOT NULL,
  periodo_hasta  DATE NOT NULL,
  total          NUMERIC(10, 2) NOT NULL DEFAULT 0,
  estado         VARCHAR(10) NOT NULL DEFAULT 'borrador' CHECK (estado IN ('borrador', 'cerrada', 'pagada')),
  cerrada_at     TIMESTAMPTZ,
  pagada_at      TIMESTAMPTZ,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT chk_liquidacion_periodo CHECK (periodo_hasta >= periodo_desde),
  CONSTRAINT uq_liquidacion_periodo UNIQUE (barbero_id, periodo_desde, periodo_hasta)
);

CREATE TABLE IF NOT EXISTS liquidacion_citas (
  liquidacion_id  UUID NOT NULL REFERENCES liquidaciones(id) ON UPDATE CASCADE ON DELETE CASCADE,
  cita_id         UUID NOT NULL REFERENCES citas(id) ON UPDATE CASCADE ON DELETE RESTRICT,
  monto_base      NUMERIC(10, 2) NOT NULL,
  monto_comision  NUMERIC(10, 2) NOT NULL,
  PRIMARY KEY (liquidacion_id, cita_id),
  CONSTRAINT uq_cita_liquidada UNIQUE (cita_id)             -- una cita se liquida una sola vez
);
```

```
ALTER TABLE barbero_comisiones ENABLE ROW LEVEL SECURITY;
ALTER TABLE liquidaciones ENABLE ROW LEVEL SECURITY;
ALTER TABLE liquidacion_citas ENABLE ROW LEVEL SECURITY;
```

0016_cola_notificaciones.sql

```
-- Migración 0016: cola de notificaciones (patrón outbox). La petición solo escribe aquí; otra tarea envía.
CREATE TABLE IF NOT EXISTS cola_notificaciones (
  id                 UUID PRIMARY KEY,
  tipo               VARCHAR(30) NOT NULL,
  user_id            UUID REFERENCES users(id) ON UPDATE CASCADE ON DELETE CASCADE,
  email              VARCHAR(255),
  payload            JSONB NOT NULL DEFAULT '{}'::jsonb,
  estado             VARCHAR(12) NOT NULL DEFAULT 'pendiente' CHECK (estado IN ('pendiente', 'enviado',
'fallido')),
  intentos           INTEGER NOT NULL DEFAULT 0,
  proxima_ejecucion  TIMESTAMPTZ NOT NULL DEFAULT now(),
  enviado_at         TIMESTAMPTZ,
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS idx_cola_pendientes ON cola_notificaciones (proxima_ejecucion) WHERE estado =
'pendiente';

ALTER TABLE cola_notificaciones ENABLE ROW LEVEL SECURITY;
```

0017_lista_espera.sql

```
-- Migración 0017: lista de espera para horarios liberados.
CREATE TABLE IF NOT EXISTS lista_espera (
  id                UUID PRIMARY KEY,
  cliente_id        UUID NOT NULL REFERENCES clientes(id) ON UPDATE CASCADE ON DELETE CASCADE,
  servicio_id       UUID NOT NULL REFERENCES servicios(id) ON UPDATE CASCADE ON DELETE CASCADE,
  barbero_id        UUID REFERENCES barberos(id) ON UPDATE CASCADE ON DELETE CASCADE,  -- NULL = cualquier
barbero
  fecha             DATE NOT NULL,
  hora_desde        TIME NOT NULL,
  hora_hasta        TIME NOT NULL,
  estado            VARCHAR(12) NOT NULL DEFAULT 'activa'
                    CHECK (estado IN ('activa', 'ofrecida', 'aceptada', 'vencida', 'cancelada')),
  oferta_expira_at  TIMESTAMPTZ,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT chk_espera_horas CHECK (hora_hasta > hora_desde)
);
CREATE INDEX IF NOT EXISTS idx_lista_espera_busqueda ON lista_espera (fecha, servicio_id) WHERE estado =
'activa';

ALTER TABLE lista_espera ENABLE ROW LEVEL SECURITY;
```

0018_auditoria.sql

```
-- Migración 0018: bitácora de auditoría para operaciones sensibles (pagos, liquidaciones, cambios de rol).
CREATE TABLE IF NOT EXISTS auditoria (
  id             UUID PRIMARY KEY,
  actor_user_id  UUID REFERENCES users(id) ON UPDATE CASCADE ON DELETE SET NULL,
  accion         VARCHAR(60) NOT NULL,
  entidad        VARCHAR(40) NOT NULL,
  entidad_id     UUID,
  datos          JSONB NOT NULL DEFAULT '{}'::jsonb,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS idx_auditoria_entidad ON auditoria (entidad, entidad_id, created_at);

ALTER TABLE auditoria ENABLE ROW LEVEL SECURITY;
```

Anexo B. Diagramas BPMN

Vista general de los seis procesos futuros. Cada diagrama se muestra completo y a tamaño legible en la sección 3.4.

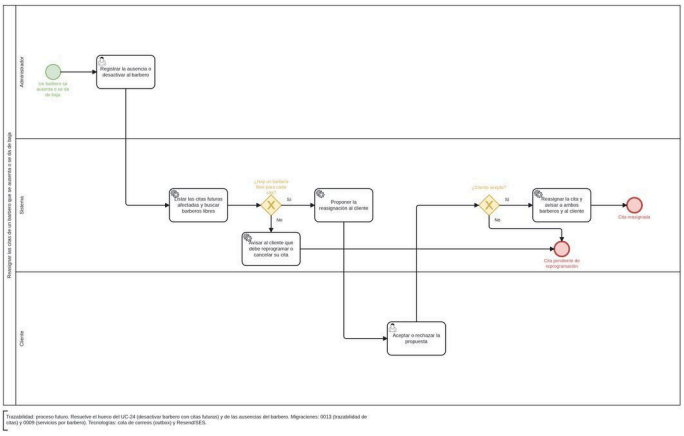
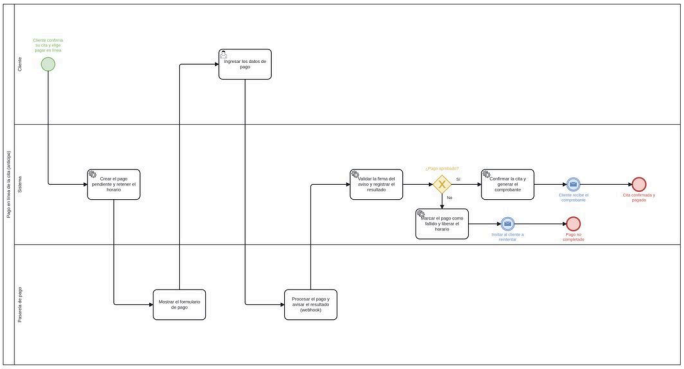
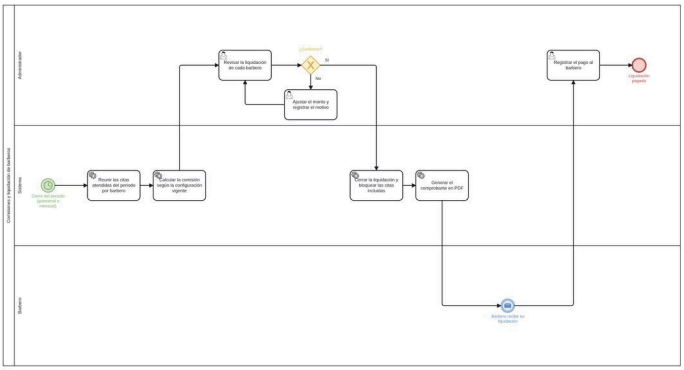
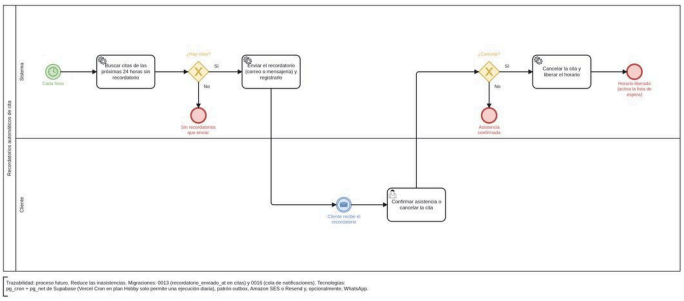
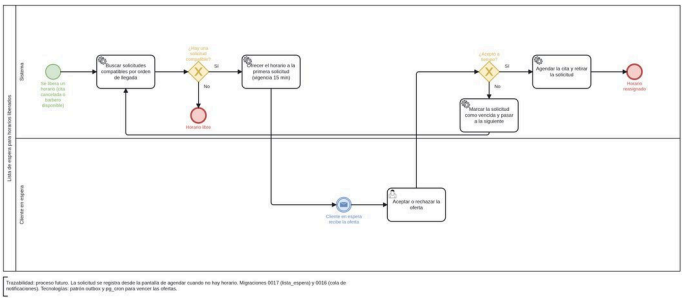
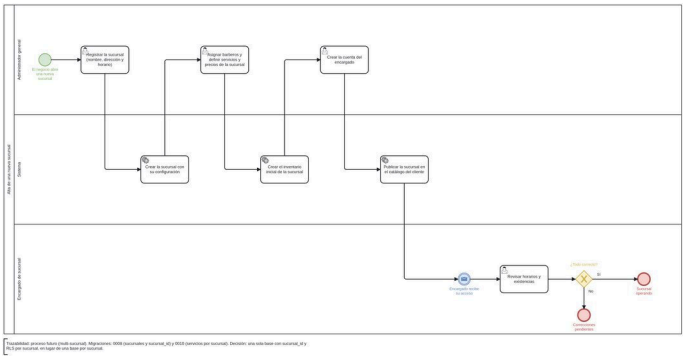
Diagrama	Proceso
	**Reasignar las citas de un barbero que se ausenta o se da de baja** Disparador: El administrador registra una ausencia o desactiva a un barbero. Figura 1, sección 3.4.
	**Pago en línea de la cita (anticipo)** Disparador: El cliente elige pagar al confirmar. Figura 2, sección 3.4.
	**Comisiones y liquidación de barberos** Disparador: Cierre del periodo (quincenal o mensual). Figura 3, sección 3.4.
	**Recordatorios automáticos de cita** Disparador: Temporizador cada hora. Figura 4, sección 3.4.

Diagrama	Proceso
	**Lista de espera para horarios liberados** Disparador: Se libera un horario. Figura 5, sección 3.4.
	**Alta de una nueva sucursal** Disparador: El negocio abre una sucursal. Figura 6, sección 3.4.

Manual de Usuario

Campo	Detalle
Proyecto	Barber Kongs — Sistema de Gestión para Barbería (Fase 3)
Equipo	Miguel Angel Quemé Gómez · Brandon Gustavo Güinac Román
Versión del documento	2.0 — 25 de octubre de 2026
Dirigido a	Clientes, barberos y administradores de la barbería
Capturas	Tomadas de la aplicación en funcionamiento con datos de demostración; los nombres, correos y citas que aparecen son ficticios.

1. Roles de usuario y requisitos

1.1 Roles de usuario

Lo que cada persona puede hacer depende de su rol. Al iniciar sesión, el sistema muestra solo las opciones de ese rol.

Rol	Quién es	Qué puede hacer
Cliente	Persona que se atiende en la barbería. Crea su propia cuenta.	Agendar, reprogramar y cancelar sus citas; ver sus próximas citas; recibir notificaciones; activar la verificación en dos pasos.
Barbero	Cuenta creada por el administrador.	Consultar su agenda diaria y semanal, marcar citas como atendidas, registrar clientes sin cita (walk-in) e indicar los días en que no estará disponible.
Administrador	Responsable del negocio.	Todo lo anterior sobre cualquier barbero, además de gestionar barberos, clientes, servicios e inventario, y generar reportes.

1.2 Antes de empezar

- **Navegador:** una versión actual de Chrome, Firefox, Edge o Safari, con conexión a internet.
- **Horario de atención:** de domingo a jueves, de 9:00 a. m. a 7:00 p. m., con receso de almuerzo de 1:00 a 2:00 p. m. Viernes y sábado no se agendan citas.
- **Cuenta:** los clientes crean su cuenta desde la pantalla de inicio. Las cuentas de barbero las crea el administrador; la cuenta de administrador la entrega quien instala el sistema.

2. Funciones comunes a todos los roles

2.1 Pantalla de inicio

La pantalla de inicio es pública: muestra los servicios con su duración y precio, una galería de cortes realizados, los barberos del equipo y el horario de atención. Desde ella se puede agendar una cita, ingresar o crear una cuenta.

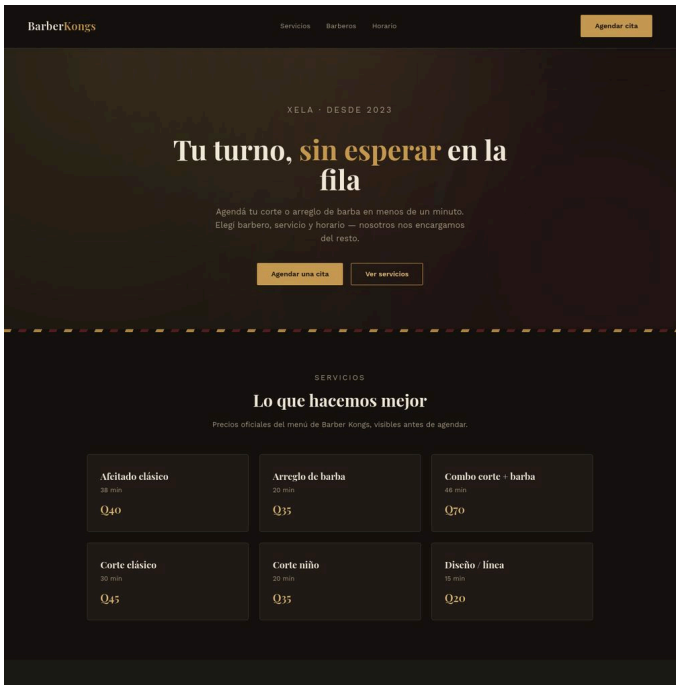


Figura 1. Pantalla de inicio: encabezado y catálogo de servicios.

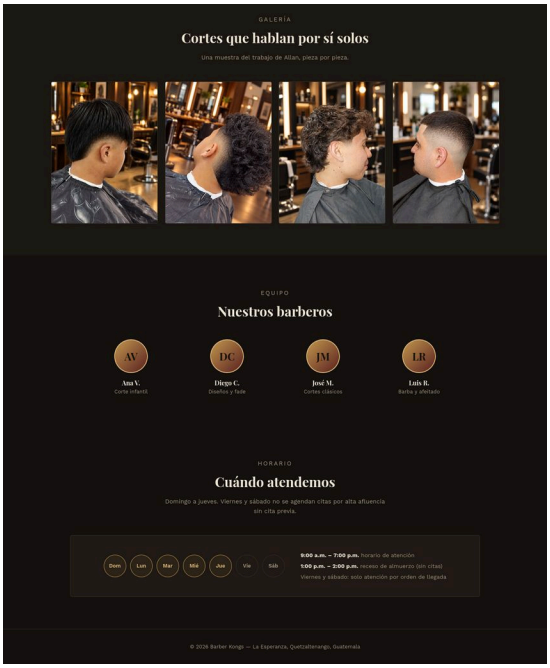


Figura 2. Pantalla de inicio: galería de cortes, equipo de barberos y horario de atención.

2.2 Crear una cuenta (clientes)

Para agendar citas es necesario tener una cuenta de cliente.

1. En la pantalla de inicio, seleccione **Ingresar** y luego la pestaña **Crear cuenta**.
2. Escriba su **nombre completo**, su **correo electrónico**, su **teléfono** (opcional) y una **contraseña** de al menos 8 caracteres.
3. Presione **Crear cuenta**. El sistema lo lleva a la pantalla de ingreso para que inicie sesión con los datos que acaba de registrar.

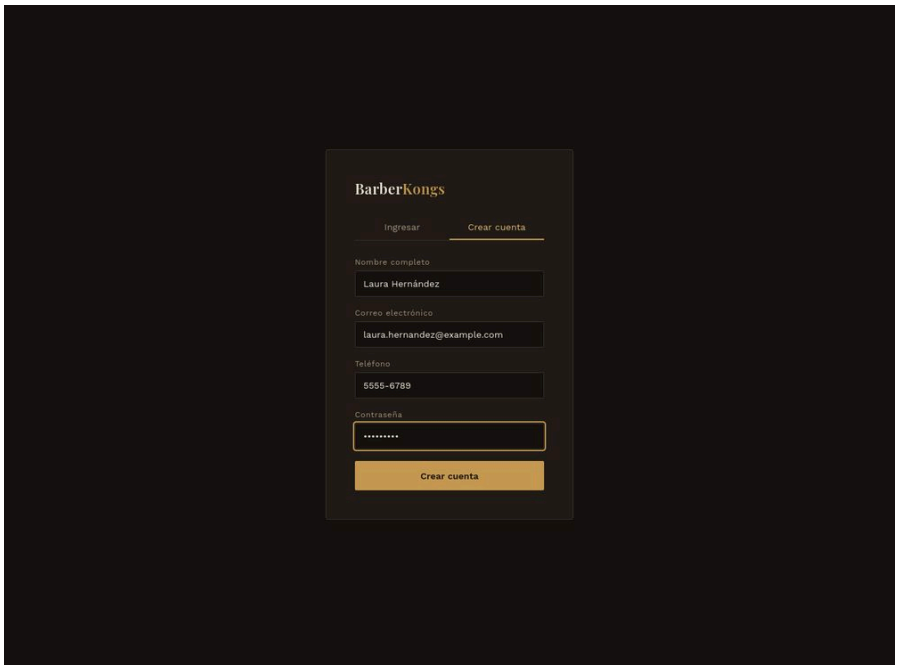


Figura 3. Formulario de creación de cuenta.

> Cada correo puede registrarse una sola vez. Si ya existe una cuenta con ese correo, el sistema lo indica y se debe usar otro.

2.3 Iniciar sesión

1. Abra la pantalla **Ingresar**, escriba su correo y su contraseña y presione **Ingresar**.
2. Si la cuenta tiene verificación en dos pasos, el sistema pide además un código (sección 2.4).
3. Al entrar, cada rol llega a su pantalla principal: el cliente a **Agendar cita**, el barbero a su **agenda** y el administrador al **panel de administración**.

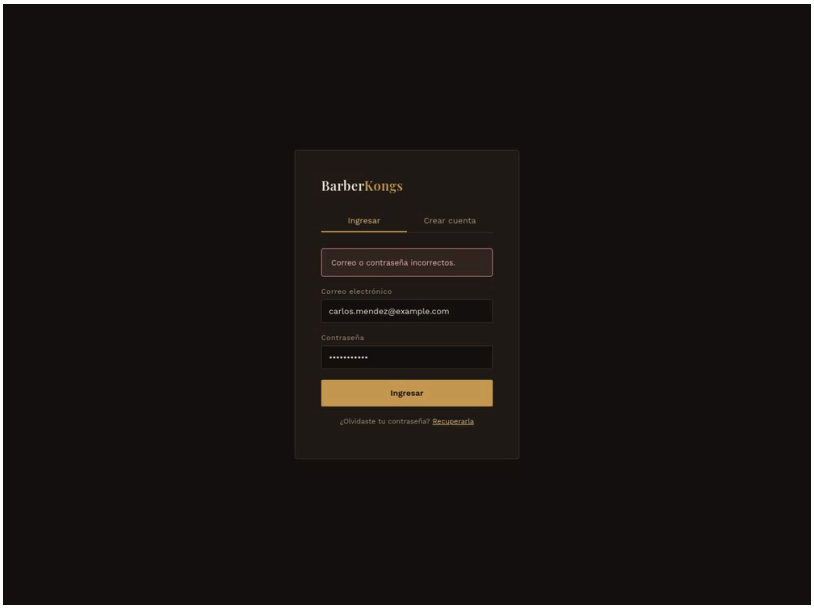


Figura 4. Si los datos no son correctos, el sistema muestra un aviso sin indicar cuál de los dos falló.

2.4 Verificación en dos pasos (opcional)

La verificación en dos pasos añade una capa de seguridad: además de la contraseña, cada ingreso pide un código de 6 dígitos que se envía al correo de la cuenta. Es opcional y la puede activar cualquier rol.

Activarla: entre a **Mi cuenta**, escriba su contraseña actual y presione **Activar 2FA**.

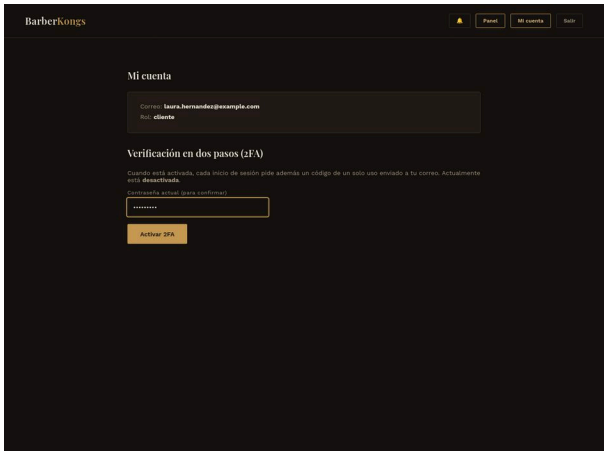


Figura 5. Mi cuenta: activar la verificación en dos pasos.

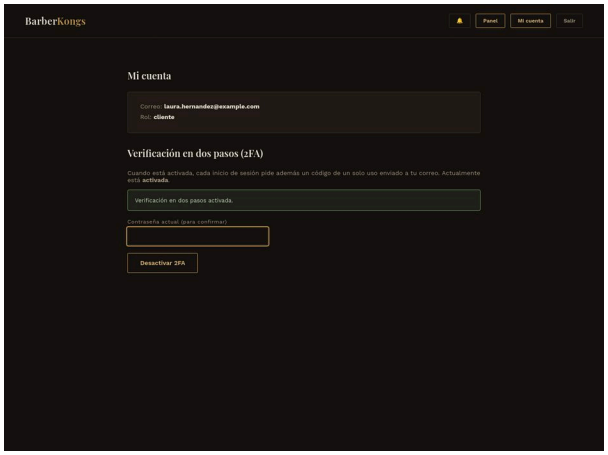


Figura 6. Confirmación de que quedó activada.

Usarla al ingresar: después de escribir correo y contraseña, el sistema envía el código por correo y muestra la pantalla de verificación. Escriba el código y presione **Verificar**. El código es de un solo uso y vence a los 10 minutos.

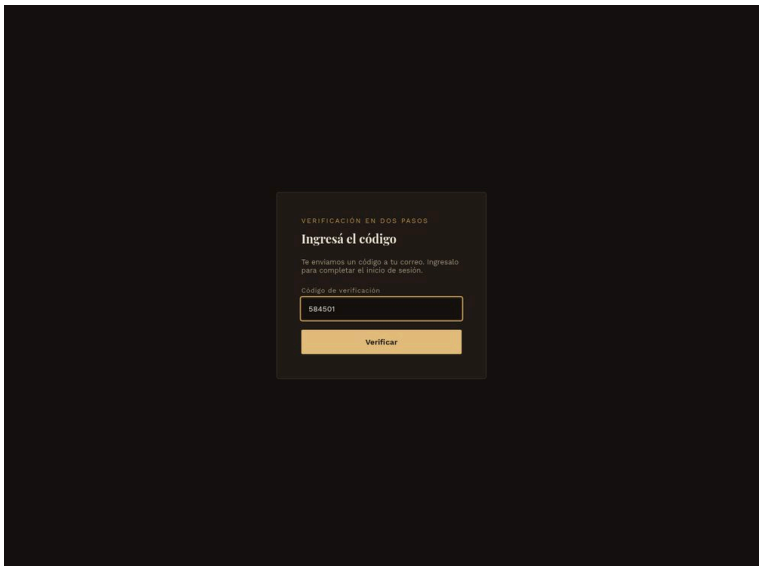


Figura 7. Pantalla donde se escribe el código recibido por correo.

Desactivarla: en **Mi cuenta**, escriba su contraseña actual y presione **Desactivar 2FA**.

2.5 Recuperar la contraseña

1. En la pantalla de ingreso, presione **Recuperarla** (junto a «¿Olvidaste tu contraseña?»).
2. Escriba su correo y presione **Enviar enlace**. Por seguridad, el sistema responde lo mismo exista o no la cuenta.
3. Abra el correo recibido y siga el enlace. El enlace vence en 1 hora.
4. Escriba la nueva contraseña, confírmela y presione **Guardar contraseña**.

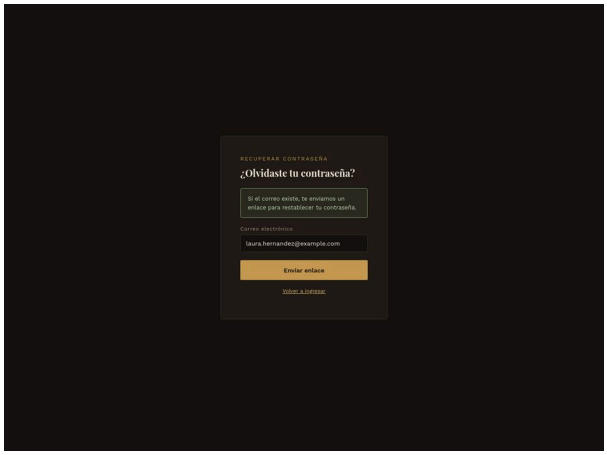


Figura 8. Solicitud del enlace de recuperación.

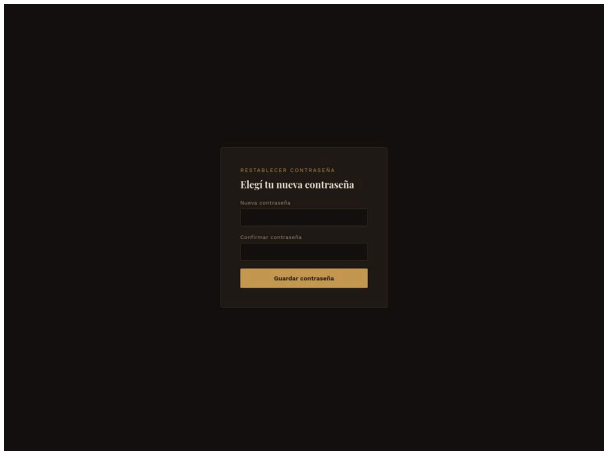


Figura 9. Pantalla para elegir la nueva contraseña.

2.6 Notificaciones

La campana de la barra superior muestra cuántas notificaciones no ha leído. Al presionarla se despliega la lista con las más recientes: confirmaciones y cancelaciones de cita, códigos de verificación y, para el administrador, alertas de inventario. Presione **Marcar leída** en cada una; el contador disminuye. Las mismas notificaciones también llegan al correo.

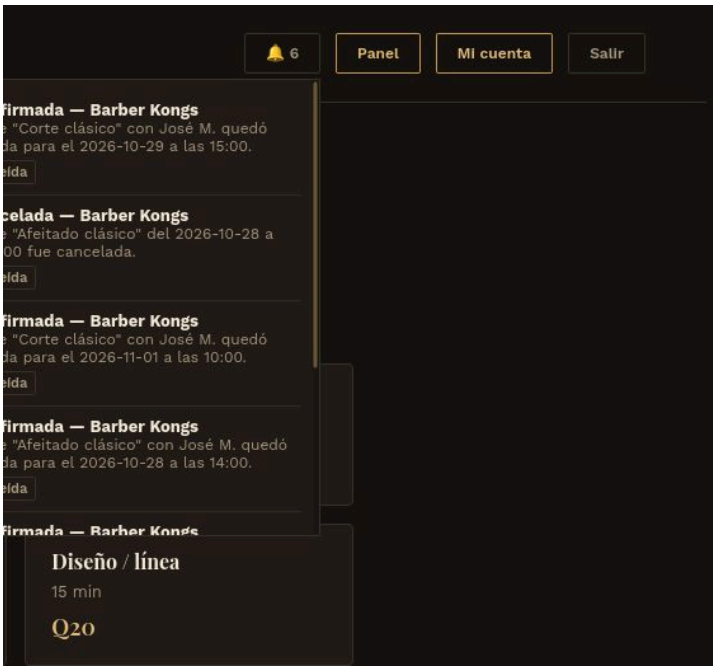


Figura 10. Campana de notificaciones desplegada.

2.7 Mi cuenta y cerrar sesión

Mi cuenta muestra el correo y el rol de la sesión y permite activar o desactivar la verificación en dos pasos. Para terminar la sesión, presione **Salir** en la barra superior; hágalo siempre que use un equipo compartido.

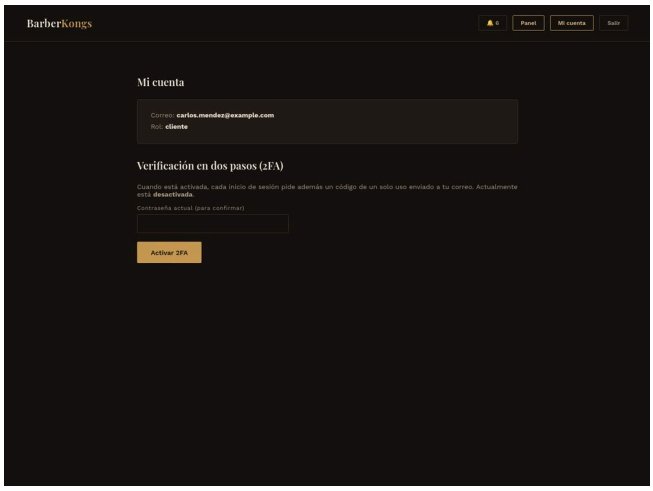


Figura 11. Pantalla Mi cuenta.

3. Guía del cliente

3.1 Agendar una cita

La pantalla **Agendar cita** es un asistente de cuatro pasos. En la parte superior se ve en cuál va, y con **Atrás** se puede volver a corregir.

Paso 1 — Servicio. Elija el servicio; cada tarjeta muestra su duración y su precio. Presione **Continuar**.

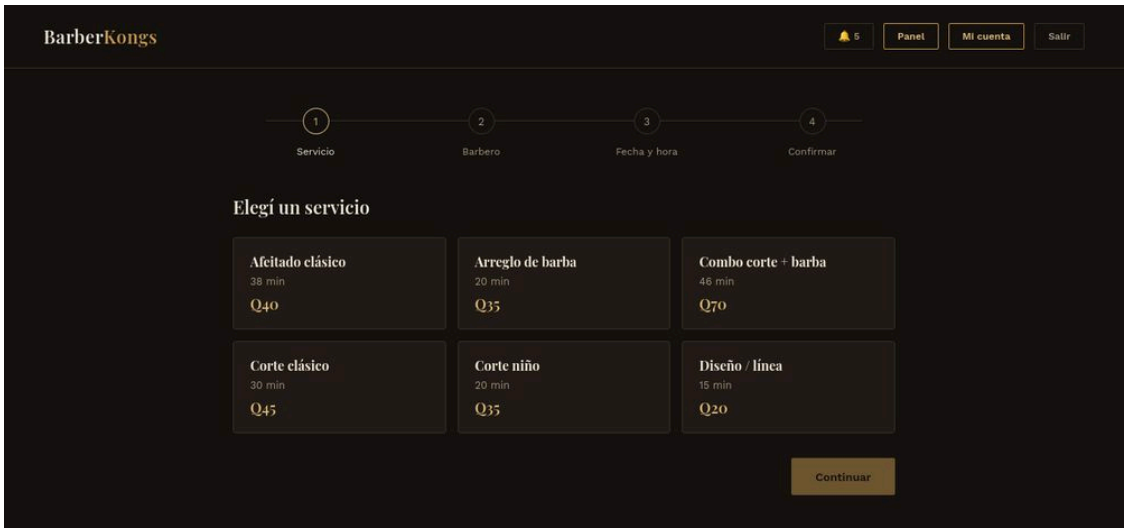


Figura 12. Paso 1: elegir un servicio.

Paso 2 — Barbero. Elija con quién quiere atenderse y presione **Continuar**.

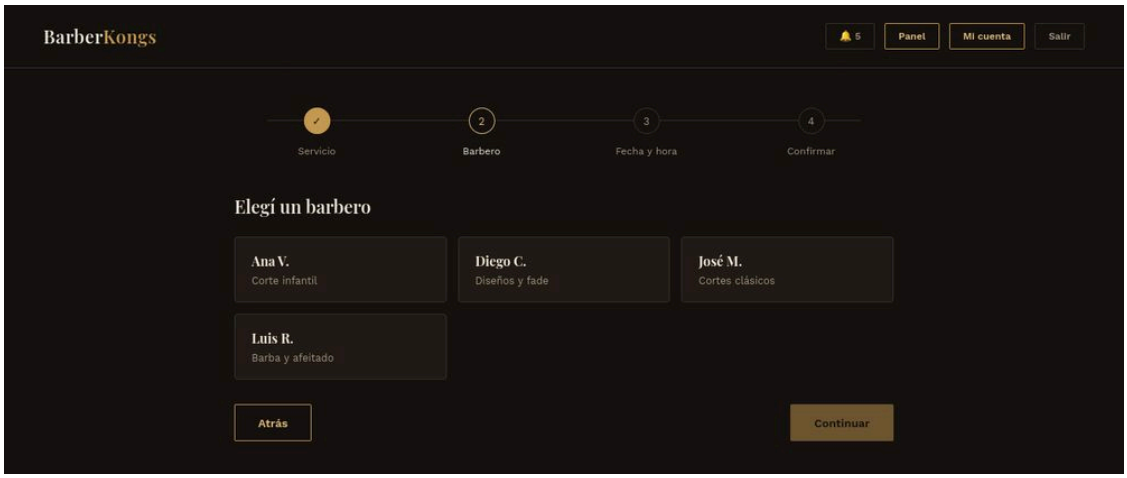


Figura 13. Paso 2: elegir un barbero.

Paso 3 — Fecha y hora. Elija el día entre los siete que se muestran. Los días en que el barbero no atiende, o en que no estará disponible, aparecen atenuados y no se pueden elegir. Luego elija una hora libre; las horas ocupadas y el receso de almuerzo aparecen deshabilitadas. Presione **Continuar a confirmación**.

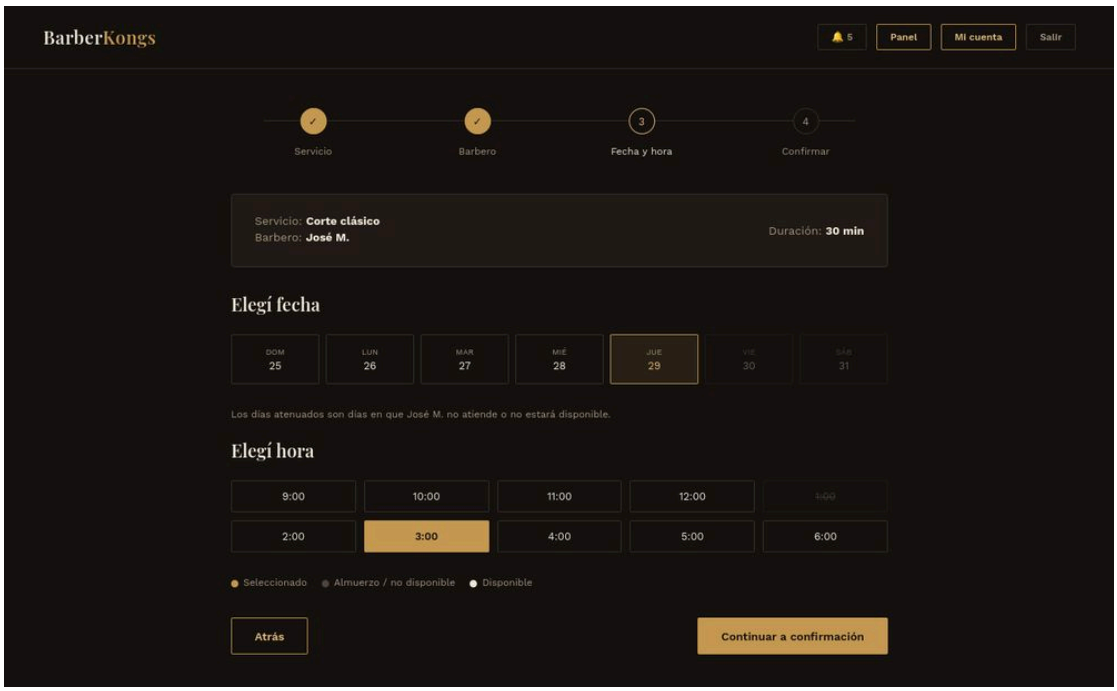


Figura 14. Paso 3: elegir fecha y hora disponibles.

Paso 4 — Confirmar. Revise el resumen (servicio y precio, barbero, fecha y hora) y presione **Confirmar cita**.

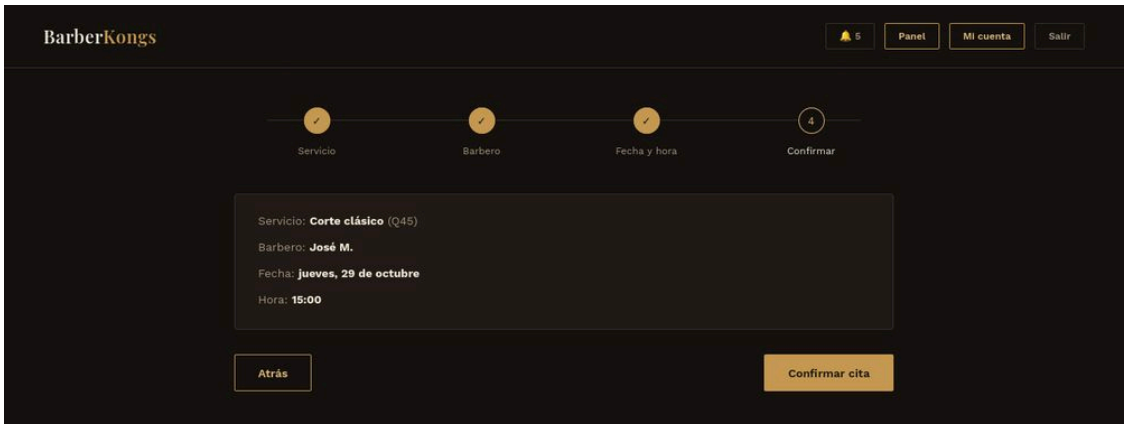


Figura 15. Paso 4: resumen de la cita antes de confirmar.

La cita queda **confirmada** de inmediato, sin necesidad de que el barbero la acepte. Aparece en **Mis próximas citas**, se envía una confirmación al cliente y el barbero la ve reflejada en su agenda y recibe un aviso.

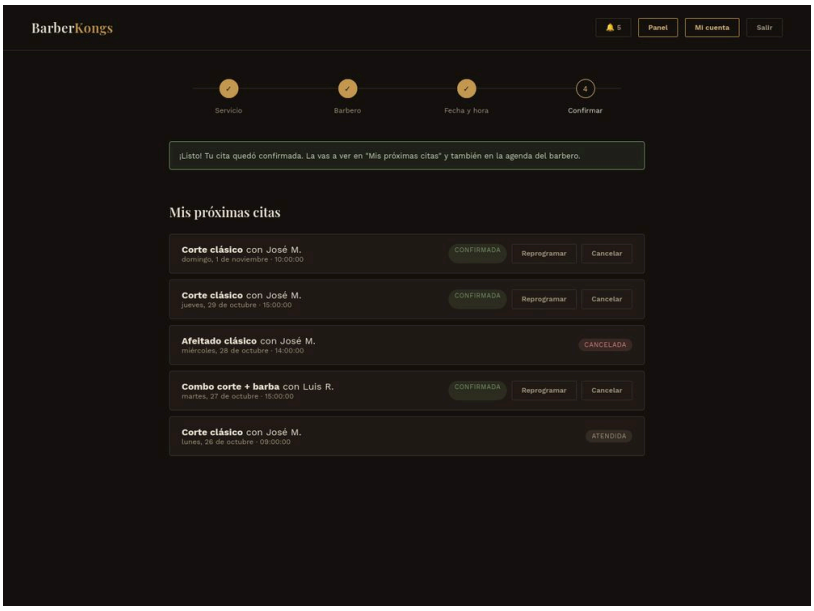


Figura 16. Mensaje de confirmación y la cita en la lista de próximas citas.

> Si otra persona toma el horario antes de que usted confirme, el sistema avisa que ya no está disponible y se debe elegir otro.

3.2 Ver, reprogramar y cancelar mis citas

Mis citas (también visible debajo del asistente de agendar) lista las citas del cliente, con su estado, ordenadas por fecha con la más reciente primero. Las citas confirmadas tienen dos botones.

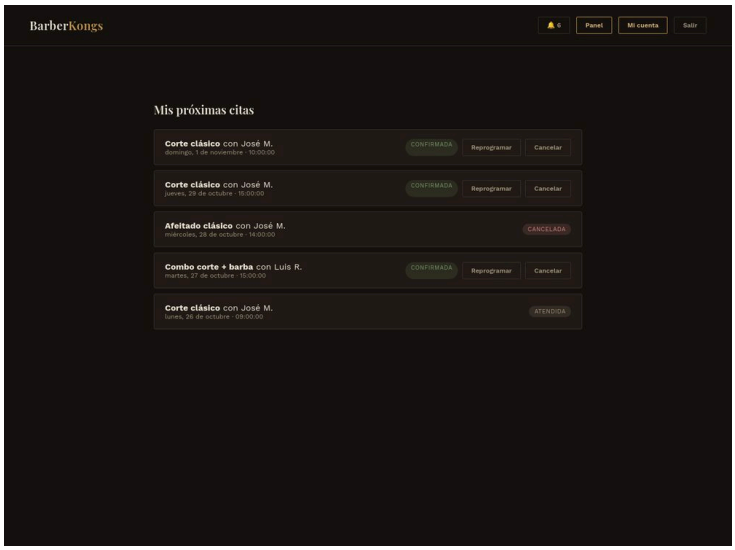


Figura 17. Lista de citas del cliente.

Reprogramar:

1. Presione **Reprogramar** en la cita que desea cambiar.
2. Elija una nueva fecha y una nueva hora entre las disponibles del mismo barbero.
3. Presione **Confirmar cambio**. Para no cambiar nada, presione **Cancelar** o la ×.



Figura 18. Ventana para elegir la nueva fecha y hora.

Cancelar: presione **Cancelar** en la cita. La cita queda cancelada y el horario vuelve a estar disponible para otros clientes. Una cita que ya fue atendida no se puede cancelar ni reprogramar.

3.3 Estados de una cita

Estado	Significado
Confirmada	La cita está agendada y pendiente de atenderse. Se puede reprogramar o cancelar.
Atendida	El barbero ya atendió al cliente. Queda en el historial.
Cancelada	La cita se canceló, por el cliente o por el administrador, y el horario quedó libre.

4. Guía del barbero

El barbero solo ve su propia agenda: no puede consultar la de otros barberos ni administrar clientes, servicios o inventario.

4.1 Agenda diaria

Al ingresar se abre la **agenda**, en vista **Diaria**. Muestra las franjas de una hora de la jornada con la cita de cada una (cliente, servicio y estado) o «Sin cita agendada», y el receso de almuerzo. Con ← **Anterior** y **Siguiente** → se cambia de día.

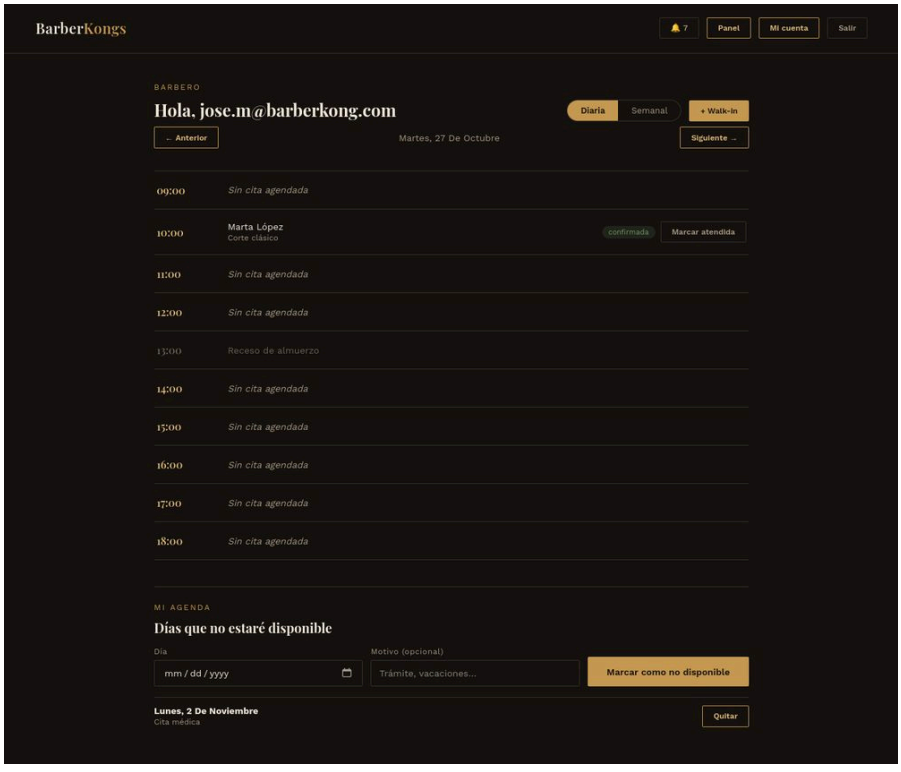


Figura 19. Agenda diaria con una cita confirmada.

4.2 Agenda semanal

El botón **Semanal** muestra la semana completa en una cuadrícula de horas por día (domingo a jueves). Los viernes y sábados aparecen rayados porque no se atiende. Las citas se identifican por color de estado.

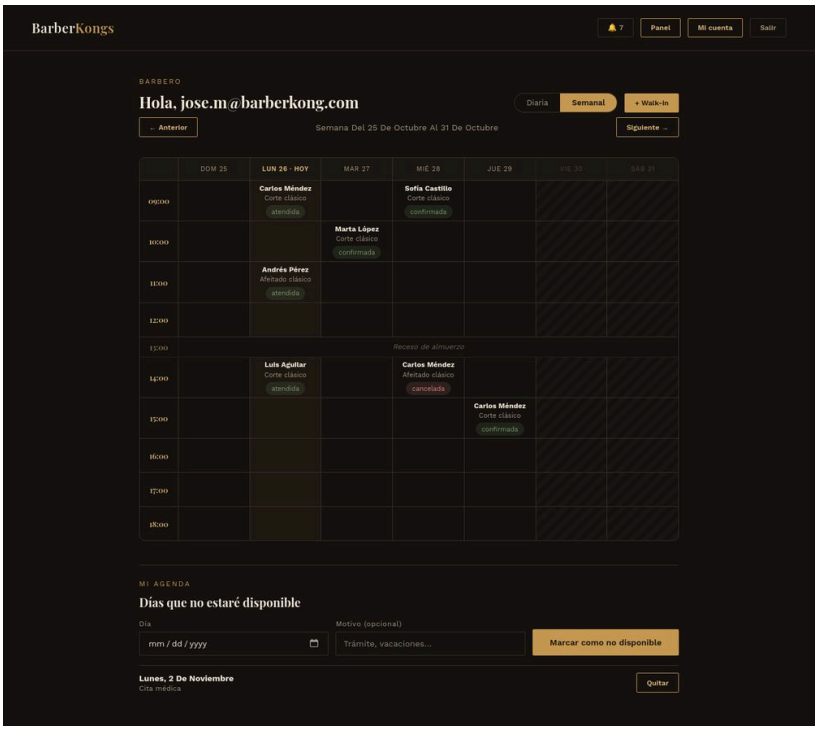


Figura 20. Agenda semanal.

4.3 Marcar una cita como atendida

Cuando termina de atender a un cliente, presione **Marcar atendida** en su cita. Al hacerlo, el sistema descuenta automáticamente del inventario los insumos que consume ese servicio. Si algún insumo queda por debajo de su mínimo, el administrador recibe una alerta. Solo se pueden marcar las citas propias, y una cita cancelada no se puede marcar.

4.4 Registrar un cliente sin cita (walk-in)

Para atender a alguien que llega sin cita, presione **+ Walk-in**, elija el servicio y la hora, escriba el nombre del cliente (opcional) y presione **Registrar**. La atención se registra en la agenda del barbero para el día que está viendo (por defecto, hoy). Si no se escribe nombre, queda registrada como «Walk-in».



Figura 21. Ventana para registrar un walk-in.

4.5 Días en que no estaré disponible

Debajo de la agenda, la sección **Días que no estaré disponible** permite bloquear un día completo (vacaciones, trámites): elija el día, escriba el motivo si lo desea y presione **Marcar como no disponible**. Ese día deja de ofrecerse a los clientes. Con **Quitar** se vuelve a habilitar. No se puede bloquear un día pasado ni un día que ya tiene citas: primero deben cancelarse o reprogramarse.

5. Guía del administrador

El administrador entra al panel de administración, cuyo menú lateral da acceso a las secciones **Citas**, **Barberos**, **Clientes**, **Servicios**, **Inventario** y **Reportes**, además de **Seguridad** (verificación en dos pasos) y **Mi cuenta**.

5.1 Citas del día

La pantalla inicial resume el día: citas de hoy, ingresos estimados (suma del precio de las citas del día que no están canceladas) y barberos con agenda, y lista las citas con su estado. Desde aquí se puede **Cancelar** una cita confirmada (el sistema pide confirmar) y registrar una atención sin cita con **+ Walk-in**, eligiendo servicio, barbero, hora y, si se desea, el nombre del cliente.

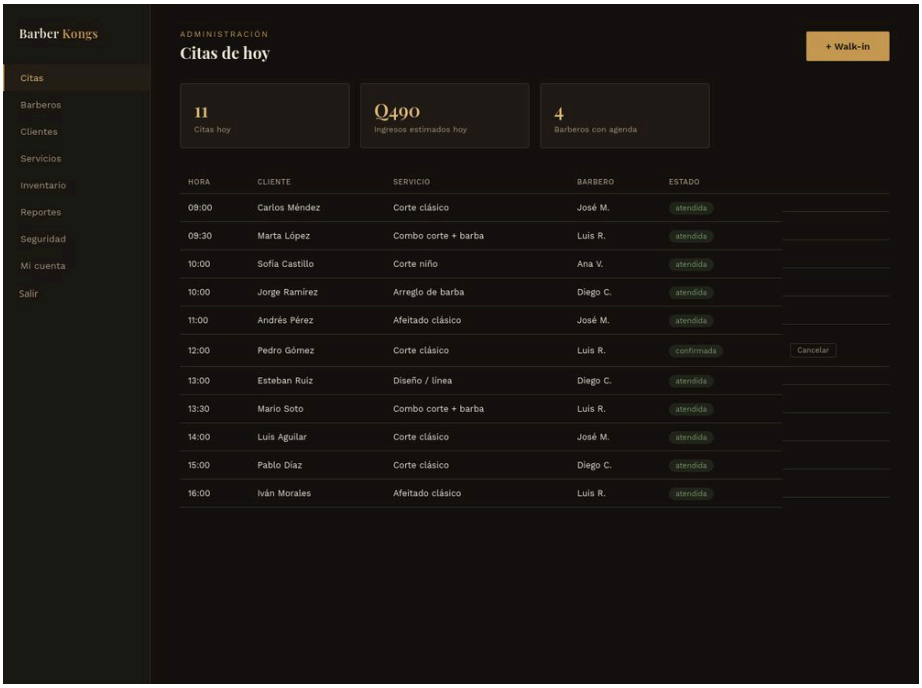


Figura 22. Panel de citas del día.

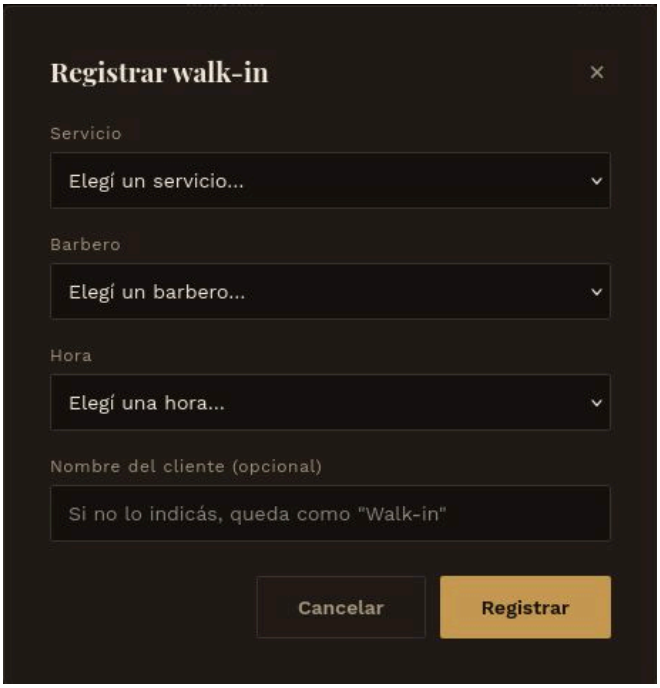


Figura 23. Registro de un walk-in desde el panel.

5.2 Barberos

Lista a todos los barberos con su especialidad y su estado. Desde aquí se crean, editan y desactivan.

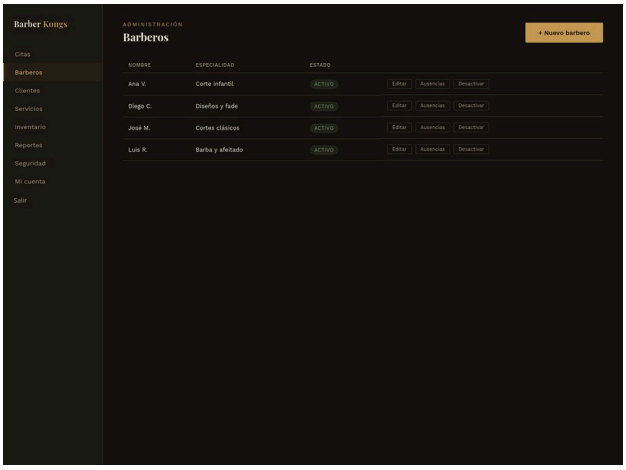


Figura 24. Lista de barberos.

Crear un barbero:

1. Presione **+ Nuevo barbero**.
2. Escriba el nombre, la especialidad, el correo con el que iniciará sesión y una contraseña inicial.
3. Revise el **horario de atención**: por defecto, domingo a jueves de 9:00 a. m. a 7:00 p. m. Cambie las horas, agregue otro bloque en el mismo día con **+ Agregar bloque** (por ejemplo, para dejar un receso) o quite un día con la ×. Los días sin horario aparecen como «No atiende».
4. Presione **Guardar**. El sistema avisa qué bloque está mal si las horas se traslapan o terminan antes de empezar.

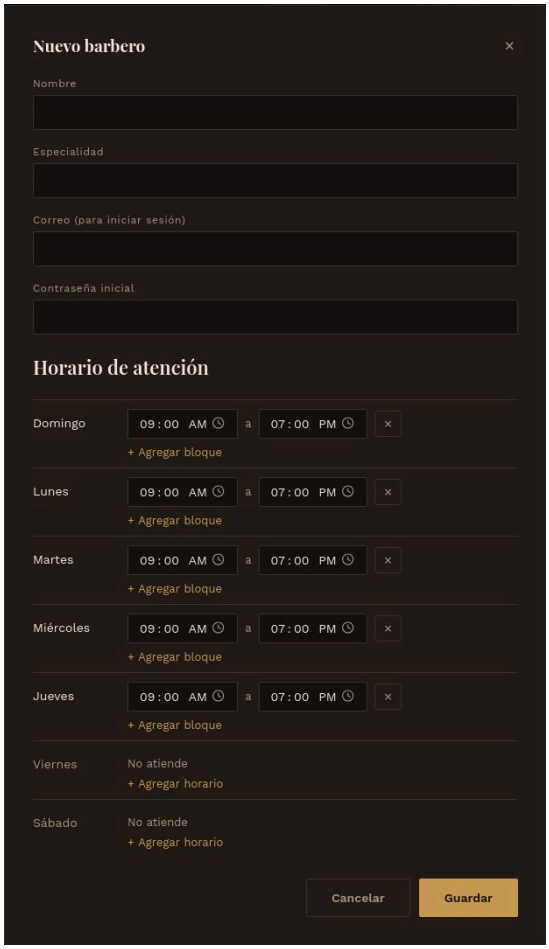


Figura 25. Alta de un barbero con su horario.



Figura 26. Edición de un barbero.

Editar: con **Editar** se cambia el nombre, la especialidad y el horario de un barbero (segunda ventana de la figura anterior). El correo y la contraseña no se modifican desde aquí.

Ausencias: con **Ausencias** el administrador bloquea o habilita días completos de cualquier barbero, igual que el propio barbero en su agenda.



Figura 27. Días sin disponibilidad de un barbero.

Desactivar: presione **Desactivar** y confirme. El barbero deja de aparecer para los clientes, no se le pueden agendar nuevas citas y su cuenta ya no puede iniciar sesión. El registro y su historial se conservan.

5.3 Clientes

Lista los clientes con su teléfono, correo, total de citas y fecha de última visita. Incluye tanto a quienes crearon su cuenta como a los clientes registrados como walk-in con nombre.

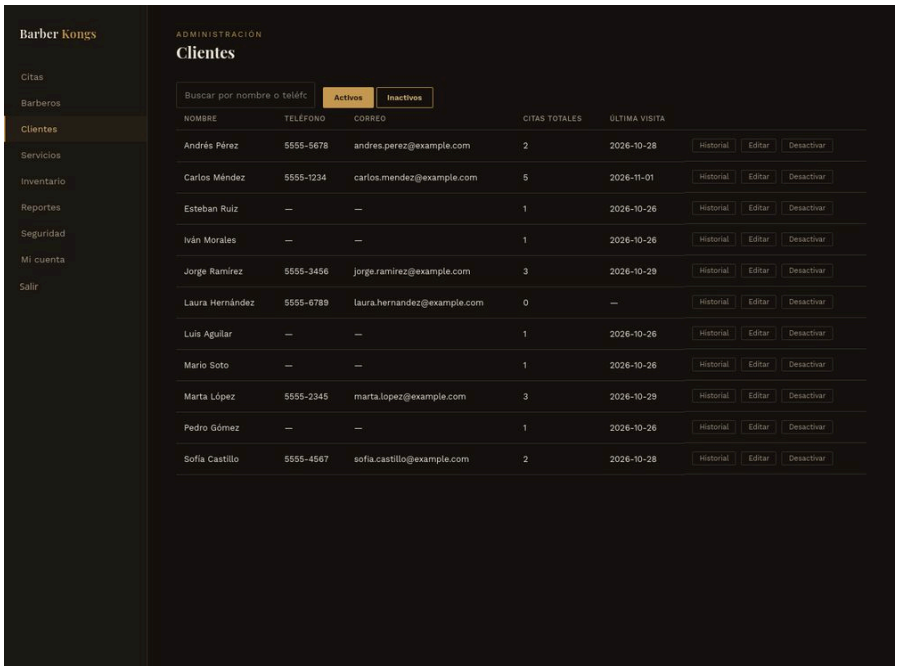


Figura 28. Lista de clientes.

- **Buscar:** escriba parte del nombre o del teléfono en el campo de búsqueda; la lista se filtra.
- **Historial:** muestra todas las citas del cliente con fecha, hora, servicio, barbero y estado.
- **Editar:** permite corregir nombre, teléfono y correo.
- **Desactivar y reactivar: Desactivar** (con confirmación) oculta al cliente del listado sin borrar su historial. El botón **Inactivos** muestra a los desactivados y **Reactivar** los devuelve a la lista de activos.



Figura 29. Historial de citas de un cliente.

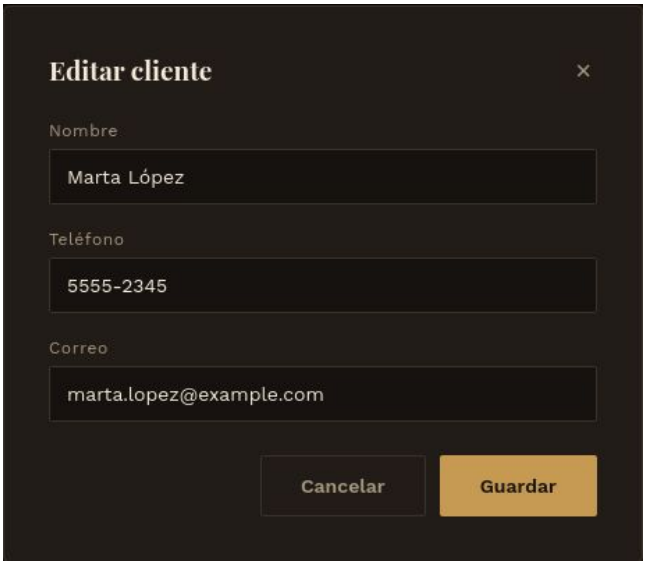


Figura 30. Edición de los datos de un cliente.

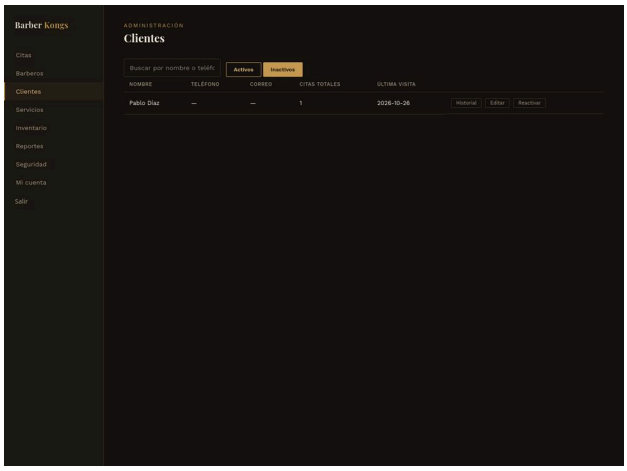


Figura 31. Clientes inactivos, con el botón Reactivar.

5.4 Servicios

El catálogo de servicios es lo que ven los clientes al agendar. Cada servicio tiene nombre, duración en minutos y precio en quetzales.

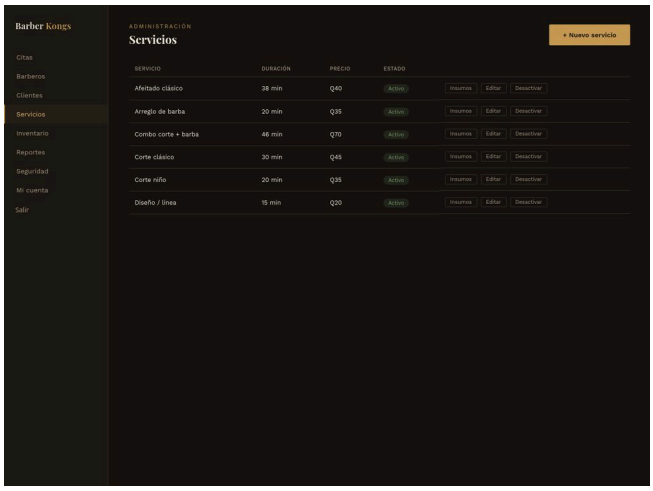


Figura 32. Catálogo de servicios.

- **+ Nuevo servicio:** escriba nombre, duración y precio, y presione **Guardar**.
- **Editar:** cambia los datos de un servicio existente.
- **Desactivar:** el servicio deja de ofrecerse a los clientes; las citas ya agendadas no se alteran.
- **Insumos:** define qué insumos consume el servicio y cuánto de cada uno. Elija el insumo, escriba la cantidad y presione **Agregar**; con **Quitar** se elimina una asociación. Estas cantidades son las que se descuentan del inventario cuando se marca una cita como atendida.



Figura 33. Alta de un servicio.



Figura 34. Insumos que consume un servicio.

5.5 Inventario

Muestra cuántos insumos hay registrados, cuántos están con **stock bajo** y, por insumo, su cantidad, unidad y mínimo. Un insumo cuya cantidad es menor que su mínimo aparece resaltado como **Stock bajo**.

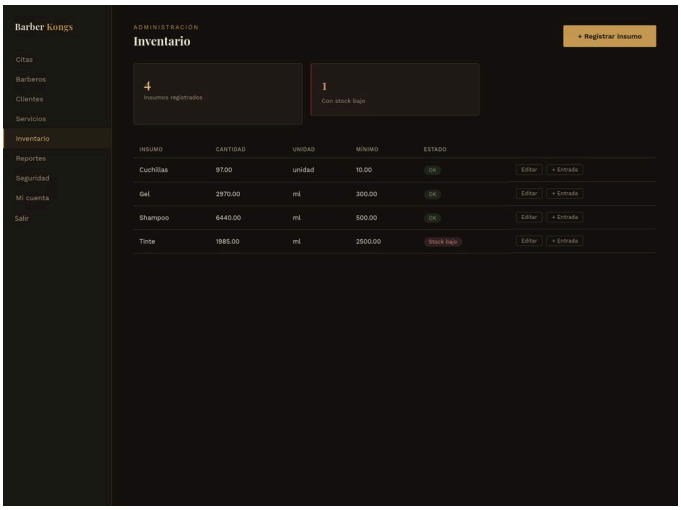


Figura 35. Inventario con un insumo en stock bajo.

- **+ Registrar insumo:** escriba nombre y unidad de medida (litros, unidades, ml…) y, si desea recibir alertas, la **cantidad mínima**. Todo insumo nuevo empieza con cantidad cero.
- **Editar:** cambia el nombre, la unidad de medida y la cantidad mínima de un insumo. Si se deja vacía la cantidad mínima, deja de generarse la alerta de ese insumo. La cantidad actual no se edita aquí: se corrige con **+ Entrada**.
- **+ Entrada:** registra un reabastecimiento; la cantidad ingresada se suma a la existencia y queda en el historial de movimientos.
- **Alertas:** cuando una atención deja un insumo por debajo de su mínimo, el sistema avisa a los administradores con una notificación y un correo.



Figura 36. Alta de un insumo.



Figura 37. Edición de un insumo.



Figura 38. Registro de una entrada (reabastecimiento).

5.6 Reportes

La sección **Reportes** genera tres reportes para el rango de fechas elegido en **Desde** y **Hasta**: **Citas por rango de fechas** (atendidas, canceladas y confirmadas), **Ingresos** (estimados por servicio y por barbero) y **Consumo de insumos**. Cada uno se descarga en **PDF** o en **Excel** con los botones de su tarjeta; el archivo lleva en el nombre el tipo de reporte y el periodo.

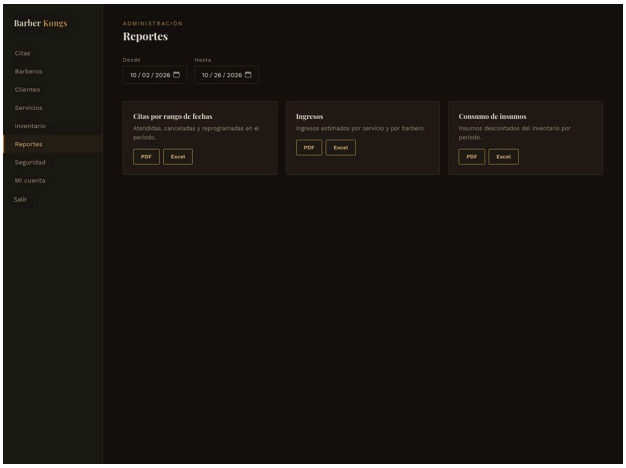
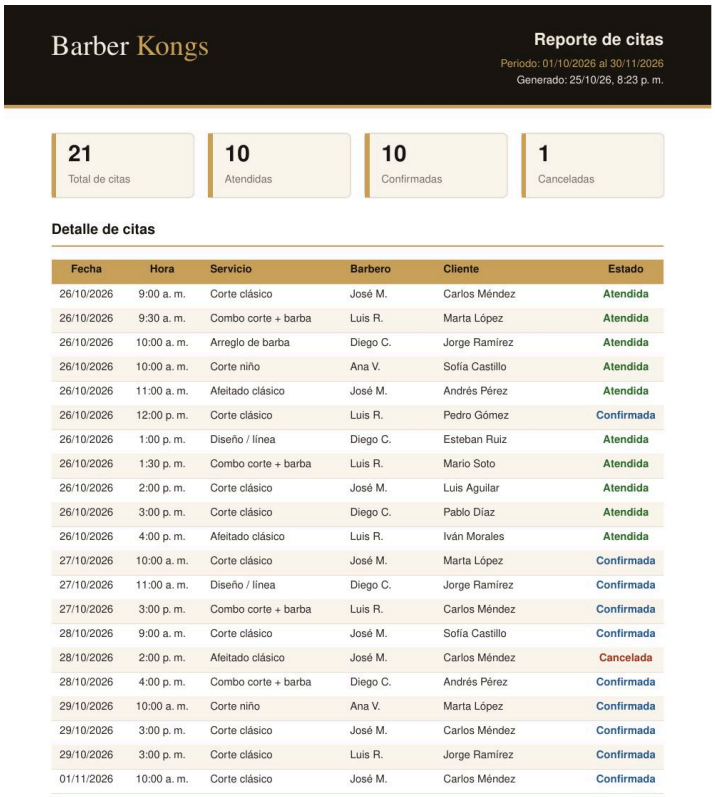


Figura 39. Pantalla de reportes.

Ejemplos de lo que se obtiene:



Barber Kongs

Reporte de citas

Período: 01/10/2026 al 30/11/2026

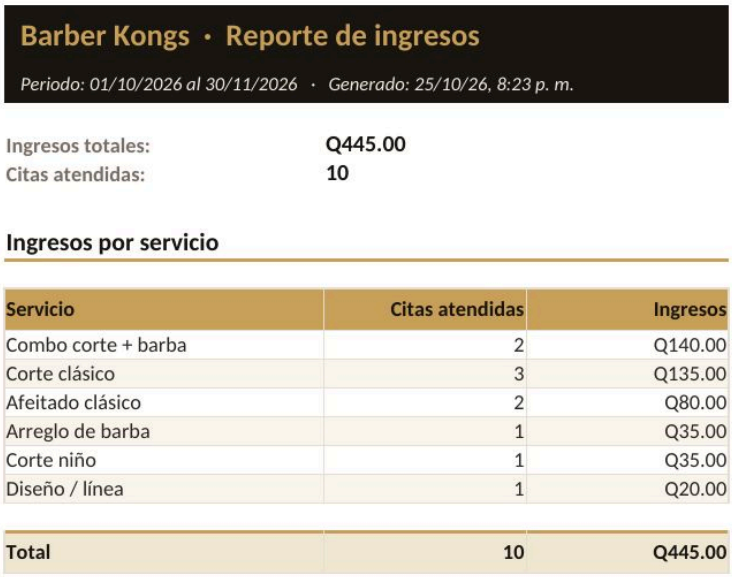
Generado: 25/10/26, 8:23 p. m.

21	10	10	1
Total de citas	Atendidas	Confirmadas	Canceladas

Detalle de citas

Fecha	Hora	Servicio	Barbero	Cliente	Estado
26/10/2026	9:00 a. m.	Corte clásico	José M.	Carlos Méndez	Atendida
26/10/2026	9:30 a. m.	Combo corte + barba	Luis R.	Marta López	Atendida
26/10/2026	10:00 a. m.	Arreglo de barba	Diego C.	Jorge Ramírez	Atendida
26/10/2026	10:00 a. m.	Corte niño	Ana V.	Sofía Castillo	Atendida
26/10/2026	11:00 a. m.	Afeitado clásico	José M.	Andrés Pérez	Atendida
26/10/2026	12:00 p. m.	Corte clásico	Luis R.	Pedro Gómez	Confirmada
26/10/2026	1:00 p. m.	Diseño / línea	Diego C.	Esteban Ruiz	Atendida
26/10/2026	1:30 p. m.	Combo corte + barba	Luis R.	Mario Soto	Atendida
26/10/2026	2:00 p. m.	Corte clásico	José M.	Luis Aguilar	Atendida
26/10/2026	3:00 p. m.	Corte clásico	Diego C.	Pablo Díaz	Atendida
26/10/2026	4:00 p. m.	Afeitado clásico	Luis R.	Iván Morales	Atendida
27/10/2026	10:00 a. m.	Corte clásico	José M.	Marta López	Confirmada
27/10/2026	11:00 a. m.	Diseño / línea	Diego C.	Jorge Ramírez	Confirmada
27/10/2026	3:00 p. m.	Combo corte + barba	Luis R.	Carlos Méndez	Confirmada
28/10/2026	9:00 a. m.	Corte clásico	José M.	Sofía Castillo	Confirmada
28/10/2026	2:00 p. m.	Afeitado clásico	José M.	Carlos Méndez	Cancelada
28/10/2026	4:00 p. m.	Combo corte + barba	Diego C.	Andrés Pérez	Confirmada
29/10/2026	10:00 a. m.	Corte niño	Ana V.	Marta López	Confirmada
29/10/2026	3:00 p. m.	Corte clásico	José M.	Carlos Méndez	Confirmada
29/10/2026	3:00 p. m.	Corte clásico	Luis R.	Jorge Ramírez	Confirmada
01/11/2026	10:00 a. m.	Corte clásico	José M.	Carlos Méndez	Confirmada

Figura 40. Reporte de citas en PDF (primera página).



Barber Kongs · Reporte de ingresos

Periodo: 01/10/2026 al 30/11/2026 · Generado: 25/10/26, 8:23 p. m.

Ingresos totales:	Q445.00
Citas atendidas:	10

Ingresos por servicio

Servicio	Citas atendidas	Ingresos
Combo corte + barba	2	Q140.00
Corte clásico	3	Q135.00
Afeitado clásico	2	Q80.00
Arreglo de barba	1	Q35.00
Corte niño	1	Q35.00
Diseño / línea	1	Q20.00
Total	**10**	**Q445.00**

Figura 41. Reporte de ingresos en Excel: totales y detalle por servicio.



Figura 42. Reporte de consumo de insumos en PDF.

> Si en el periodo elegido no hay datos, el reporte se descarga igual, con las cifras en cero.

6. Mensajes que puede encontrar

Situación	Mensaje	Qué hacer
Datos de ingreso incorrectos	Correo o contraseña incorrectos.	Revisar los datos o usar «Recuperarla».
Correo ya registrado	Ya existe una cuenta con ese correo.	Ingresar con esa cuenta o crear la cuenta con otro correo.
Código de dos pasos inválido	El código es incorrecto o ya expiró.	Escribirlo de nuevo o ingresar otra vez para recibir uno nuevo.
Enlace de recuperación vencido	El enlace no es válido o ya expiró.	Solicitar un enlace nuevo.
Horario ocupado al confirmar	Ese horario ya no está disponible. Elegí otro.	Volver y elegir otra hora.
Fecha pasada	No se puede agendar en una fecha pasada.	Elegir hoy o una fecha posterior.
Cita ya atendida	Esta cita ya fue atendida, no se puede cancelar.	No requiere acción; queda en el historial.
Día no disponible del barbero	Ese día ya hay N cita(s) agendada(s) con este barbero. Cancélalas o reprográmalas antes de marcarlo como no disponible.	Cancelar o reprogramar esas citas antes de bloquear el día.

7. Preguntas frecuentes

¿Por qué no puedo elegir viernes o sábado?

La barbería atiende de domingo a jueves. Esos dos días aparecen atenuados en el asistente de agenda.

¿Por qué un día aparece atenuado si es de domingo a jueves?

Porque el barbero elegido marcó ese día como no disponible o no tiene horario ese día. Elija otro día u otro barbero.

¿Qué pasa con mi cita si la reprogramo?

Se mantiene como una sola cita con la nueva fecha y hora. El horario anterior queda libre.

No me llegó el correo de recuperación o el código. ¿Qué hago?

Revise la carpeta de correo no deseado. Si el mensaje no llega, vuelva a ingresar para recibir un código nuevo o solicite otro enlace de recuperación.

Olvidé mi contraseña y la cuenta es de barbero o administrador.

El procedimiento es el mismo que para los clientes (sección 2.5).

¿Puedo usar el sistema desde el celular?

Sí. La aplicación se adapta al tamaño de la pantalla y se usa desde el navegador del teléfono.